

# Everyday Git for Overworked Mac Admins

PSU MacAdmins 2026

Beth Johnson



Welcome to Everyday Git for Overworked Mac Admins. My name is Beth Johnson, if you're not able to see me, I am an average height, average weight, middle-aged woman, wearing *whatever I'm wearing today*, and I manage computers at Mount Holyoke College in Western Massachusetts.

# First Things First



Edmonia Lewis, The Old Indian Arrow Maker and His Daughter, 1867

I'm going to start off with some housekeeping notes while we all finish settling in.

## About this presentation

A little bit about this presentation to set expectations.

- The QR code goes to the resources for this presentation, which will also be shown on multiple slides at the end so don't feel like you need to scramble to get it now.
- There's a shared Google Doc for note-taking. I've linked it in the resources and it is also shared in the Slack channel for this room.
- What I'm going to cover are some very simple use cases, with the idea of getting you started if you don't know where to start. And also helping you avoid some of the rabbit holes that I fell down.
- I'm sure there are mistakes here. It's the nature of being a busy Mac Admin that you will be interrupted when you are trying to learn something new or get important work done. We can only do our best. Perfection is not in the job description.
- I like puns, people. I didn't intentionally seek puns, but they happen.
- My bumper images are the works of artists who are women, whose work you may not be familiar with. I use them because the works are beautiful and more people should know these artists.
- In related news, I don't use any form of generative "AI" in my presentations. So, no artists were harmed in the course of developing this.





## About this presentation



Alice Aycock, Three Fold Manifestation, 1987

## About this presentation

- Links to resources



Alice Aycock, Three Fold Manifestation, 1987

## About this presentation

- Links to resources
- Shared note Google Doc -  
#psumac-208



Alice Aycock, Three Fold Manifestation, 1987

## About this presentation

- Links to resources
- Shared note Google Doc - #psumac-208
- Introduction and basics



Alice Aycock, Three Fold Manifestation, 1987

## About this presentation

- Links to resources
- Shared note Google Doc - #psumac-208
- Introduction and basics
- Mistakes? We got 'em!



Alice Aycock, Three Fold Manifestation, 1987

## About this presentation

- Links to resources
- Shared note Google Doc - #psumac-208
- Introduction and basics
- Mistakes? We got 'em!
- Content warning: puns



Alice Aycock, Three Fold Manifestation, 1987



## About this presentation

- Links to resources
- Shared note Google Doc - #psumac-208
- Introduction and basics
- Mistakes? We got 'em!
- Content warning: puns
- Bumper art - women artists



Alice Aycock, Three Fold Manifestation, 1987

## About this presentation

- Links to resources
- Shared note Google Doc - #psumac-208
- Introduction and basics
- Mistakes? We got 'em!
- Content warning: puns
- Bumper art - women artists
- No "AI"/LLMs, etc.



Alice Aycock, Three Fold Manifestation, 1987



# What you'll see



Georgia O'Keeffe, Part of the Cliff, 1946

What I'm going to show you more specifically is:

- How to check for and install Git on your computer.
- Some local and cloud tools you can use
- How to make your first repository
- How to manage everyday functions to store changes, update your repository, and create documentation.
- When you might want to branch and merge.
- How to interact with other folks' projects on GitHub or GitLab.

# What you'll see

- How to get Git



Georgia O'Keeffe, Part of the Cliff, 1946

# What you'll see

- How to get Git
- Local & cloud tools and apps



Georgia O'Keeffe, Part of the Cliff, 1946

# What you'll see

- How to get Git
- Local & cloud tools and apps
- Repository creation



Georgia O'Keeffe, Part of the Cliff, 1946

# What you'll see

- How to get Git
- Local & cloud tools and apps
- Repository creation
- Everyday Git functions



Georgia O'Keeffe, Part of the Cliff, 1946

# What you'll see

- How to get Git
- Local & cloud tools and apps
- Repository creation
- Everyday Git functions
- When to branch and merge



Georgia O'Keeffe, Part of the Cliff, 1946

# What you'll see

- How to get Git
- Local & cloud tools and apps
- Repository creation
- Everyday Git functions
- When to branch and merge
- How to collaborate



Georgia O'Keeffe, Part of the Cliff, 1946



# What we won't do

## Important message

And equally important, what this presentation is not.

I will not:

- Laboriously set up repos and work on the command line interface. I'll show you just enough of the CLI to be useful.
- Expect you have deep knowledge of Git and Git terminology
- Assume that you are an old hand at development processes
- Or imagine that your needs are complex and you need some fancy system.



# What we won't do

Important message



Käthe Kollwitz, Tower of Mothers, 1937

# What we won't do

## Important message

- Laboriously set up repos on the CLI



Käthe Kollwitz, Tower of Mothers, 1937

# What we won't do

## Important message

- Laboriously set up repos on the CLI
- Expect deep knowledge of Git



Käthe Kollwitz, Tower of Mothers, 1937

# What we won't do

## Important message

- Laboriously set up repos on the CLI
- Expect deep knowledge of Git
- Assume you're a developer



Käthe Kollwitz, Tower of Mothers, 1937

# What we won't do

## Important message

- Laboriously set up repos on the CLI
- Expect deep knowledge of Git
- Assume you're a developer
- Imagine your needs are complex



Käthe Kollwitz, Tower of Mothers, 1937

# My context

**“Venerable but hip” New England  
liberal arts college**

Now, quickly, a bit about my context as an overworked Mac Admin

We're a private college in New England and I get to work in this iconic building, which is fabulous.

- We have about 2100 workstations we manage
- They are about 45% macOS, 54% Windows, iPadOS & Linux
- We have a mix of office computers for admin and academic for teaching and research
- Our institution is academically competitive and
- We have a great many accomplished alumnae or alums . . .

# My context

"Venerable but hip" New England  
liberal arts college





# My context

**"Venerable but hip" New England liberal arts college**

- About 2100 computers





# My context

**"Venerable but hip" New England liberal arts college**

- About 2100 computers
- macOS, Windows, iPadOS & Linux



# My context

**“Venerable but hip” New England liberal arts college**

- About 2100 computers
- macOS, Windows, iPadOS & Linux
- Mix of admin and academic use



# My context

**“Venerable but hip” New England liberal arts college**

- About 2100 computers
- macOS, Windows, iPadOS & Linux
- Mix of admin and academic use
- Academically competitive



# My context

**“Venerable but hip” New England liberal arts college**

- About 2100 computers
- macOS, Windows, iPadOS & Linux
- Mix of admin and academic use
- Academically competitive
- Accomplished alums



# Frances Perkins

Class of 1902

Including Frances Perkins!

- She was the first woman to serve as a Presidential cabinet member in the United States.
- She was Secretary of Labor from 1933 to 1945 and was a major architect of the New Deal.
- If you enjoy any of these things, you can thank Frances Perkins:
  - Social Security
  - The Federal minimum wage
  - Unemployment insurance, and
  - Not making schoolchildren work in factories

# Frances Perkins

Class of 1902



# Frances Perkins

**Class of 1902**

- First woman US cabinet member





# Frances Perkins

**Class of 1902**

- First woman US cabinet member
- Secretary of Labor, 1933-1945, New Deal





# Frances Perkins

**Class of 1902**

- First woman US cabinet member
- Secretary of Labor, 1933-1945, New Deal
- Thanks to Secretary Perkins:



# Frances Perkins

**Class of 1902**

- First woman US cabinet member
- Secretary of Labor, 1933-1945, New Deal
- Thanks to Secretary Perkins:
  - Social Security



# Frances Perkins

**Class of 1902**

- First woman US cabinet member
- Secretary of Labor, 1933-1945, New Deal
- Thanks to Secretary Perkins:
  - Social Security
  - Minimum wage



# Frances Perkins

**Class of 1902**

- First woman US cabinet member
- Secretary of Labor, 1933-1945, New Deal
- Thanks to Secretary Perkins:
  - Social Security
  - Minimum wage
  - Unemployment insurance



# Frances Perkins

**Class of 1902**

- First woman US cabinet member
- Secretary of Labor, 1933-1945, New Deal
- Thanks to Secretary Perkins:
  - Social Security
  - Minimum wage
  - Unemployment insurance
  - Child labor laws



# About me

(A less famous alum)

I am a less famous alumna, and in fact was part of a program for non-traditional aged students that was named after Frances Perkins.

- This is my 3rd time presenting at PSU Mac Admins
- I've been doing endpoint engineering or architecting -- or whatever magic words get me the most money for my work -- for 12 years now, having been promoted from the Helpdesk.
- Before that we didn't have any centralized management for either macOS or Windows.
- And before that, like maybe some of you, I was a Mac Technician in retail.
- And even before that, I was a perinatal educator and doula, which is more like tech support than a lot of people think.
- I completed my Bachelor's in 2017, while I was in the early years of building out our central management. And I majored in studio art, with a minor in Asian Studies.
- And I am neurodivergent, which leads me directly into the origin story for this session . . .

# About me

(A less famous alum)



Beth Johnson, Entrainment V, 2017



# About me

(A less famous alum)

- 3rd time presenting at PSU MacAdmins



Beth Johnson, Entrainment V, 2017



# About me

(A less famous alum)

- 3rd time presenting at PSU MacAdmins
- ~12 Years managing Macs/Windows



Beth Johnson, Entrainment V, 2017

# About me

(A less famous alum)

- 3rd time presenting at PSU MacAdmins
- ~12 Years managing Macs/Windows
- Founded our central management



Beth Johnson, Entrainment V, 2017

# About me

(A less famous alum)

- 3rd time presenting at PSU MacAdmins
- ~12 Years managing Macs/Windows
- Founded our central management
- "Genius Emeritus"



Beth Johnson, Entrainment V, 2017

# About me

(A less famous alum)

- 3rd time presenting at PSU MacAdmins
- ~12 Years managing Macs/Windows
- Founded our central management
- "Genius Emeritus"
- Former perinatal educator/doula



Beth Johnson, Entrainment V, 2017

# About me

(A less famous alum)

- 3rd time presenting at PSU MacAdmins
- ~12 Years managing Macs/Windows
- Founded our central management
- "Genius Emeritus"
- Former perinatal educator/doula
- "I'm an art major, not a developer!"



Beth Johnson, Entrainment V, 2017

# About me

(A less famous alum)

- 3rd time presenting at PSU MacAdmins
- ~12 Years managing Macs/Windows
- Founded our central management
- "Genius Emeritus"
- Former perinatal educator/doula
- "I'm an art major, not a developer!"
- Neurospice level: 🌶️🌶️🌶️



Beth Johnson, Entrainment V, 2017

# Why we're here . . .

That is, why we're here . . .

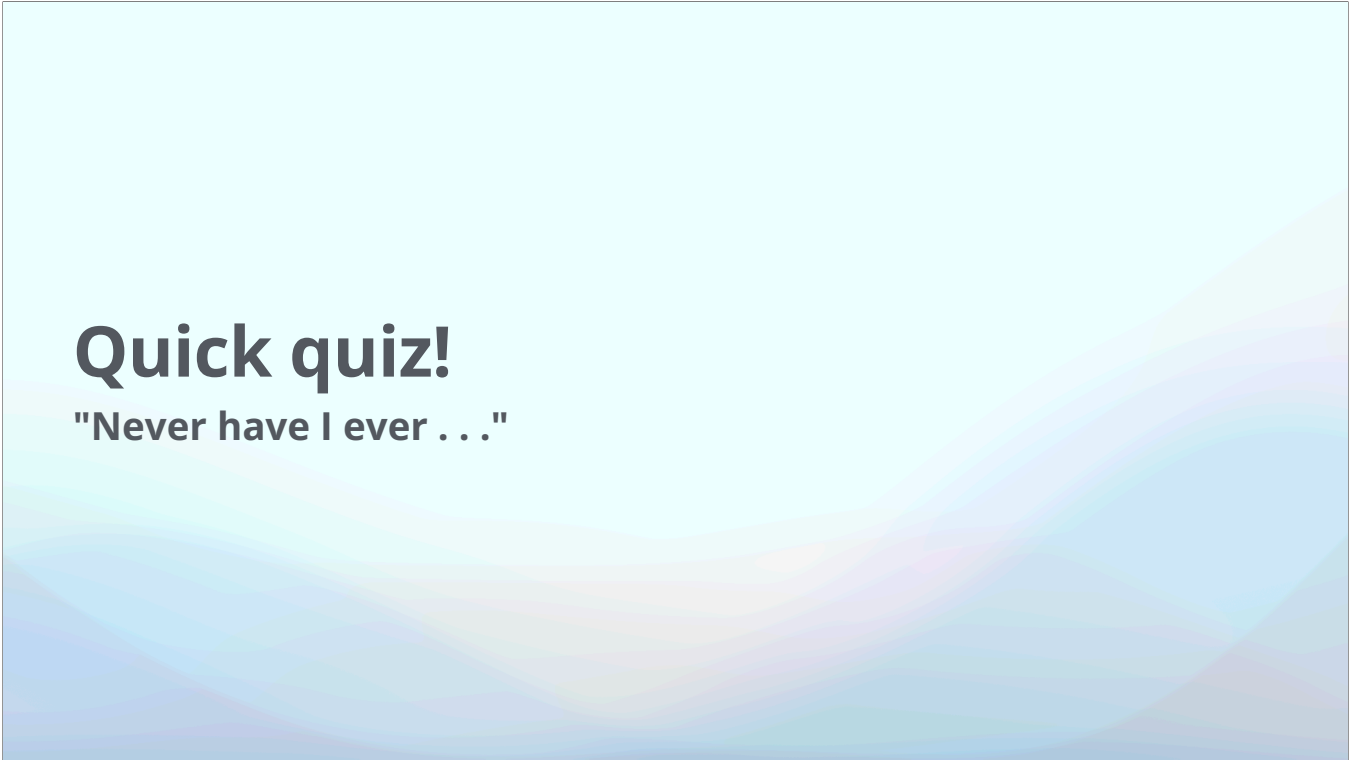


# Why we're here . . .



Rosa Bonheur, The Horse Fair, 1852-1855





## Quick quiz!

"Never have I ever ..."

This is by Artemisia Gentileschi. She was born on this day in 1593.

Show of hands: Who here has put the wrong script into production because you made a mistake about which version was the one you had tested?

Let me tell you a sad story.

## Quick quiz!

"Never have I ever . . ."



Artemesia Gentileschi, *Judith and Her Maidservant with the Head of Holofernes*, 1614

# Storytime

## The Setup

We have several different admins in our Mac management ecosystem, and we use Jamf Pro.

The other admins are deploying to computer labs and receive their computers in big batches. They'll do an inventory upload and put their computers into their own Prestage Enrollment workflows. That works great for them.

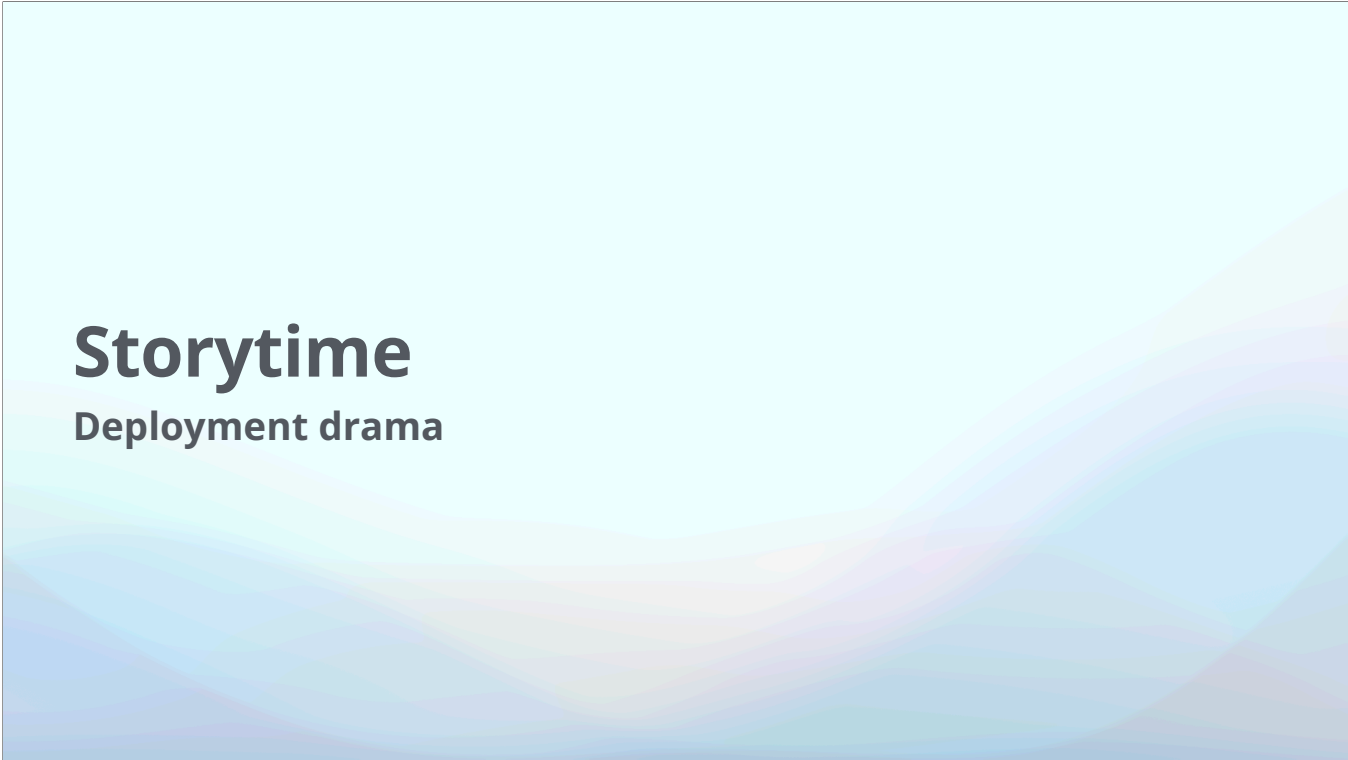
The rest of the computers are acquired singly or in small batches and automatically added to our 1:1 prestage. But the 1:1 prestage is a big bucket, and computers may need something finer grained.

# Storytime

## The Setup



Ana Mendieta, Untitled Silueta Series, 1976



# Storytime

## Deployment drama

We're not a zero-touch shop, but for many years now we've been using Dan Snelson's Setup Your Mac script to assist the technicians and their student helpers with setting up computers. And when we started experimenting with SYM,

- I was able to add some additional input fields and a defaults write command to allow them to make a more granular and definitive management group selection on the Desktop.

This has actually worked very well for us for a number of years now -- because we're making a positive choice about the computer's correct management group, we can hold off applying some of the policies and configuration profiles until after SYM is running, and differentiate between, say, a 1:1 computer for a visiting faculty member in Music, and a computer being set up as a shared workstation for student workers, all without having to move computers between Prestage Enrollment scopes or into static groups.

# Storytime

## Deployment drama

### Let's set up a new iMac for Beth



Register this computer by entering the required information below. Then select **CONTINUE** to start configuration.

If you need assistance, please consult the [MDE Deployment Help Guide](#).

- **Model Name:**  (18,1) (18,1)
- **Model Name:**  (18,1) (18,1)
- **Model Name:**  (18,1) (18,1)

**User Name:**   
**Company Name:**   
**Serial Tag \***   
**Power \***

**Configuration**

☒ **Standard**  
☐ **Power Mac G4**  
☐ **Generation**

**Building \***    
**Department \***    
**Deployment Type \***    
**Management Category \***

\* Required Fields

# Storytime

## Deployment drama

# Let's set up a new iMac for Beth



Register this computer by entering the required information below, then select **Continue to start configuration**.

If you need assistance, please contact the [MTE Technology Help Desk](#):

- Telephone: (714) 638-3600
- Email: [help@mtetech.edu](mailto:help@mtetech.edu)
- Web: [mtetech.edu](http://mtetech.edu)
- [Download User Manual PDF](#)

**Connections**

- Download image

**Customize**

- Minimal System
- Recommended
- Complete System

**Unit Name**

**Computer Name**

**Asset Tag \***

**Power \***

**Configuration**

- ☒ Standard
- ☐ Performance
- ☐ General

**Building \***

**Department \***

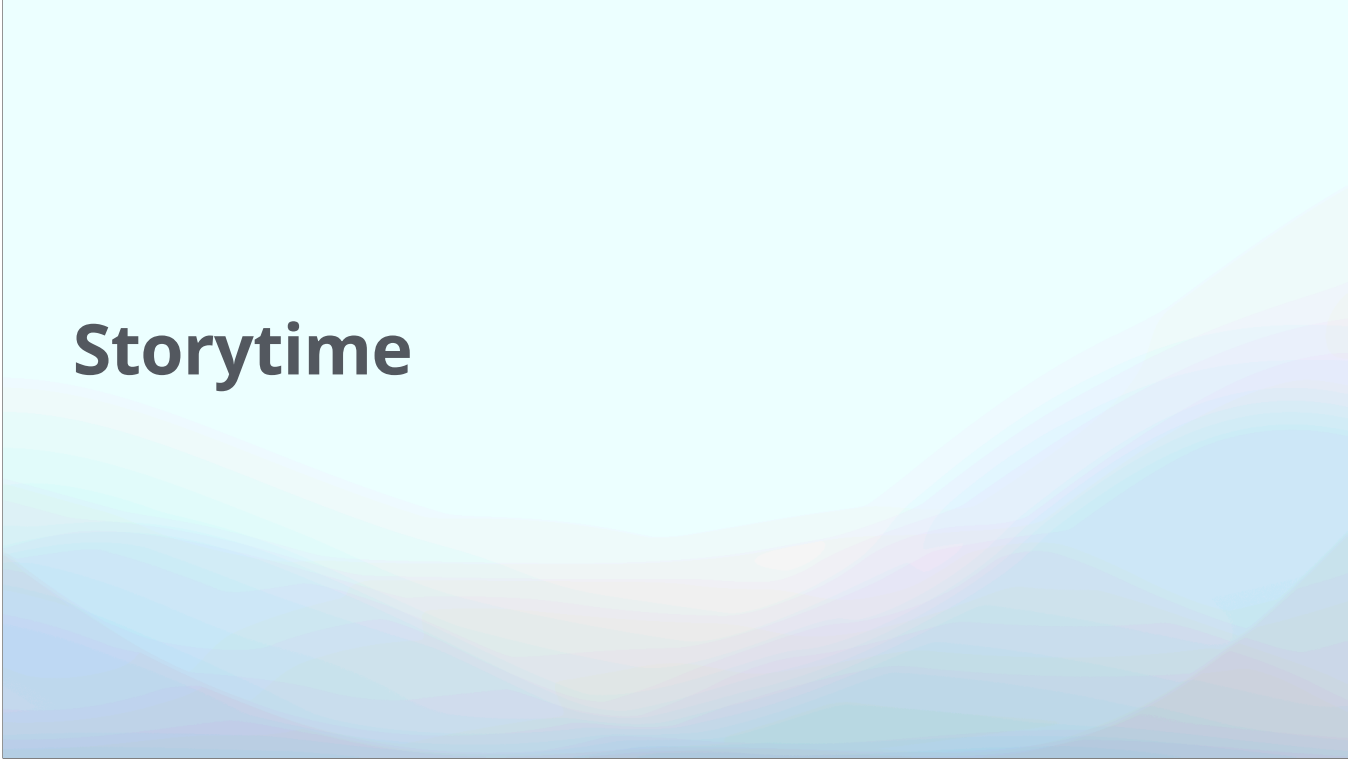
**Room/Room Type \***

**Management Category \***

\* Search Your...

1/19/20





## Storytime

I say it's worked quite well, until I noticed that it wasn't -- and in fact it **hadn't**, for some time. I clocked a pattern where computers were missing from a status collection they should have been members of.

- When I looked at the logs, I found that SYM wasn't doing the thing that I had modified it to do some years ago. And when I ran that SYM script from production, I found that it in fact was **not** a version that I had tested, because it didn't work: My custom input fields did not render.

Because of that, technicians were not able to make the selection to set the management group. And computers were not getting all the post-setup policies and profiles they should have been in scope for.

How long?



# Storytime



Jenny Holzer, Light Line, 1989

# Storytime

"sometimes there is a  
luxurious amount of time  
before anything bad  
happens"



Jenny Holzer, Light Line, 1989

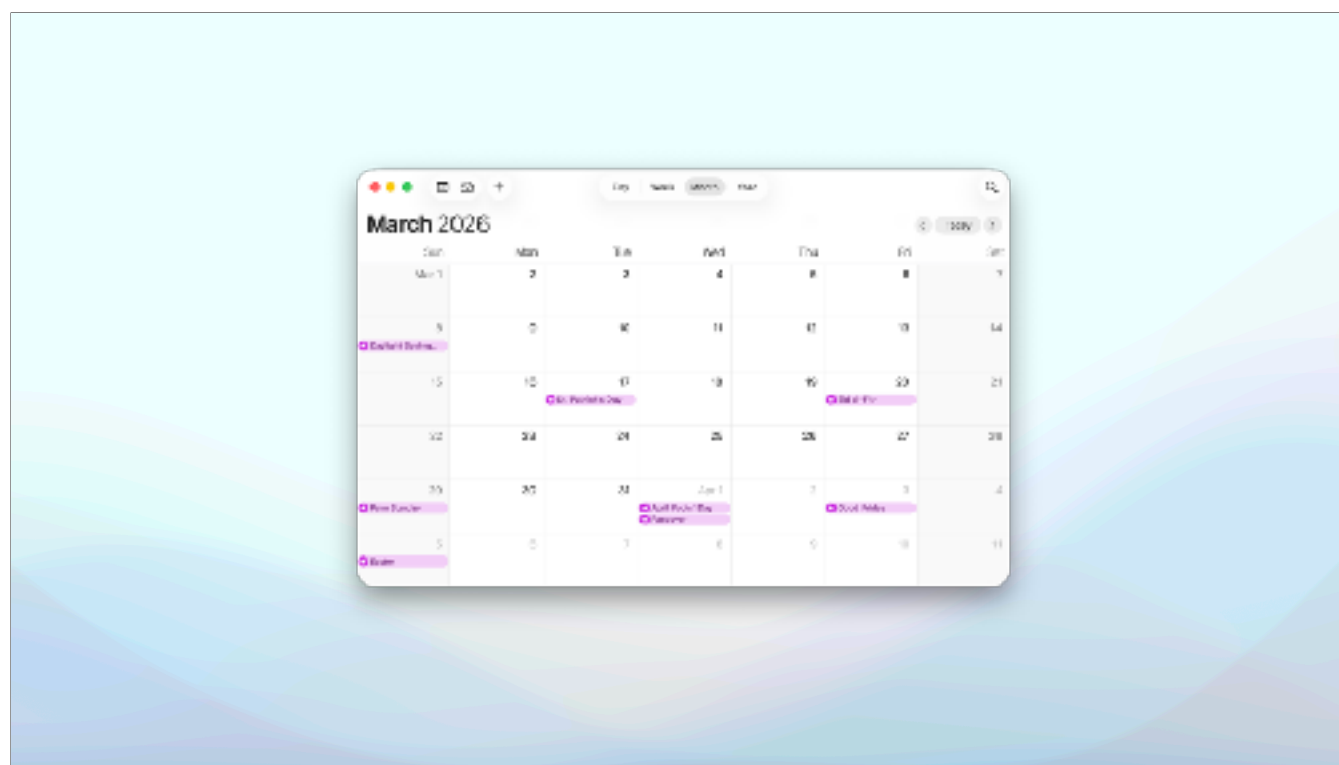


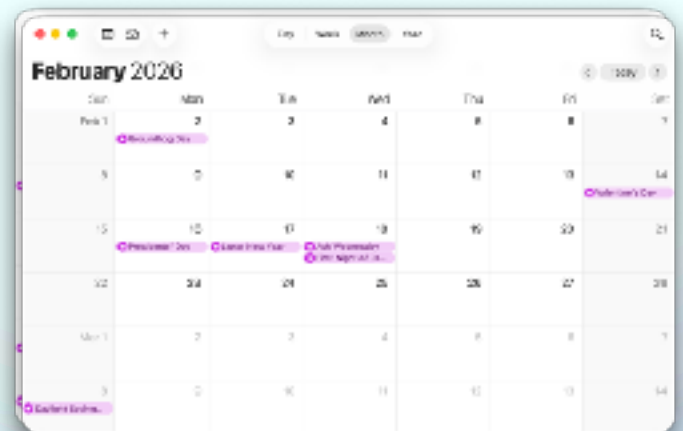
Well, I discovered the problem in March.

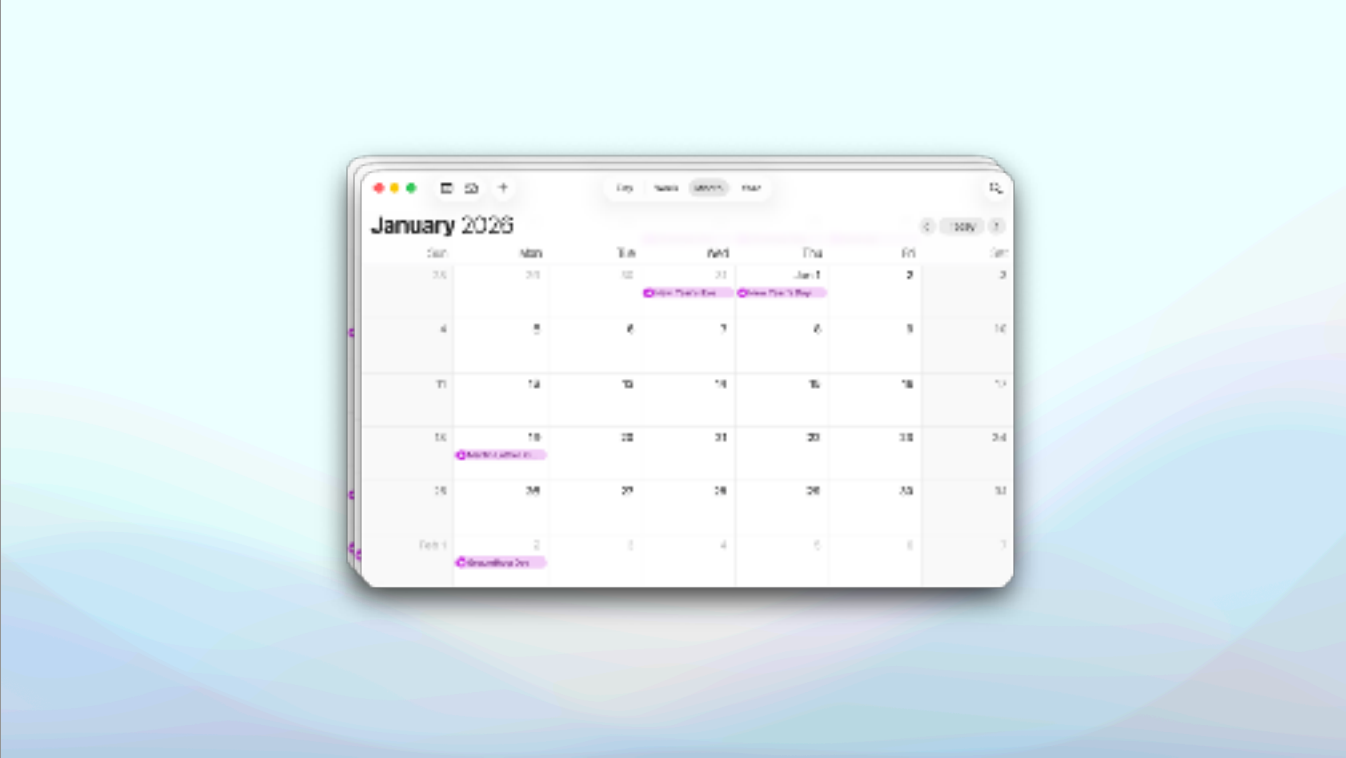
And I started going back . . .

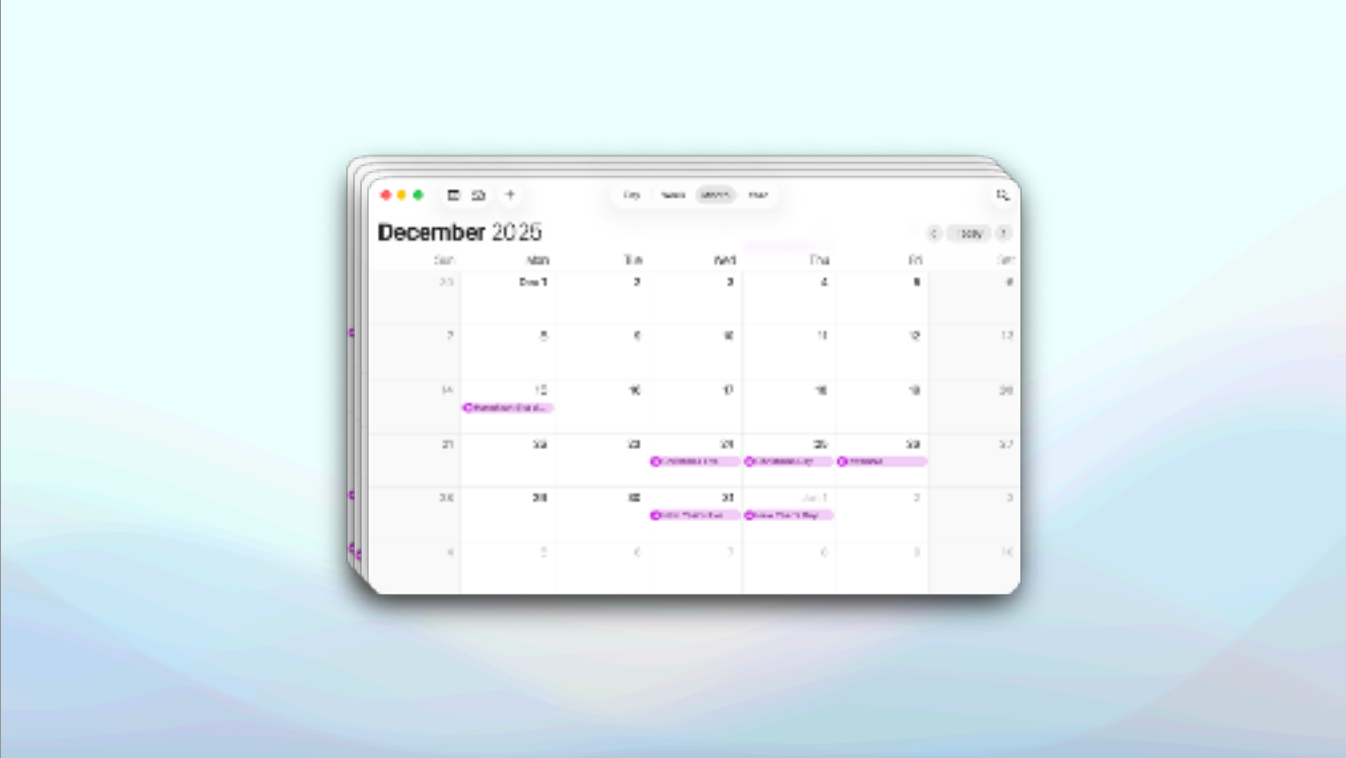
- February
- January
- December
- November
- October
- September . . .
- August.

So after several dozens of minutes of heart in my throat while I assessed the situation -- and confirmed that thankfully most of the really important stuff was scoped by major OS version and other mechanisms -- I shifted into remediation, then prevention mode.

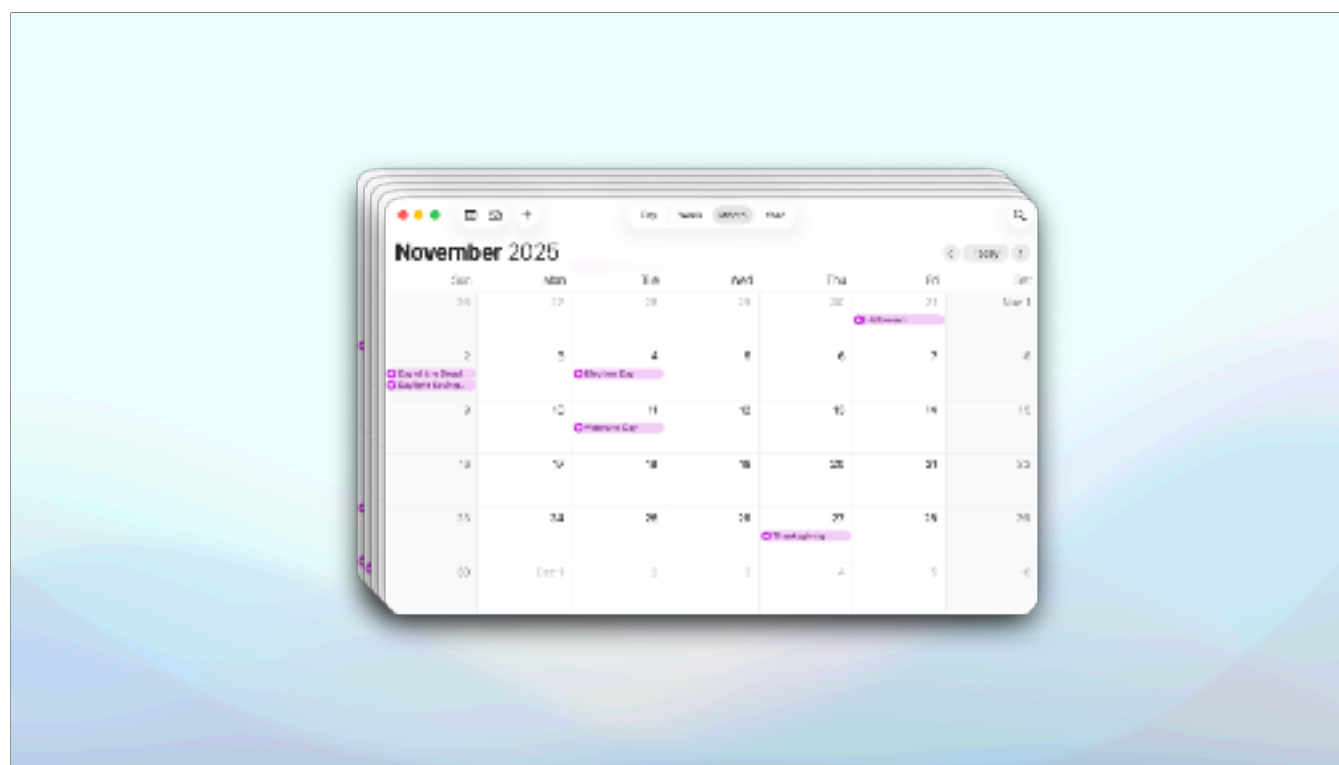




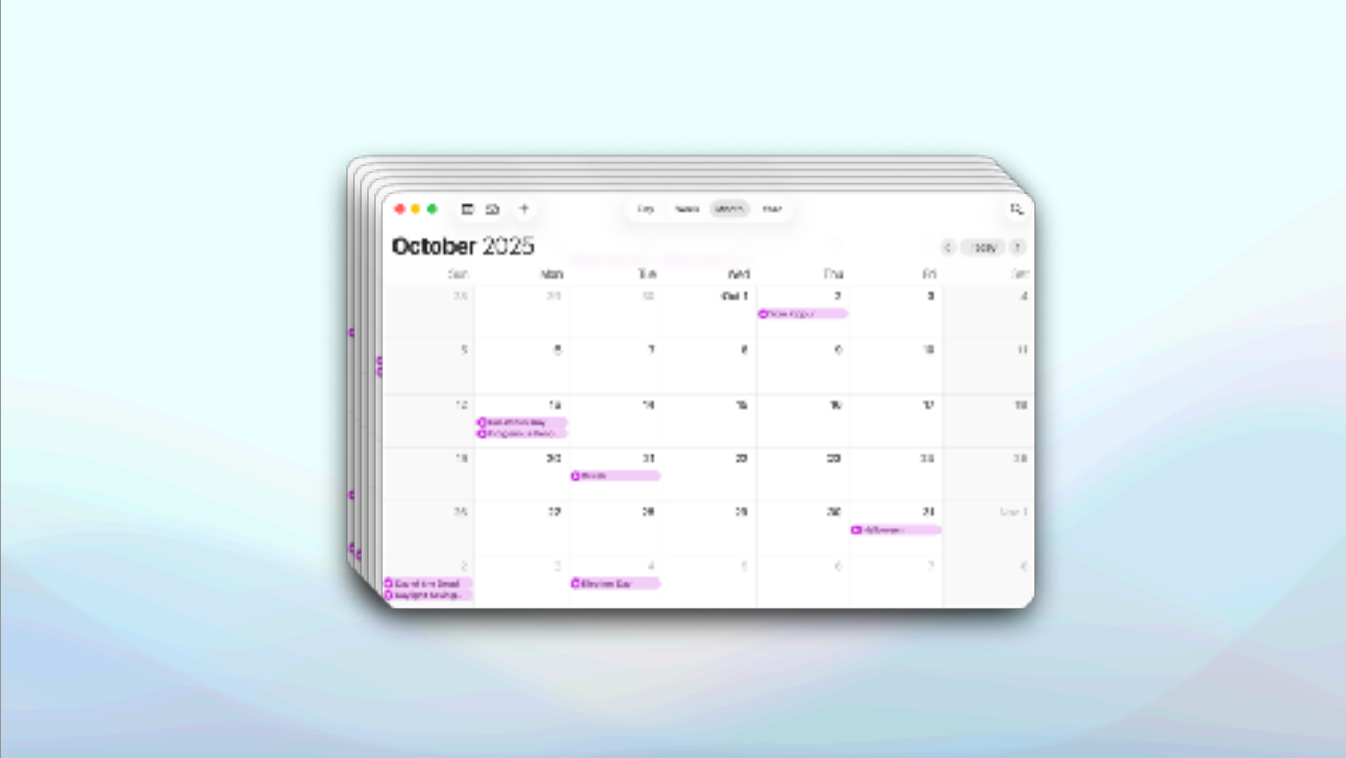


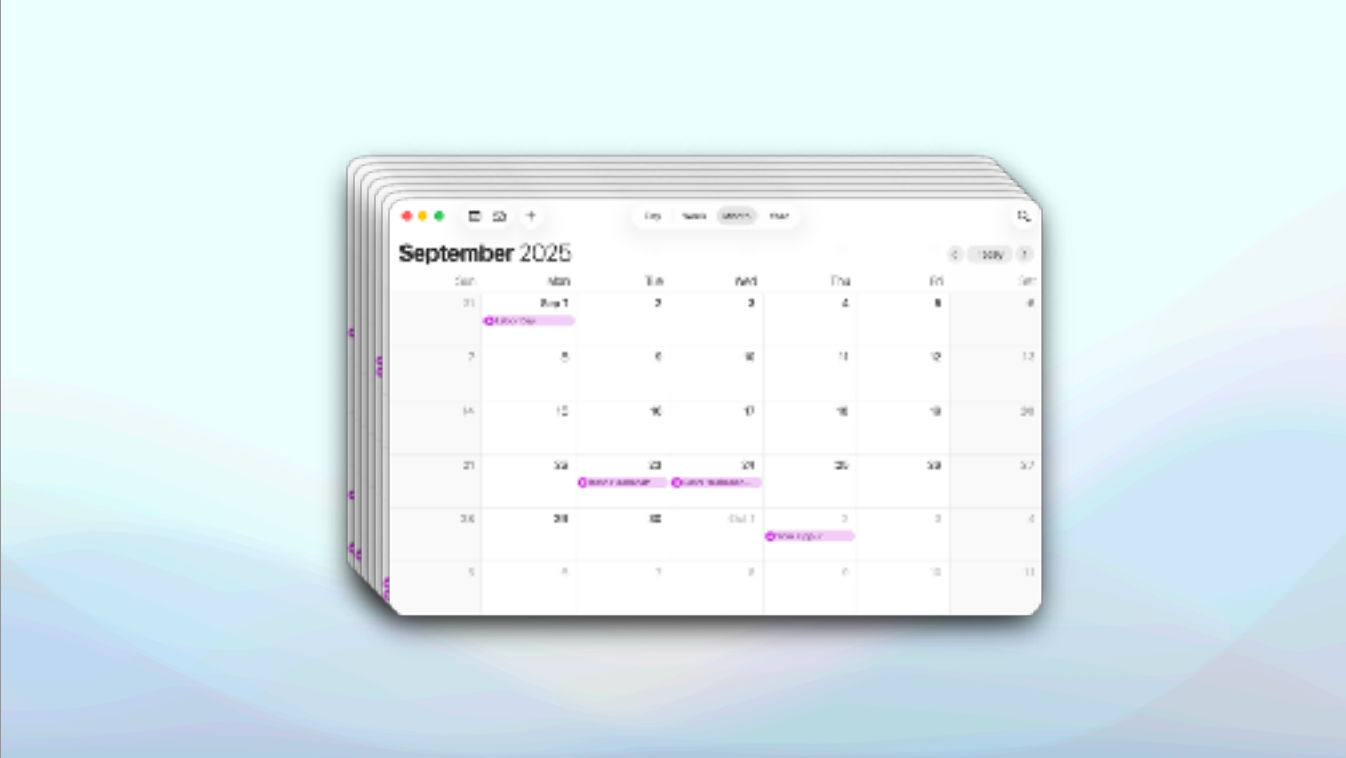














# What the heck?

## Forensics

I had two questions: How had it happened that I uploaded the wrong version?

And what could I do to reduce the risk of this -- or worse -- happening again?

I did not have to go too far afield to find the answer to the first question. Immediately after last year's PSU MacAdmins, I was thrown into extra projects so I was spinning way more plates than usual. And because I was primarily working from a single administrative machine, just making new copies of scripts with new names when I had a revision, I was not actually making a distinction between testing and test-**ed**.

Though I **do** know better than this.

The result was that I had too many copies of scripts in various states, and perhaps I had grabbed what I thought was the most recent **tested** version but was maybe only the most *recent*.

So that's the first question answered.

And for the second?

# What the heck?

Forensics



Regina Cassolo Bracchi, Poccola italiana, 1935



Well, I use this tag line a lot as a reminder for myself. It's a quote from James Clear (of Atomic Habits fame). "You do not rise to the level of your goals. You fall to the level of your systems."

My "system" (or lack thereof) had failed me. But as it turned out, I was already working on building the system that I needed, and in fact by the time I discovered the error, I had already pitched this session.

The solution was to implement a versioning system for scripts that would allow me to separate a development script from a stable production script, back it all up, and only put the things back together when the new version was proven to work.

**"You do not rise to the  
level of your goals.  
You fall to the level of  
your systems."**

James Clear





In other words, my solution was Git.





**git**

# Git, the basics



So, the basics.

# Why is Git?

**Blame Tux**

I'll get into what Git is in a moment, but first I'm going to answer the why.

- Git was developed by the Linux kernel community and Linus Torvalds.

You can imagine why a globally diverse collection of kernel developers needed a system to share and approve changes to code. They required something simple and fast. They had been using a product called BitKeeper for free, but BitKeeper wanted to start charging them. So in true Linux fashion, they built their own system and open sourced it.

# Why is Git?

Blame Tux



# Git is . . .

## Distributed Version Control

What they built is Git, a distributed version control system--that is, it's a system that tracks changes on the files (mostly text-based ones) you tell it to track.

- Git works as a local binary that you install on your computer
- or computers, that can
- communicate with compatible
- hosting services that are also running Git.

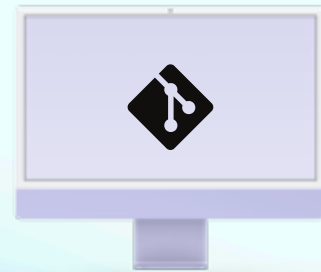
# Git is . . .

Distributed Version Control



# Git is . . .

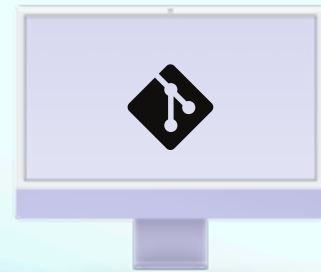
Distributed Version Control





# Git is . . .

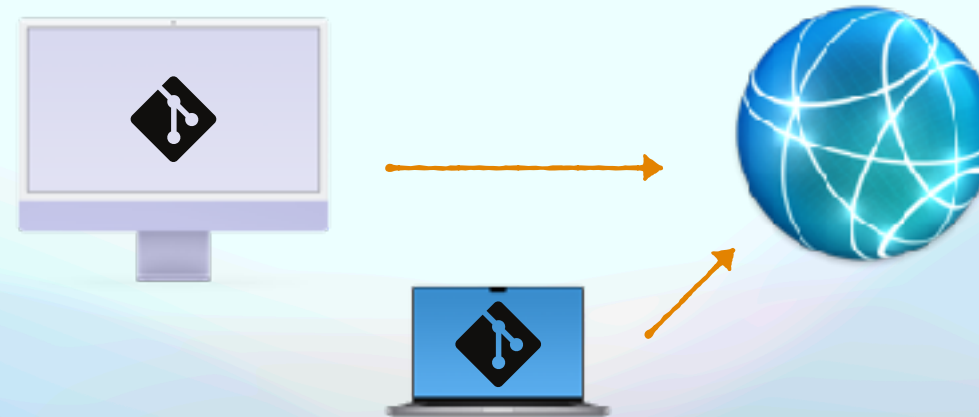
Distributed Version Control





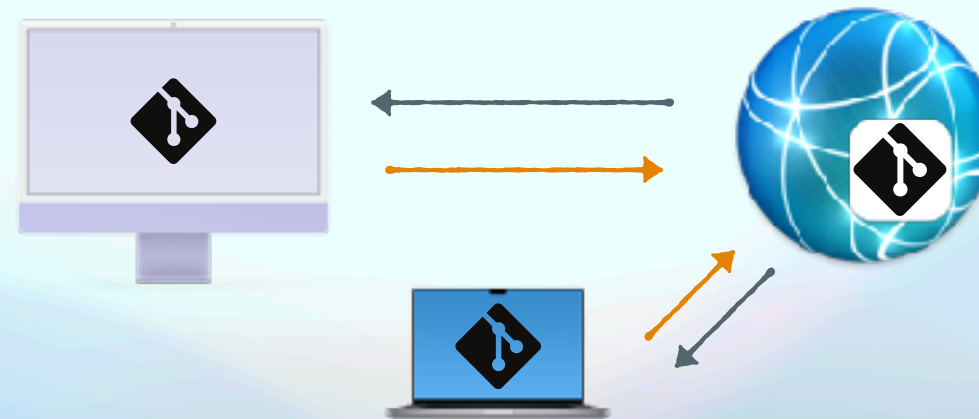
# Git is . . .

Distributed Version Control



# Git is . . .

Distributed Version Control



# Git is . . .

A distributed version control system



Git is a distributed version control system.

- It's very broad, in that it can tell you which files out of all of the files in your **repository** have changes.
- It's also extremely granular, in that it can show you if you changed a single character from lower case to upper case.
- Git allows you to copy or **clone** a repository from an **origin**
- Git lets you **branch** off some changes and
- later splice them back into the repository.
- Git can work entirely remotely, entirely locally, or a combination.
- And importantly, since it's a distributed system, it copies the entire repository when you clone it -- so if the host is lost, you still have the entire repo from the last time it was **fetched**, not just the individual files you were working on.

# Git is . . .

A distributed version control system

- Broad: tracks all files in *repository*



# Git is . . .

A distributed version control system

- Broad: tracks all files in *repository*
- Granular: tracks tiny changes



# Git is . . .

## A distributed version control system

- Broad: tracks all files in *repository*
- Granular: tracks tiny changes
- Copy or *clone* a repository from *origin*



# Git is . . .

## A distributed version control system

- Broad: tracks all files in ***repository***
- Granular: tracks tiny changes
- Copy or ***clone*** a repository from ***origin***
- ***Branch*** a repository





# Git is . . .

## A distributed version control system

- Broad: tracks all files in ***repository***
- Granular: tracks tiny changes
- Copy or ***clone*** a repository from ***origin***
- ***Branch*** a repository
- Splice or ***merge*** changes back





# Git is . . .

## A distributed version control system

- Broad: tracks all files in ***repository***
- Granular: tracks tiny changes
- Copy or ***clone*** a repository from ***origin***
- ***Branch*** a repository
- Splice or ***merge*** changes back
- Work remotely, locally, or both



# Git is . . .

## A distributed version control system

- Broad: tracks all files in **repository**
- Granular: tracks tiny changes
- Copy or **clone** a repository from **origin**
- **Branch** a repository
- Splice or **merge** changes back
- Work remotely, locally, or both
- Distributed: mirrors entire repository when **fetched**



# Main functions

Git going

I've already previewed some of the actions that Git can do. Let's get into more detail about the main ones.

# Main functions

Git going



Marcelle Cahn, Spatial-Mobile, 1967-69



Git actions revolve around a

- repository, which is a collection of files that you want to track

And the first and most important action is

- `init` to Initialize a repository.

This creates the supporting resources, such as a configuration file, folders for objects, references, and change tracking.

The Git repository is a just directory with benefits -- and the directory can exist anywhere.

Whenever you select New Repository in a Git-enabled desktop app or on the web, it is running a **`git init`** for you.



# repository repo

**Noun: Files + Git supports**





repository  
repo

**Noun: Files + Git supports**



init

**Verb: Initialize a repository**





It's worth noting that we use most of these words as both nouns and verbs.

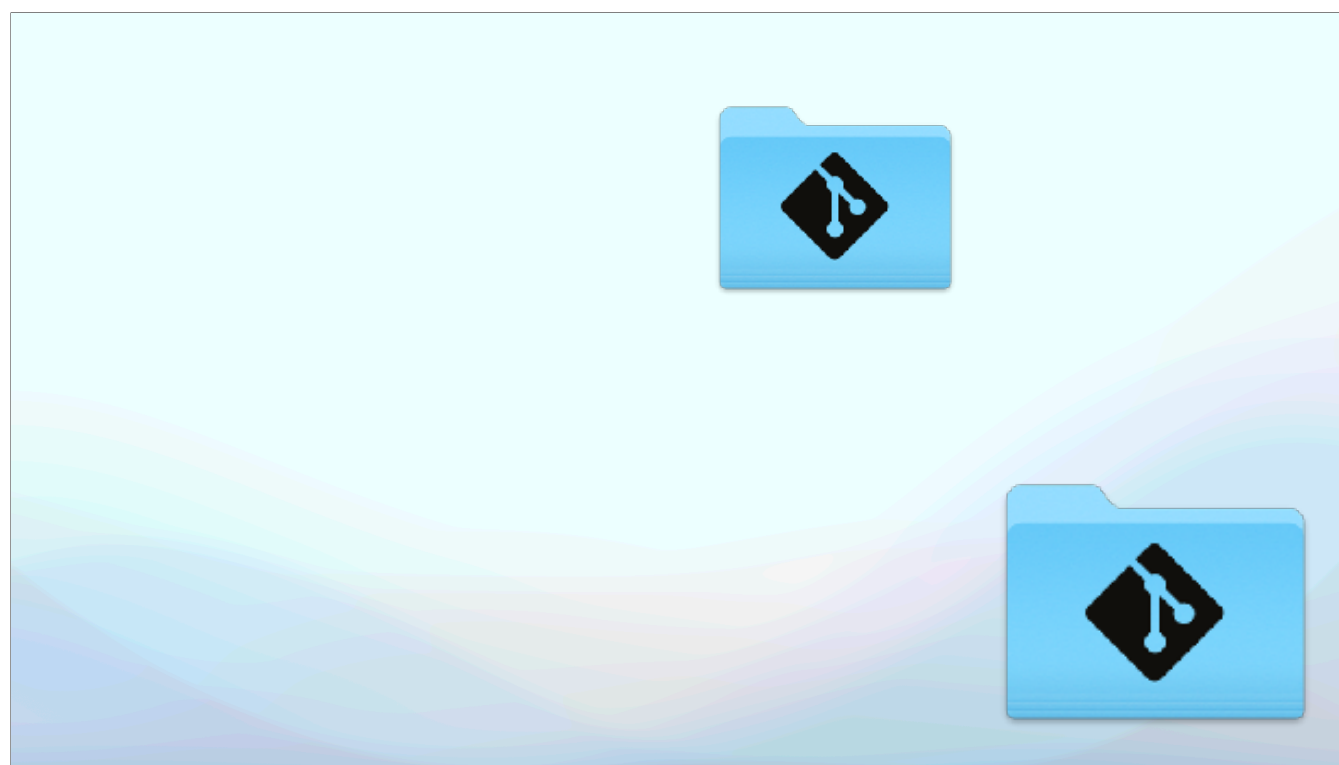
- Something you'll do more frequently is clone a repository to another location.

In order to do work on your workstation, you may need to clone the repo to it.

To do work between different workstations, you can clone the repo to a new workstation.

To work on someone else's project, you can clone a local copy to your own computer or your own host.

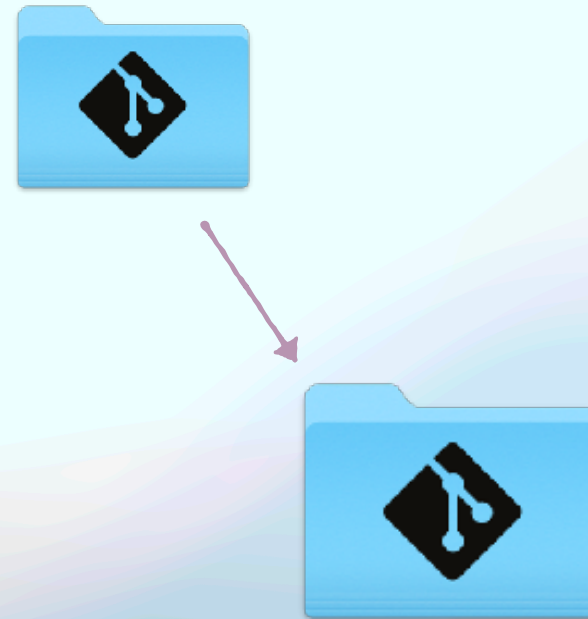
- The origin is the source for the clone



# clone

**Noun:** An exact copy of a repo

**Verb:** Copy a repo to another location



# clone

Noun: An exact copy of a repo

Verb: Copy a repo to another location



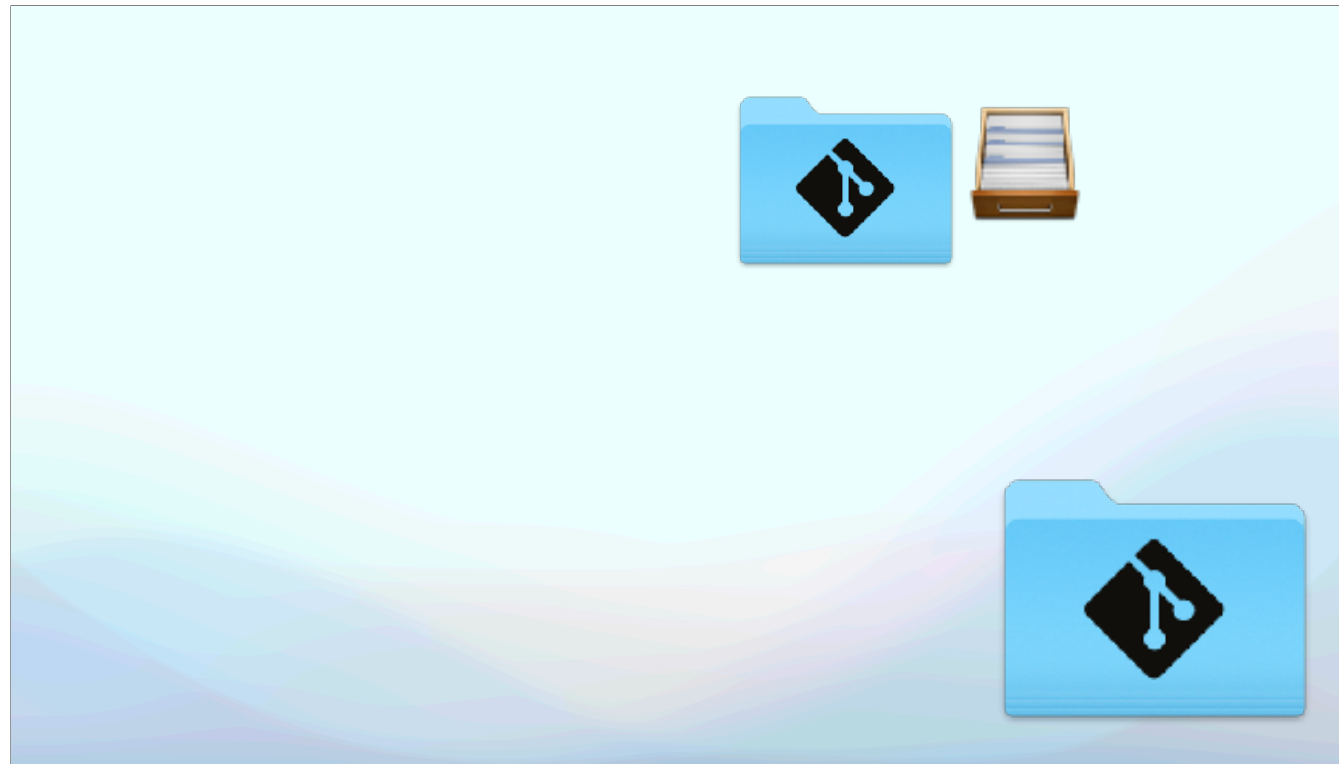
# origin

Noun: the clone's source



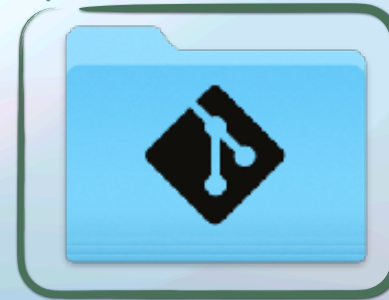
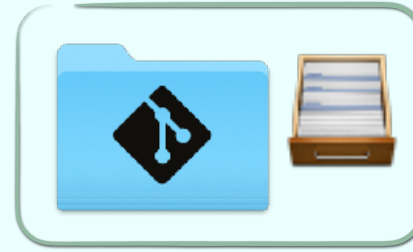
- Fetch grabs any changes from a repository origin and brings them to you.

It's initiated from the working location. And I'd just like to note that this is not the last time we'll consider directionality.



# fetch

Verb: Collect changes from origin

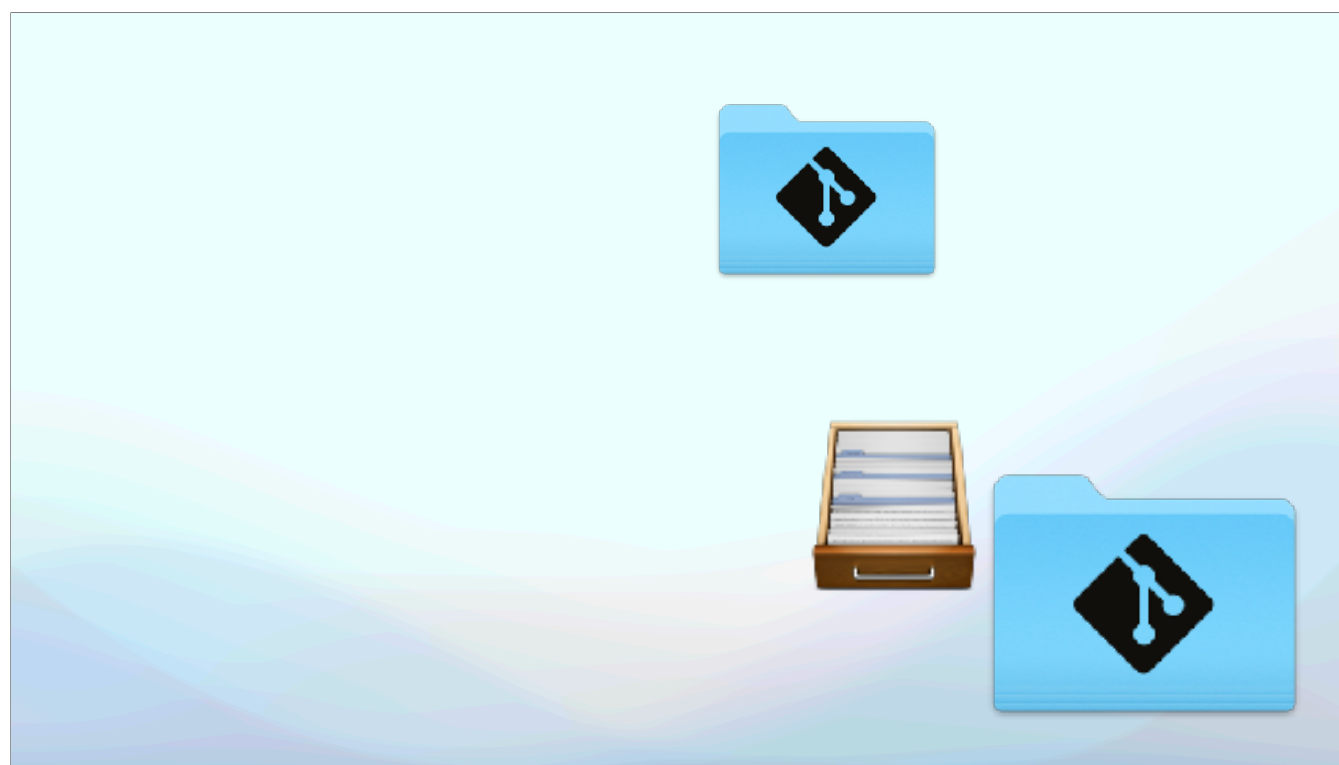






- Commit gathers up changes to be logged to a repository. A commit creates a snapshot or a marker to say, "here's a significant group of changes, and here's what they are." You don't do a commit every time you have changes, but every time you want the state of the repository to be noted.

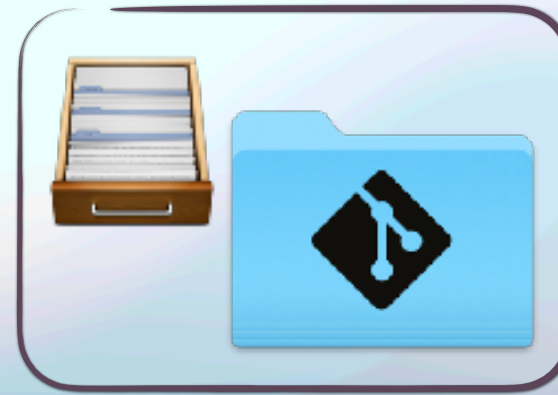




# commit

**Noun:** Snapshot of changes to a repository

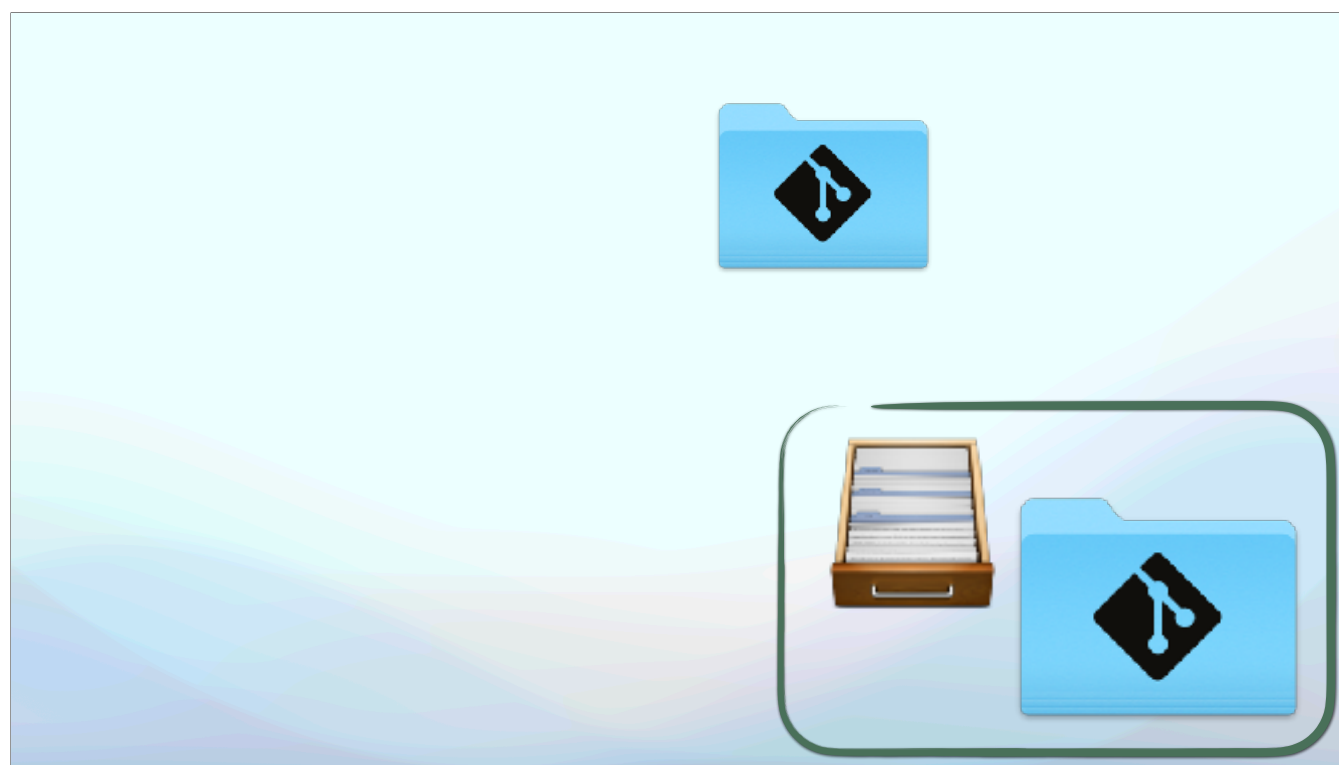
**Verb:** Make a snapshot of changes to be sent to a repository





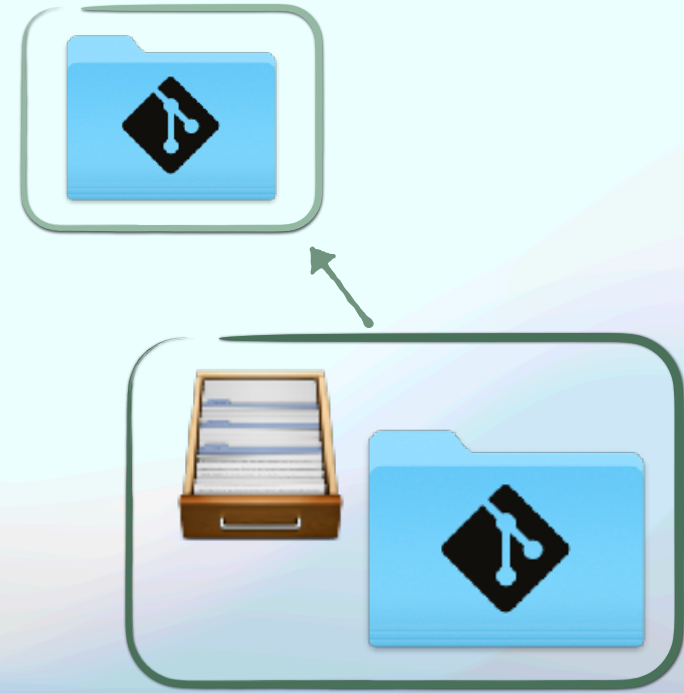
Here's some more directionality.

- A push sends the committed changes to a repository. You usually do this when you've cloned your repository to your workstation and want to save your new changes to the copy at origin.



# push

Verb: Send committed  
changes to a repository





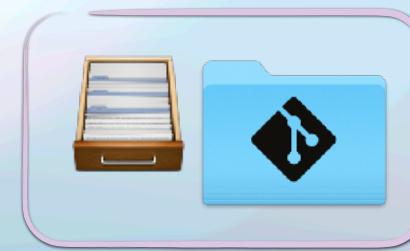
- And a Pull grabs changes from a commit that is logistically elsewhere and pulls them into the repository where you're working. A "pull request," which we'll see later, happens when you're collaborating with other folks and you have changes to incorporate, but you don't have permission to push. You ask them to pull.





# pull

Verb: Pull changes from a commit into a repository

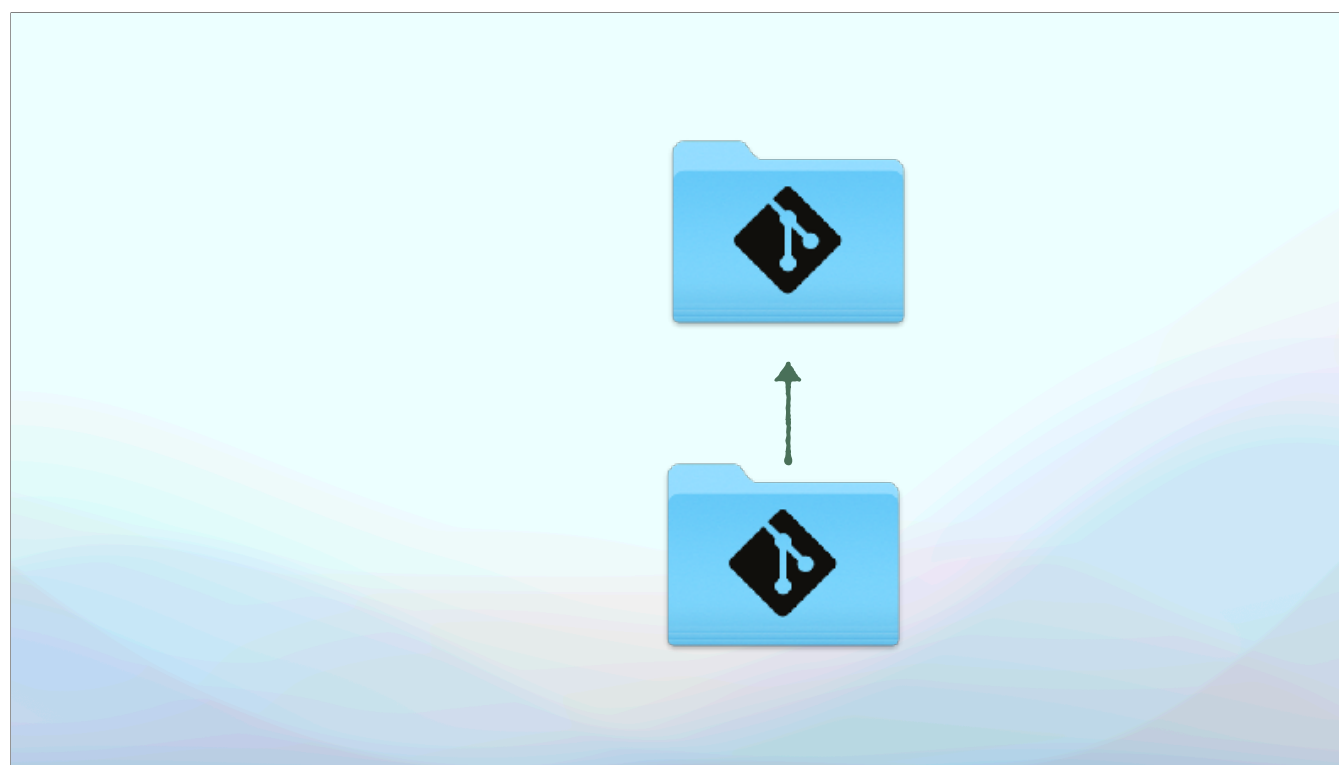






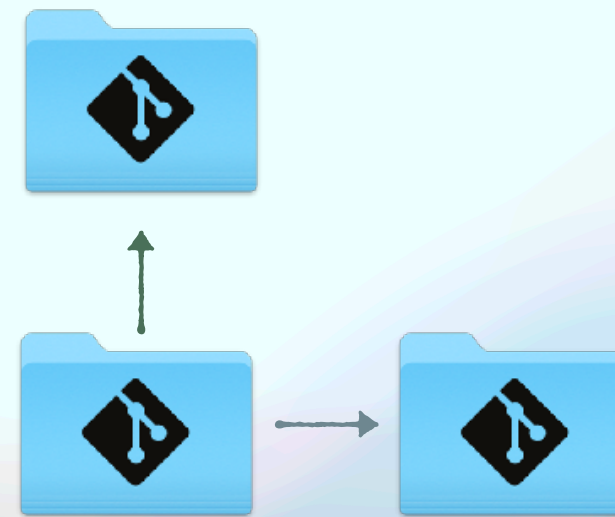
- A Branch makes a separate copy to
- track changes where you are not ready to put them directly in your **main** branch

It has its own tracking and commits.



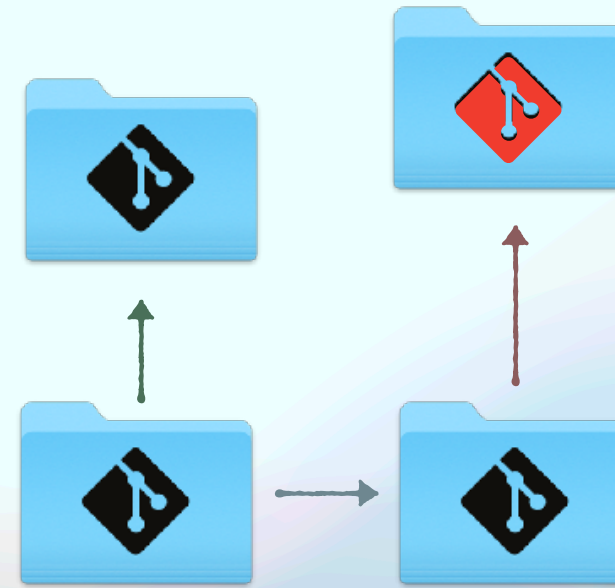
# branch

Noun: A separate copy within a repo  
Verb: Make a copy within a repo



# branch

Noun: A separate copy within a repo  
Verb: Make a copy within a repo

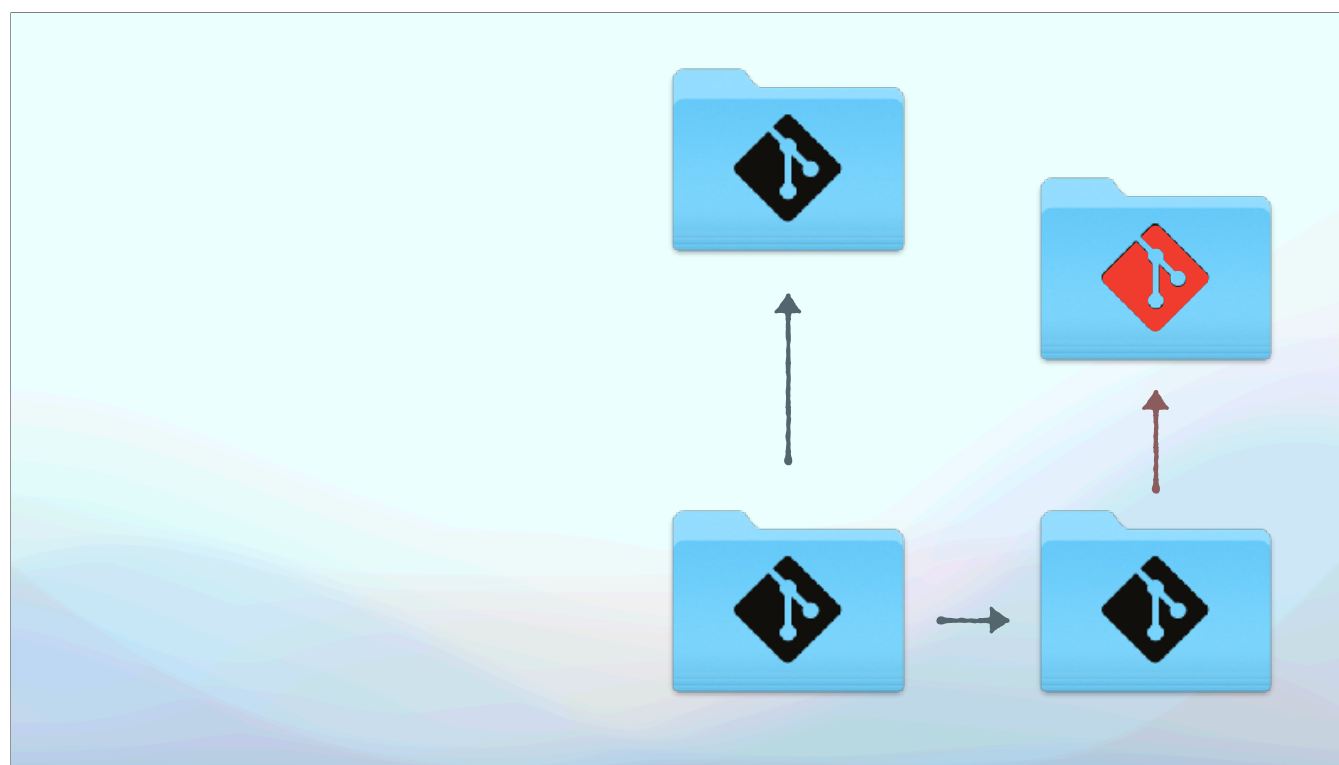




And finally, when you've tested and are ready,

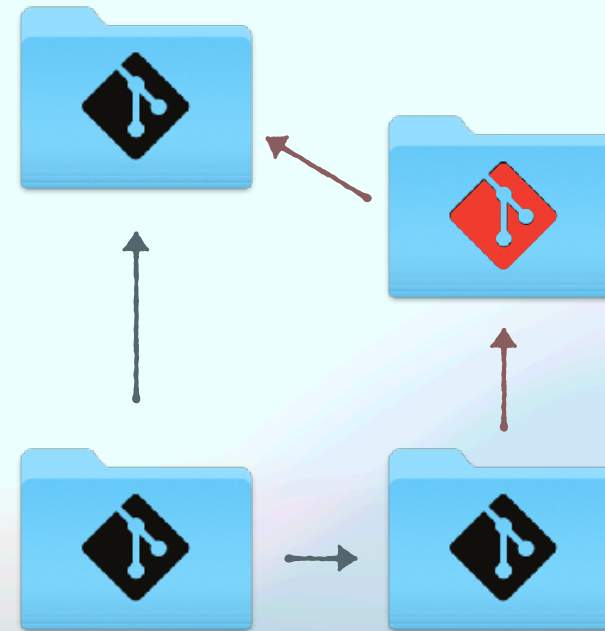
- you merge changes from a side branch into the main branch

Well, it's usually main, but it could be any other branch.



# merge

Verb: Splice changes from a branch

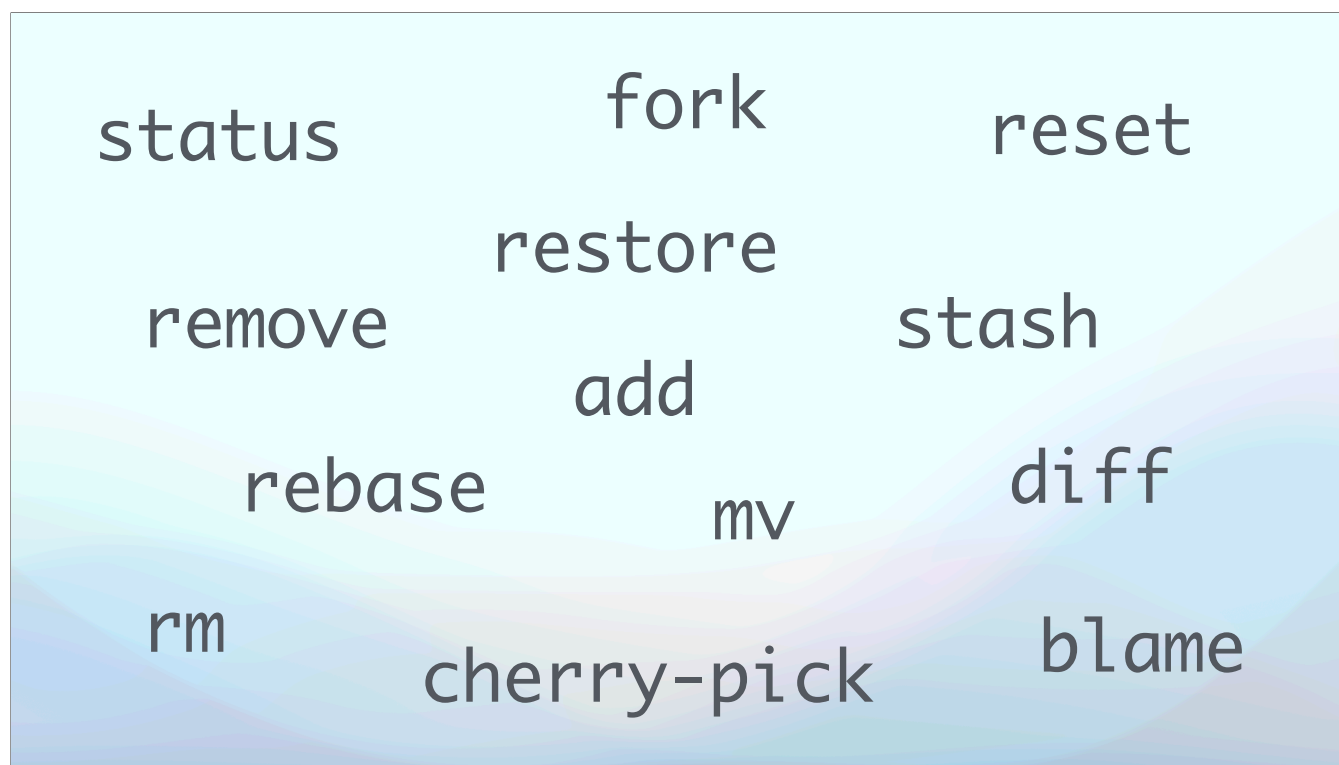






- Those are just the primary terms we'll be using. There are in fact many more that you can explore on your own.





# Getting Git

Mild puns continue until  
morale improves . . .

"This all sounds great!" You say. "Let's get it!"

You might not need to get Git, you might already have it installed.

# Getting Git

Mild puns continue until  
morale improves . . .



Zubeida Agha, Abstract Study, 1967

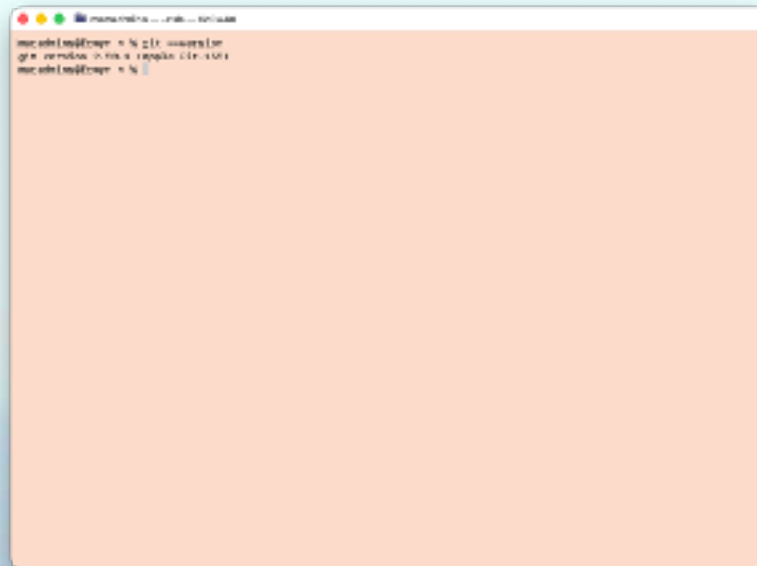


```
git --version
```

Here's how to check: open a terminal window (just trust me here)

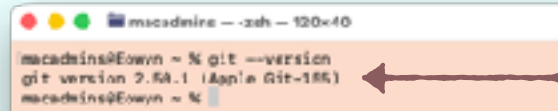
- and type `git --version` This is Apple's Git that comes as part of the Xcode command line tools. So if you don't have it . . .

# git --version

A terminal window with a light orange background and a white title bar. The title bar contains the text "macos-10.15.7 - ssh://root@192.168.1.100". The terminal shows the command "git --version" being executed, followed by the output "git version 2.20.1 (Apple Git 11.5)".

```
macos-10.15.7 - ssh://root@192.168.1.100  
git --version  
git version 2.20.1 (Apple Git 11.5)  
macos-10.15.7 - ssh://root@192.168.1.100
```

# git --version



macadmins@Eowyn ~ % git --version  
git version 2.58.1 (Apple Git-186)  
macadmins@Eowyn ~ %

A terminal window with a light orange background. The title bar shows 'macadmins -- zsh -- 120x40'. The command 'git --version' is entered and executed, resulting in the output 'git version 2.58.1 (Apple Git-186)'. A red arrow points to the output line.



macOS will offer to install it for you as part of the command line developer tools.

The installer will give wildly inaccurate estimates of download times, then settle down into something reasonable.

And takes probably under 5 minutes on an M1 iMac to install.



```
macos-12.0 -- ash -- 123456
Last login: Fri Jun 12 07:27:54 on ttyd00
macos-12.0 ~ % git --version
git-select: error: No developer tools were found, requesting install.
If developer tools are located at a non default location on disk, use 'sudo xcode-select --switch path/to/Xcode.xc' to
specify the Xcode that you wish to use for command line developer tools, and cancel the installation dialog.
See 'man xcode-select' for more details.
macos-12.0 ~ %
```



The "git" command requires the command line developer tools. Would you like to install the tools now?

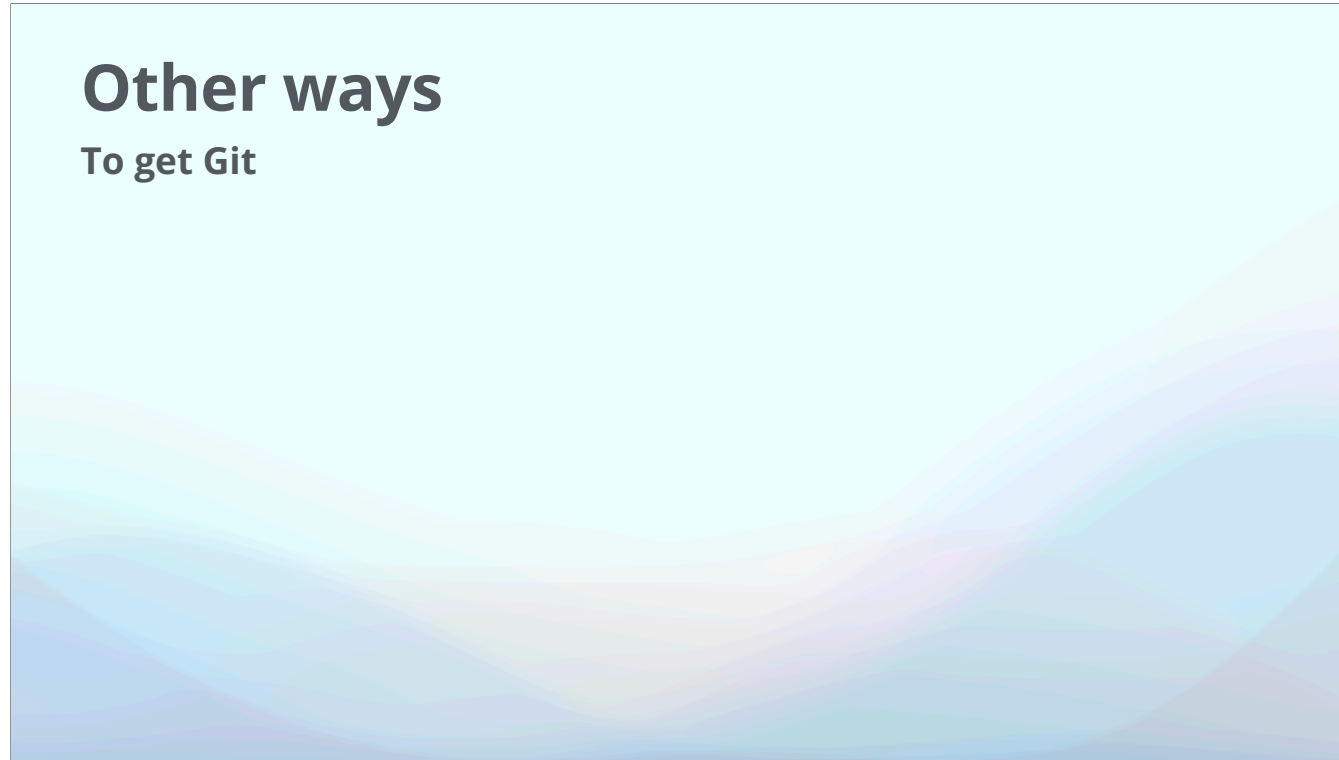
Click on Install to download and install the command line developer tools now.

Cancel Install



# Other ways

To get Git



There are other ways to get Git on your system.

- If it's already available via your existing package management service, that's a great option as long as it's kept up to date.
- And of course you can download an installer from the main Git website, [git-scm.com/download/mac](https://git-scm.com/download/mac)
  - this is completely vendor-agnostic
  - And always the latest version, obviously -- the Apple version tends to fall behind

# Other ways

## To get Git



# Other ways

## To get Git

- Existing package management



# Other ways

## To get Git

- Existing package management
- Acquire from git: <https://git-scm.com/download/mac>



# Other ways

## To get Git

- Existing package management
- Acquire from git: <https://git-scm.com/download/mac>
- Vendor-agnostic



# Other ways

## To get Git

- Existing package management
- Acquire from git: <https://git-scm.com/download/mac>
- Vendor-agnostic
- Always the latest





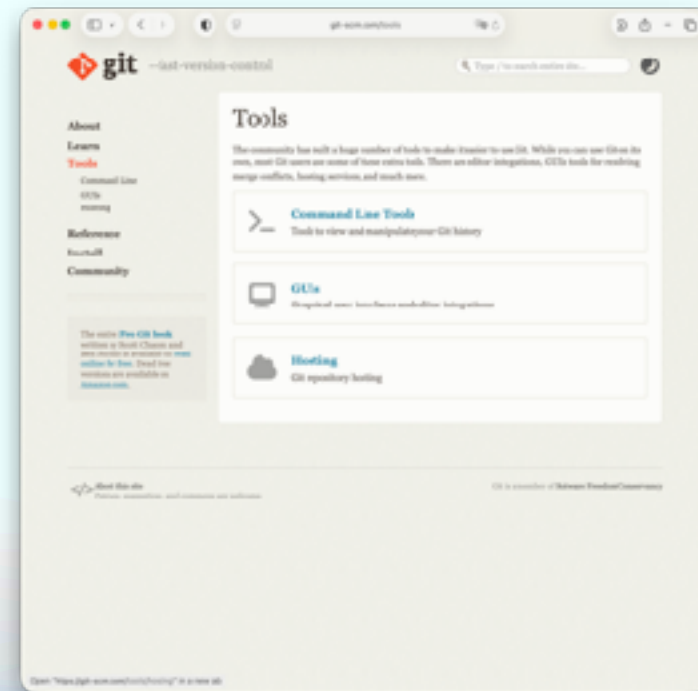
# Tools

While we're here, it's a good idea to take a look at some of the tools that are available for use with Git.

These include tools for the command line, the GUI, and hosting.



# Tools





# Tools

## Command line

I'm not big on the command line for my Git use. Git does provide a ton of information about what it's doing or did, and I make better sense of it right now in the GUI. It's likely that unless you're a developer, you won't have a lot of use for many of these, either. But we can just do a speed run of the main types.

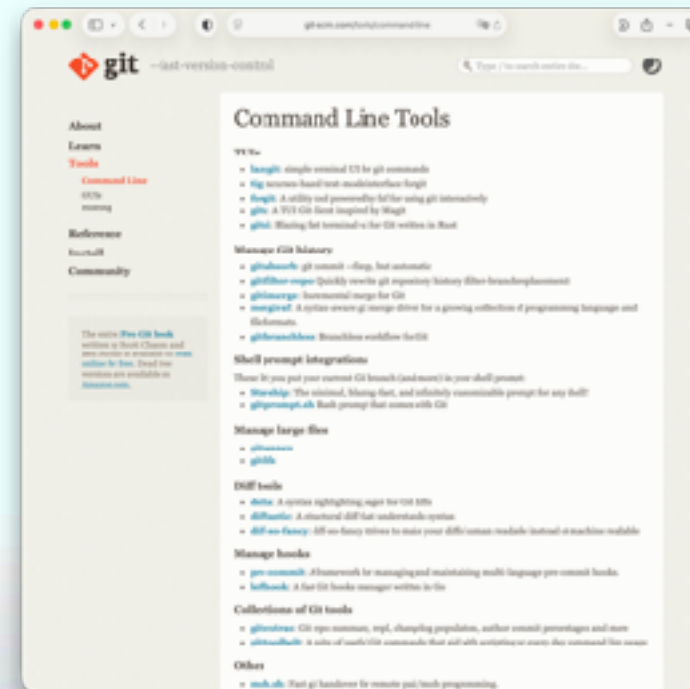
We're got things here that can

- do things you can't do in Apple's Terminal,
- manage history,
- Deal with large files
- do advanced rendering of differences between files
- And many more . . .

I don't use any of these! You might not, either. That's fine.

# Tools

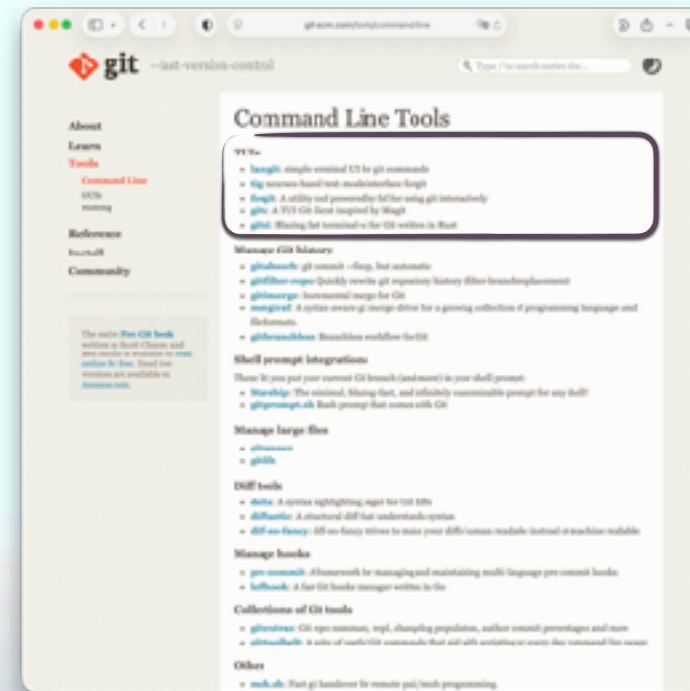
## Command line



## Tools

### Command line

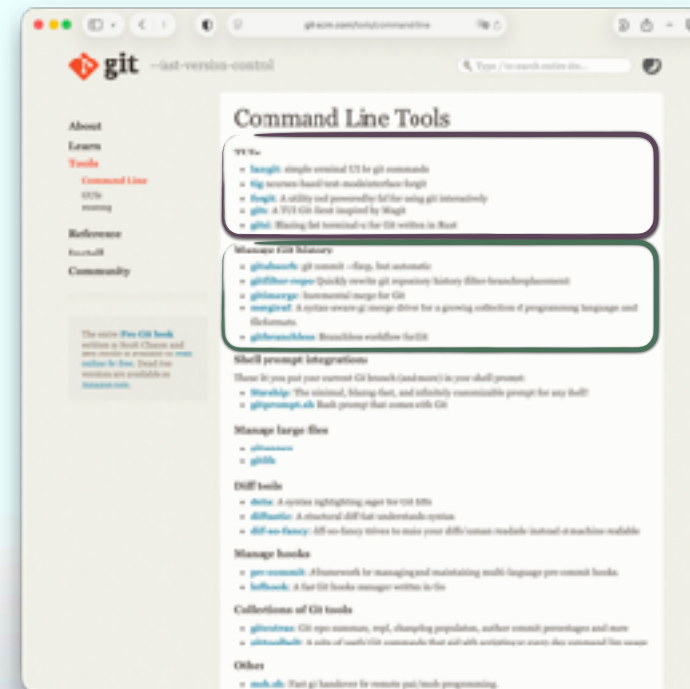
- Add features vs. Terminal



# Tools

## Command line

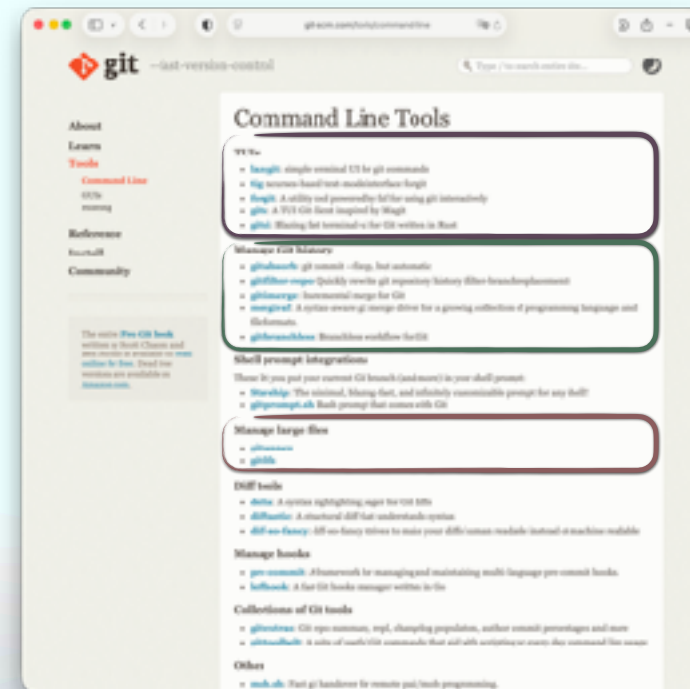
- Add features vs. Terminal
- Manage history



## Tools

## Command line

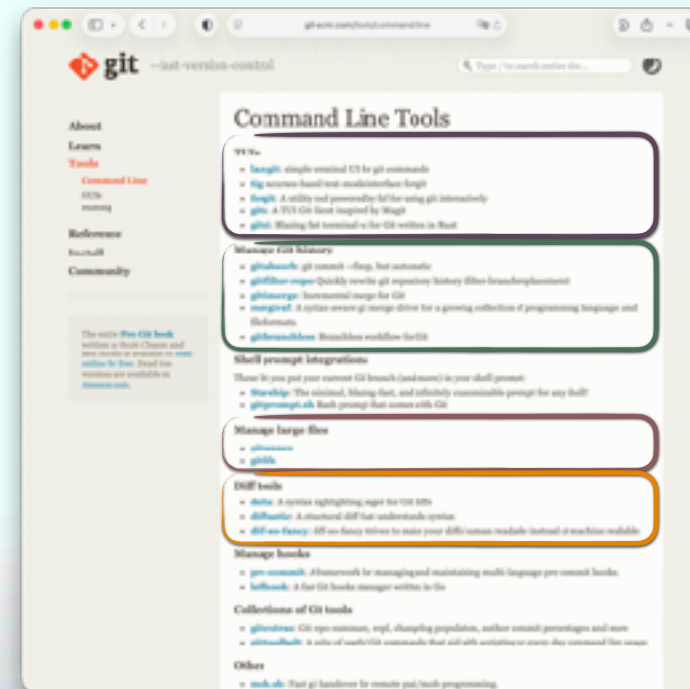
- Add features vs. Terminal
- Manage history
- Large file handling



## Tools

## Command line

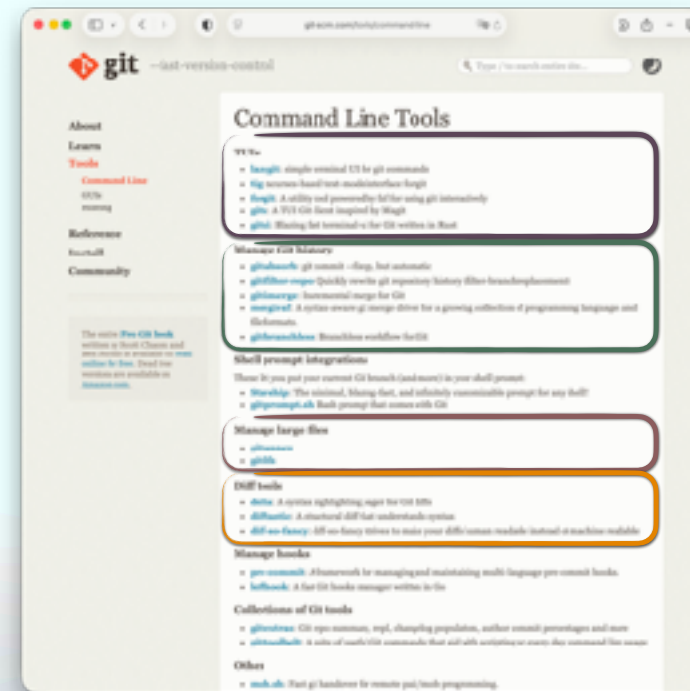
- Add features vs. Terminal
- Manage history
- Large file handling
- Do **diffs** (show differences)



## Tools

## Command line

- Add features vs. Terminal
- Manage history
- Large file handling
- Do **diffs** (show differences)
- And more . . . .







As for GUI tools,

- They are available for every platform,

Including mobile. I wouldn't trust myself to see everything I needed to manage a merge on my iPhone, but that's me.

- They list 55 tools here,
  - not all of which are actively maintained.
  - The license type varies. Some are free and open source, some are freemium, etc.



## Tools

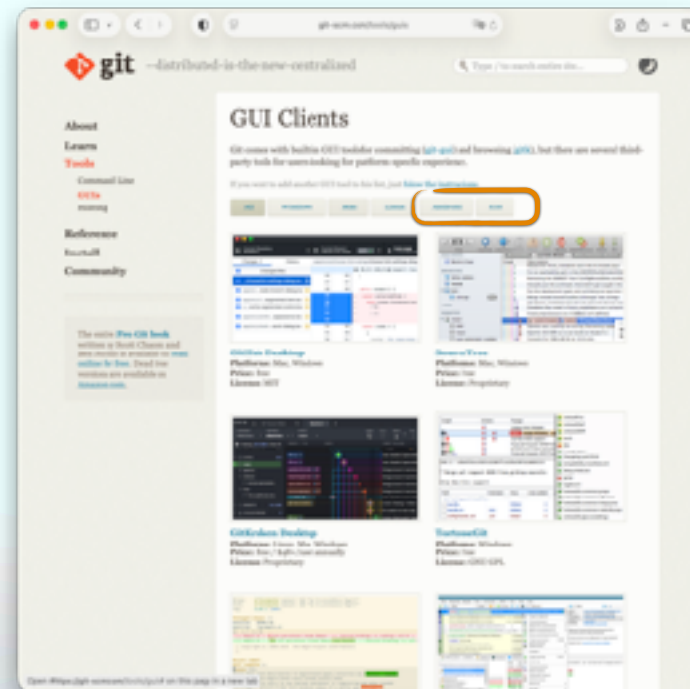
### GUI



# Tools

## GUI

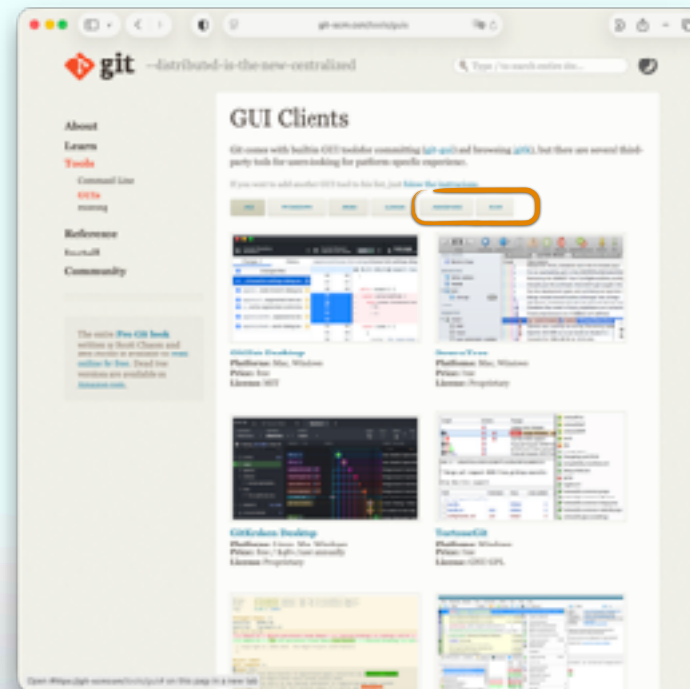
- Every platform
- Including mobile



# Tools

## GUI

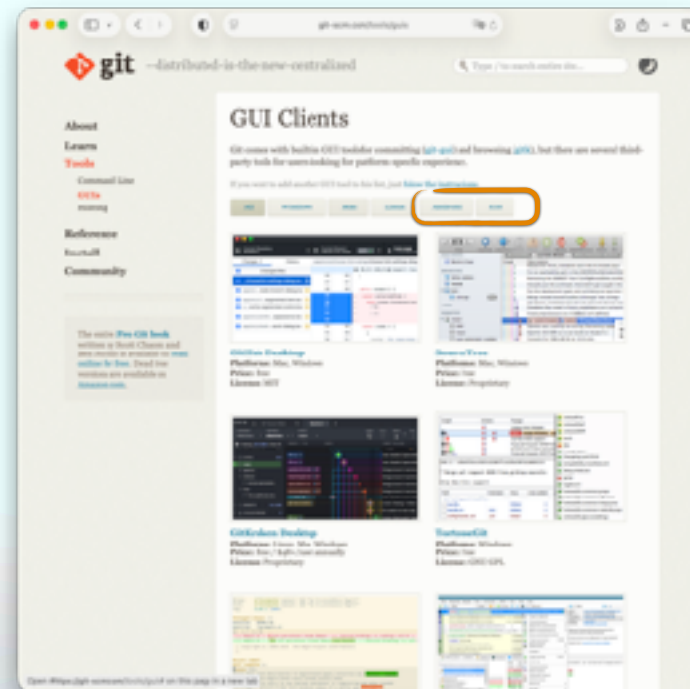
- Every platform
- Including mobile
- 55 tools listed



# Tools

## GUI

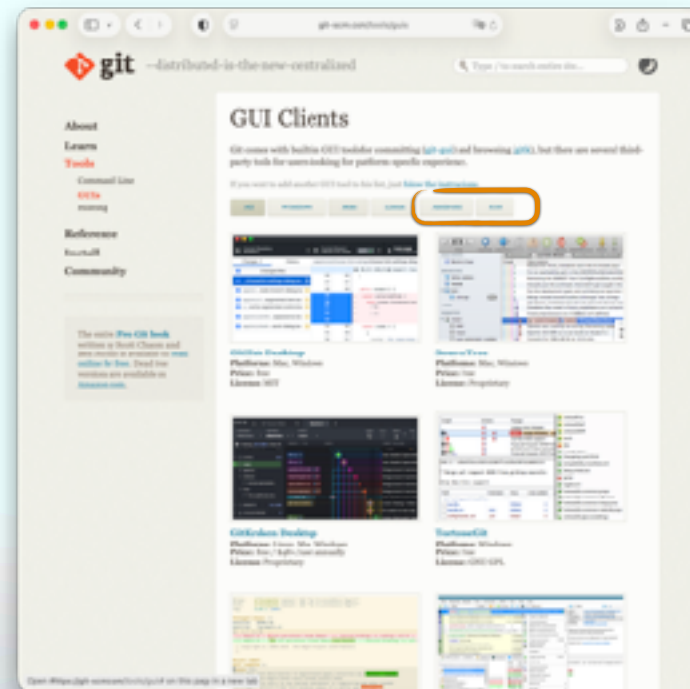
- Every platform
  - Including mobile
- 55 tools listed
  - Not all are maintained



# Tools

## GUI

- Every platform
  - Including mobile
- 55 tools listed
  - Not all are maintained
  - License type varies



# Tools

## Hosting

Ideally, if you want to use Git like I do, to track changes and create a shared backup of your scripts and related material, then you need a remote repository host. This could be in the cloud, hosted by a service, or installed on your own server on premises.

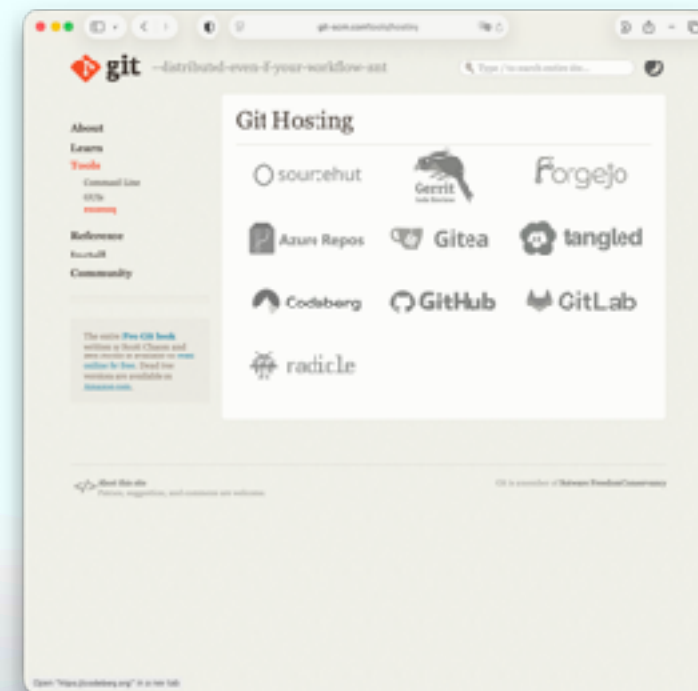
There are a number of these, and the most popular are

- GitHub, a subsidiary of Microsoft, and
- GitLab, which is independent.
- It may also be worth checking your org's Microsoft license to see if you have Azure Repos or whatever they're calling it this week

Not knocking any of these other hosts, but I'm not familiar with them.

# Tools

## Hosting

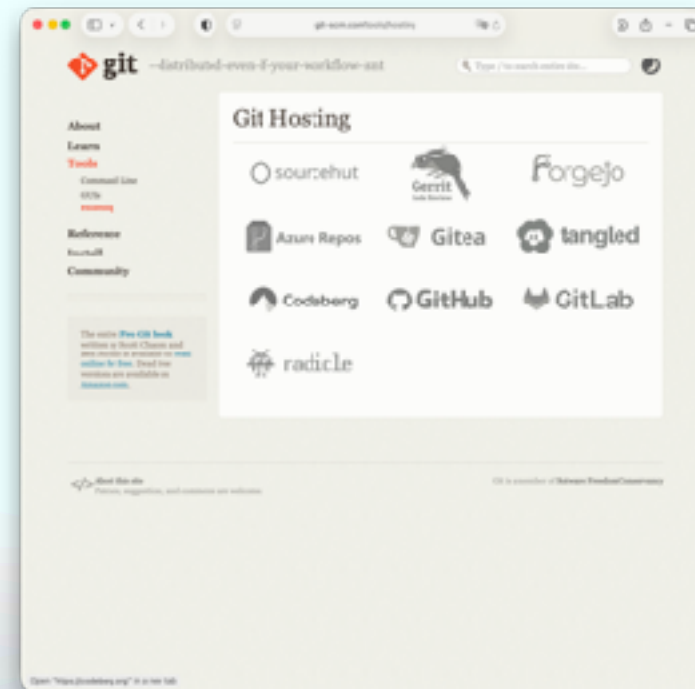




# Tools

## Hosting

- GitHub, Microsoft subsidiary

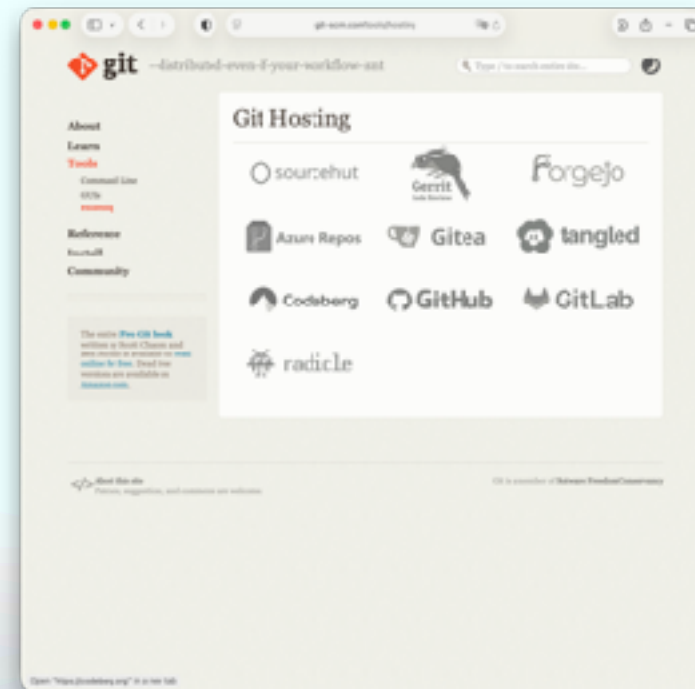




# Tools

## Hosting

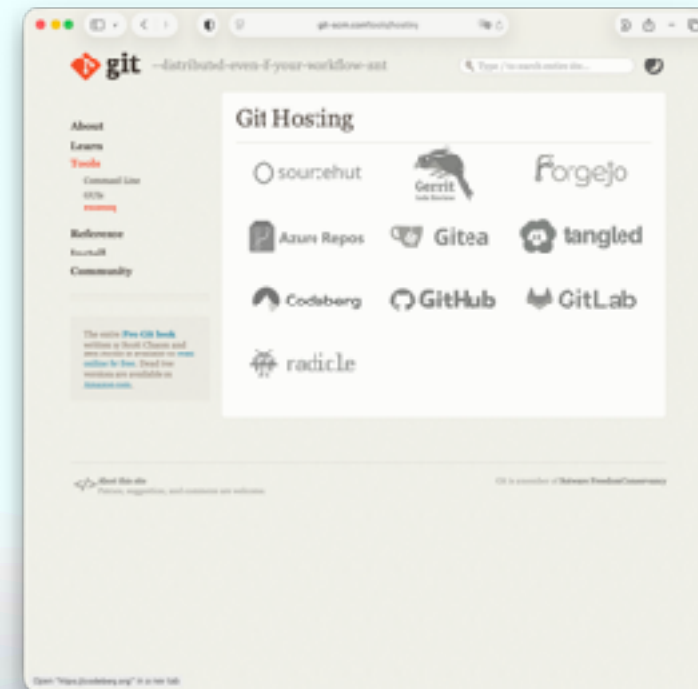
- GitHub, Microsoft subsidiary
- GitLab, GitLab Inc.



# Tools

## Hosting

- GitHub, Microsoft subsidiary
- GitLab, GitLab Inc.
- Others, including Azure Repos



# Git Setup

And go?

Hopefully that's enough terminology to let us move on to setting up Git on our computer.

# Git Setup

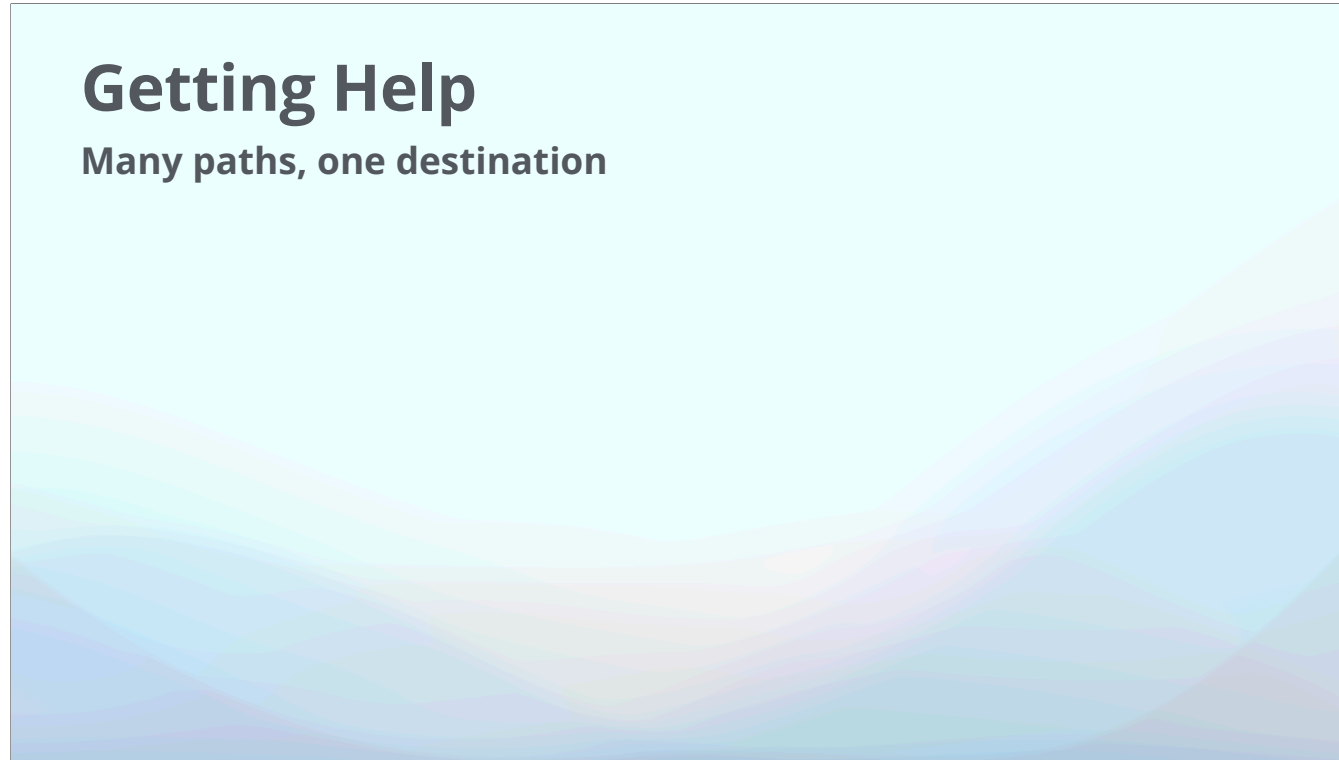
And go?



Rosa Bonheur, Retour de Champs, 1854

# Getting Help

Many paths, one destination



## Arguments vs. Options

I know what you're thinking when you see this image, but I promise, this is the most command line heavy section of this session, because you need to know how to get help with Git.

Say you run into a term you haven't heard before, or you want some clarification about specifics. Obviously, you can use the documentation of the platform that you decide to use. And I've put a link to a starter book on Git in the resources. But the command line is a really quick and easy way to access the built-in manual, or man pages.

If you're not familiar with Linux style help, that's what this is, and it's the same process you'd use to get help with any text command on your Mac via Terminal.

- You can type `git` with the `help` verb, then the verb you need help with, like "commit"
- You can type `git` and the verb, then add the option to say show me the help on this command
- And you can straight up type `man git - verb`

They will all get you to the man page for the action.

## Many paths, one destination

## Many paths, one destination

[illegible]

## Many paths, one destination

- `git help <verb>`

[illegible]



## Many paths, one destination

- `git help <verb>`
- `git <verb> --help`





## Many paths, one destination

- `git help <verb>`
- `git <verb> --help`
- `man git-<verb>`



# Configuration

## Three configuration locations

First thing to do is configure Git. I'm showing you the command line again. Why am I doing this?

It's because you need to know that Git stores configuration information at three levels in any operating system.

- You can use the `git config` command with arguments to list all the configs that Git knows about:
- There's a global config at `et c/gitconfig`
  - But if you installed via that macOS prompt, it's deep in the Developer folder in Library
- Per user
  - At the top level of the user's home in a hidden file called **gitconfig** or
  - Within the user's config if you're not working on macOS
- And Per repository, in the hidden `.git` directory within the repository.
- The proximal configuration wins! Git reads the config within a repository last. This means you can set a configuration value that you'd normally use for all your repositories in your global or user configuration, but if you want to -- say, use a different identity for a particular repository -- you can specify a change at that level by putting those settings in the repo's own config. That's why it's a good idea to be at least familiar with the configuration.

# Configuration

## Three configuration locations

```
macos@macos:~$ cat ~/.gitconfig
[core]
editor = vim
[user]
name = John Doe
email = john.doe@example.com
[alias]
lg = log --graph
[diff]
tool = diff-so-fancy
[mergetool]
prompt = true
[commit]
gpg = true
[push]
default = simple
[remote "origin"]
url = https://github.com:john.doe/example.git
fetch = +refs/heads/*:refs/remotes/origin/*
[remote "staging"]
url = https://github.com:john.doe/staging.git
fetch = +refs/heads/*:refs/remotes/staging/*
```

## Configuration

## Three configuration locations

[illegible]

## Configuration

## Three configuration locations

- Global, default: `/etc/gitconfig`

[illegible]

## Configuration

## Three configuration locations

- Global, default: `/etc/gitconfig`
- Xcode: within Developer Library

[illegible]

# Configuration

## Three configuration locations

- Global, default: `/etc/gitconfig`
  - Xcode: within Developer Library
- User:
  - `~/.gitconfig`

[illegible]



## Configuration

## Three configuration locations

- Global, default: /etc/gitconfig
  - Xcode: within Developer Library
- User:
  - ~/.gitconfig
  - ~/.config/git/config

[illegible]



## Configuration

## Three configuration locations

- Global, default: /etc/gitconfig
- Xcode: within Developer Library
- User:
  - ~/.gitconfig
  - ~/.config/git/config
- Within the repository at .git/config

[illegible]

## Configuration

## Three configuration locations

- Global, default: `/etc/gitconfig`
- Xcode: within Developer Library
- User:
  - `~/.gitconfig`
  - `~/.config/git/config`
- Within the repository at `.git/config`
- Most local wins!

[illegible]

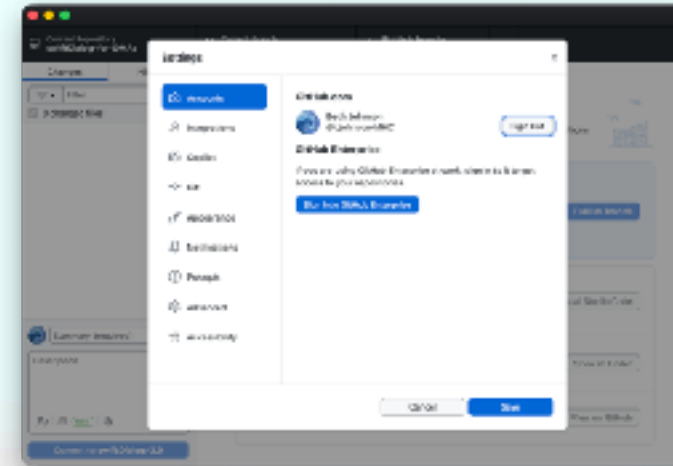
# GUI configuration

Seems obvious . . .

- The minimum configuration you need is to set your name and email.
- It seems perhaps obvious to say, you can configure the basics within a GUI tool, by signing in to your account, or
- By simply entering your information in the settings.

# GUI configuration

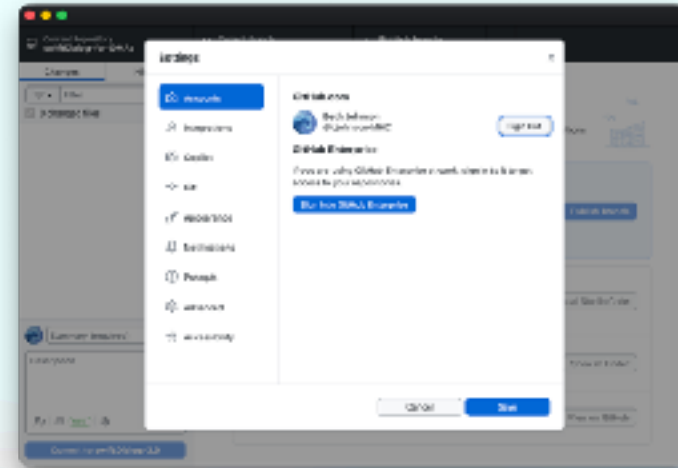
Seems obvious . . .



## GUI configuration

## Seems obvious . . .

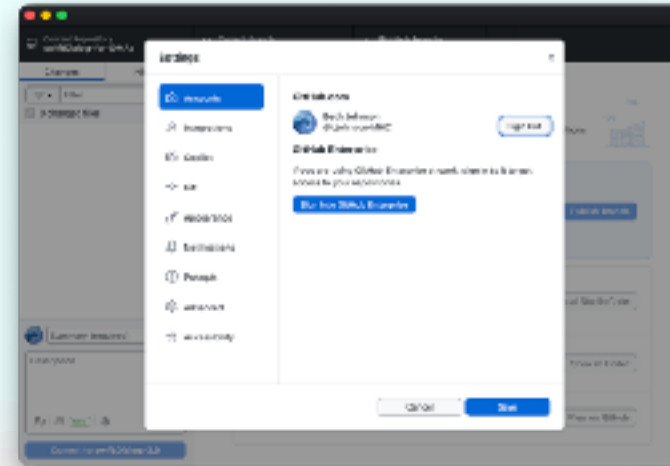
- Minimum configuration is name and email



# GUI configuration

Seems obvious . . .

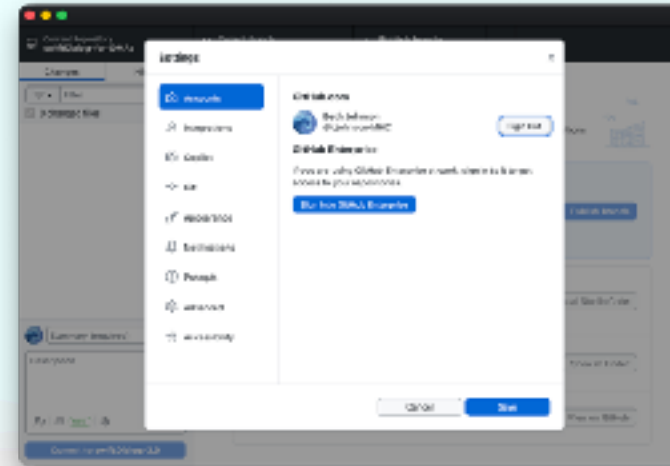
- Minimum configuration is name and email
- Sign in via app, or



# GUI configuration

Seems obvious . . .

- Minimum configuration is name and email
- Sign in via app, or
- Enter email and name in setup





# Identify yourself

Name and email

You can also do it on the command line using these Git commands.

- Use git with the config verb and specify the global config for the user name, with your full name: `$ git config --global user.name "John Doe"`
- Use git with the config verb, specify global config for user email, with your email: `$ git config --global user.email johndoe@example.com`

There are lots of other things you can set, but we'll avoid that rabbit hole, with one important exception . . .



# Identify yourself

Name and email

A terminal window with a light orange background and a white title bar. The title bar contains three colored window control buttons (red, yellow, green) and the text 'mscodemine - ssh - 105x22'. The terminal content shows a command prompt 'me@codin.us\$' followed by the command 'git config --global user.name "Jo Doe"'.

```
me@codin.us$ git config --global user.name "Jo Doe"
```

# Identify yourself

## Name and email

- `$ git config --global user.name "Jo Doe"`

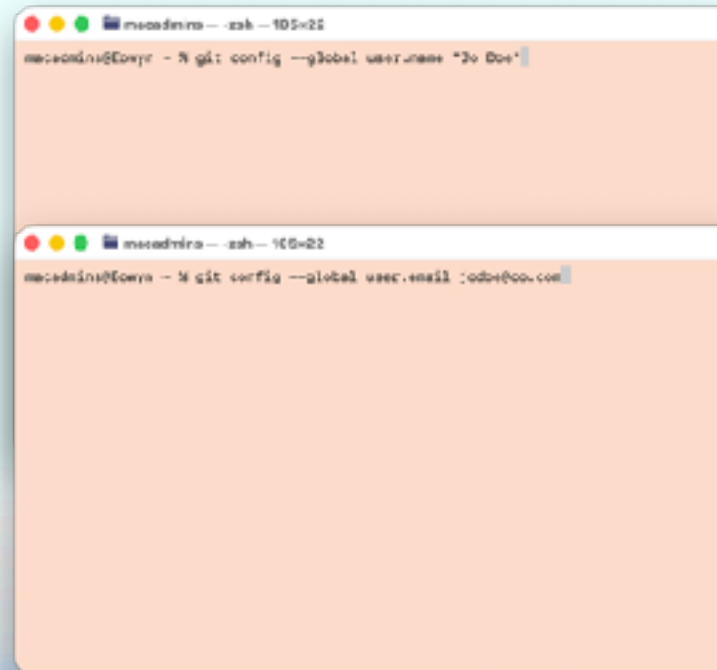
A terminal window with a light orange background and a white title bar. The title bar contains the text 'mscodemine - zsh - 105x22'. The terminal shows the command 'me@codemine:~\$ git config --global user.name "Jo Doe"' with a cursor at the end of the line.

```
me@codemine:~$ git config --global user.name "Jo Doe"
```

# Identify yourself

## Name and email

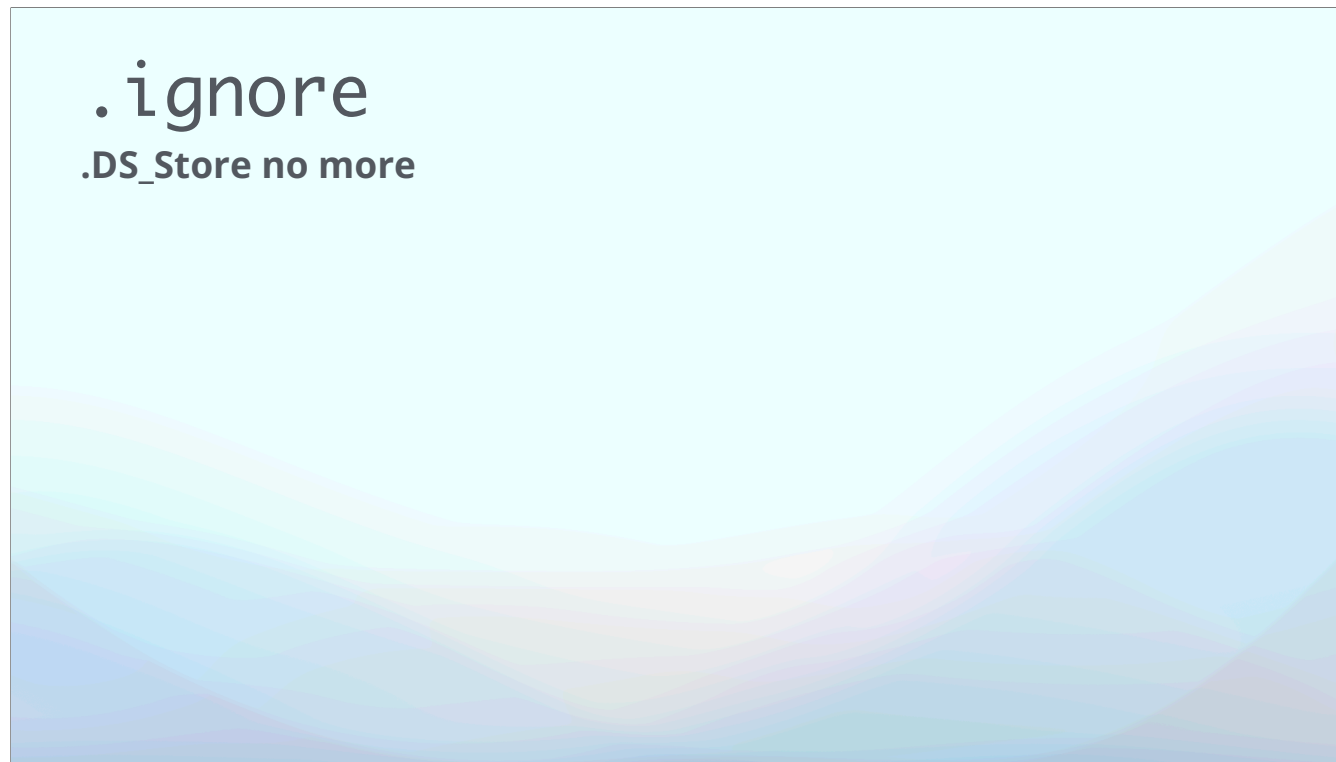
- `$ git config --global user.name "Jo Doe"`
- `$ git config --global user.email jodoe@co.com`



The image shows two terminal window screenshots. The top window has a title bar with three colored circles (red, yellow, green) and the text 'macedonino - ssh - 105x22'. The terminal text is 'macedonino@Corya ~ % git config --global user.name "Jo Doe"'. The bottom window has a title bar with the same three colored circles and the text 'macedonino - ssh - 105x22'. The terminal text is 'macedonino@Corya ~ % git config --global user.email jodoe@co.com'.

```
macedonino - ssh - 105x22
macedonino@Corya ~ % git config --global user.name "Jo Doe"

macedonino - ssh - 105x22
macedonino@Corya ~ % git config --global user.email jodoe@co.com
```



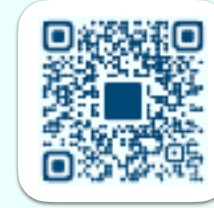
This the step that will make future you love present you, if you're working on macOS, and one thing that I couldn't get configured via GUI:

Set Git to globally ignore macOS's Desktop Services Store metadata file, which ends up in every folder and if you don't exclude it from tracking will end up in every commit. GUI tools let you choose files to ignore, but not globally.

- Yes, the link is in the resources -- don't try to copy this down or do this now.
- First, create a hidden file at the top level of your user directory
- Add `".DS_Store"` to it `~/.gitignore_global` file.
- Use the git command with the config verb to tell Git to use that gitignore file for its global exclusions

And there we go! Minimal configuration.

.ignore  
.DS\_Store no more



# .ignore

**.DS\_Store no more**

1. touch ~/.gitignore\_global



# .ignore

**.DS\_Store no more**



1. `touch ~/.gitignore_global`

2. `echo ".DS_Store" >> ~/.gitignore_global`



# .ignore

**.DS\_Store no more**



1. `touch ~/.gitignore_global`
2. `echo ".DS_Store" >> ~/.gitignore_global`
3. `git config --global core.excludesfile  
~/.gitignore_global`



# Git to work

Now that we've got Git configured, what the heck are we going to do with it?

# Git to work



Anneke van der Feer, Washerwomen, 1938

# Git Goals



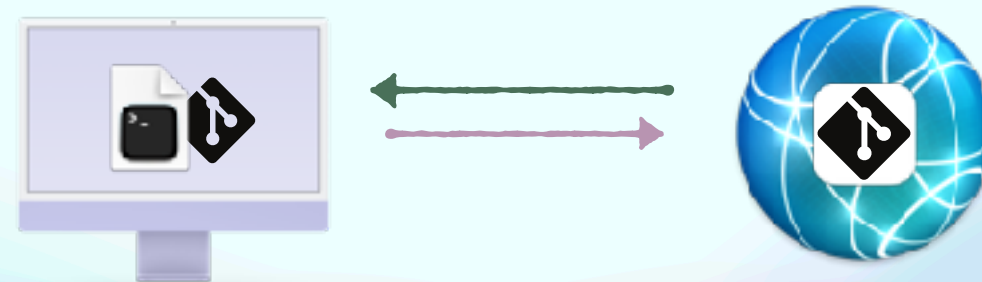
Let's go over my goals. I want to:

- Safely sync my management scripts someplace off my admin computer
- Access them from another computer if necessary
- Make a new branch to work on changes to the scripts while keeping them separate from production
- Neatly merge those changes in when I'm done
- Allow other admins to collaborate with scripts and documentation

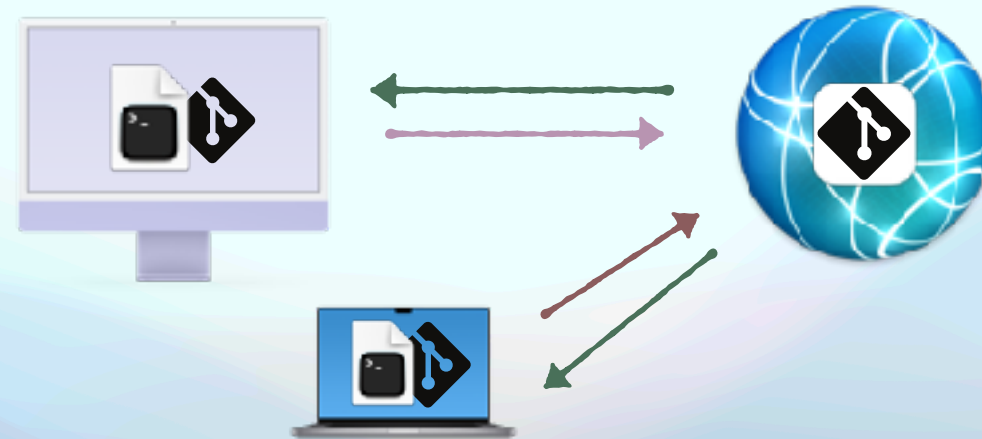
# Git Goals



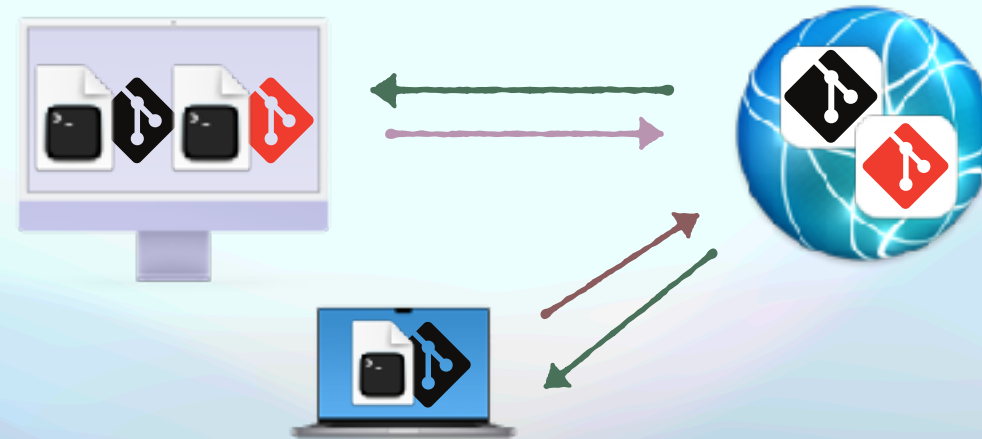
## Git Goals



# Git Goals

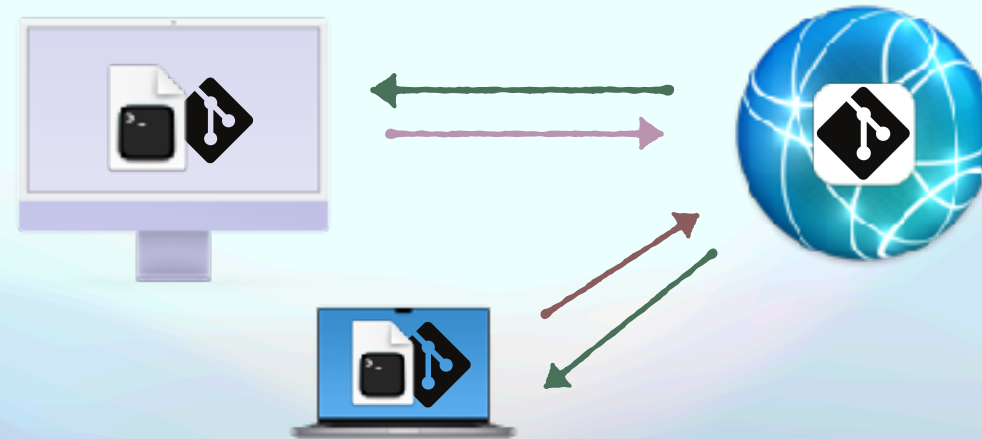


## Git Goals



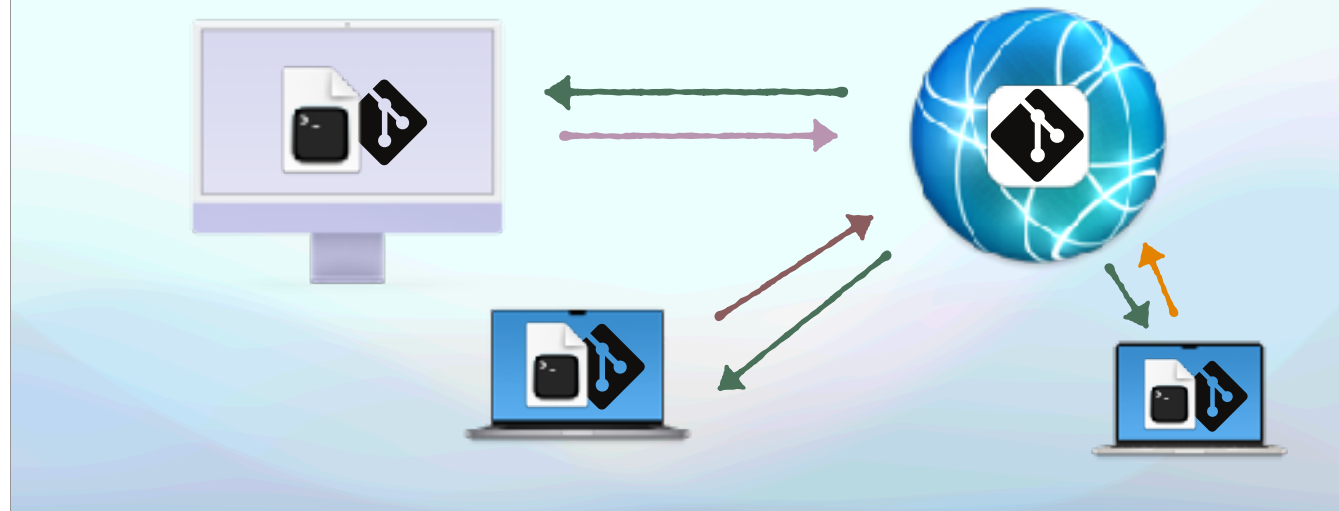


# Git Goals





## Git Goals



# Getting Ready

## Hosting considerations

First we need a hosting service. Obviously, if your organization already has a service, that's probably your best choice. But if you don't, give some thought as we go through the next 30 minutes or so about what features you think would help you out the most, and would be dealbreakers if they are missing.

For instance, in our production admin repository,

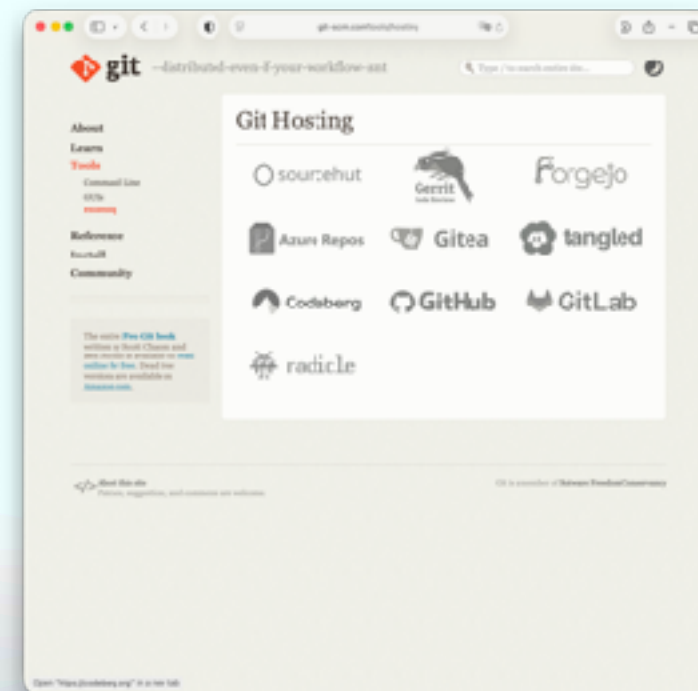
- the ability to define roles on a granular basis is very important.
- Account tiers give access to new features and integrations as you move up from a solo, private account.
- Tools may limit the size of files or have additional hurdles if you need to store large ones.
- A Wiki may or may not be available by account tier or conditions of the repository -- for instance, GitHub offers a wiki at the free personal account tier, but only if the repository is public.
- Automation tools can help do some of the routine work for you -- so I'm told.
- And it's good to think about whether your situation suggests self-hosting vs. vendor-hosted.

I use both GitHub and GitLab. We have an on-campus GitLab instance which is where I store my admin-work.

And I use a free GitHub account to interact with other Mac Admins on that service and to host the resources for my conference sessions. I guess you could call that my social-work.

# Getting Ready

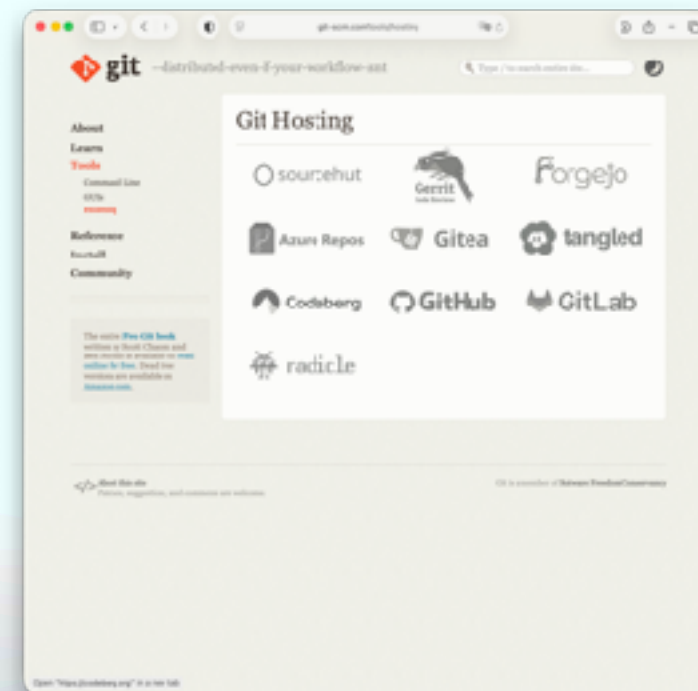
## Hosting considerations



# Getting Ready

## Hosting considerations

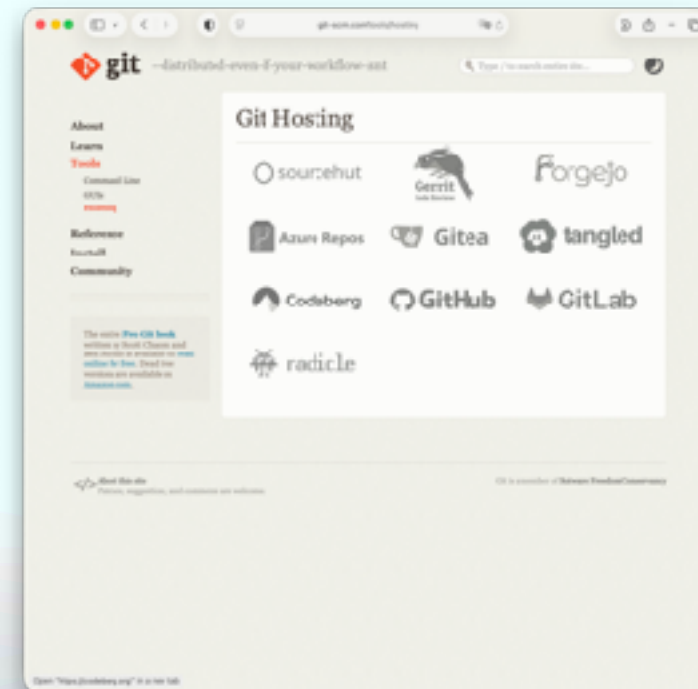
- Roles and permissions



# Getting Ready

## Hosting considerations

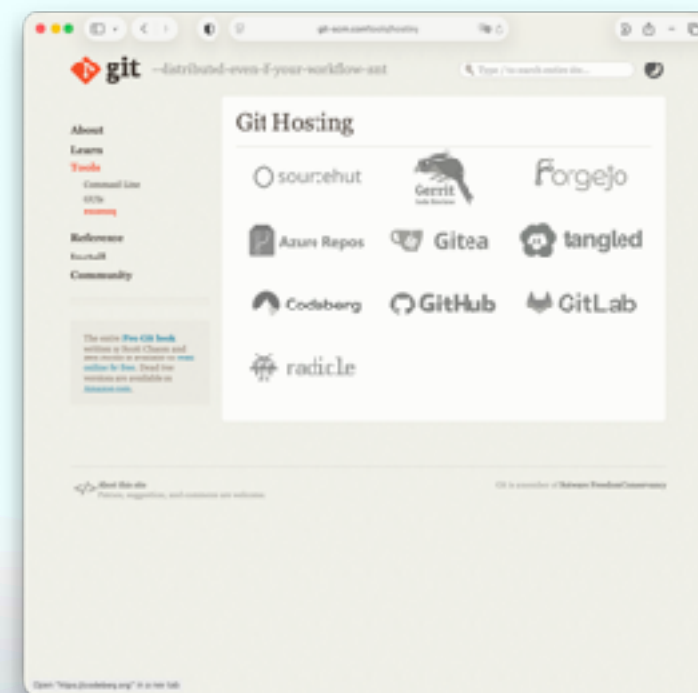
- Roles and permissions
- Account tiers



# Getting Ready

## Hosting considerations

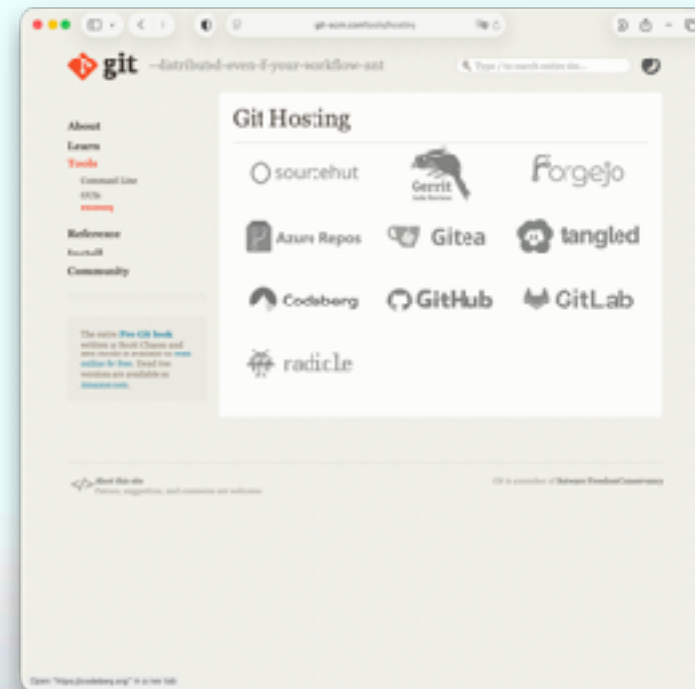
- Roles and permissions
- Account tiers
- File size constraints



# Getting Ready

## Hosting considerations

- Roles and permissions
- Account tiers
- File size constraints
- Available Wiki

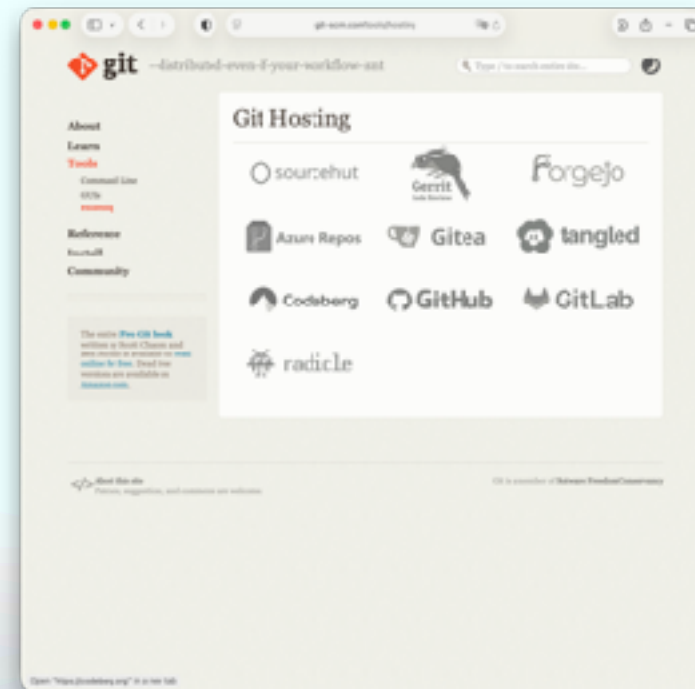




# Getting Ready

## Hosting considerations

- Roles and permissions
- Account tiers
- File size constraints
- Available Wiki
- Automation tools

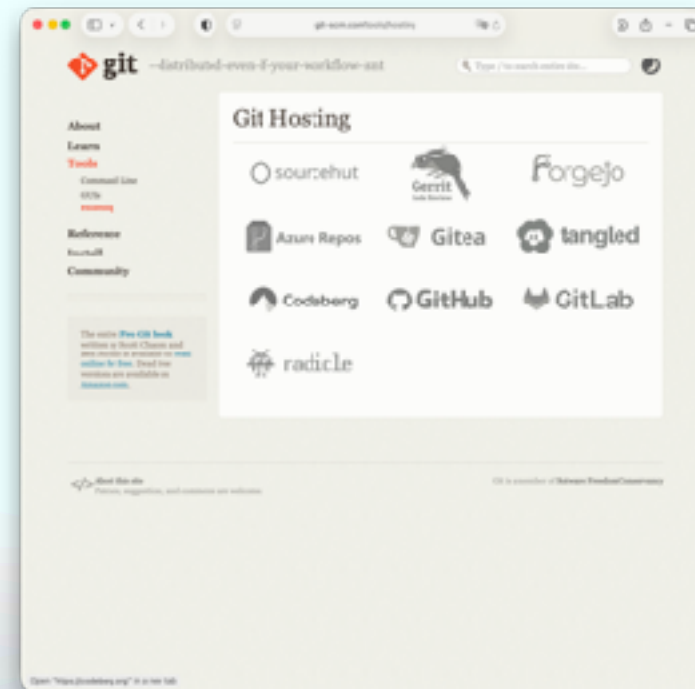


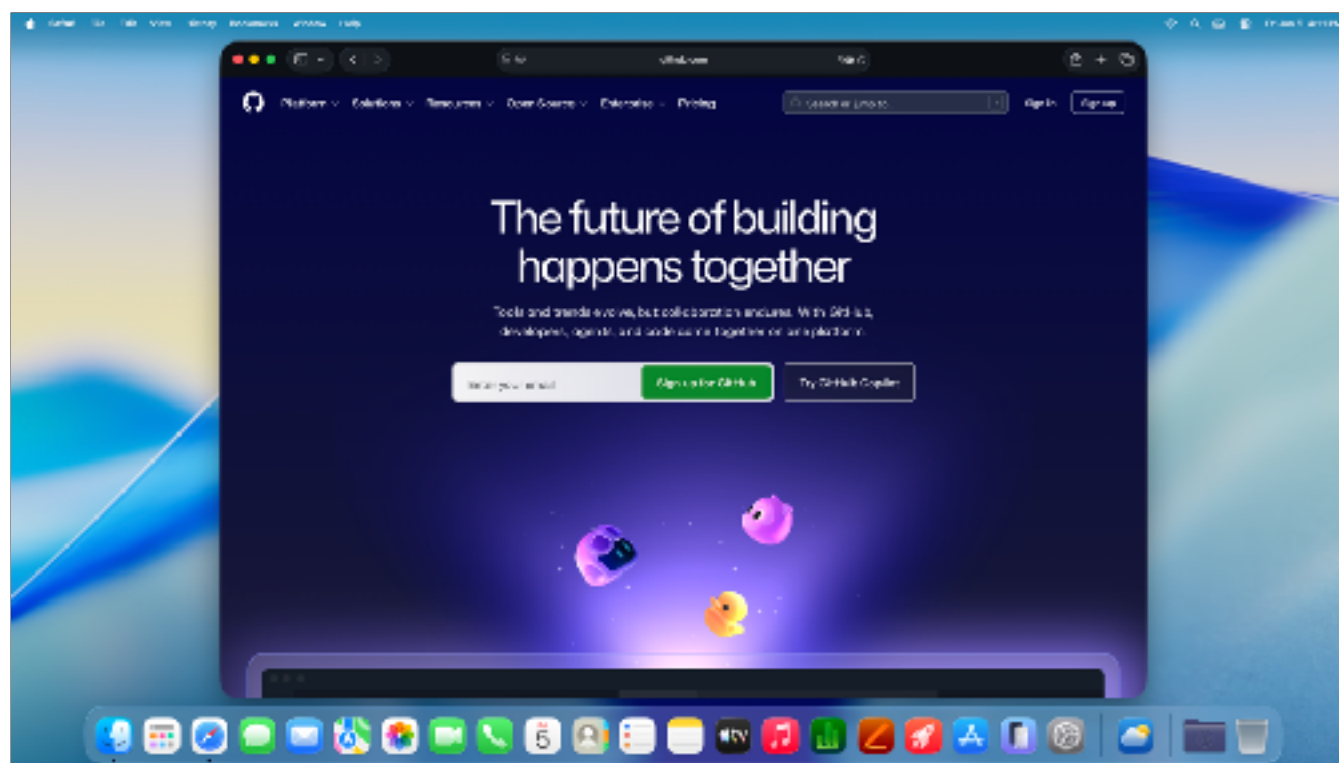


# Getting Ready

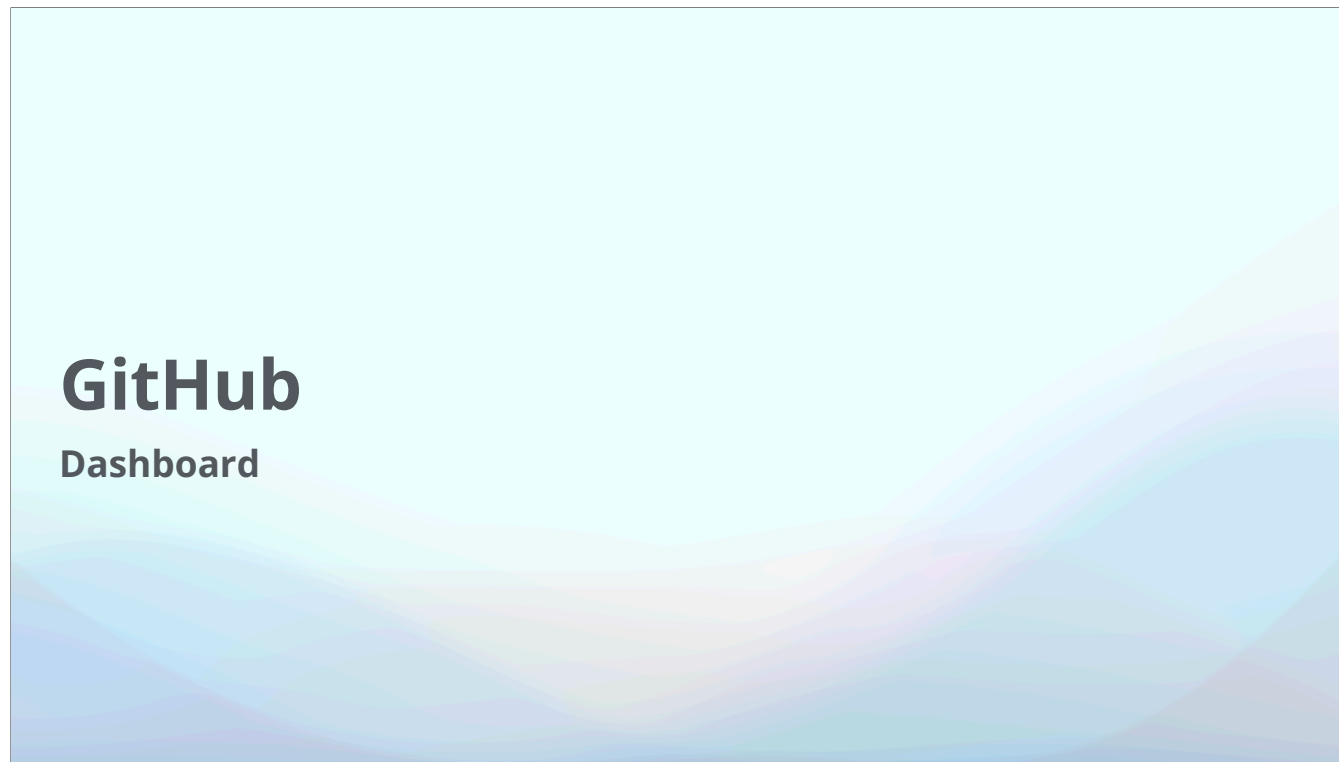
## Hosting considerations

- Roles and permissions
- Account tiers
- File size constraints
- Available Wiki
- Automation tools
- Self-hosted vs. vendor-hosted





Speaking of which, since this session is part of that work, I'll mostly be showing GitHub here.



Once you have an account, you have a home page and dashboard for yourself.

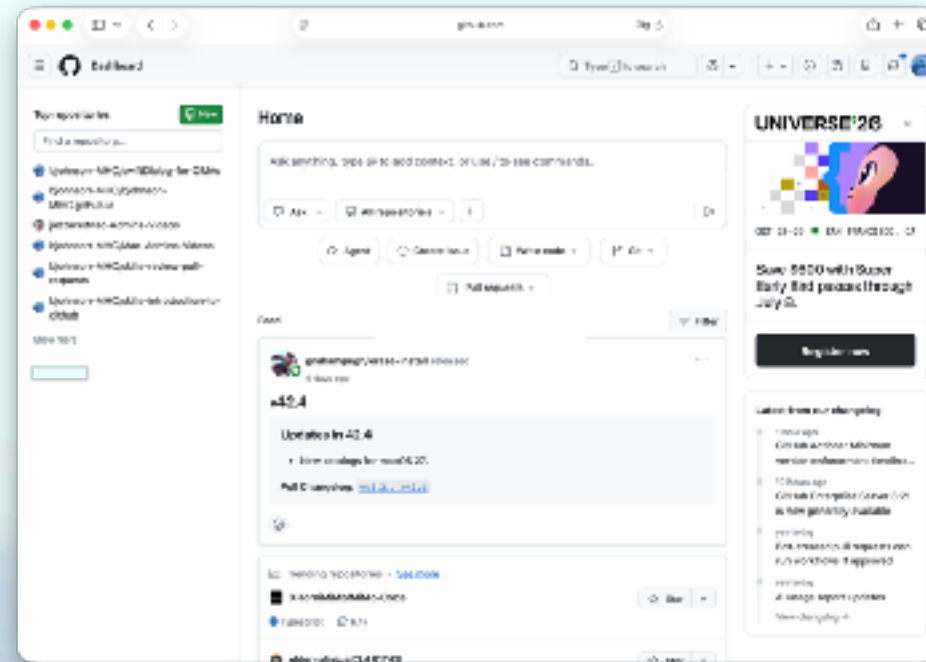
- In the center we have a feed.
- If you subscribe to notifications about other projects, you see updates from those here as well, like our erase-install update.
- On the right we've got some news from GitHub.
- And on the left we've got repositories, including my own, that I've interacted with the most.

Most of these have been for learning git, but the top one is the repository for stuff from my session last year.

We want to start a new repository, and there are a number of ways to do that.

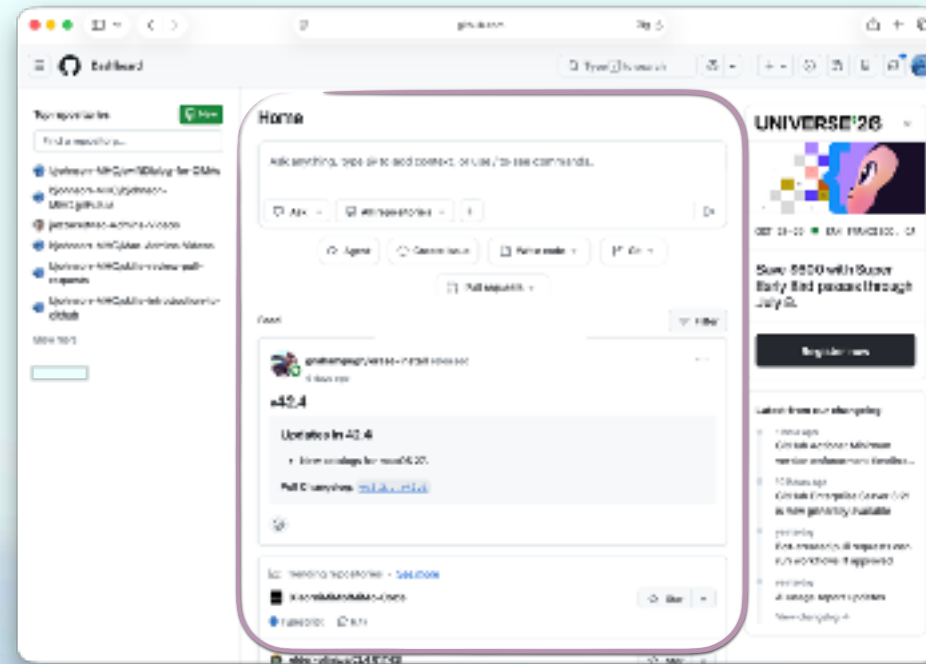
# GitHub

## Dashboard



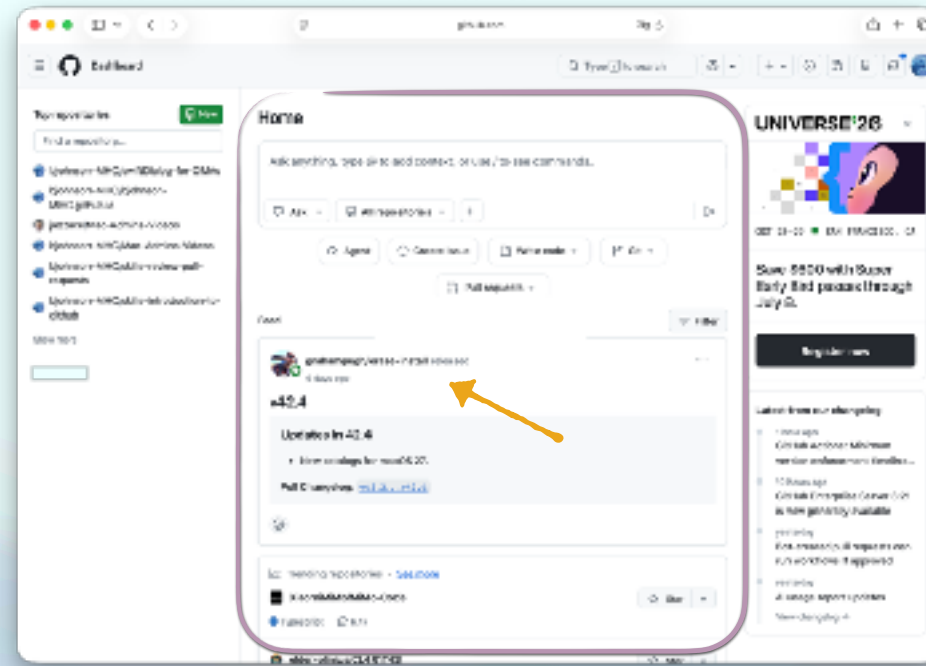
# GitHub

## Dashboard



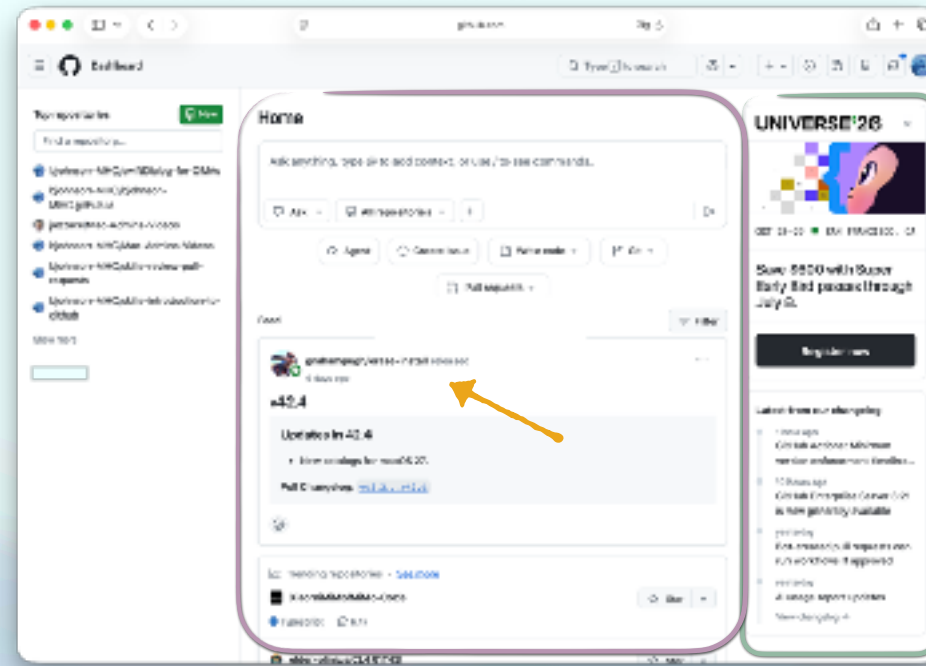
# GitHub

## Dashboard



# GitHub

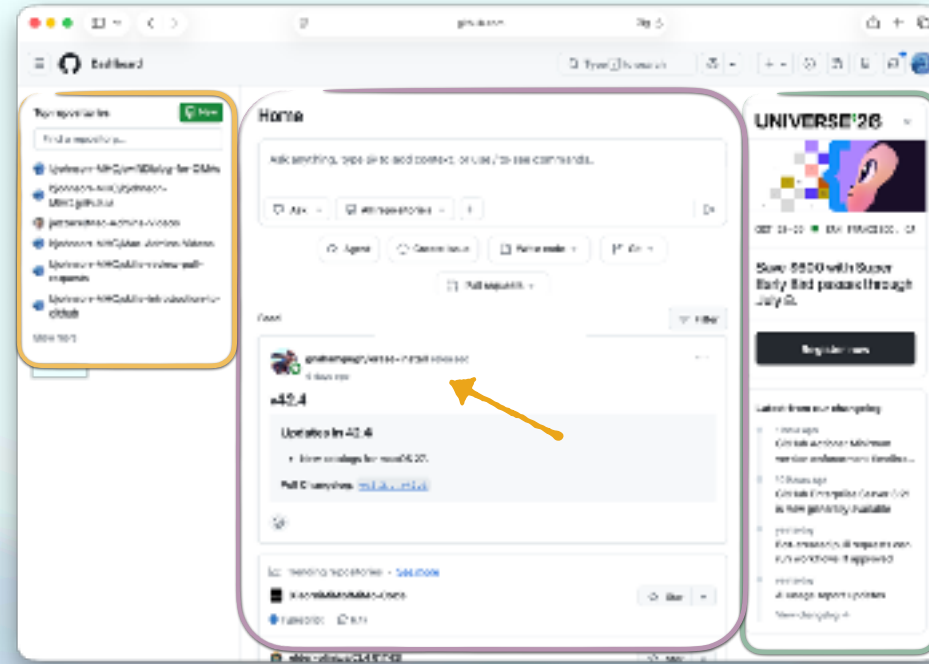
## Dashboard



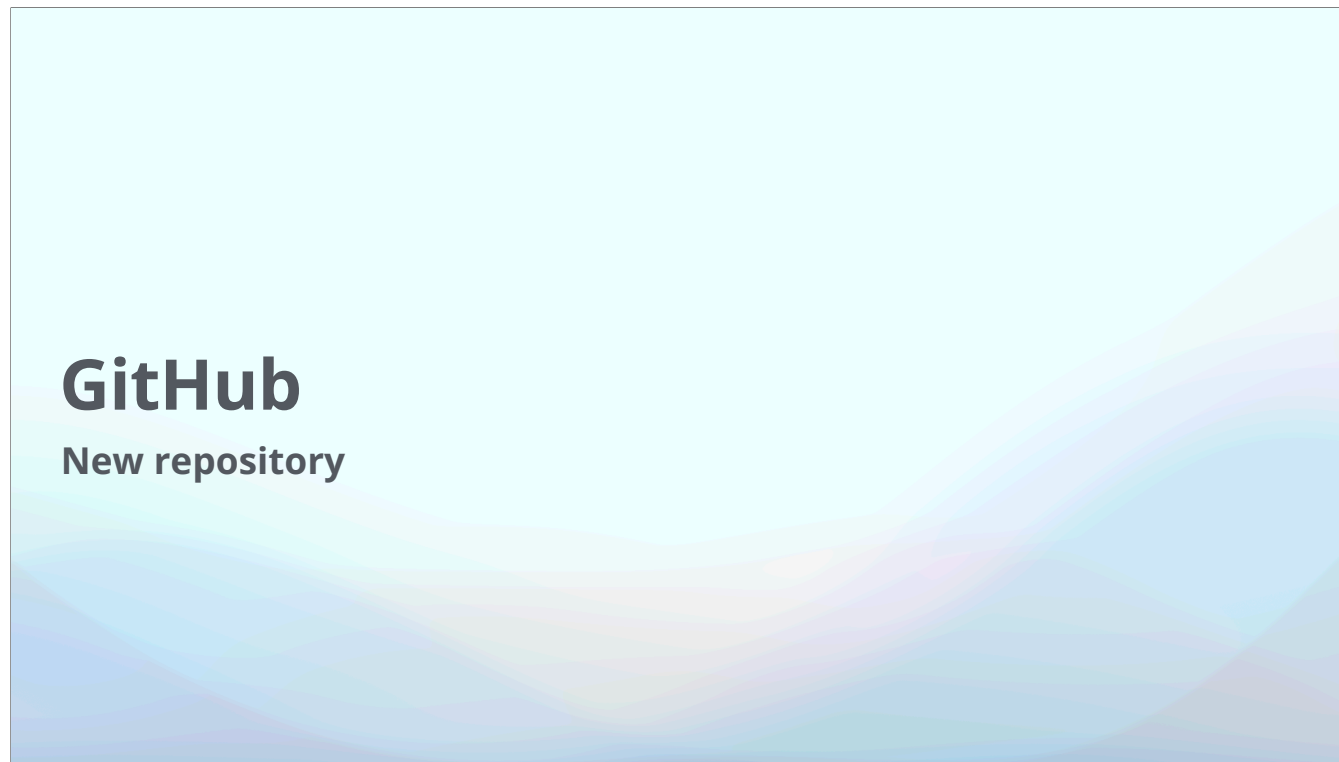


# GitHub

## Dashboard



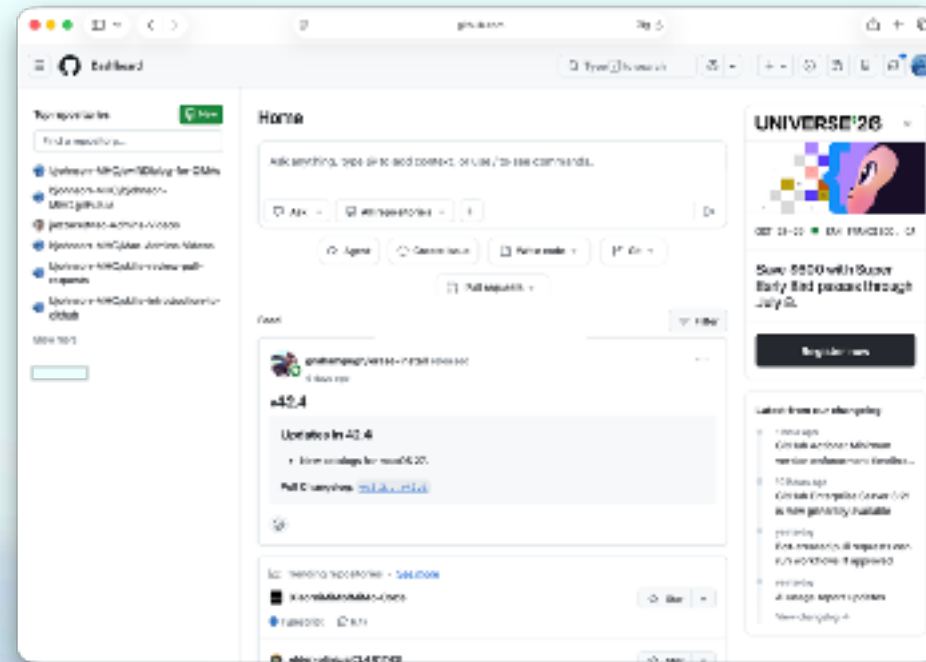




- You can use that icon that looks like a book,
  - or you can select the Add menu and select a new repository
  - You can also get there from your account overview, which you can reach from your profile. Just select your user icon in the top right and choose profile.
- Wherever we've got that new button, we can set up a new repository.

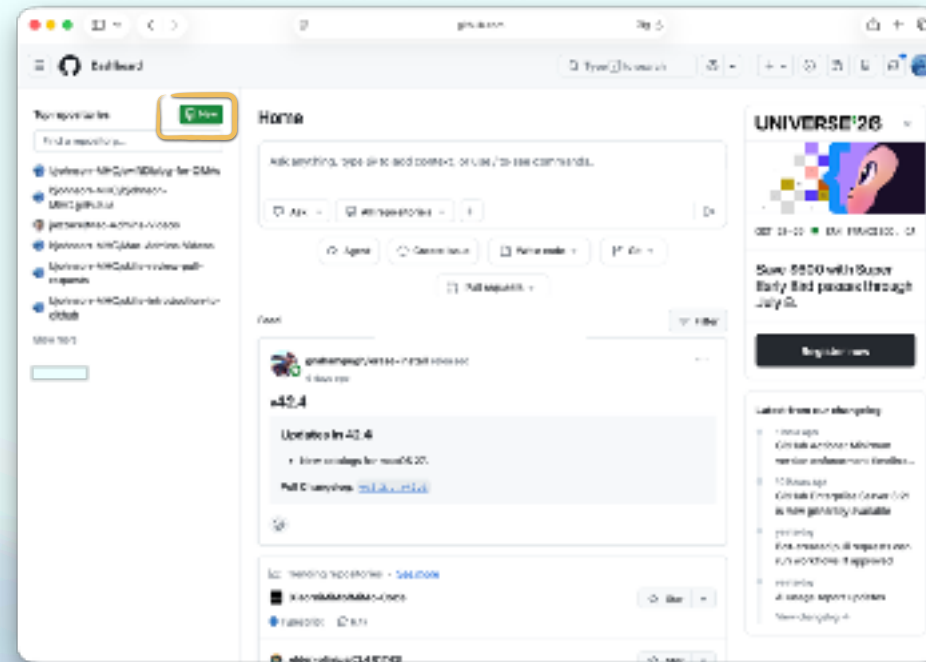
# GitHub

New repository



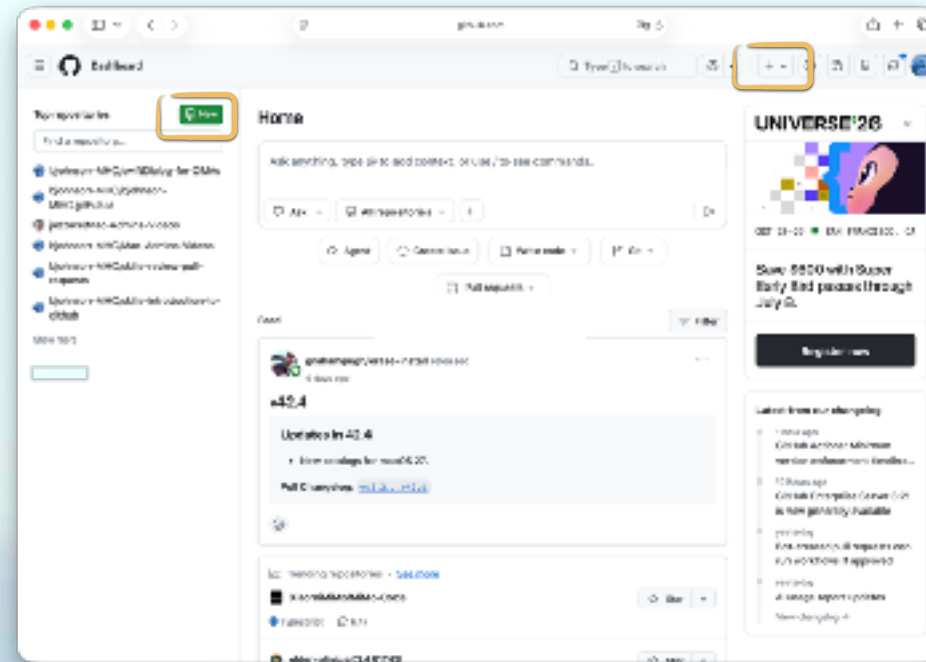
# GitHub

New repository



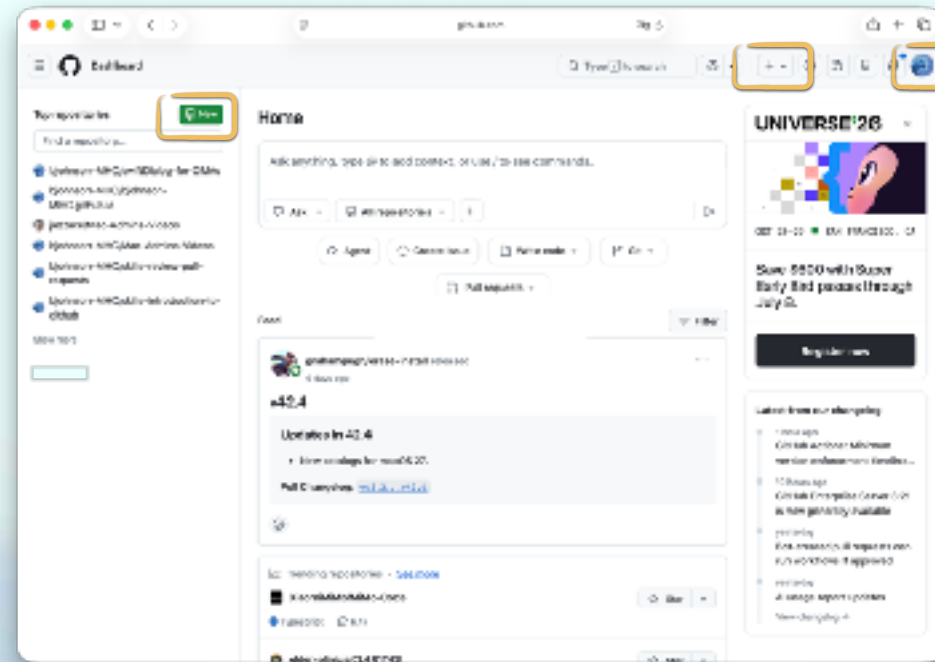
# GitHub

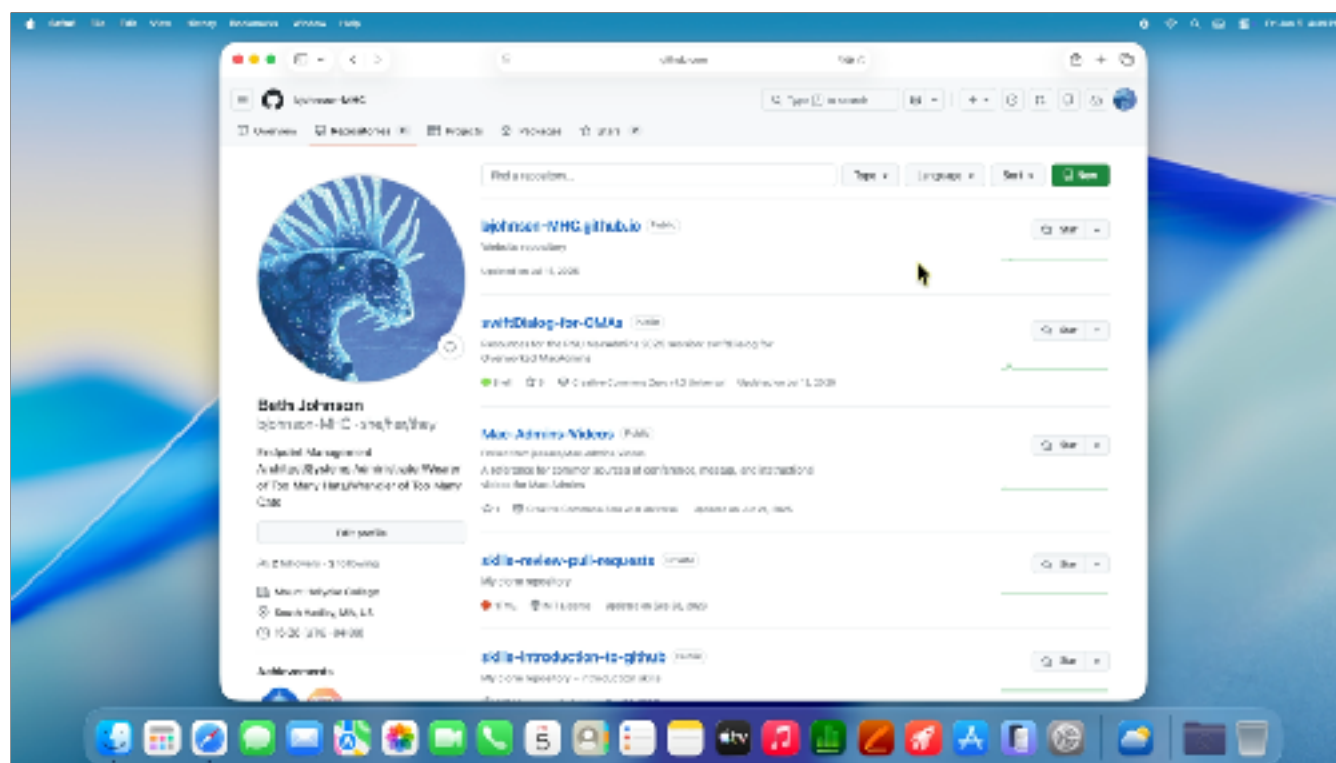
New repository



# GitHub

New repository





Let's take a look at setting up a new repo from the account profile view.

Select New, pick a name that's unique. I like to be really boring and descriptive.

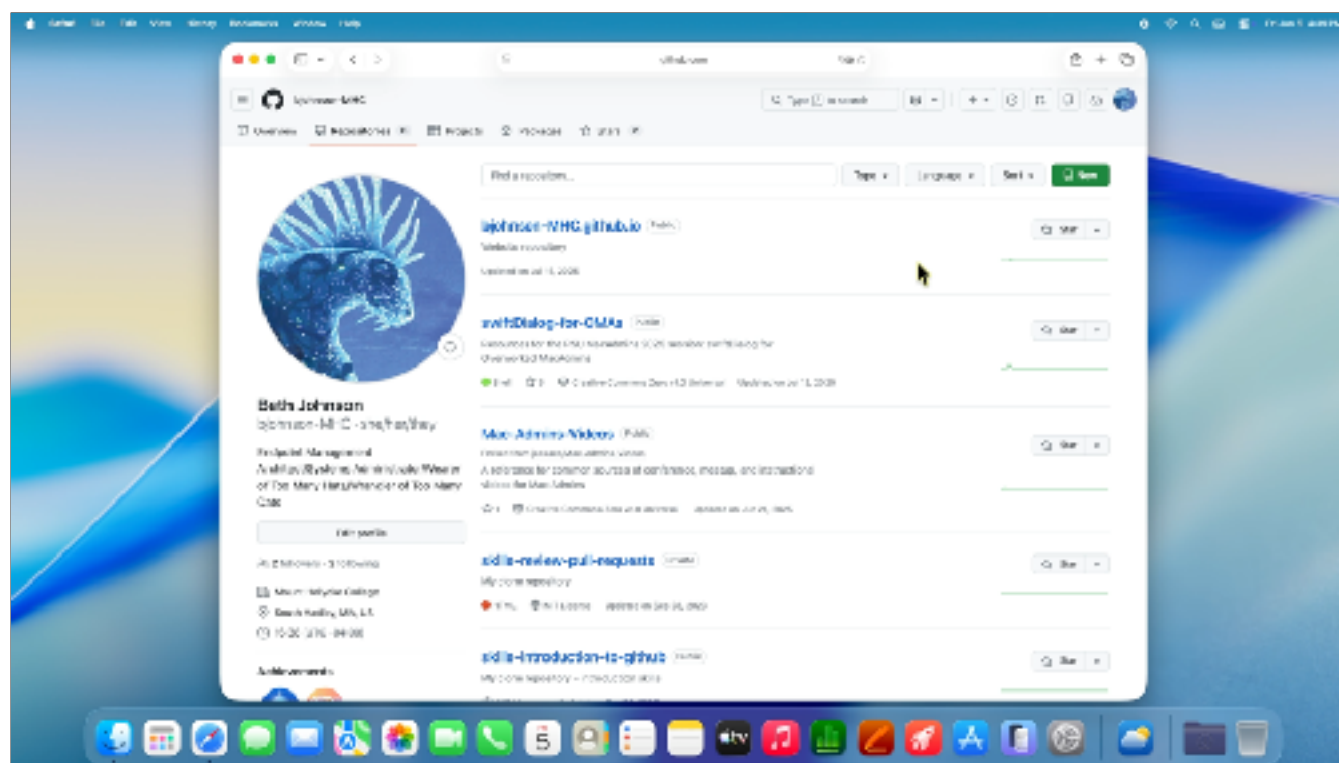
Add a description for the repo. This can be edited later.

Set your visibility. This can also be edited later. In this case, we want to set it public to get a wiki at my account tier.

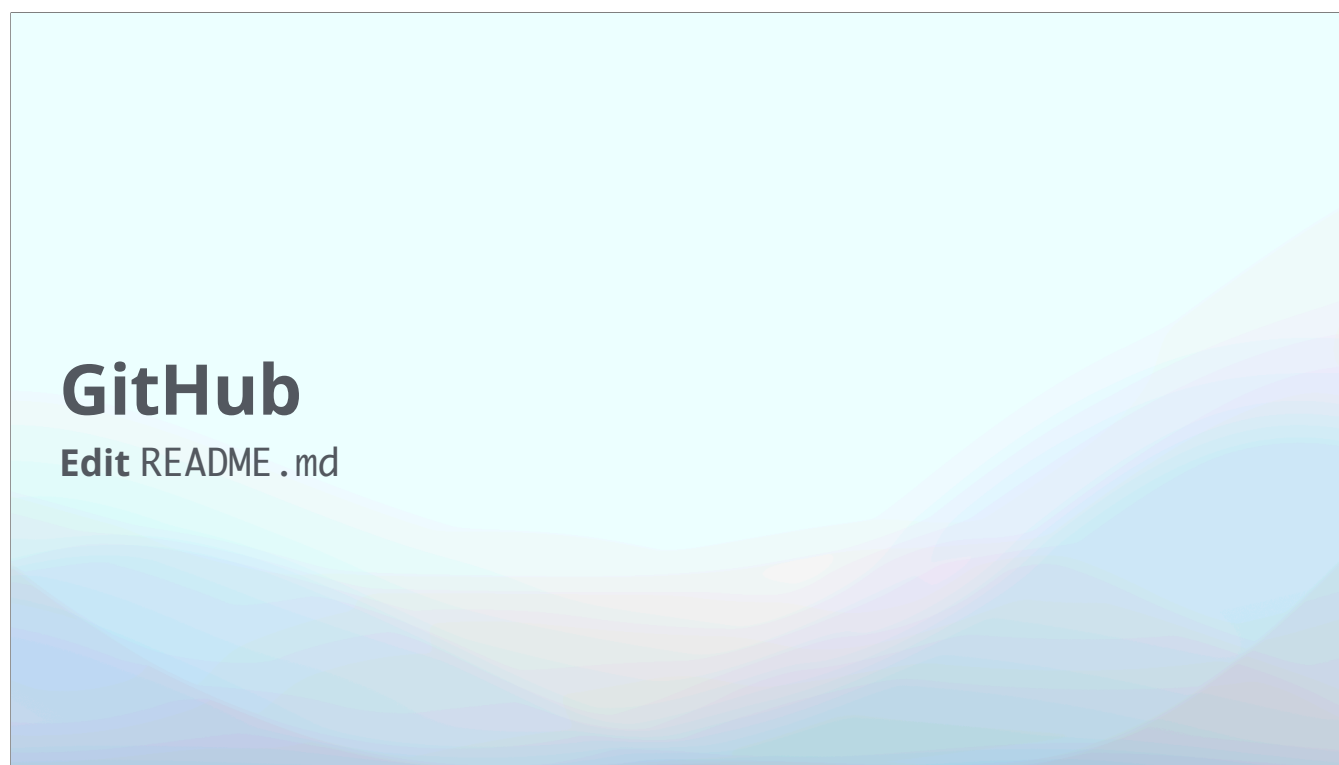
Yes, I want a to create a README file.

Select a license type. There are a lot of types you can choose from, and I'm choosing Creative Commons for this use case. If you're not sure, you can add one later.

Then select "Create repository" and it initializes the repo in the cloud.







And there it is . . . our repository.

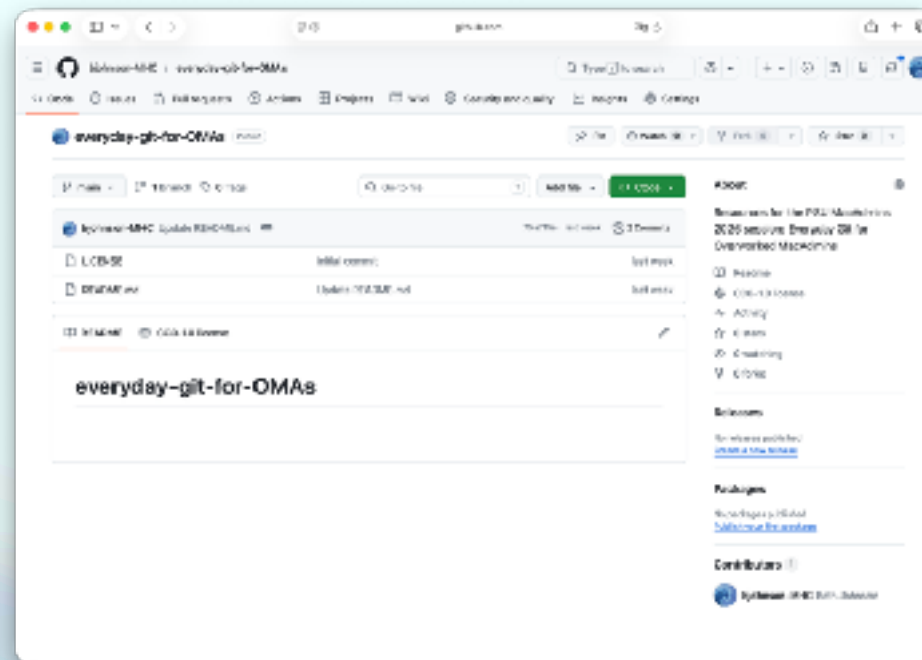
I feel like we should add some description in this readme, though, because right now it only has a title.

- So we can go ahead and select that edit pencil . . .



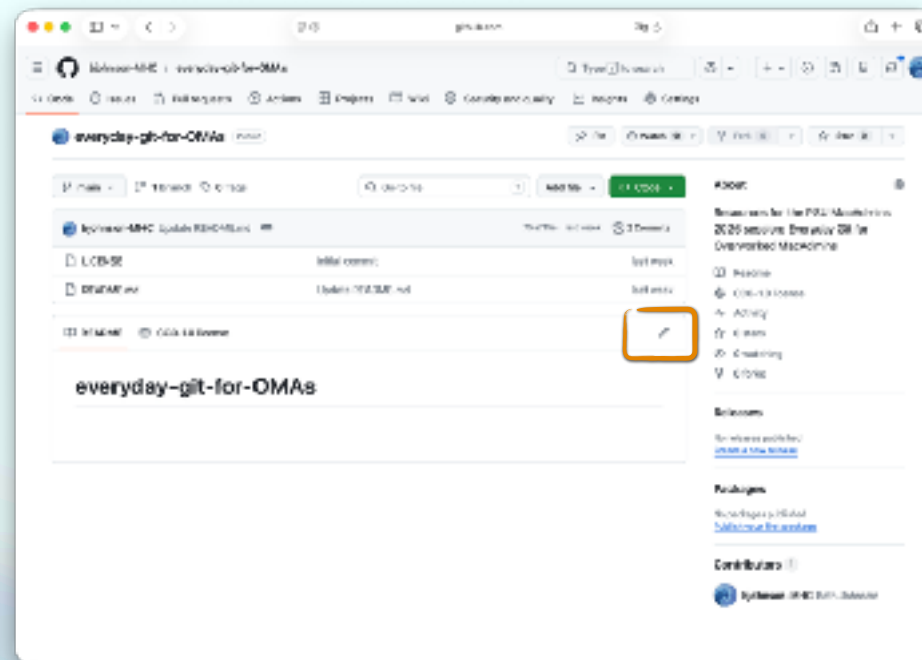
# GitHub

Edit README .md



# GitHub

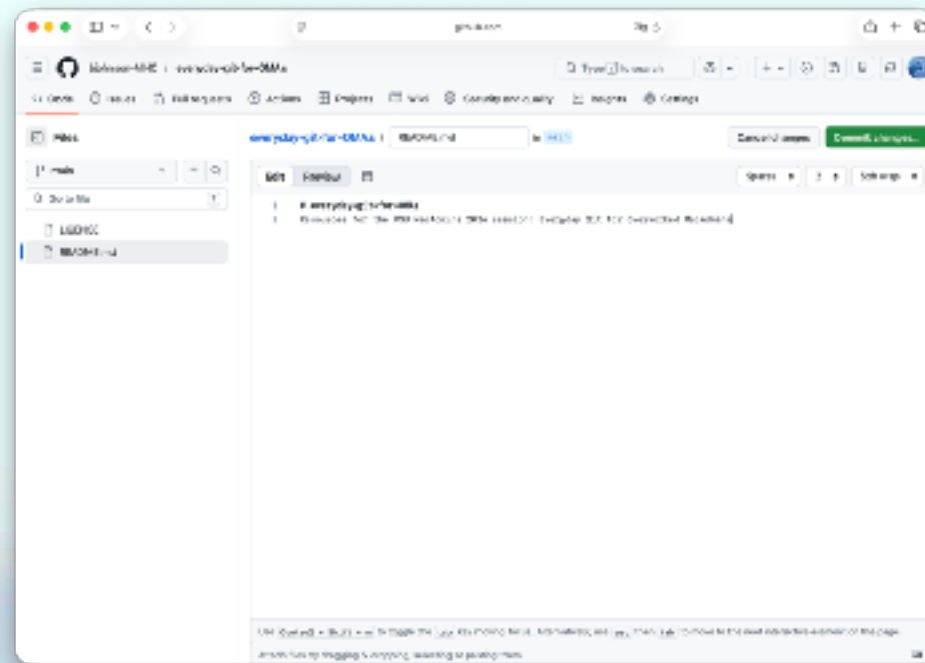
Edit README .md



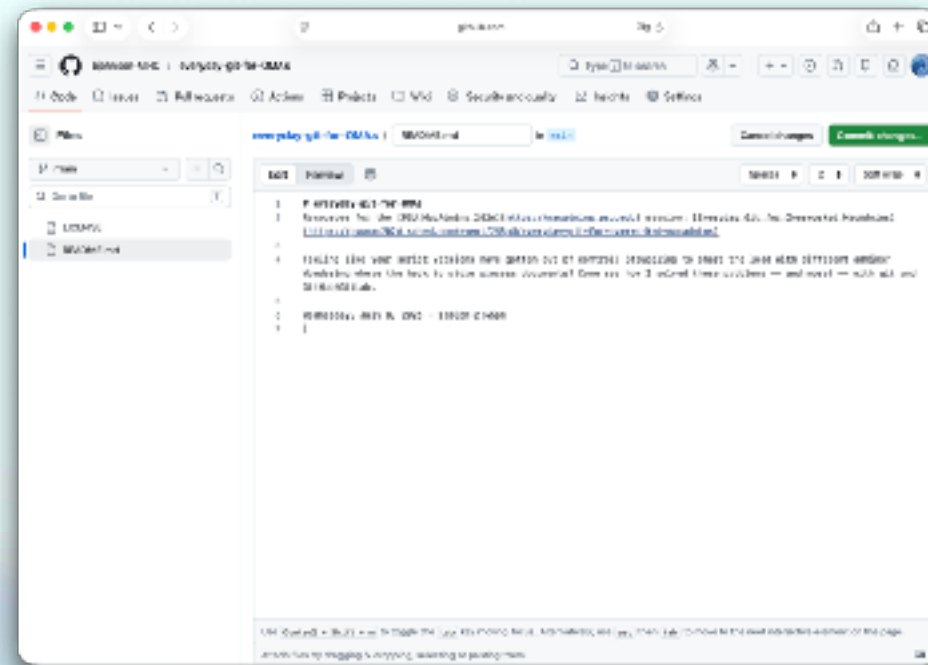


## Marking it down

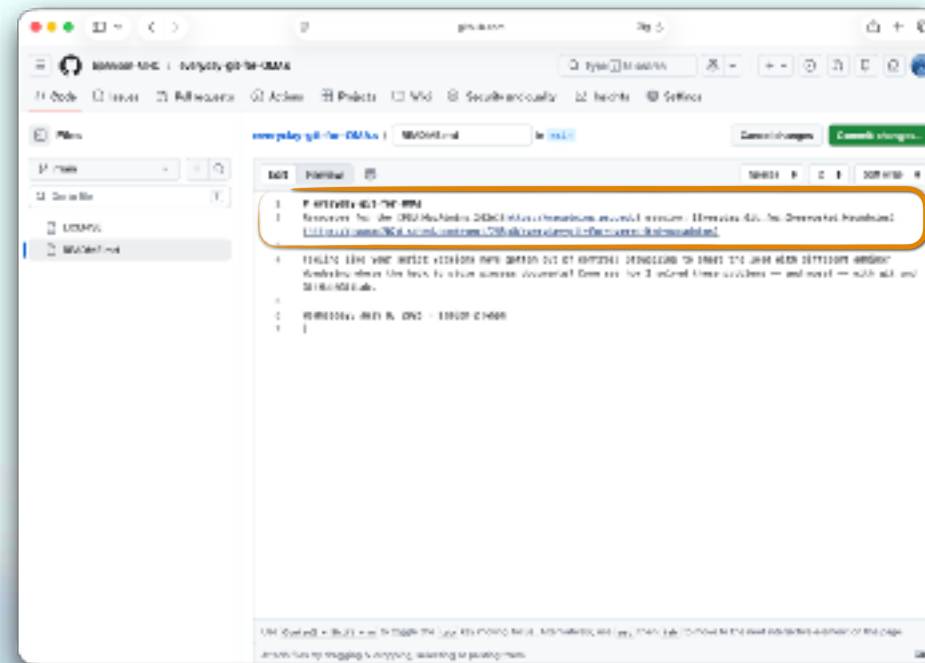
- Paste in the session information to the readme . . .
- If you know Markdown, then formatting this is a breeze. If you don't know Markdown, I have a link in the resources.
- When we're done, we'll hit that big green "Commit changes . . ." button and . . .



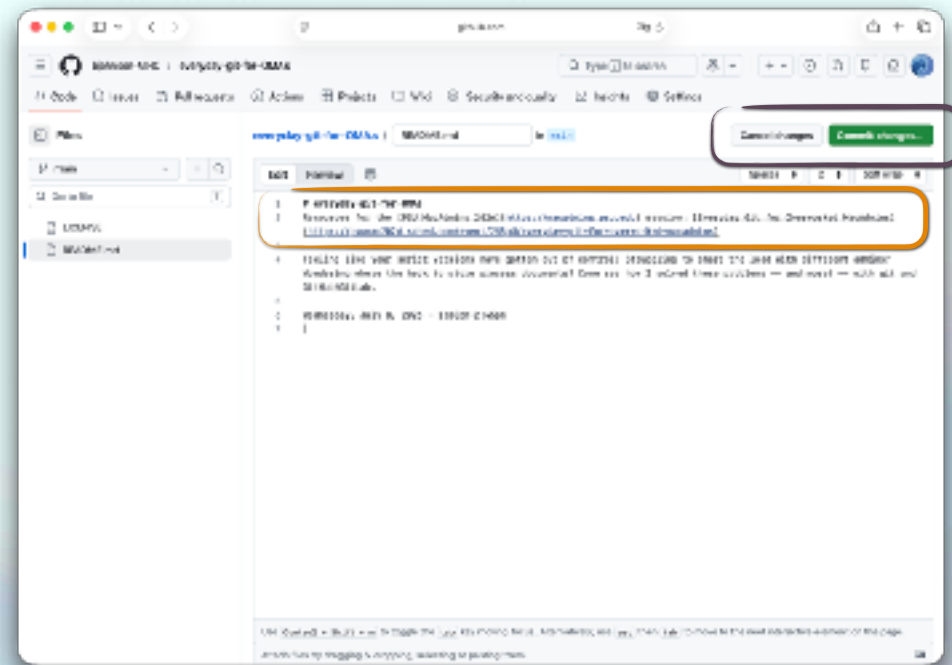
Marking it  
down



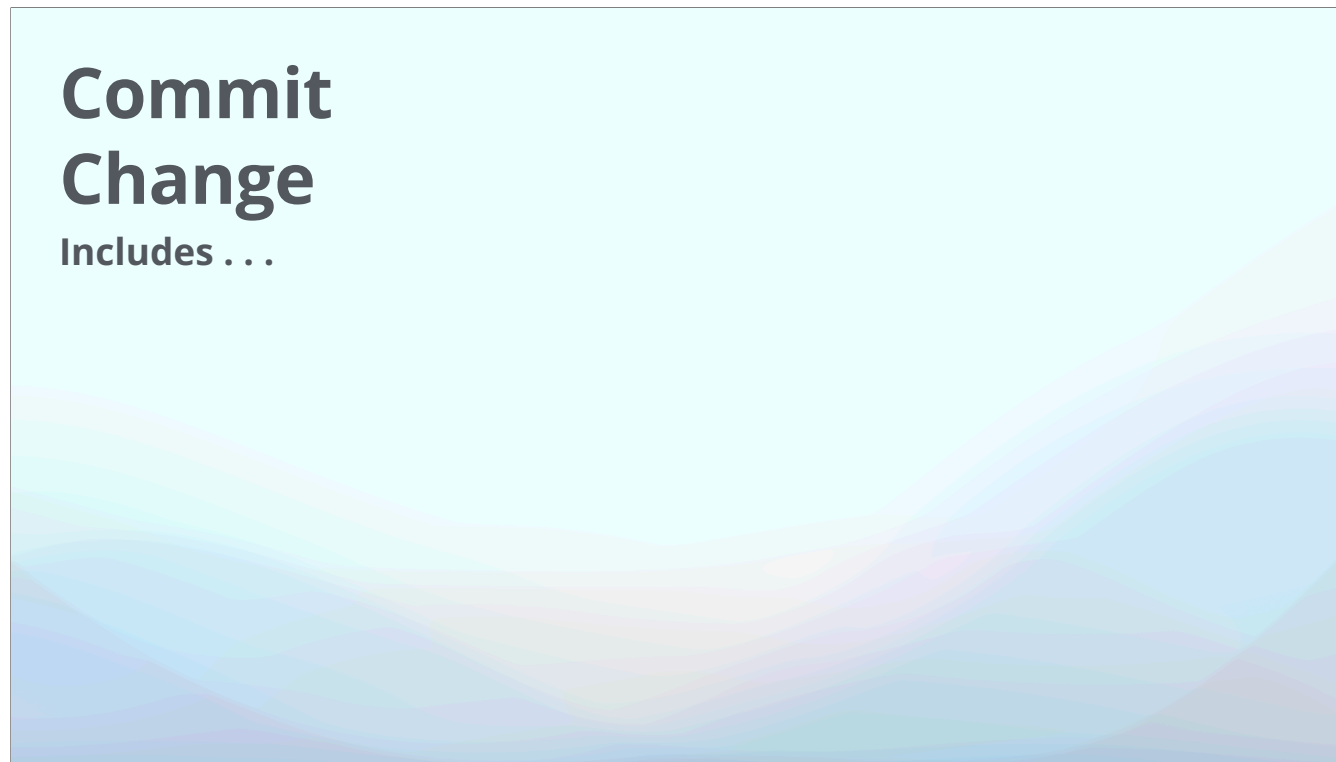
Marking it  
down



Marking it  
down







Even though we're just editing the README, it's a commit just like committing code changes.

The important parts of a commit are:

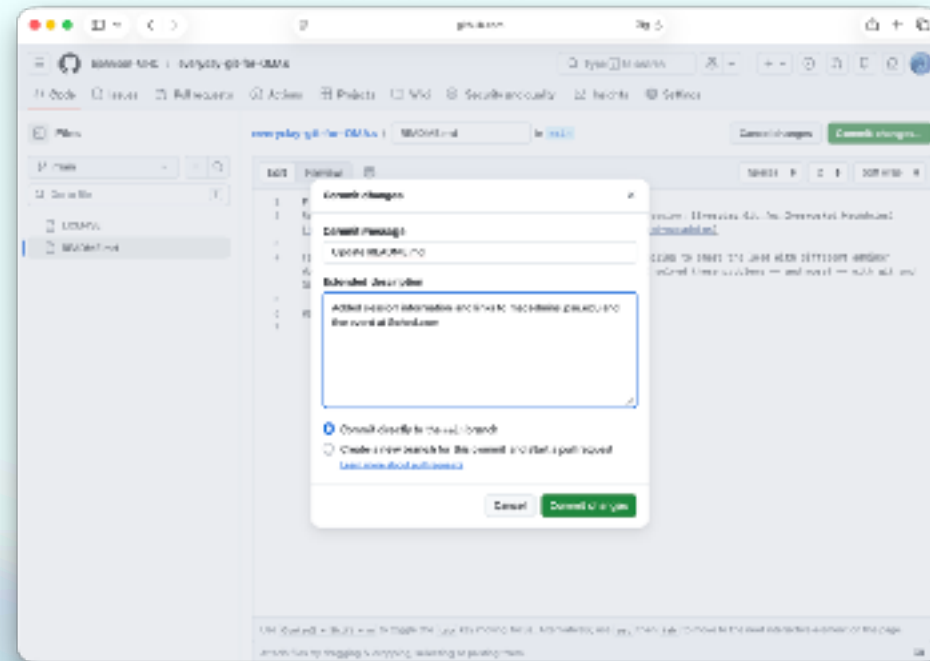
- The summary message about the changes -- and here we've got one that's auto-generated since the change to the single file is straightforward
- The description with more detail about the changes These elements are important as both reminders to yourself about what had occurred, and to collaborators.
- And of course the actual changes in the files that are being committed
- Then there's a choice about whether this should go right into the main branch — the core of our repository — or be put into a new branch.

It's just the README and I haven't done anything else, so I'll commit it right to main.



# Commit Change

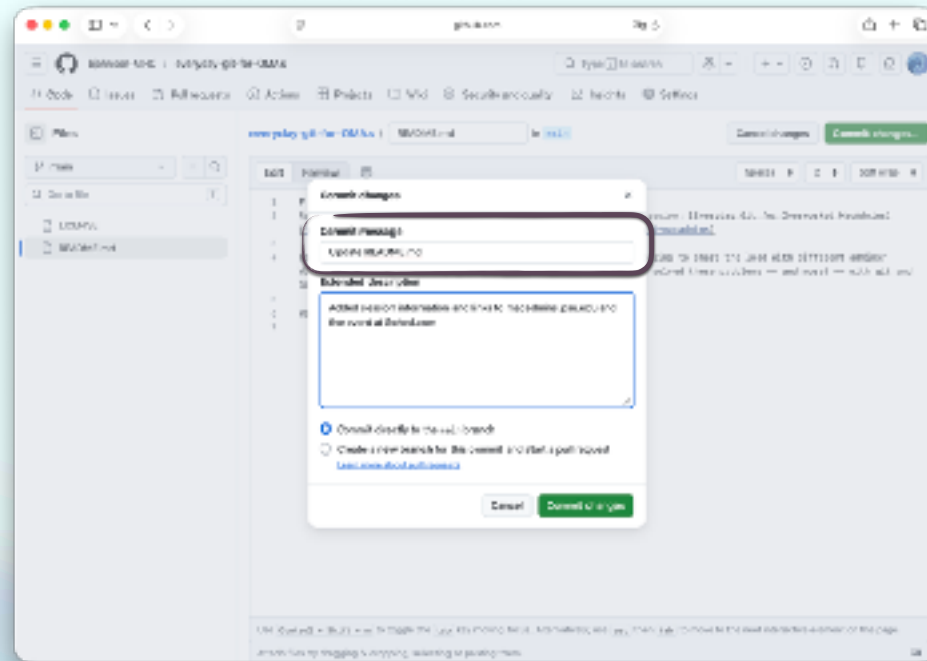
Includes . . .



# Commit Change

Includes . . .

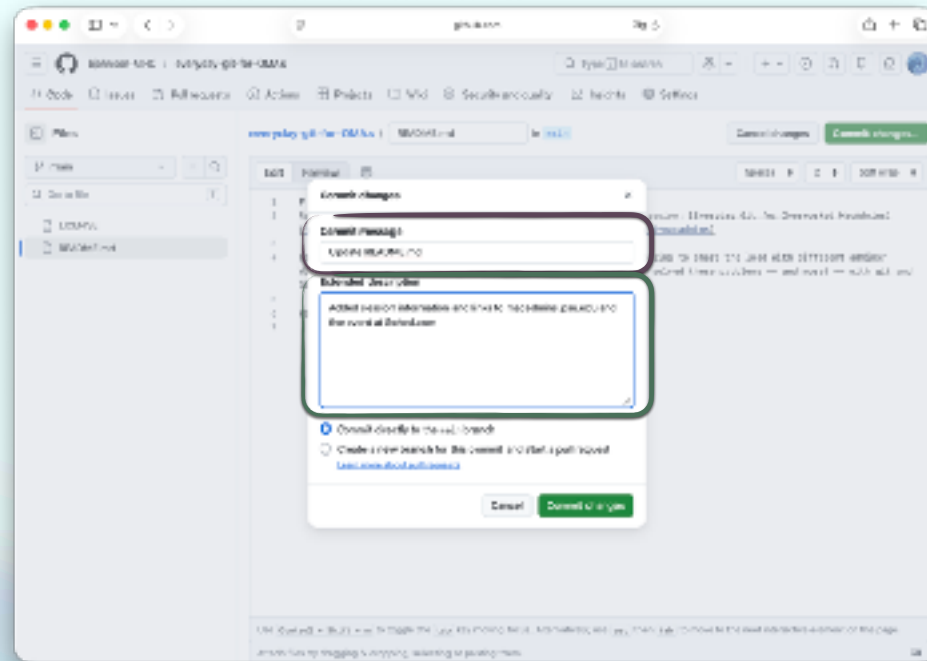
- Summary - aka, commit message



# Commit Change

Includes . . .

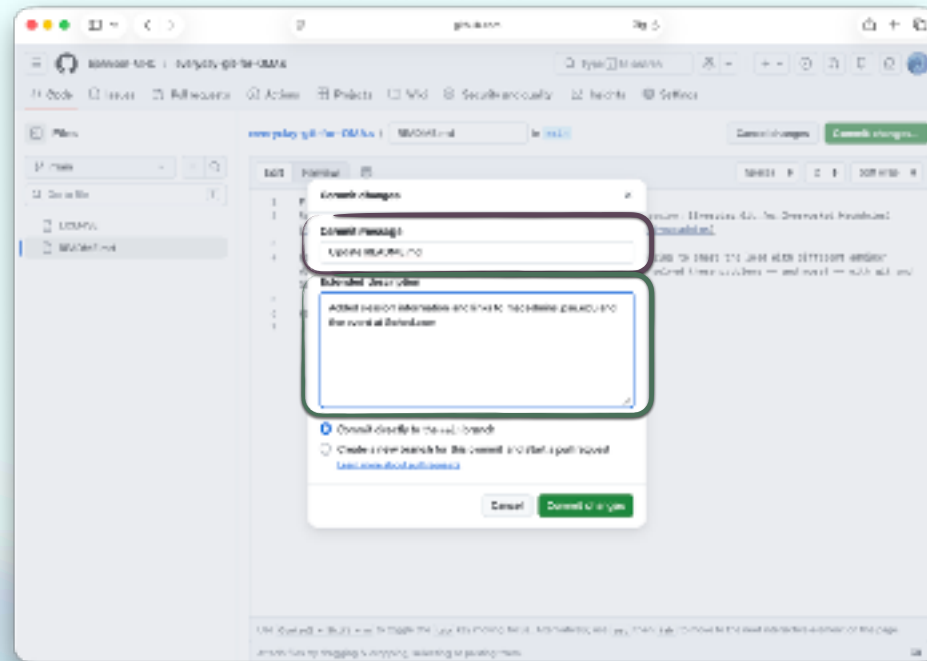
- Summary - aka, commit message
- Description



# Commit Change

Includes . . .

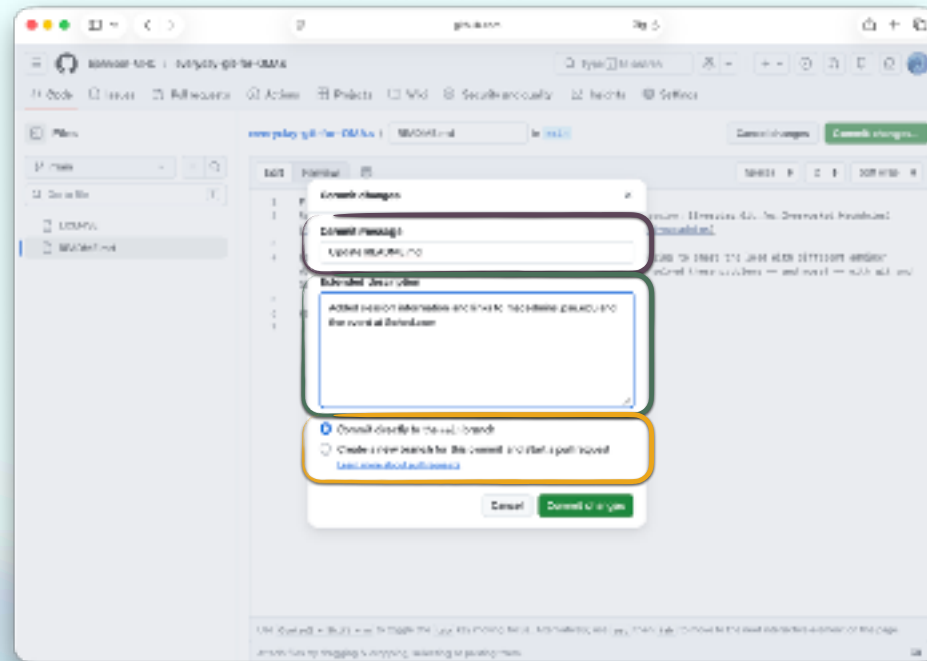
- Summary - aka, commit message
- Description
- Actual files



# Commit Change

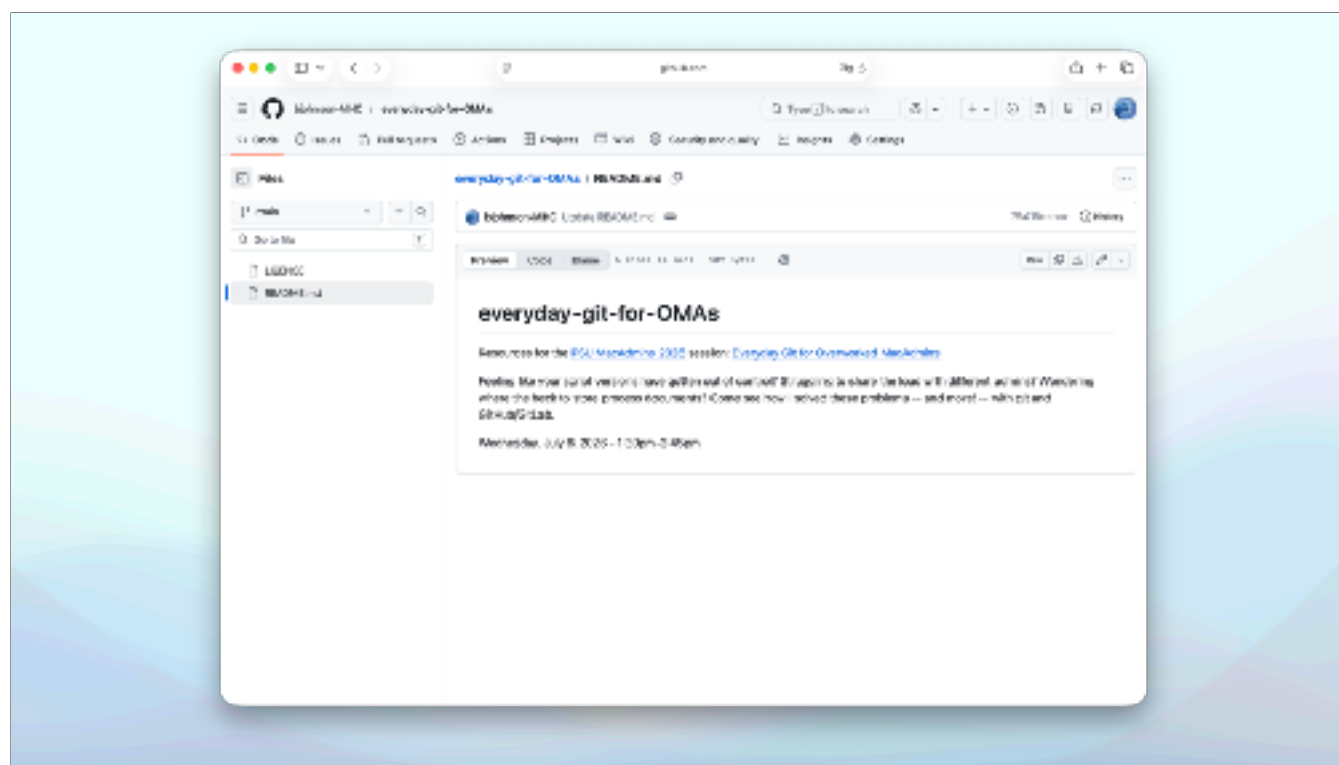
Includes . . .

- Summary - aka, commit message
- Description
- Actual files
- Destination

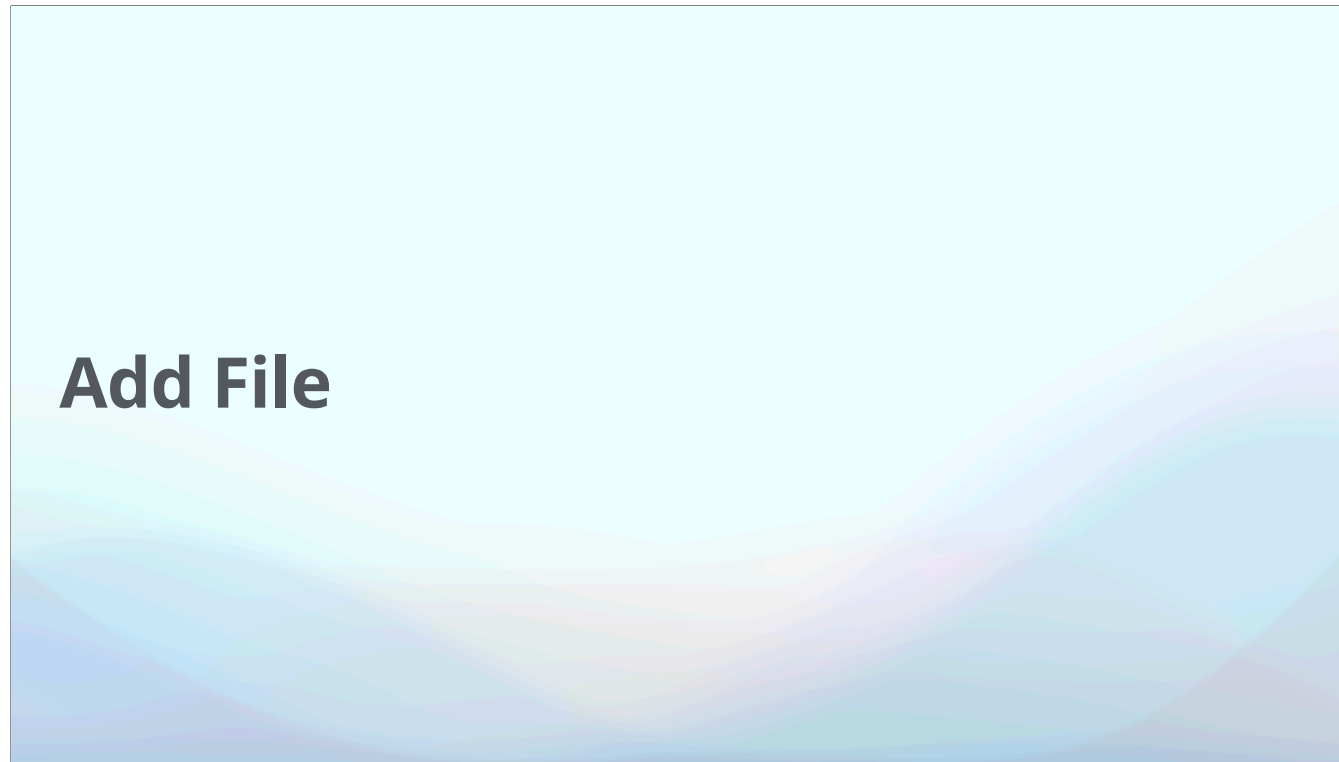




And there's our updated README with the nicely formatted information via Markdown.







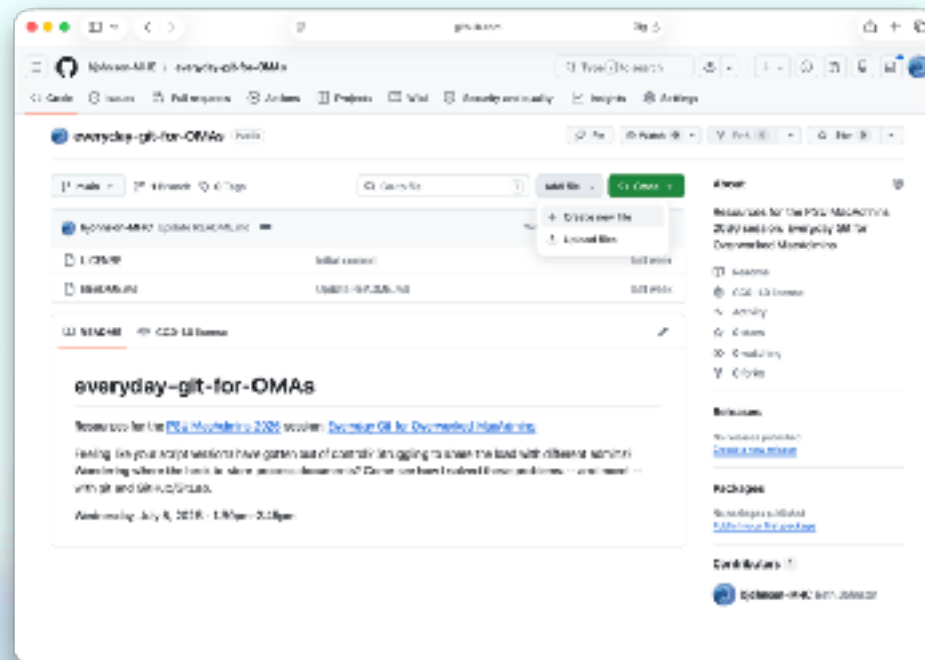
We want to do more than the README, and we can add another file via the webUI.

- Select the Add file option here. There's Create new file, for something text-based like another Markdown file or a shell script.

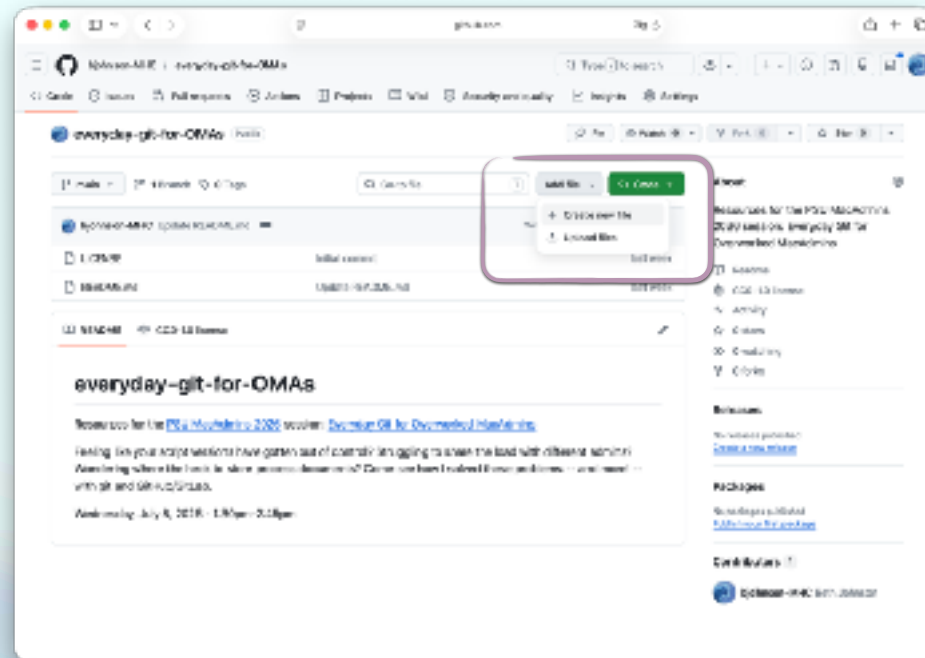
There's also "upload files." Let's pick that.



Add File



# Add File





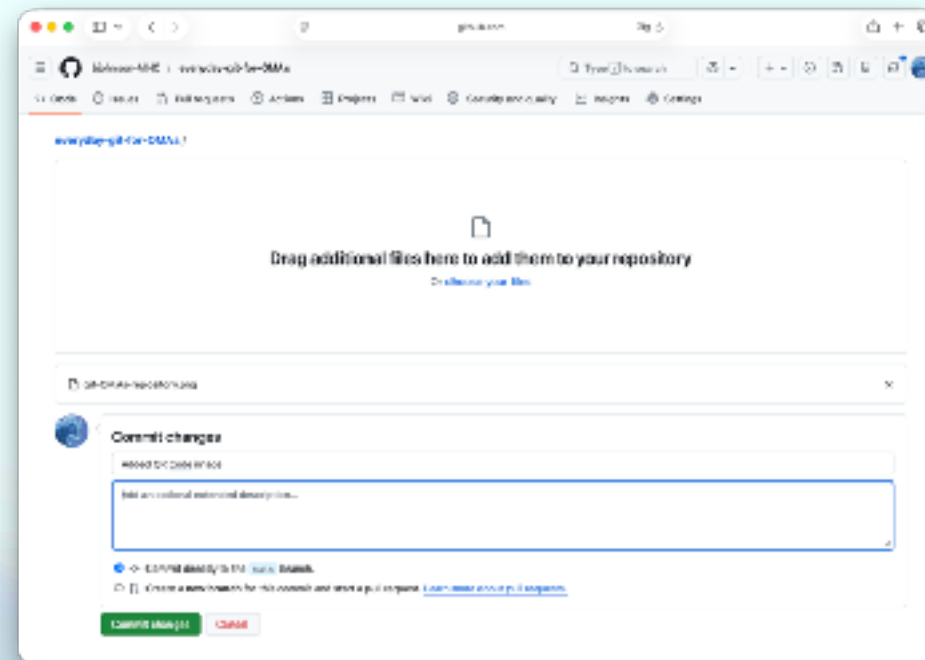
**Upload  
File**

I've chosen my

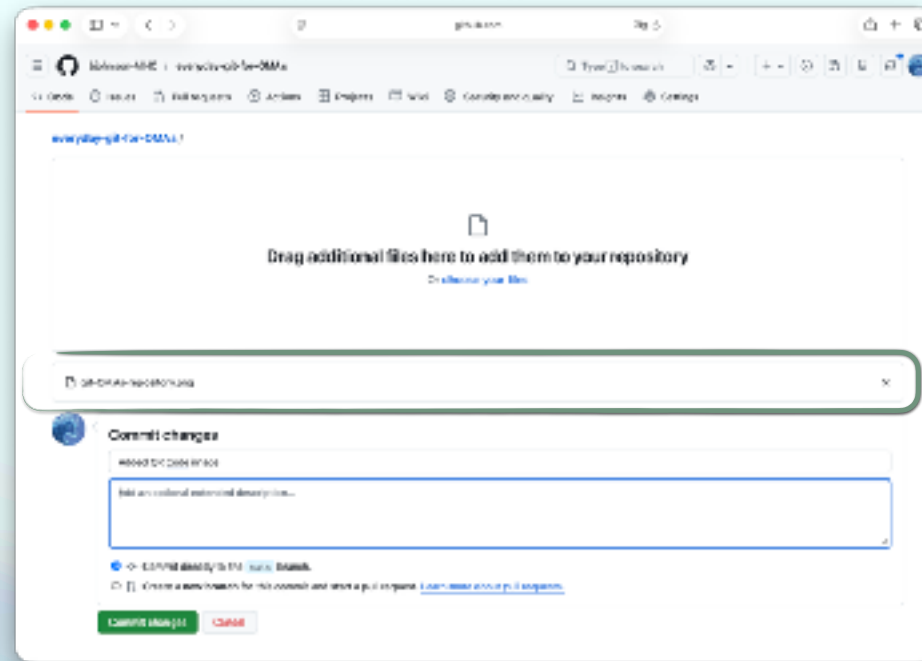
- QR code image. Git can't track changes within an image document -- only text. But it can host this image file and track whether it exists or not.
- And Oh, look! It's another commit.

We're starting to learn the routine: we make a significant change, then commit.

# Upload File



# Upload File



The screenshot shows the GitHub web interface for a repository named "my-pkg-123456789". The main area displays a message: "Drag additional files here to add them to your repository" with a link to "Choose your files". Below this is a "Commit changes" section. It includes a text input field for the commit message, which currently contains "Added my code file". Below the input field are two radio buttons: "Create a new branch for this commit and start a pull request" (selected) and "Commit to master" (unselected). At the bottom of the form are two buttons: "Commit" (green) and "Cancel" (red).

my-pkg-123456789

Drag additional files here to add them to your repository  
[Choose your files](#)

my-pkg-123456789

Commit changes

Added my code file

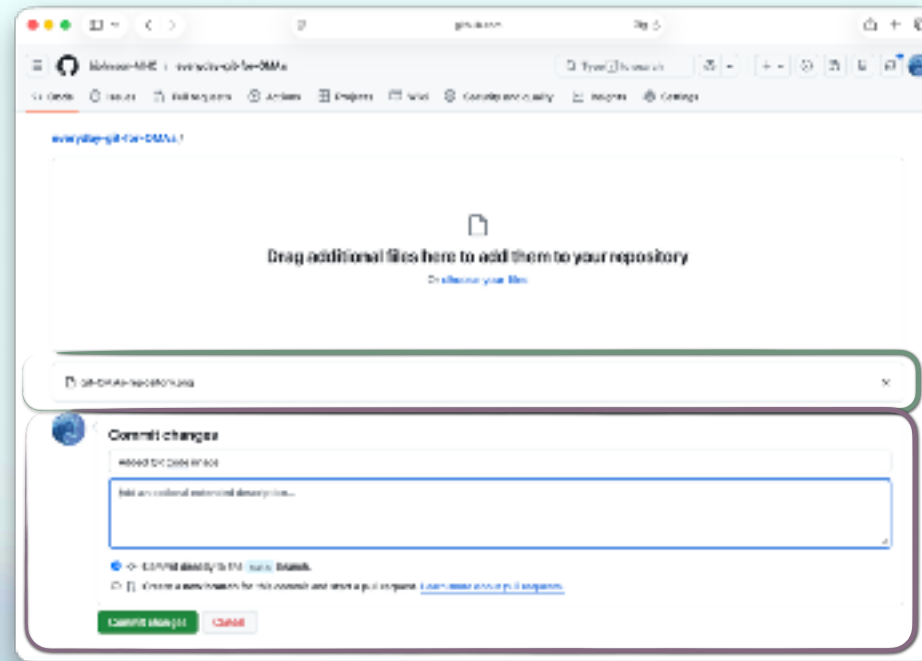
Write an optional extended description...

☒ Create a new branch for this commit and start a pull request [Learn more about pull requests](#)

☐ Commit to master

Commit Cancel

# Upload File

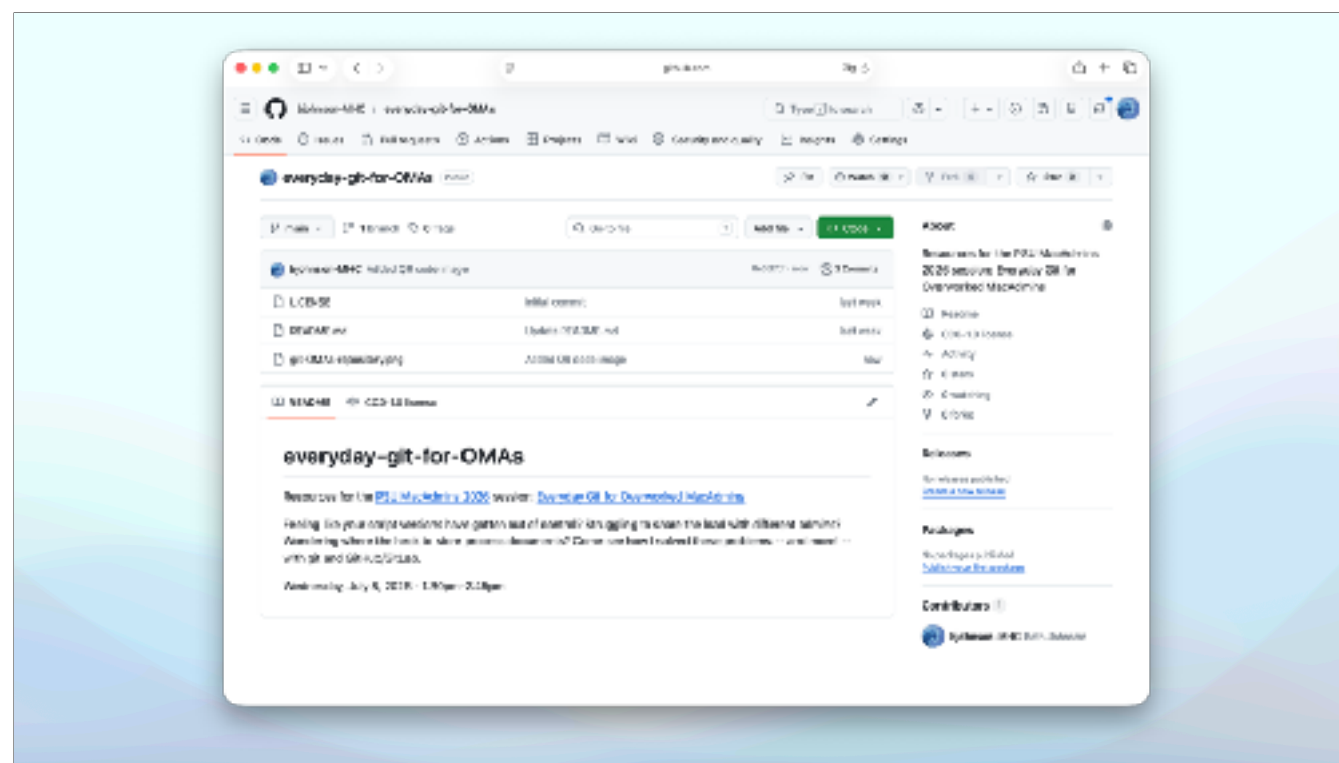




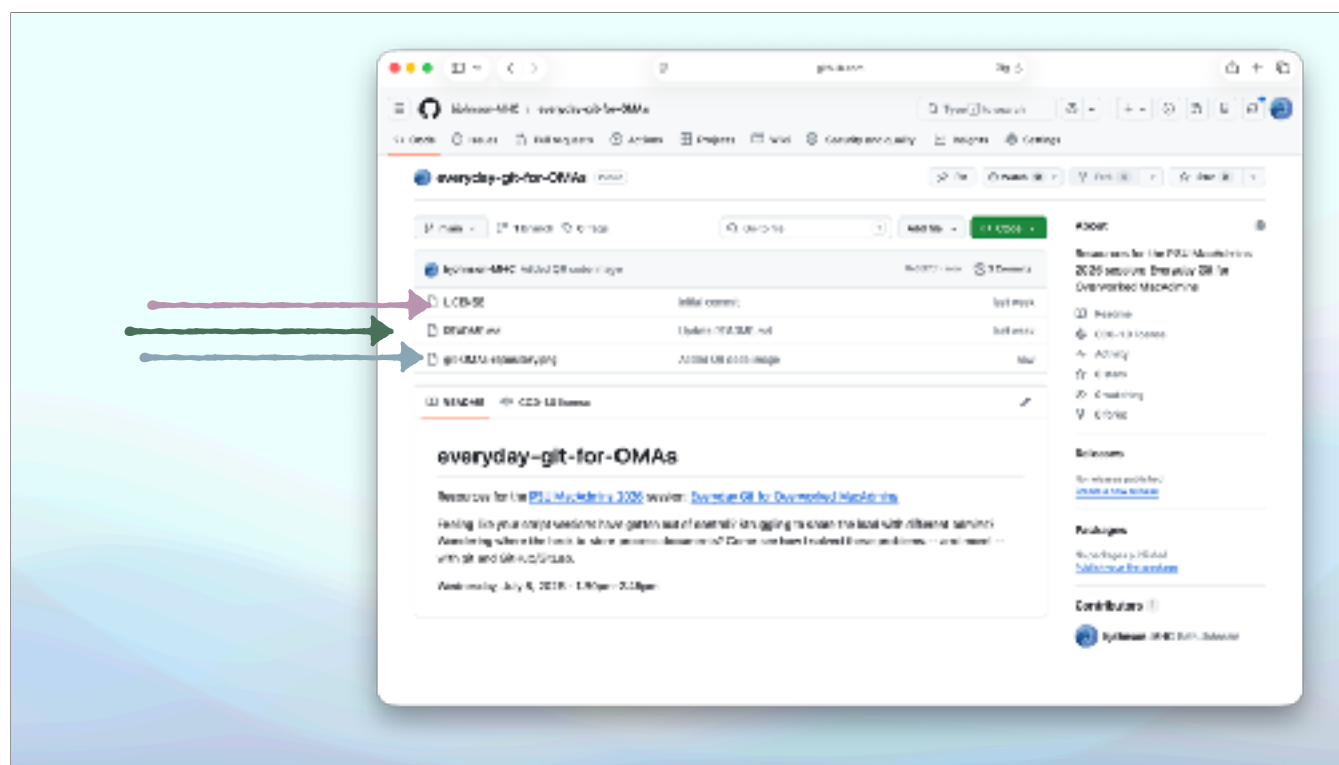
Once we've done the commit, now my repo

- has
  - a license,
  - a README,
  - and a file.











# Git Clones & Commits

**Real work**

Okay we've got a repository built. My next goal is to add files to it, and by files I mean all of my scripts from production. That would satisfy one of my goals of making a distributed backup of my management scripts.

It's all well and good to upload files through the webUI, but honestly that's not much different than using Dropbox or Google Drive. And those things are really not great for storing scripts.

Plus I also want to get the repository on my computer, in order to use a Git-enabled code editor for useful features like context-based formatting, advanced find and replace, and more.

# Git Clones & Commits

Real work



Tarsila do Amaral, Workers, 1933

# Tools

## Real talk

Friends, I have not found the command line to have the . . . ease of use that I'm looking for, for all those actions.

- So I use GitHub's desktop client. It's free, it works with GitHub natively -- and with our GitLab instance and probably other hosts as well if configured correctly. If you want to explore other tools, great! But this choice saves me time and effort.

# Tools

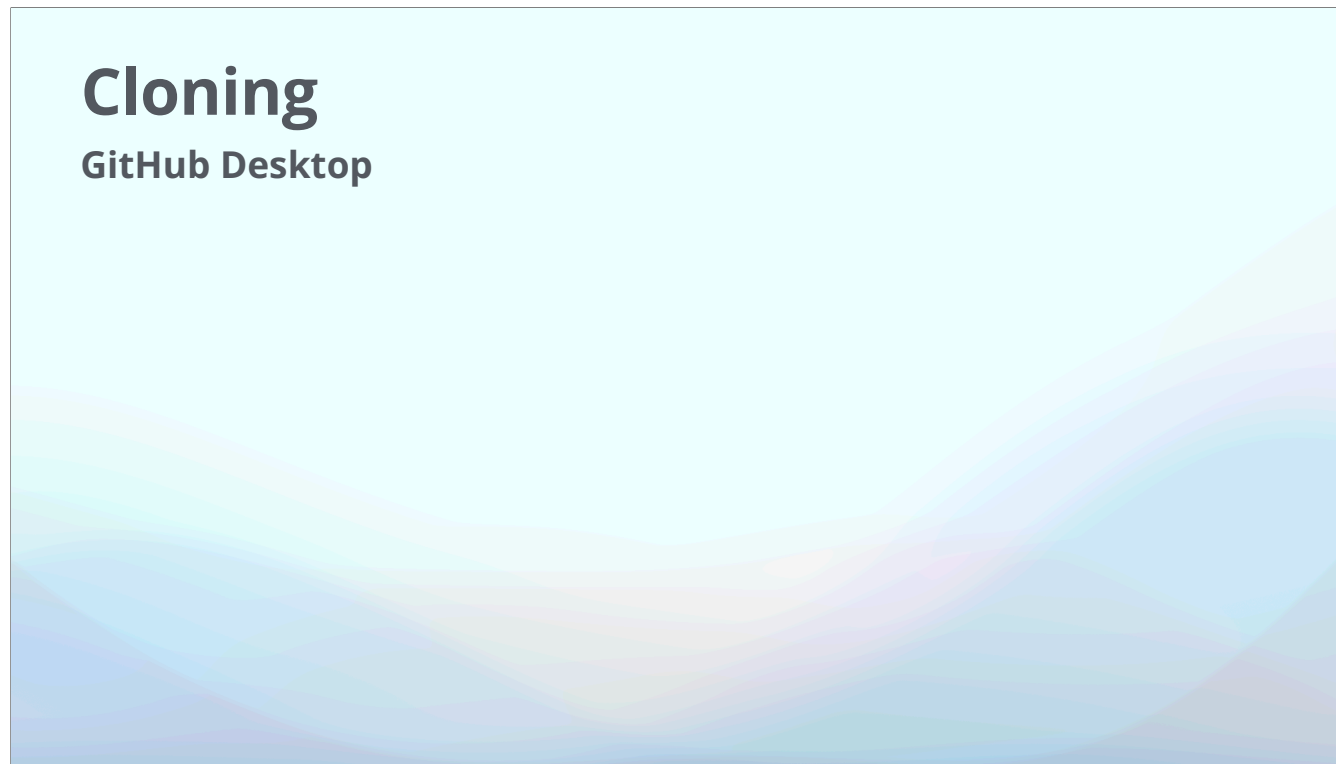
Real talk



# Tools

Real talk



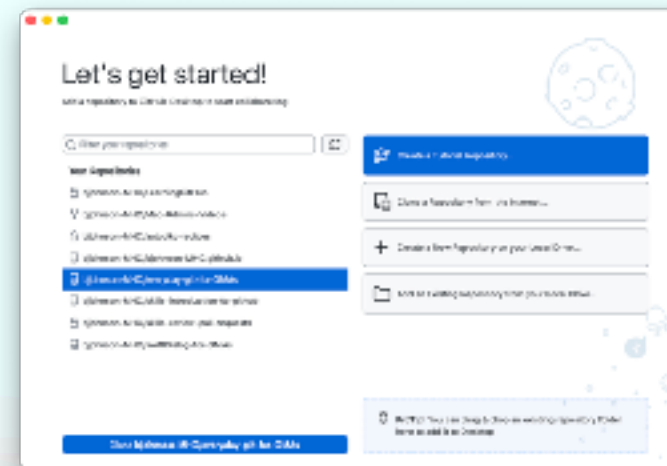


If you sign in to GitHub during setup,

- GitHub Desktop will list your repositories,
  - or you can select them later from the repository menu.
- Otherwise, you can add paste in a URL.
- To find the URL, go back to the GitHub website for the repo, select the prominent Code button, and under HTTPS you see the URL that you can plug in to GitHub Desktop,
- or simply use the Open with GitHub Desktop option.
- Either way, you'll be prompted for the local directory to save the repo into. GitHub Desktop will create a directory with the same name within that directory and store the repo there.



# Cloning GitHub Desktop

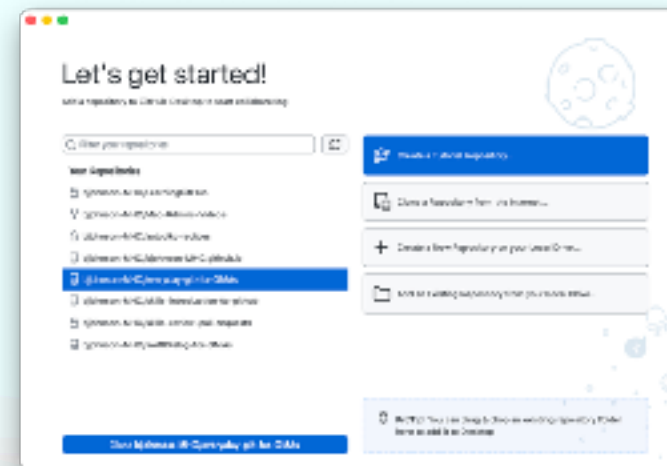




# Cloning

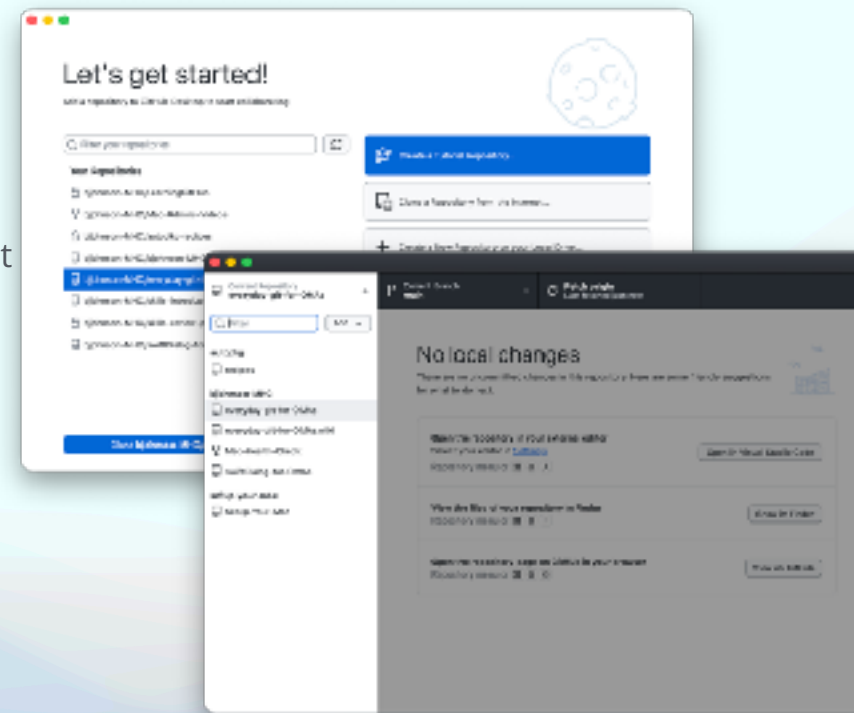
## GitHub Desktop

- Sign in via app & select



# Cloning GitHub Desktop

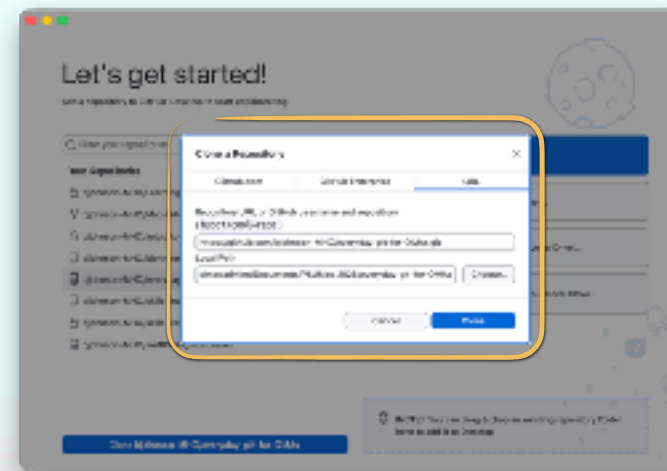
- Sign in via app & select



# Cloning

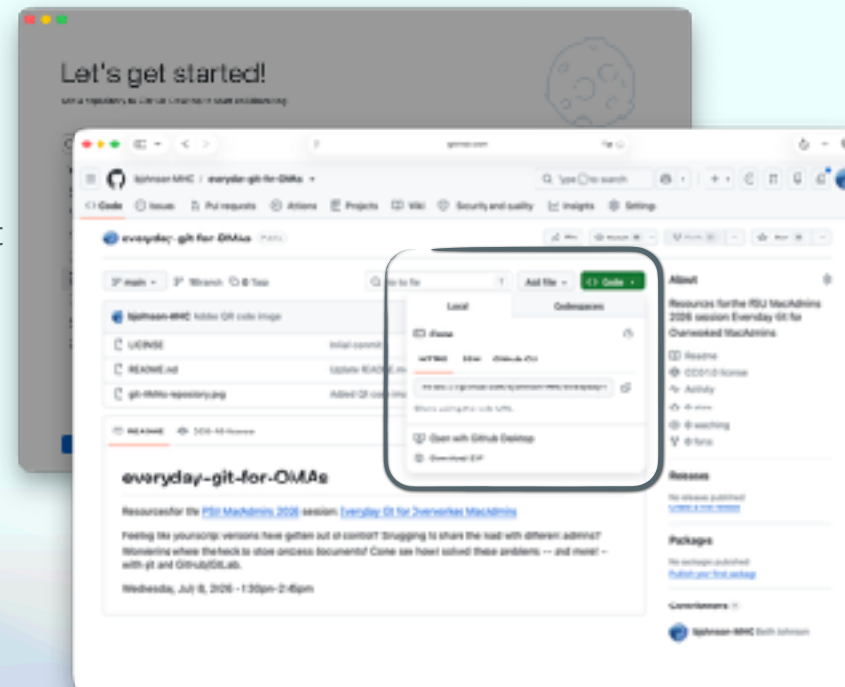
## GitHub Desktop

- Sign in via app & select
- Add via URL



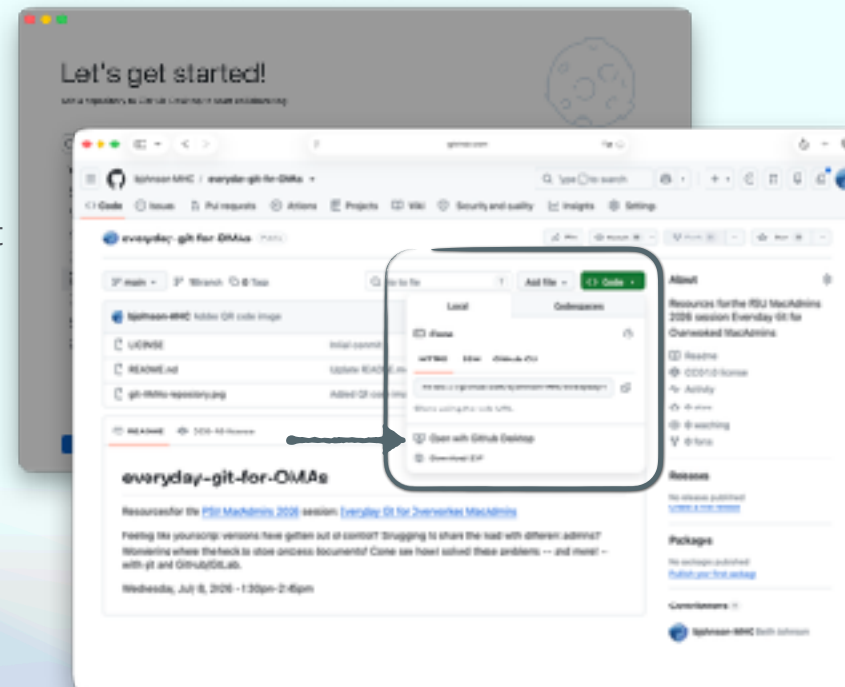
# Cloning GitHub Desktop

- Sign in via app & select
- Add via URL
- Get URL from web



# Cloning GitHub Desktop

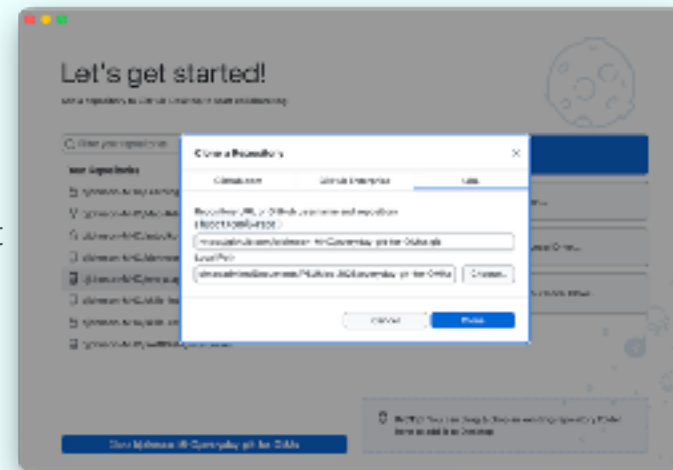
- Sign in via app & select
- Add via URL
  - Get URL from web
  - Open with Desktop



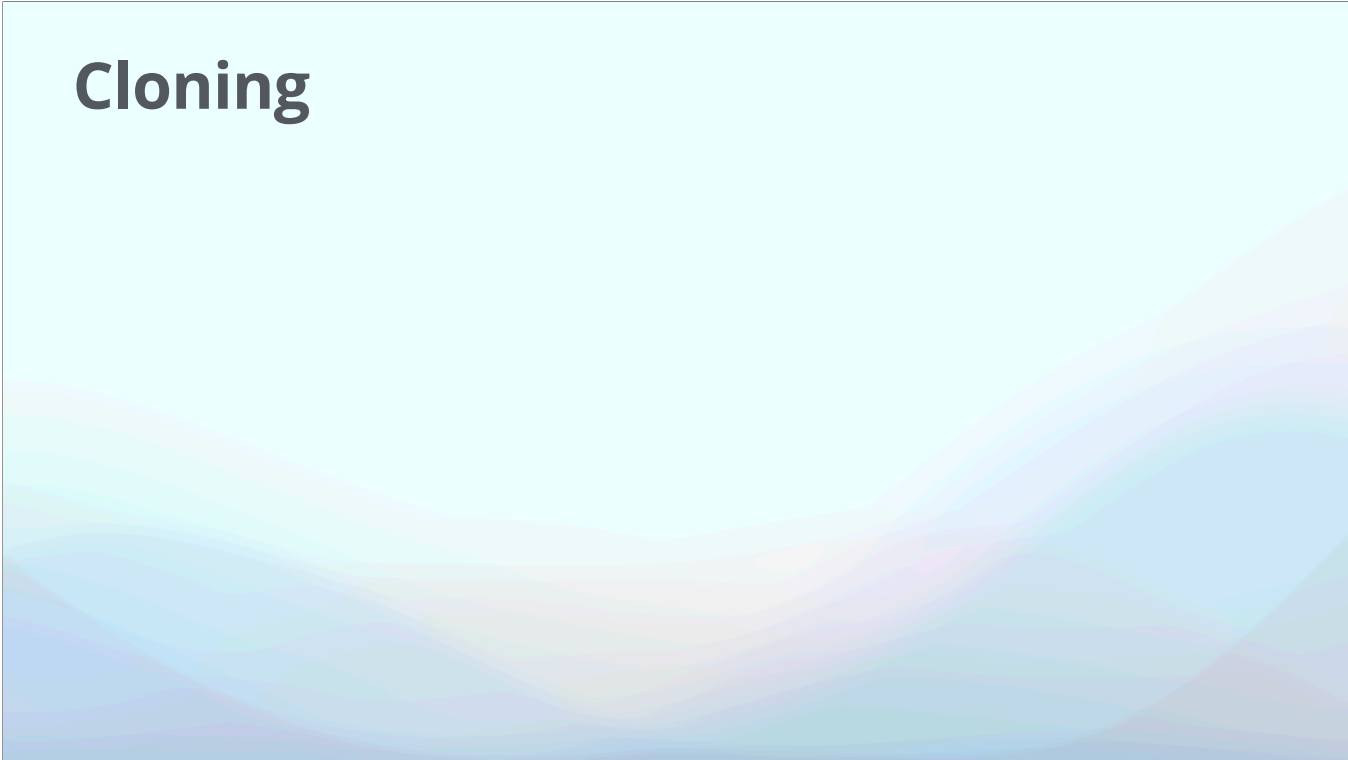
# Cloning

## GitHub Desktop

- Sign in via app & select
- Add via URL
  - Get URL from web
  - Open with Desktop
- Select your local directory



# Cloning



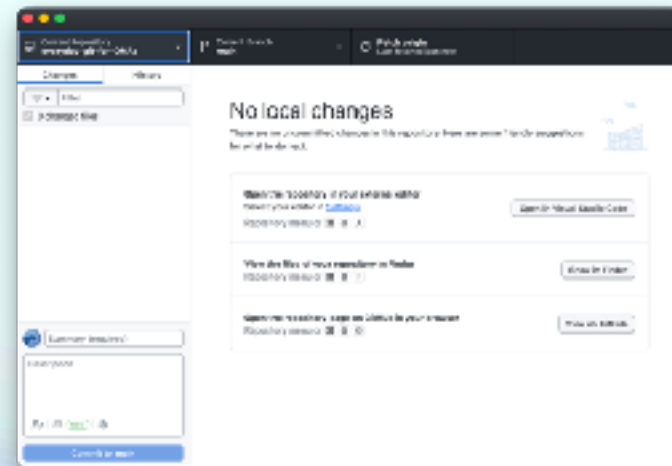
Fabulous! Now I have a complete copy of my repository on my local computer.

- As expected, there are no changes between my local version and origin.
- Which we can see if we check it out in Finder
- And if we use terminal to list the files, we can see the
- hidden git directory (as well as that pesky Directory Services Store!)
- And diving into that .git directory, we can see all the supporting structure for our repository,
- including the per-repository config.

So, let's make some changes!

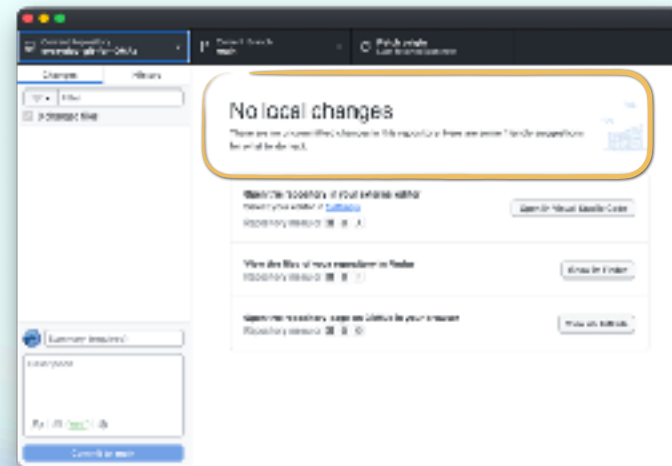


# Cloning

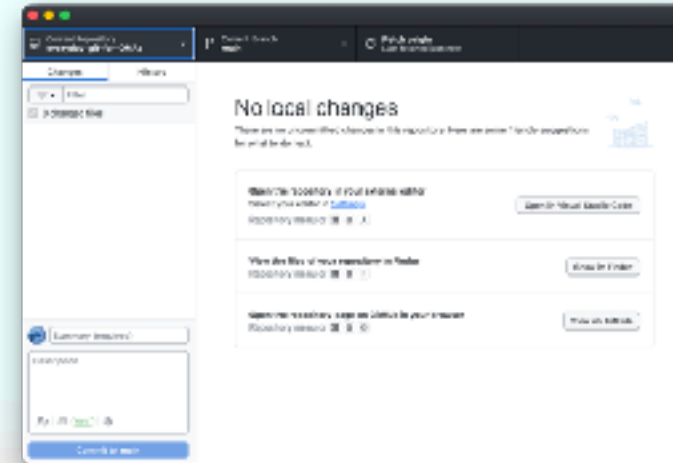


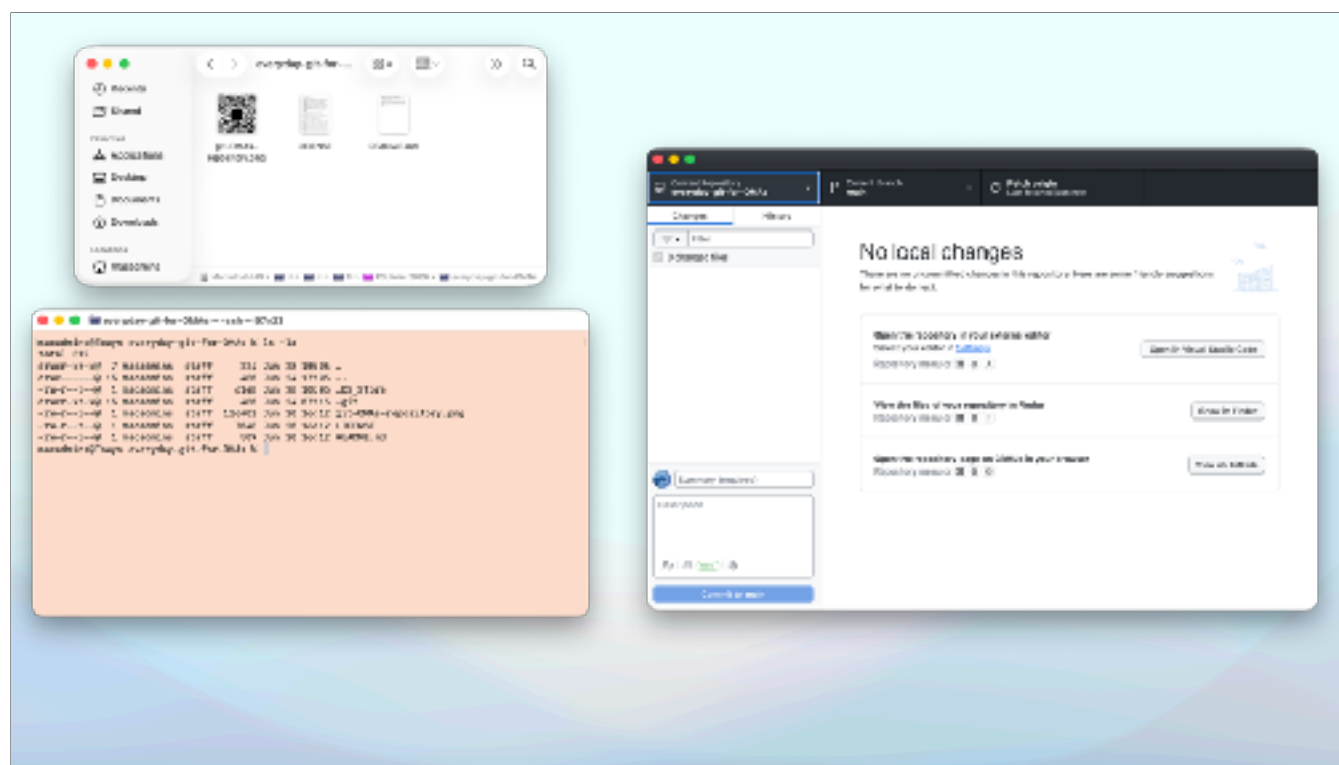


# Cloning

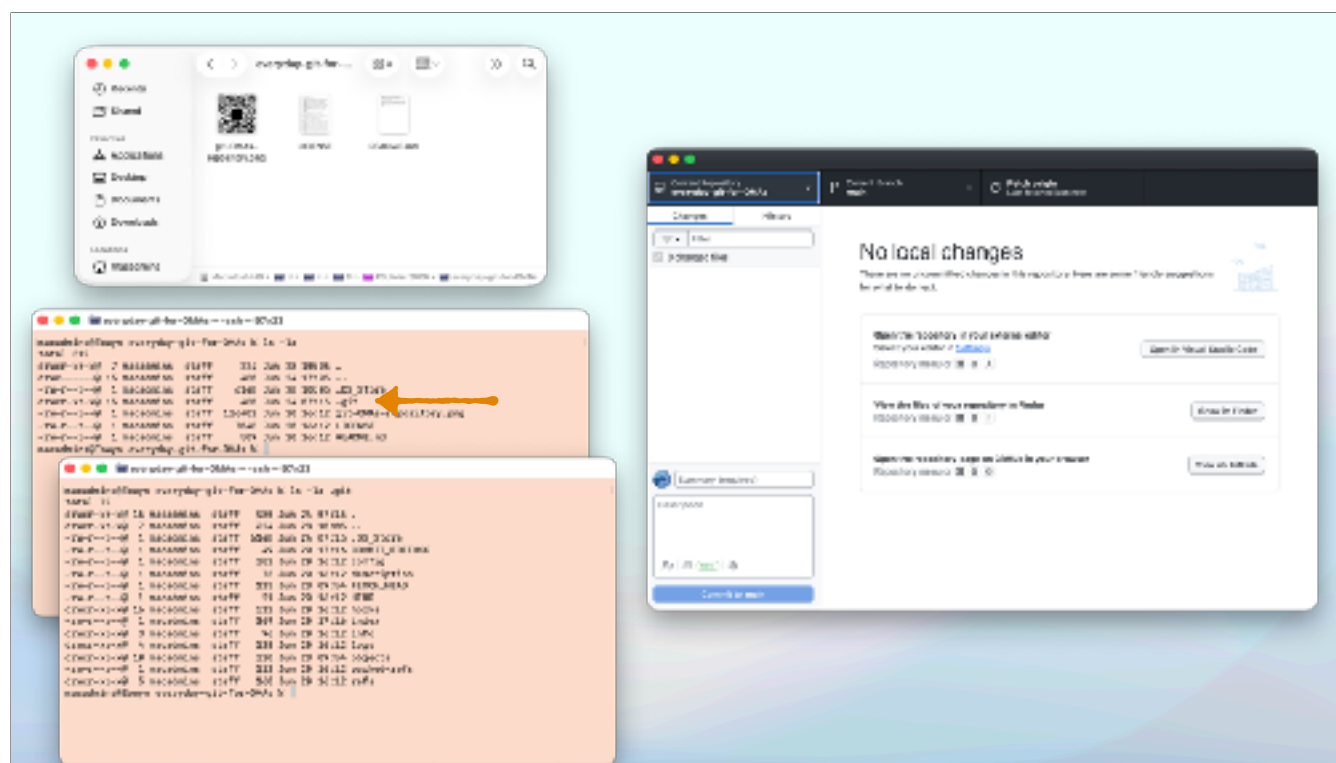


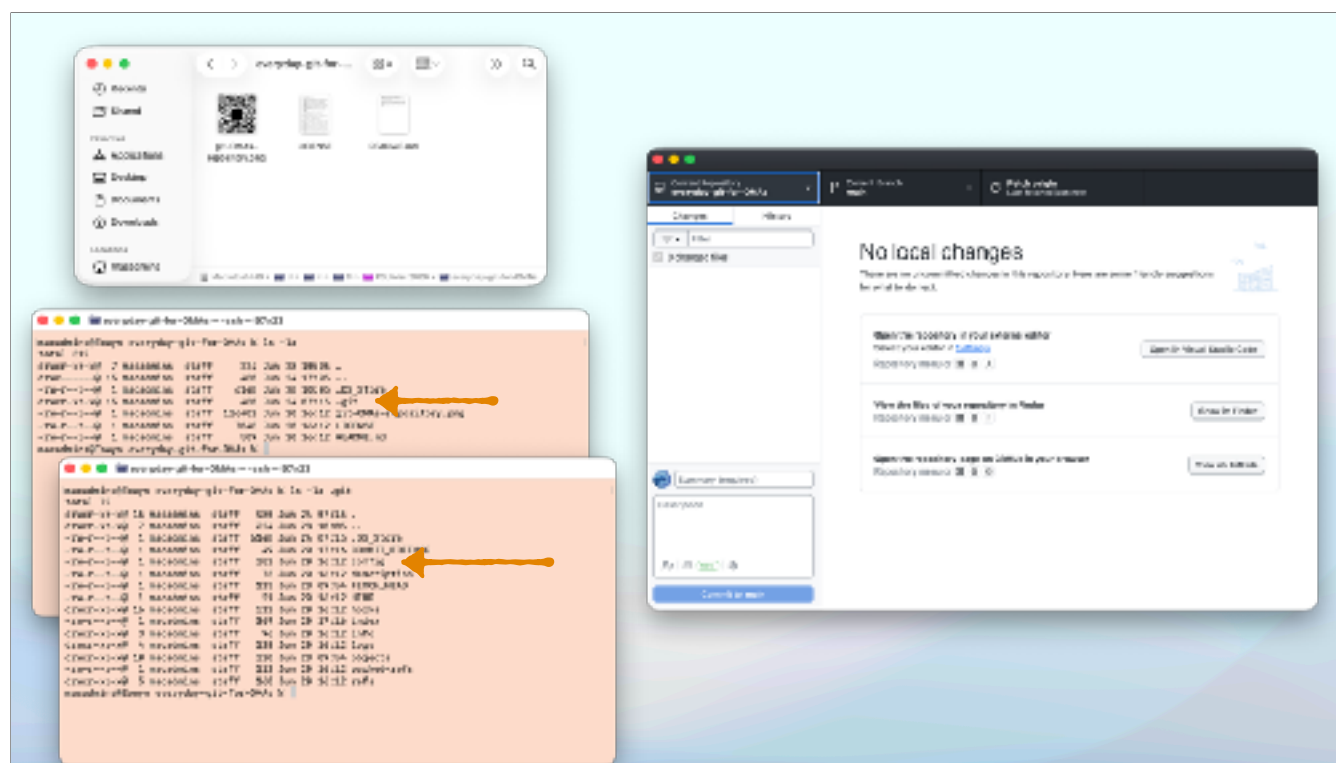
# Cloning





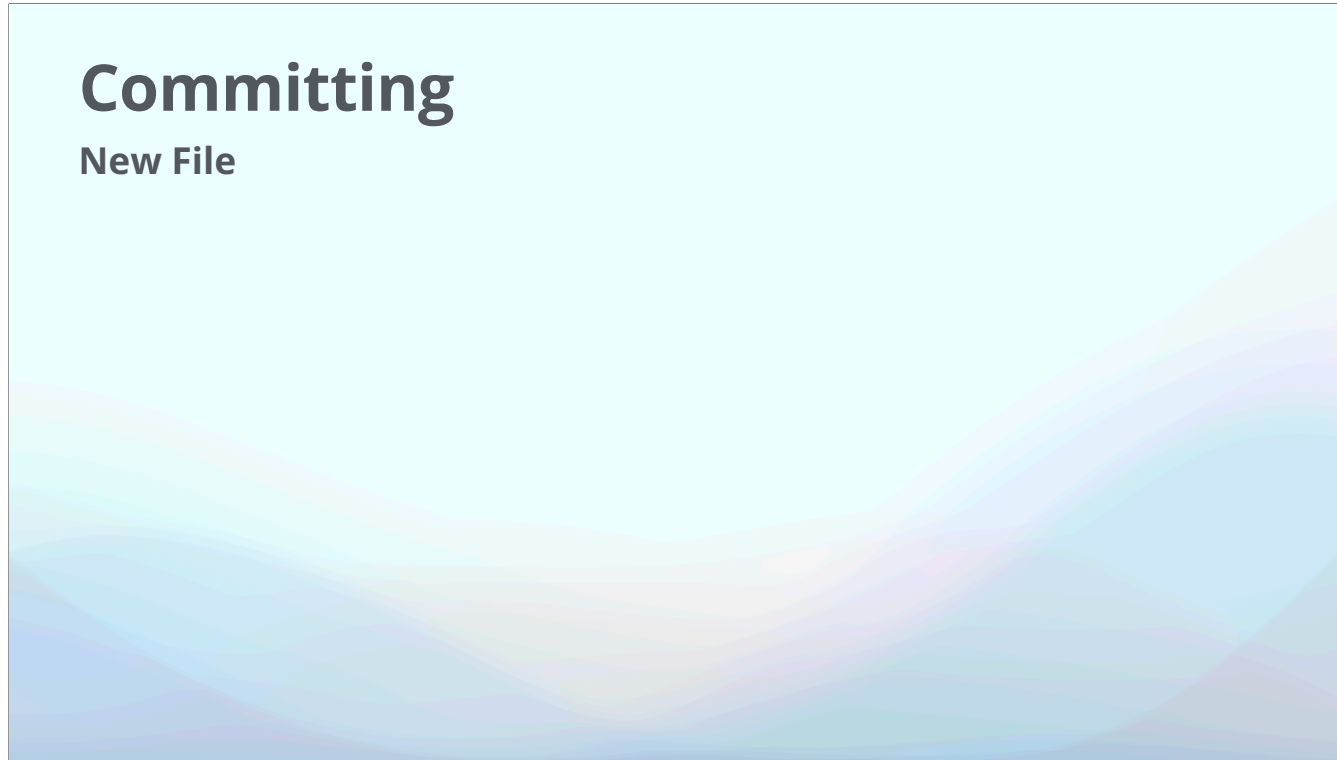






# Committing

## New File



I want a to-do list for this presentation's repository.

- So I'll make one within the repo's directory. It's just a plain text file and a few lines of Markdown saved as to-do.md. It doesn't matter how I get it there, just that it ends up in the directory.
- Git didn't recognize the change until I saved the file, then suddenly we have a list of the files that changed (one) and what changed in that file.
- Notice that summary for the commit notes the added file. I'd just need to add my detailed description. At this point we're staged, but not committed.
  - But you don't have to commit for each change. Just each change that you want to count. So let me make some more changes first.
- After I save, I can check the staged changes -- I just need to add a description.

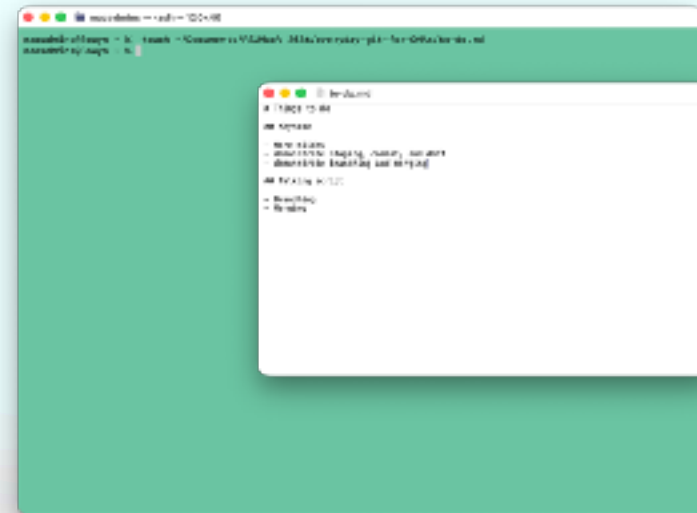
With the information updated, I'm ready to make a commit . . .



# Committing

## New File

- Create new .md file

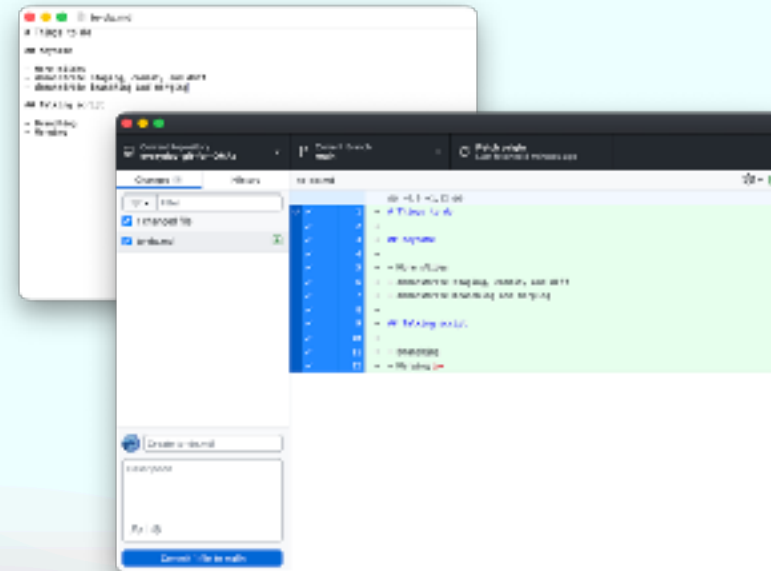




# Committing

## New File

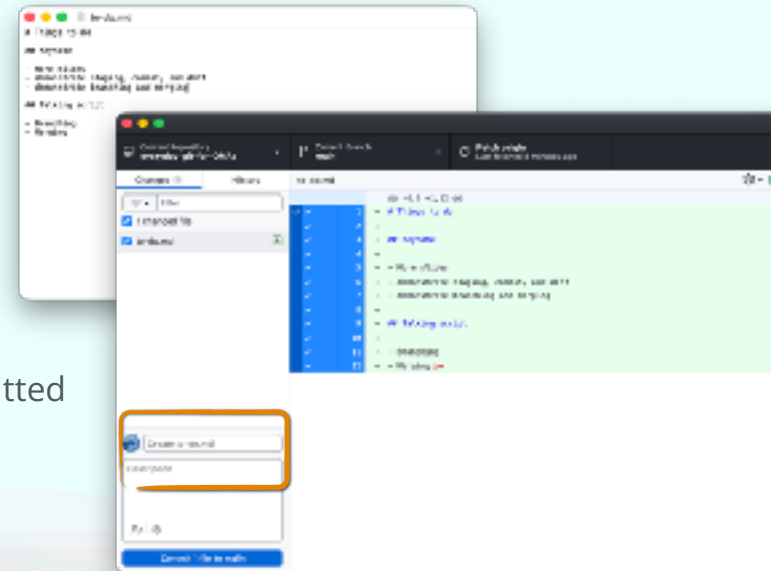
- Create new .md file
- Edit file and save



# Committing

## New File

- Create new .md file
- Edit file and save
- New is staged, not committed



## Committing New File

- Create new `.md` file
- Edit file and save
- New is staged, not committed
- Commit changes that count

```

# Import the library
import pandas as pd

# Create the data
data = {
    'Year': 2010,
    'Country': 'USA',
    'Population': 308,
    'GDP': 14.7,
    'GDP_per_capita': 47,800
}

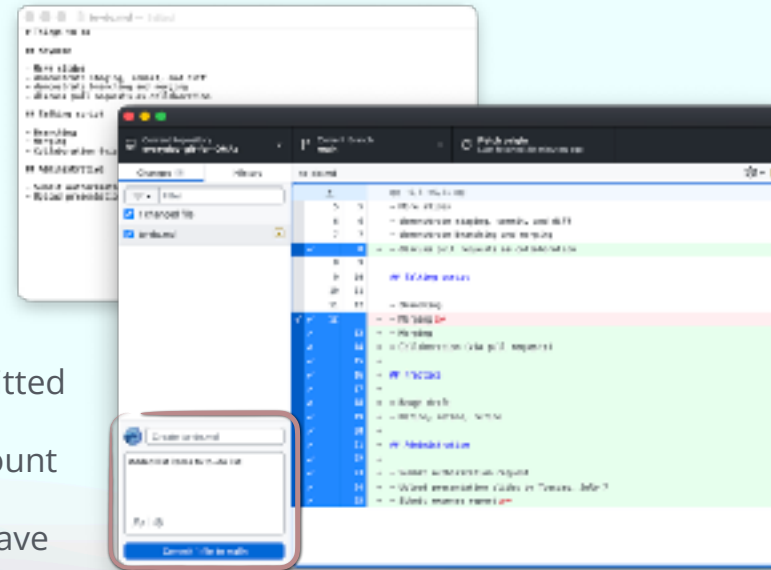
# Create the DataFrame
df = pd.DataFrame(data)

# Print the DataFrame
print(df)

```

## Committing New File

- Create new .md file
- Edit file and save
- New is staged, not committed
  - Commit changes that count
- New changes shown on save



# Real Work

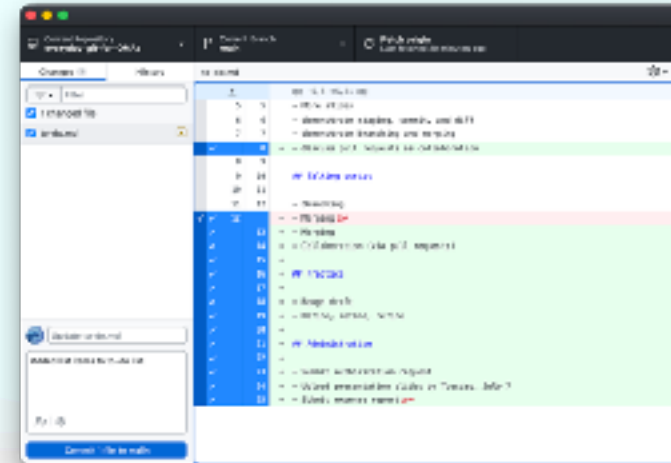
Send changes

V

- The commit is going to function the same as on the web.
- I'm going to take the suggested summary since my change is extremely straightforward.
- I'm going to add a description that says a little more about what I added.
- We do the commit, and the changes are filed.
- Which means that there are no local changes anymore--that is, there's nothing being tracked that isn't committed.
- These changes haven't been communicated to origin, though. For this, we need a push.
- Look at what happens to our
- information about origin here: it went from last fetched 20 (minutes ago) to Just now.

# Real Work

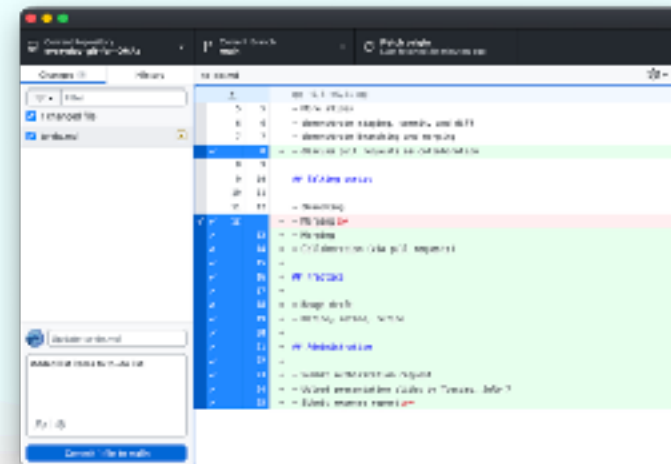
## Send changes



# Real Work

## Send changes

- Same commit process

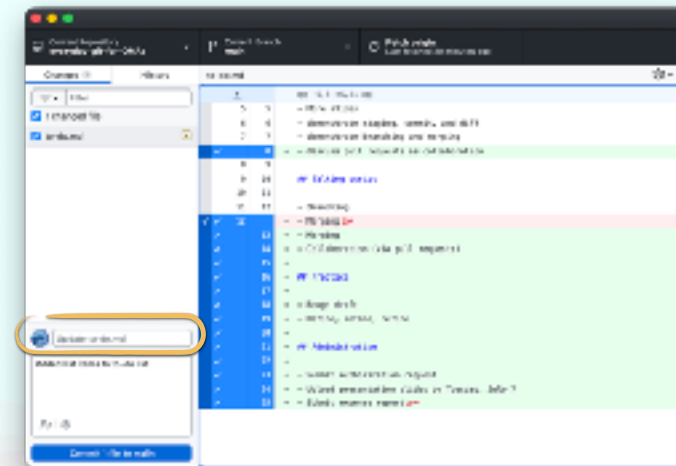




# Real Work

## Send changes

- Same commit process
- Summary

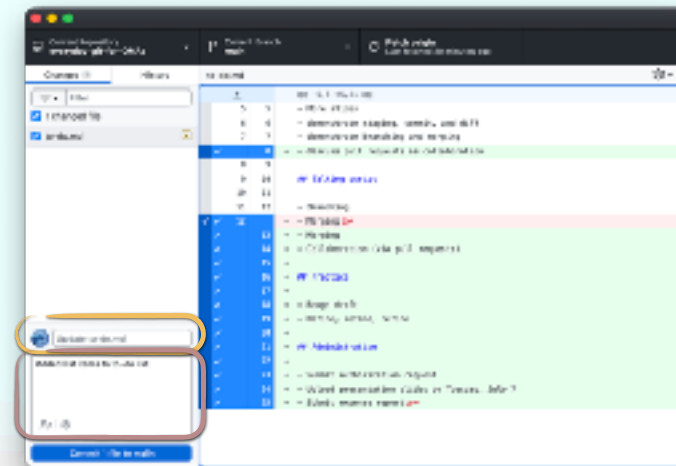




# Real Work

## Send changes

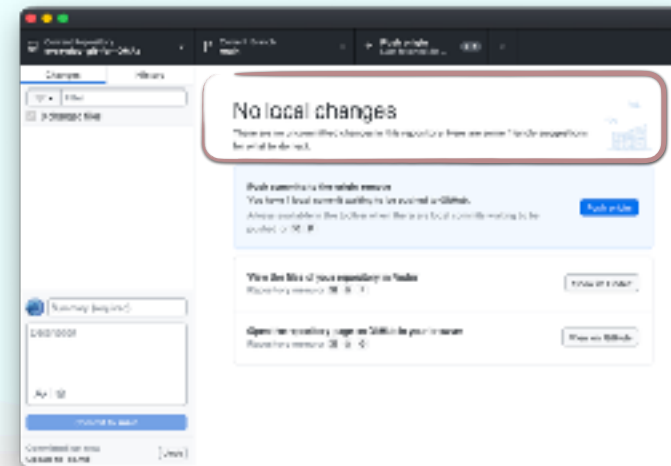
- Same commit process
- Summary
- Description



# Real Work

## Send changes

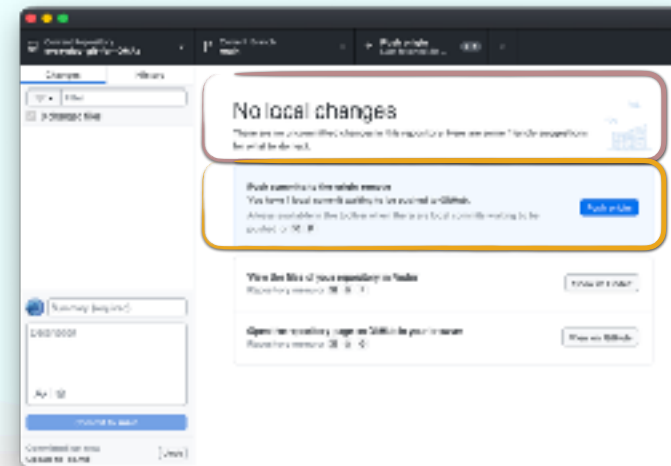
- Same commit process
- Summary
- Description
- Commit = changes are filed



# Real Work

## Send changes

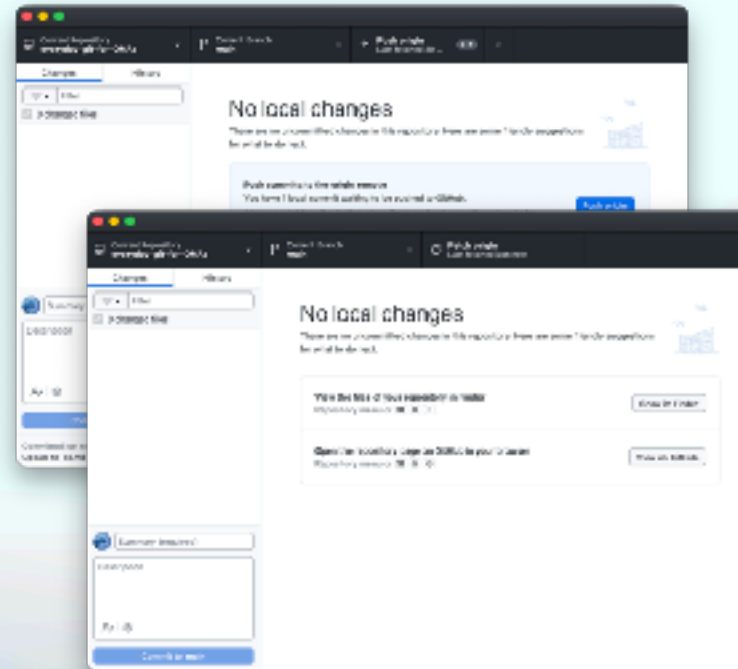
- Same commit process
  - Summary
  - Description
- Commit = changes are filed
- Push to origin



# Real Work

## Send changes

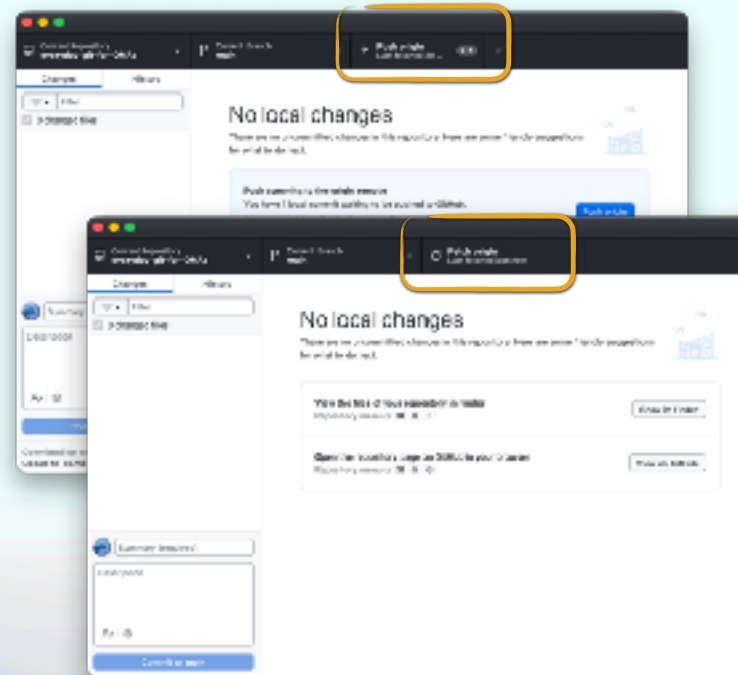
- Same commit process
  - Summary
  - Description
- Commit = changes are filed
- Push to origin
  - Note the fetch



# Real Work

## Send changes

- Same commit process
  - Summary
  - Description
- Commit = changes are filed
- Push to origin
  - Note the fetch



# Always. Fetch. First.

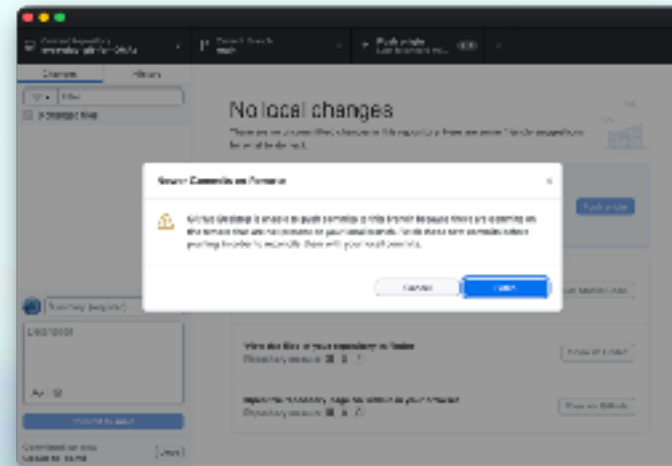
This is important, so it gets its own slide.

Always fetch before pushing. Even if you're the only person making commits.

The tool you're using may do this for you, which is great! And you should still be aware that it is doing that.

- GitHub Desktop is going to do a quick compare to see if there are any conflicts. If the same file had different changes, it will give you a conflict message like this.  
A tool may help you resolve this conflict, but you may end up having to manually copy the changes from origin and editing your local file, then making a new commit before you can push.

# Always. Fetch. First.



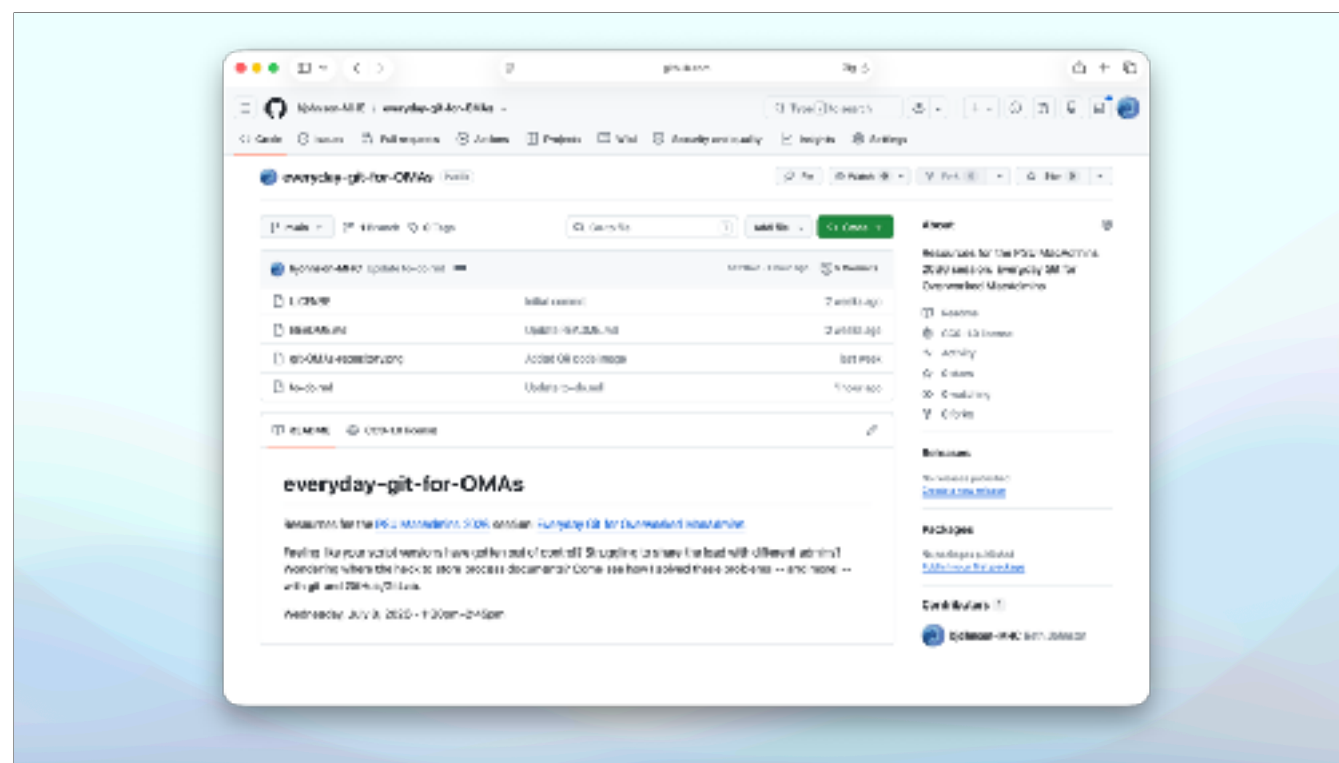


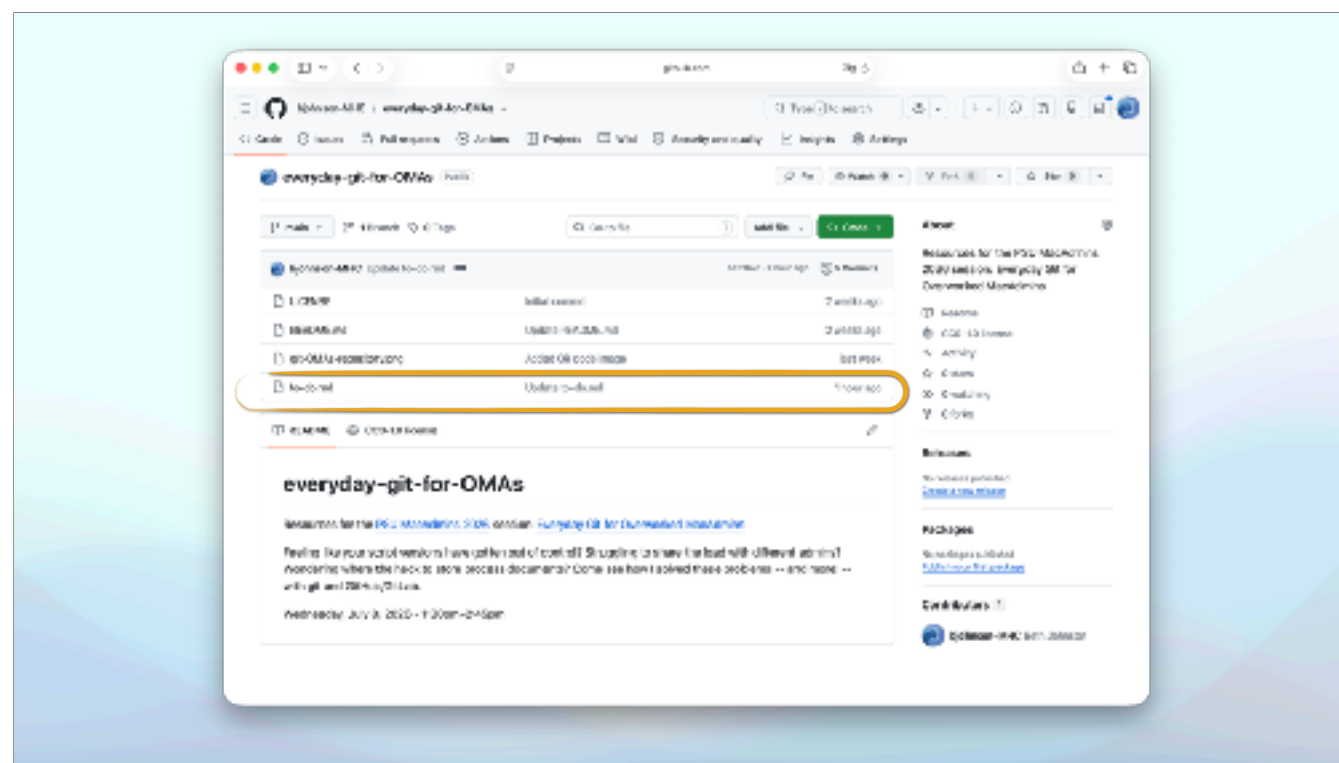


In our case, there are no conflicts, so the new file is pushed up.

- Here's what it looks like on the web, with our new file and our summary from the commit.

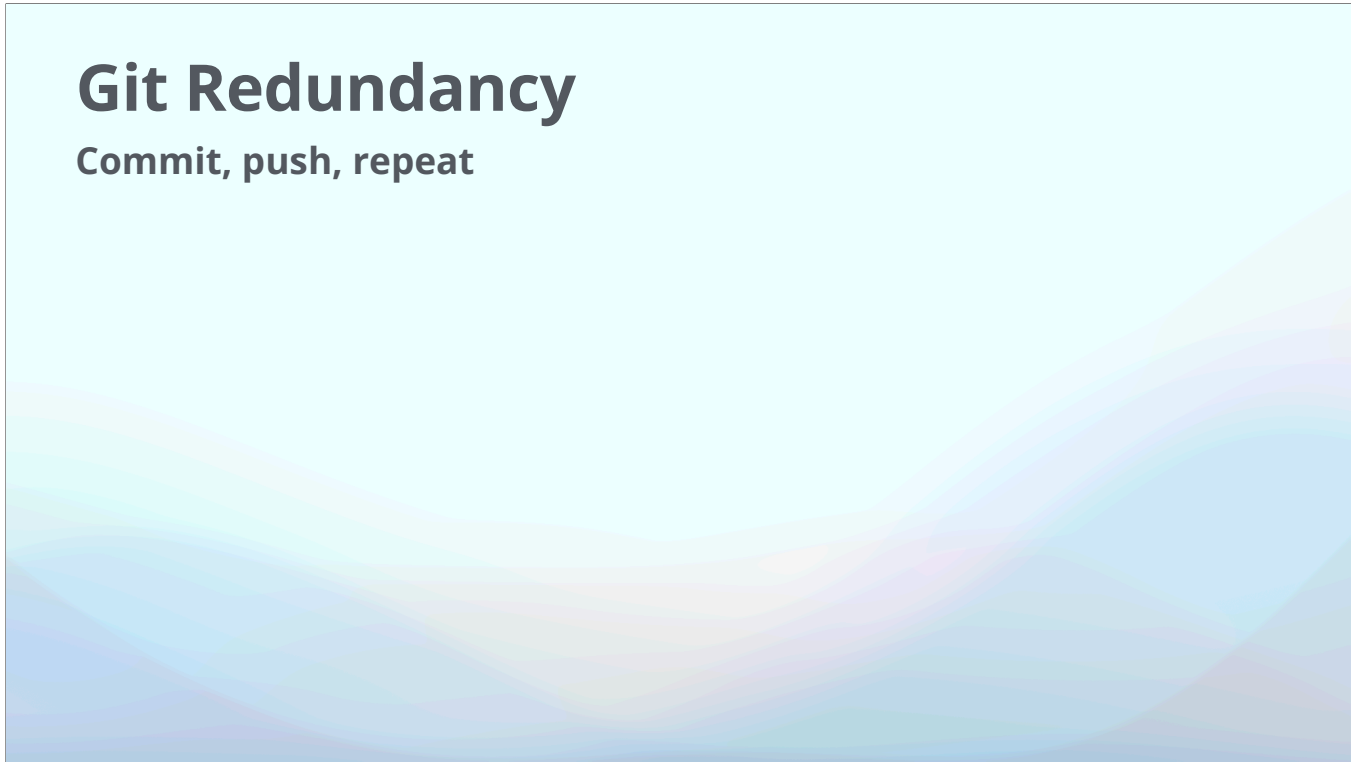






# Git Redundancy

Commit, push, repeat



So my first goal, making a working backup of my production scripts, was as easy as downloading them from Jamf, using a quick Automator action to be sure they all had appropriate file extensions, and

- dragging them to the repo folder on my computer, similar to this.
- Once there, Git picked them up and staged them for the commit.
- I completed the commit and push, and they were safely stored in my production repo on the server.
- With the cycle of change, commit, push, I was able to meet the goal of securing a redundant backup of my production scripts, with a workflow that's easy to manage.

# Git Redundancy

Commit, push, repeat

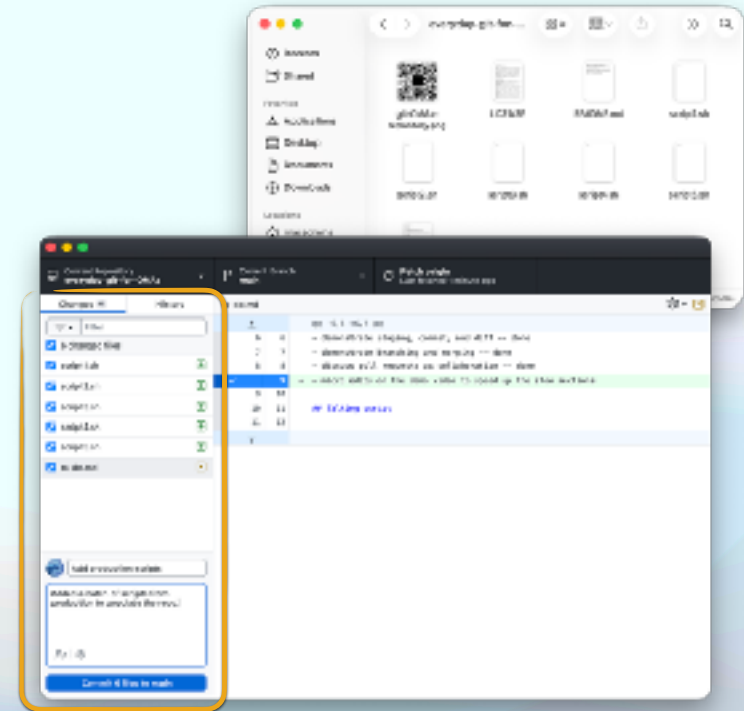
- Change or add files



# Git Redundancy

Commit, push, repeat

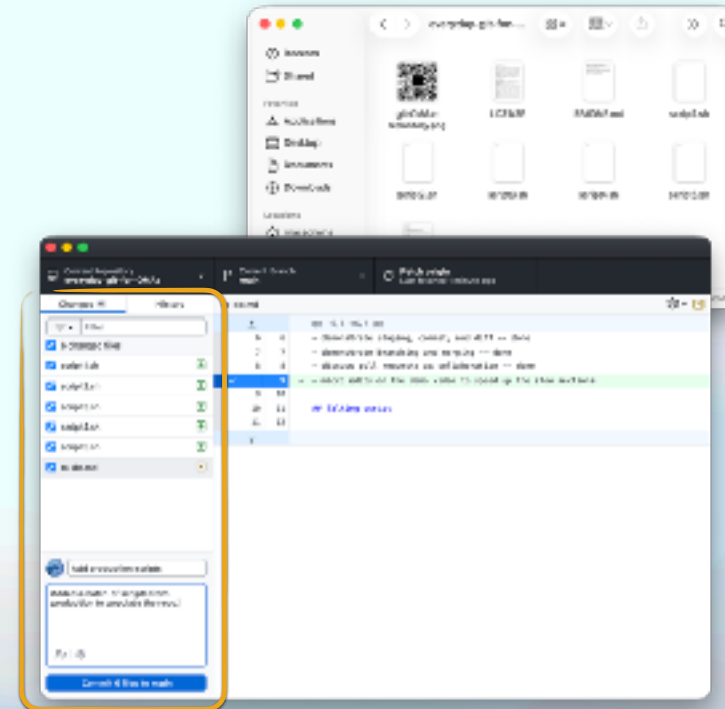
- Change or add files
- commit



# Git Redundancy

Commit, push, repeat

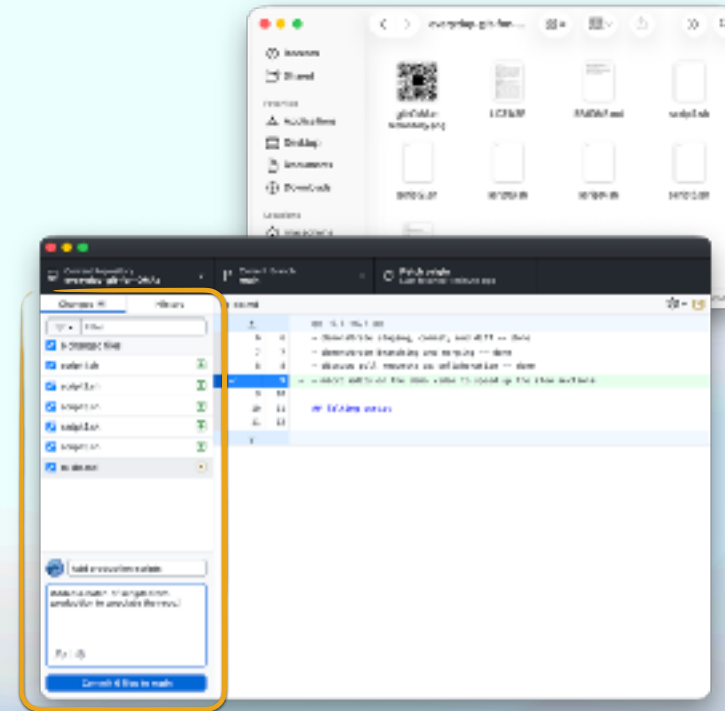
- Change or add files
- commit
- push



# Git Redundancy

Commit, push, repeat

- Change or add files
- commit
- push
- Repeat!





# Branching Out

That was great for populating the repo for production, but one of the compelling reasons to use Git is for separating work into branches, to segregate development work from production scripts. Which brings us to branching and merging.



# Branching Out



Louise Nevelson, Mirror Shadow VIII, 1985



## Working Repo

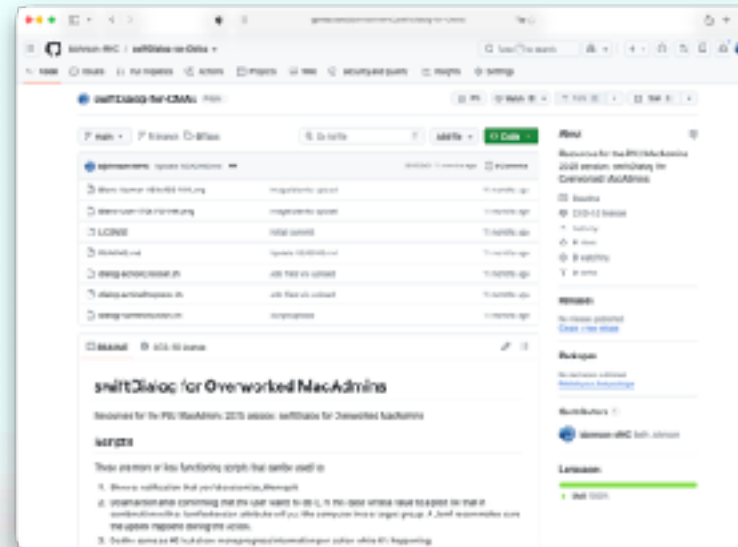
Okay, so the repo that I made for this session doesn't have a whole lot of material for demonstrating with, and I'm not going to demo with my production admin repository.

- But my repo from last year's session has more files and scripts, so I can use it to talk about branching and merging.

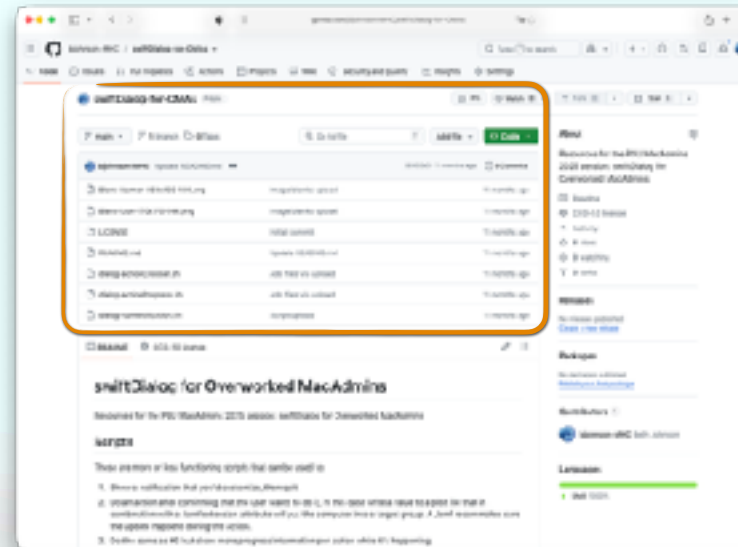
The other thing that I'm going to do is use a more robust tool for editing the script files.

- I used to use Atom for this, as I found it pretty intuitive while also being Git-aware,
- but sadly GitHub retired it.
- I've mostly switched to Visual Studio Code, which is way more than I need! But it's available and works well with GitHub for reasons that should be obvious.

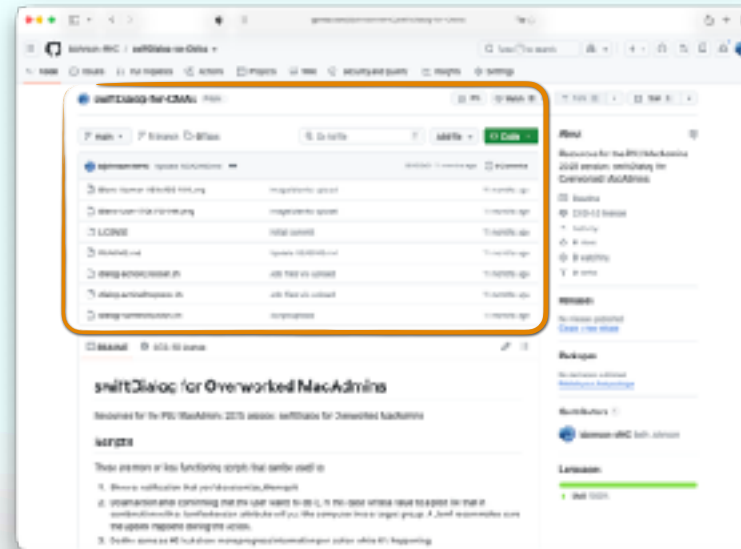
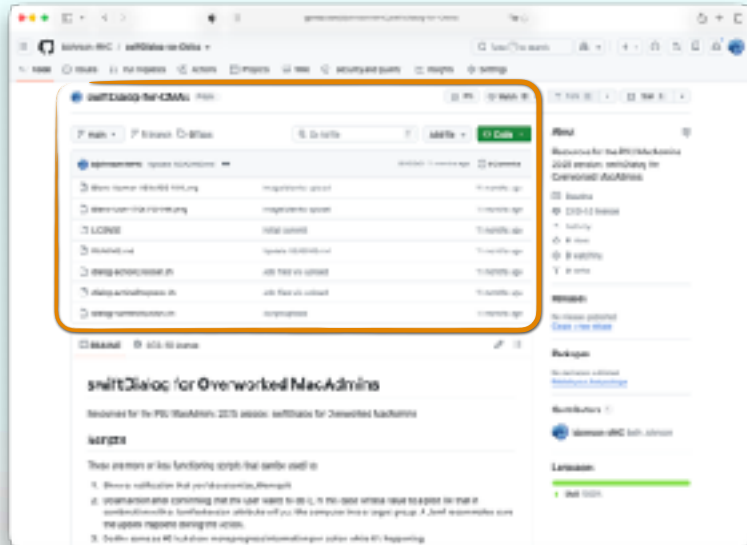
# Working Repo



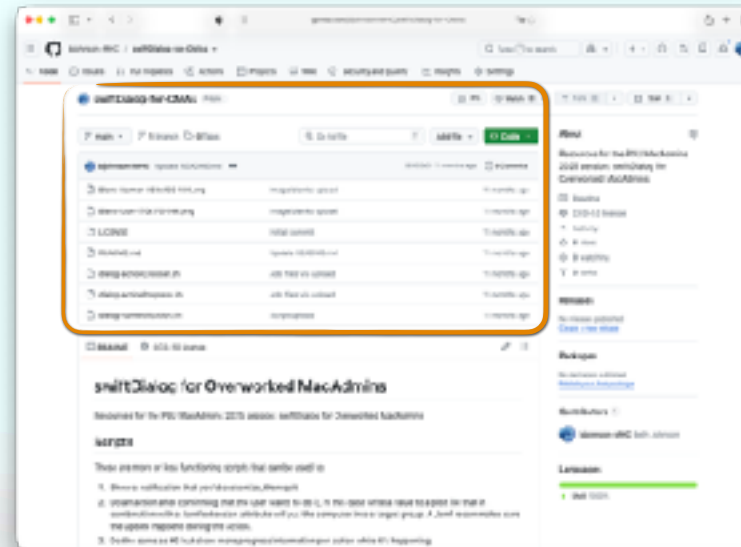
# Working Repo



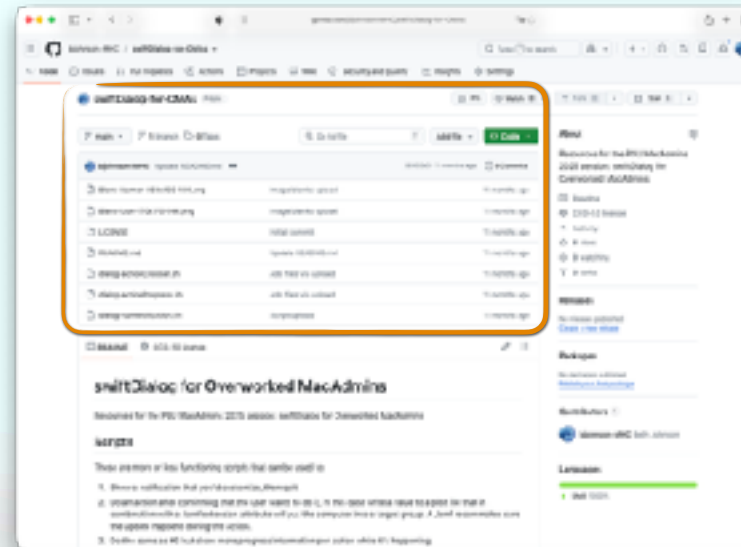
# Working Repo

# Working Repo



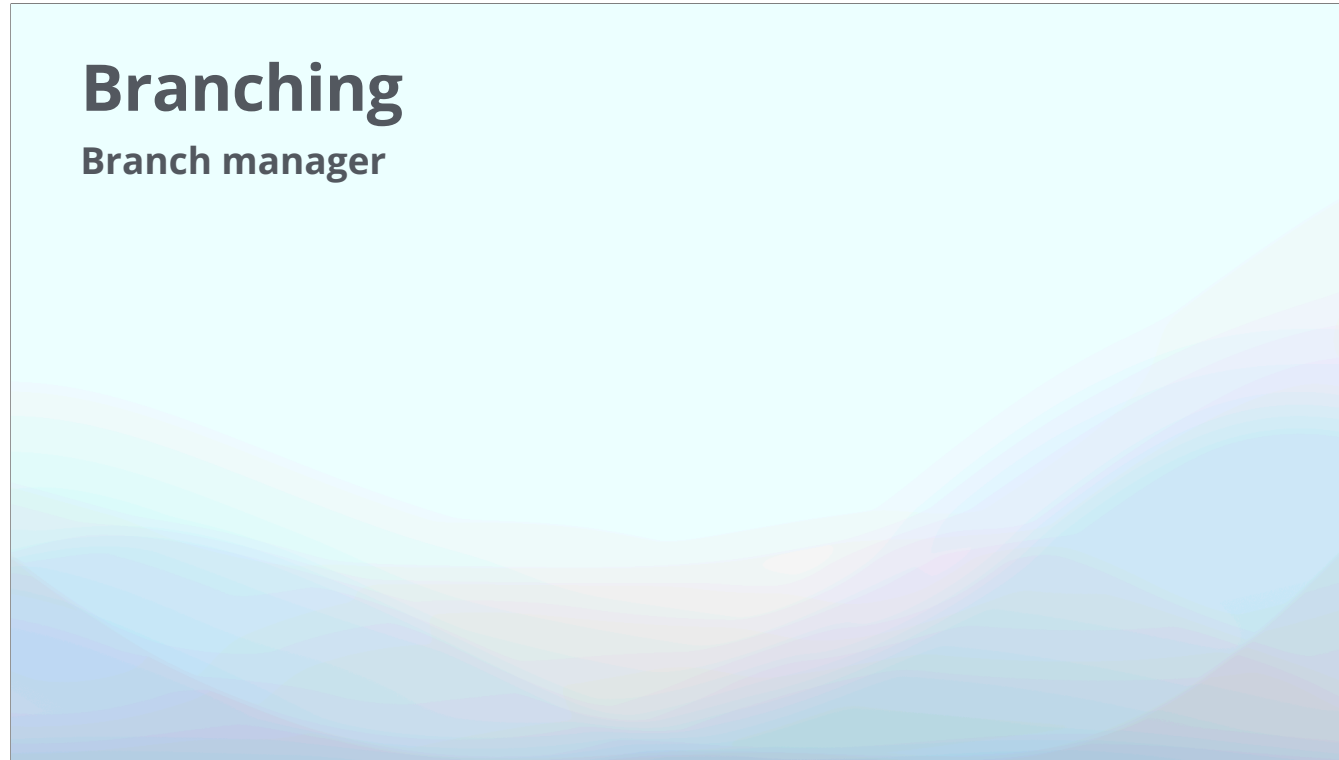
# Working Repo





# Branching

## Branch manager



In this case, I want to clone off a branch to make some changes, because I want to test these scripts with swiftDialog 3.0.

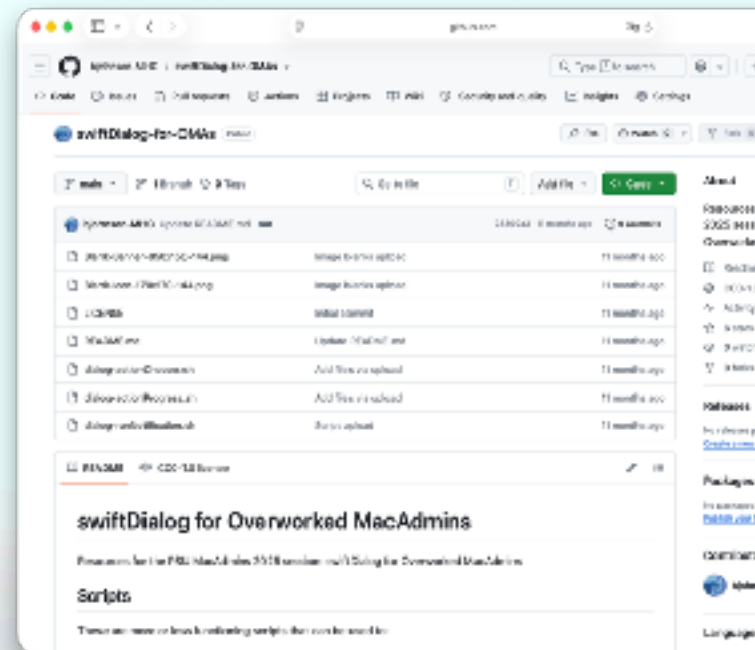
Let's take a look first at the repo on the web.

- Notice here, below the repo name? We've got a selector that right now shows "main." Next to that a branch symbol, and it says "1 branch."
- That's because "main" is also a branch. It's just the branch we started with--our default branch.
- There is nothing functionally special about main--it's just a label.



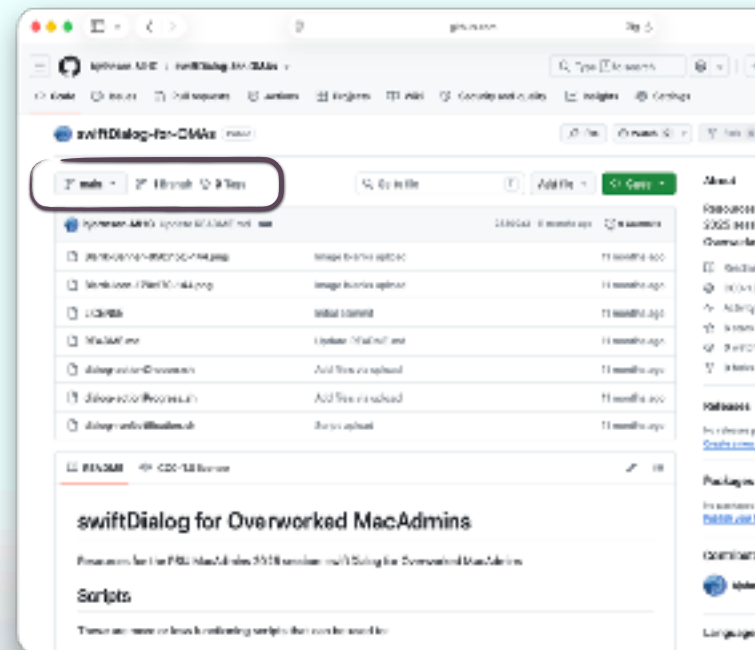
# Branching

## Branch manager



# Branching

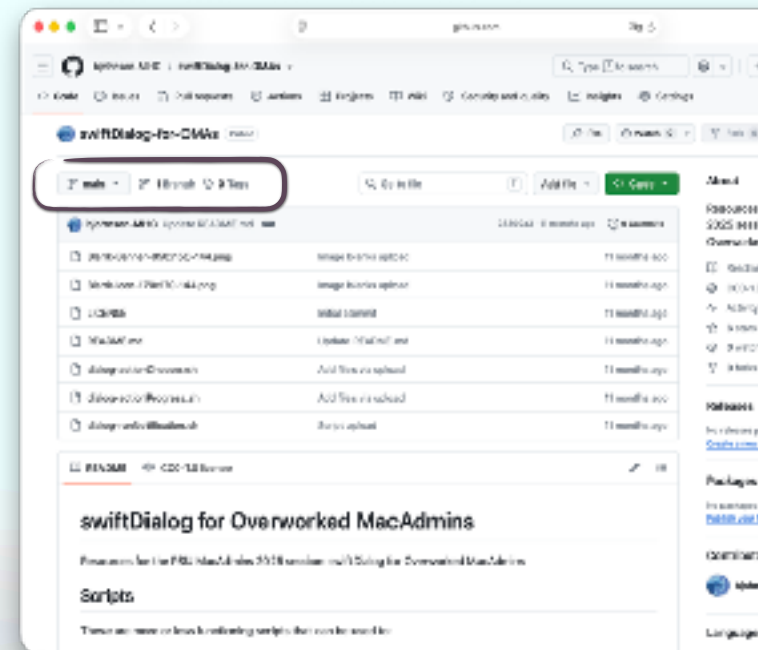
## Branch manager



# Branching

## Branch manager

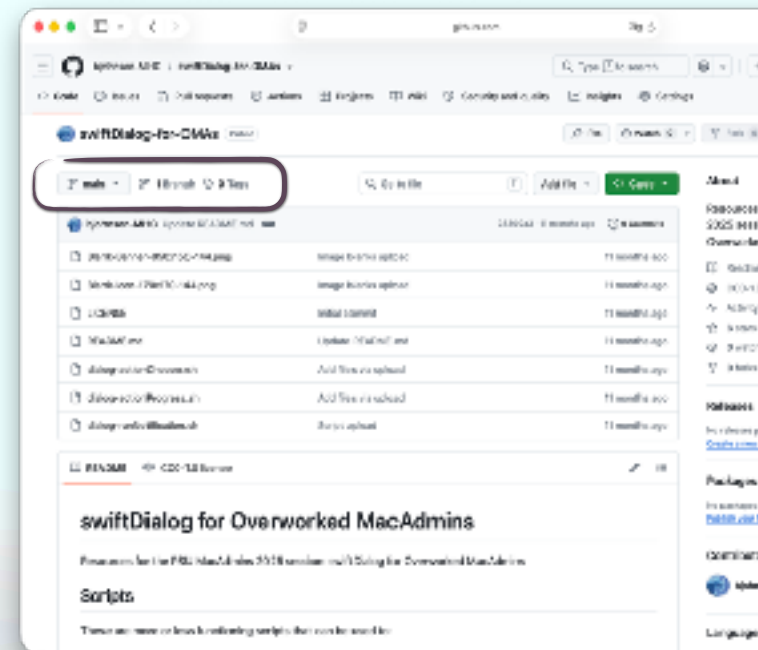
- "main" is a branch



# Branching

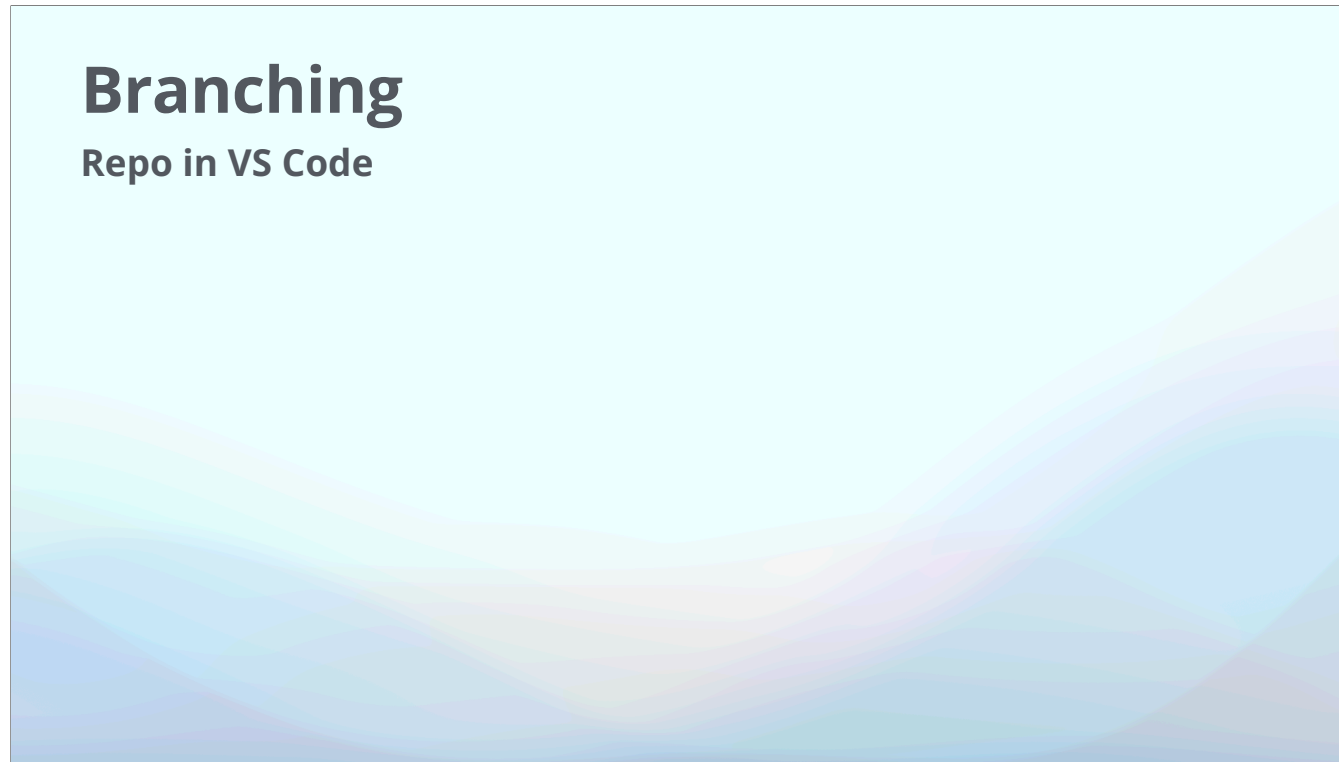
## Branch manager

- "main" is a branch
- "main" is just a label



# Branching

Repo in VS Code

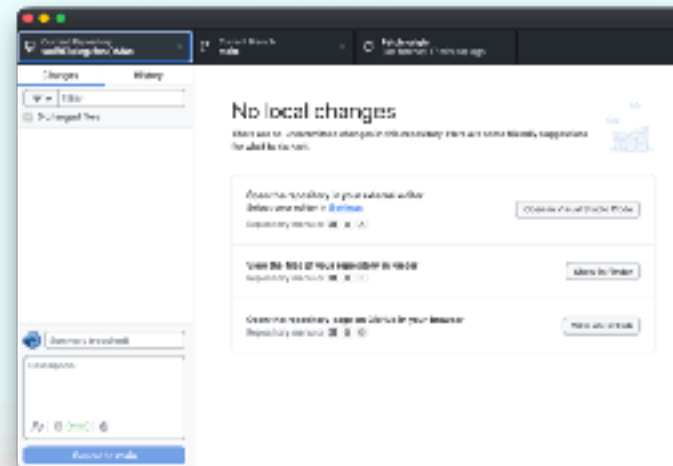


In the GitHub Desktop app,

- I have some options for accessing the files in the repo
  - I can open them via my default external editor
  - Show them in Finder
  - And go to GitHub to view them, which we've already seen.

# Branching

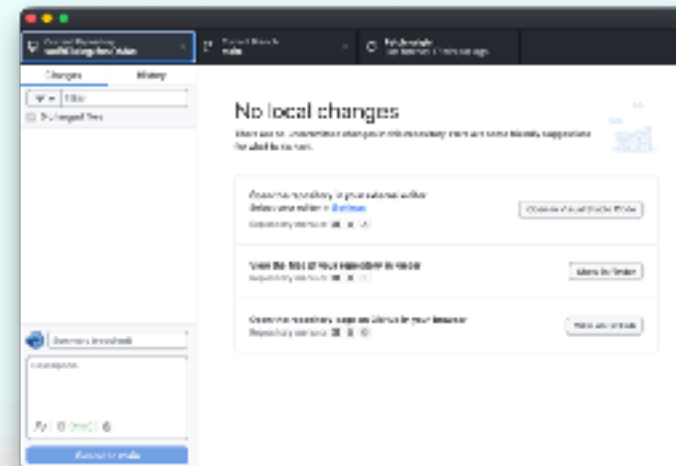
## Repo in VS Code



# Branching

## Repo in VS Code

- Repo actions via app

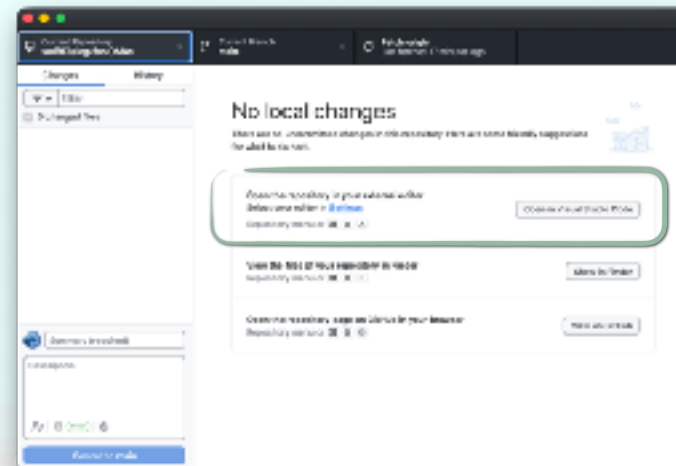




# Branching

## Repo in VS Code

- Repo actions via app
- Open in editor

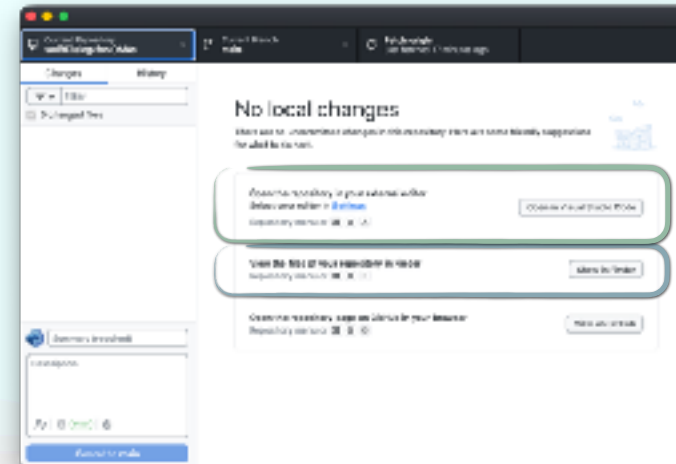




# Branching

## Repo in VS Code

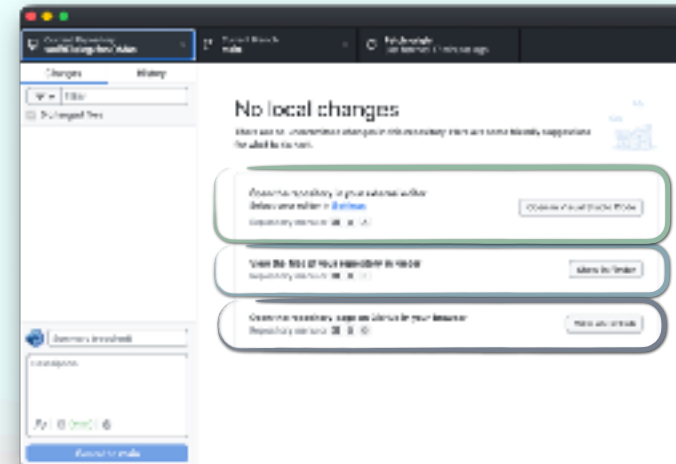
- Repo actions via app
- Open in editor
- Show in Finder



# Branching

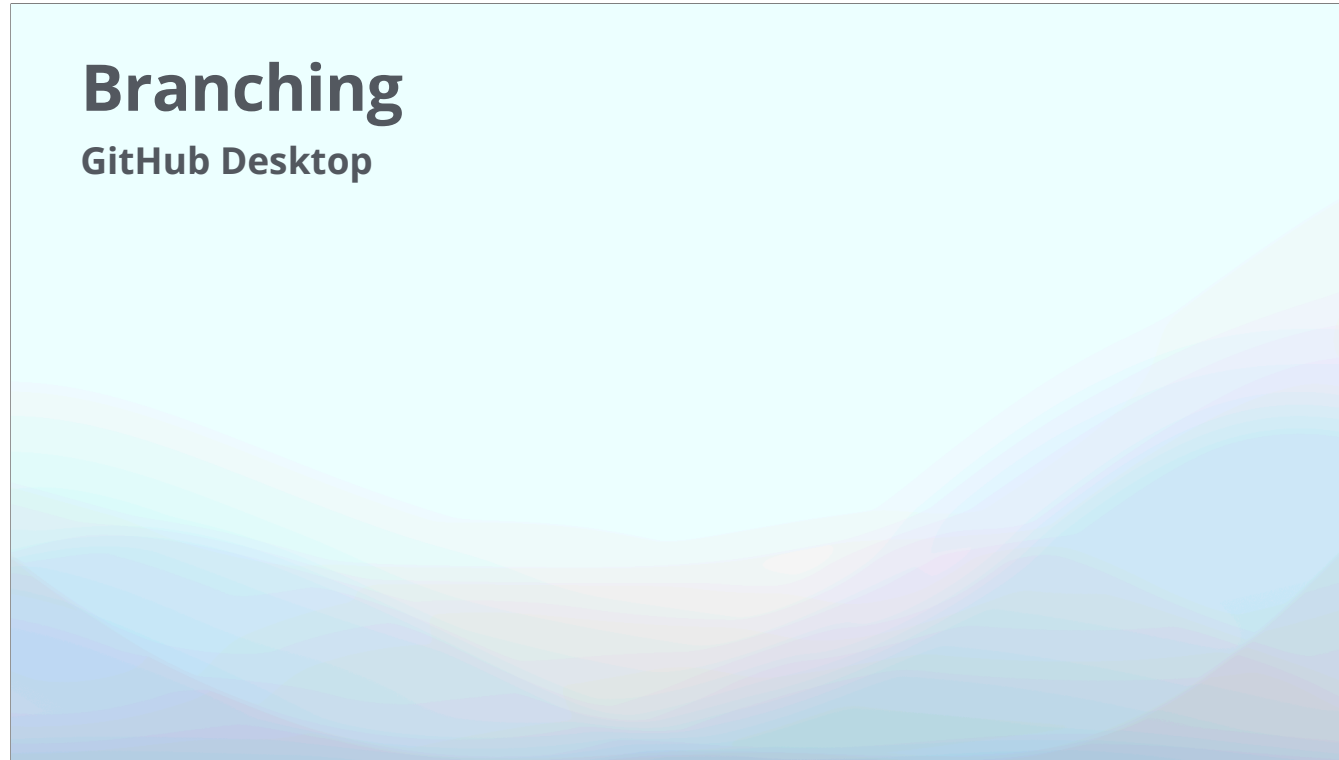
## Repo in VS Code

- Repo actions via app
  - Open in editor
  - Show in Finder
  - View on GitHub



# Branching

GitHub Desktop

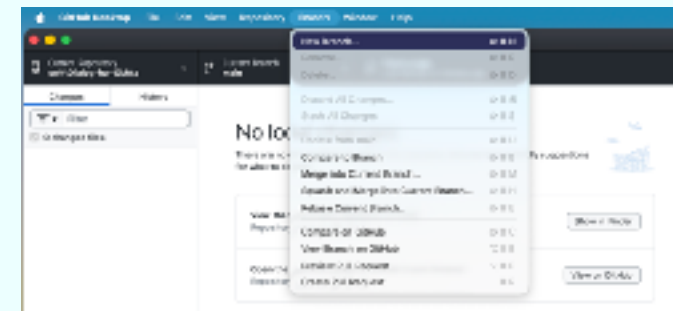


I'm going to be doing my work on my workstation.

- I have already cloned the repository to it.
- I'm going to the Branch menu and select New Branch
- I'm going to give it an accurate and boring name "swiftDialog 3.0"
- Check out this notice: "Your new branch will be based on your currently checked out branch (`main`). `main` is the default branch for your repository." I was working from `main`. So the branch is from `main`.
- This is a hint that you're not limited to branching off of `main`. Any branch can be branched.

# Branching

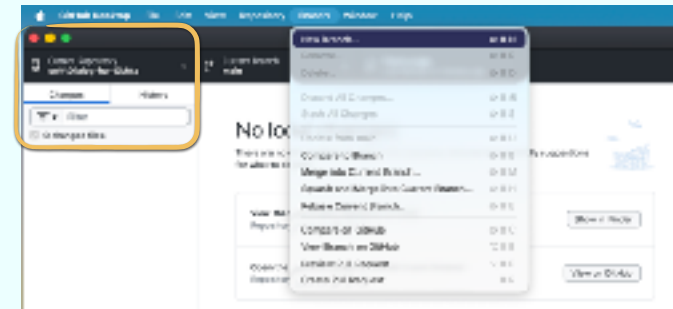
## GitHub Desktop



# Branching

## GitHub Desktop

- Clone to local



# Branching

## GitHub Desktop

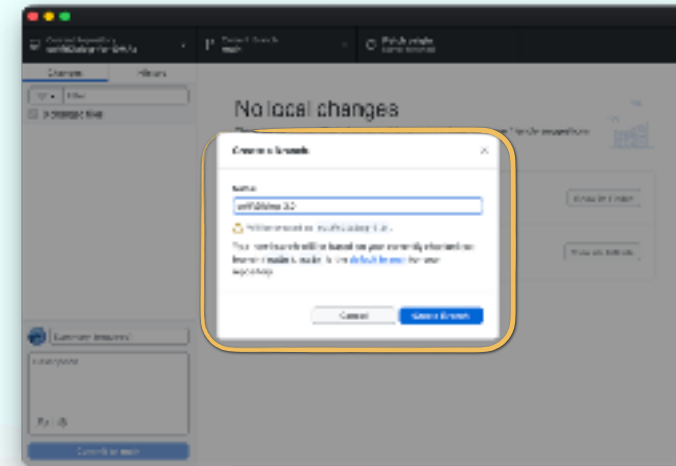
- Clone to local
- Branch menu: new branch



# Branching

## GitHub Desktop

- Clone to local
- Branch menu: new branch
- Branch name: be specific

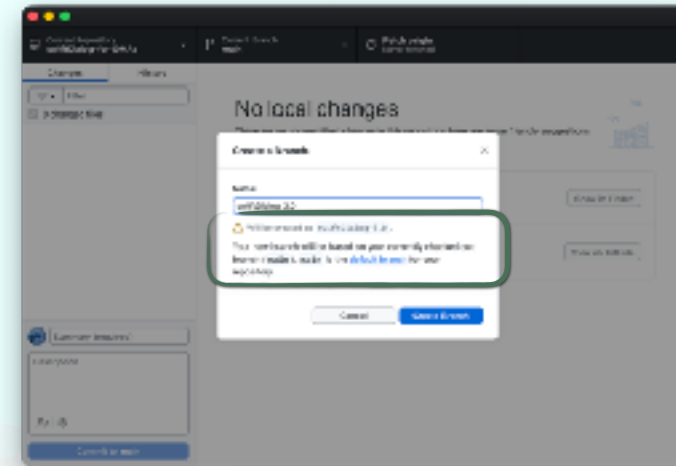




# Branching

## GitHub Desktop

- Clone to local
- Branch menu: new branch
- Branch name: be specific
- Context: active branch

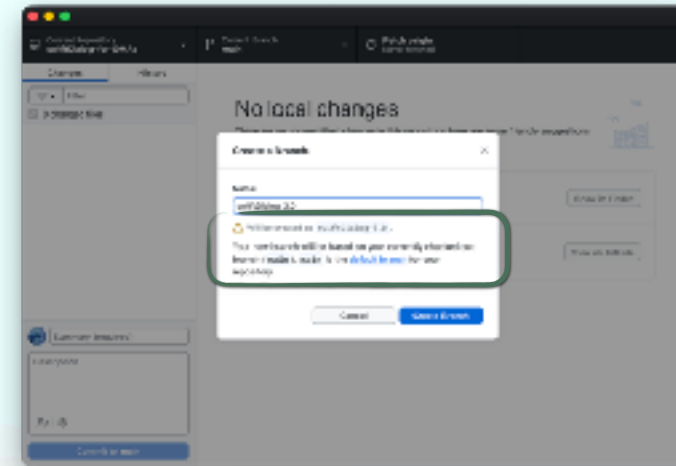




# Branching

## GitHub Desktop

- Clone to local
- Branch menu: new branch
- Branch name: be specific
- Context: active branch
- Any branch can be branched



# Local branch

Push/publish optional

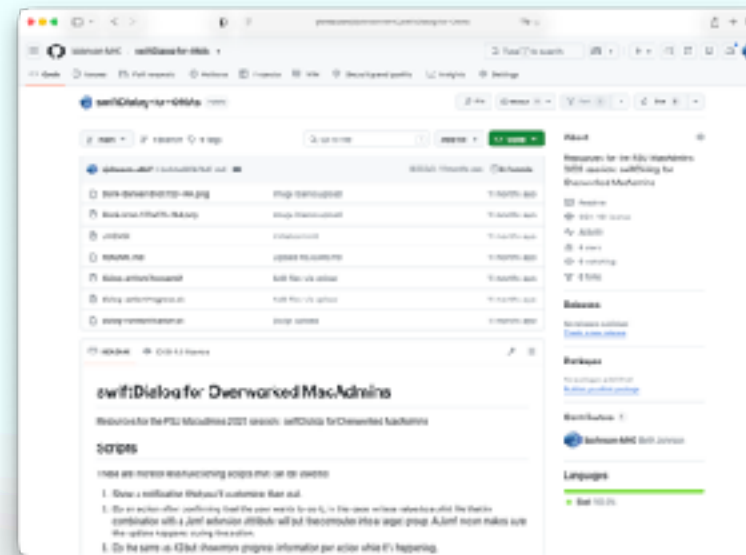
Now we have a branch, but for now it's

- only on my local machine.
- If we check out the repo on the web, there's still only main
- So we can publish the branch so it can benefit from being part of a distributed system.
- After we publish, we look again at the web and can see that we've got two branches.
- Follow the branches tab, and we have information about all of our branches.

So far, so good. Now, let's change some things up.

## Local branch

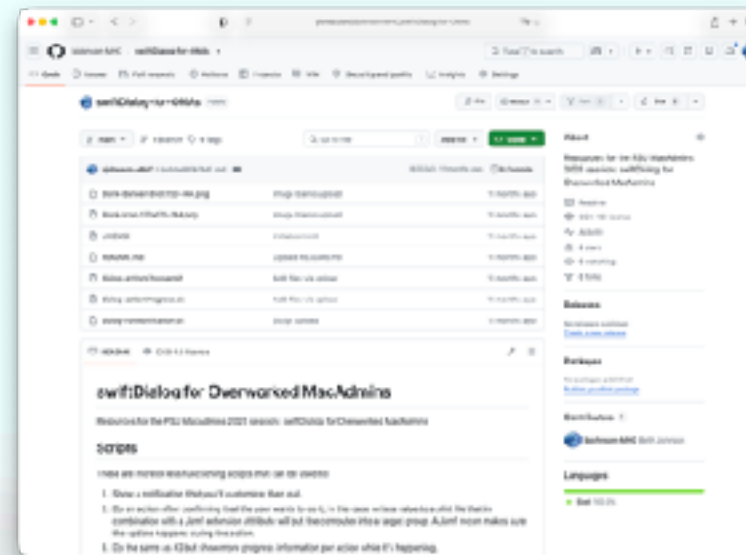
Push/publish optional



# Local branch

## Push/publish optional

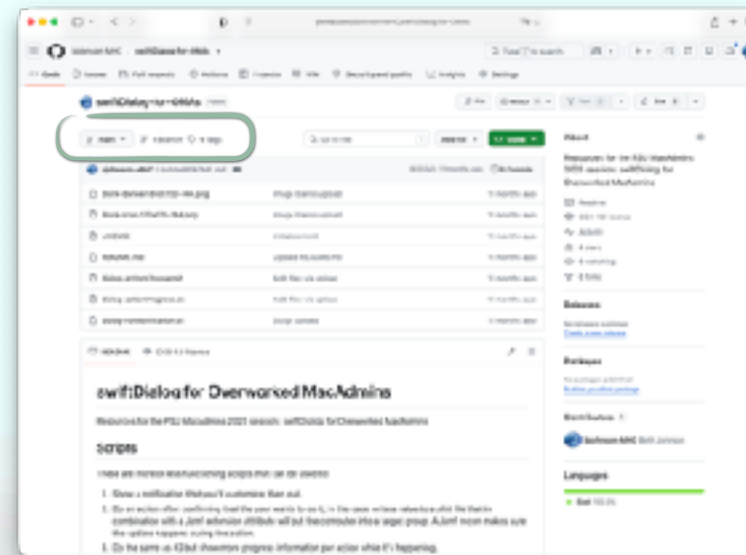
- Local only until pushed



## Local branch

Push/publish optional

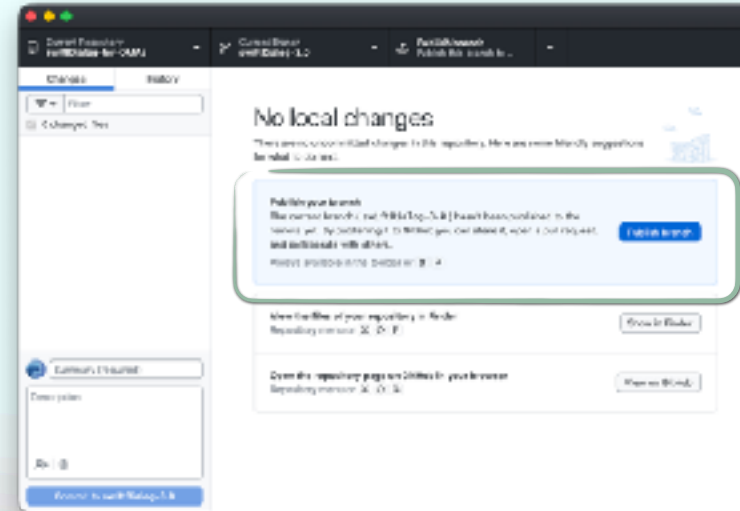
- Local only until pushed
- GitHub: one branch



# Local branch

## Push/publish optional

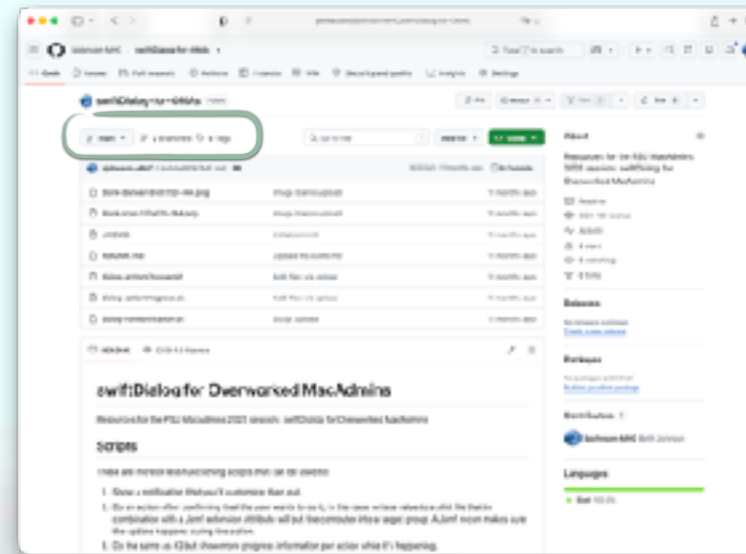
- Local only until pushed
- GitHub: one branch
- Publish to origin



# Local branch

## Push/publish optional

- Local only until pushed
  - GitHub: one branch
- Publish to origin
- All branches, both places
  - Branch page

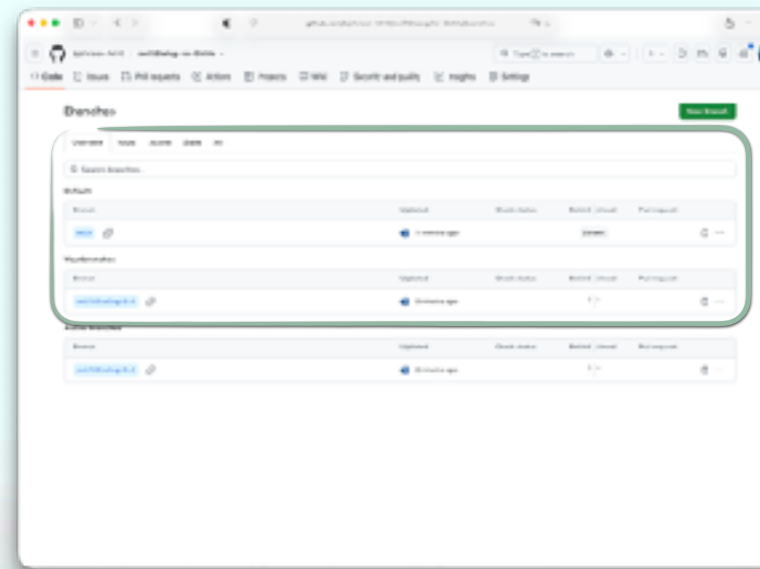




# Local branch

## Push/publish optional

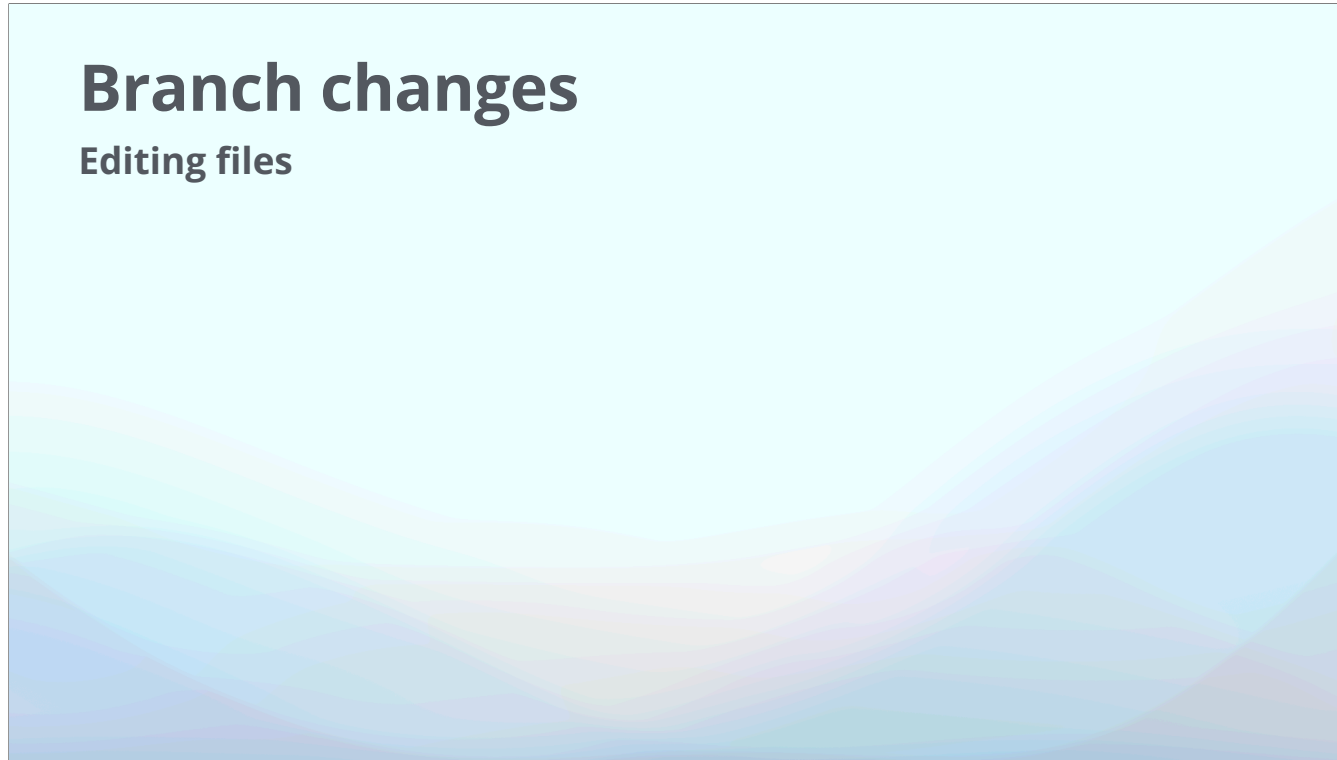
- Local only until pushed
  - GitHub: one branch
- Publish to origin
- All branches, both places
  - Branch page





# Branch changes

## Editing files

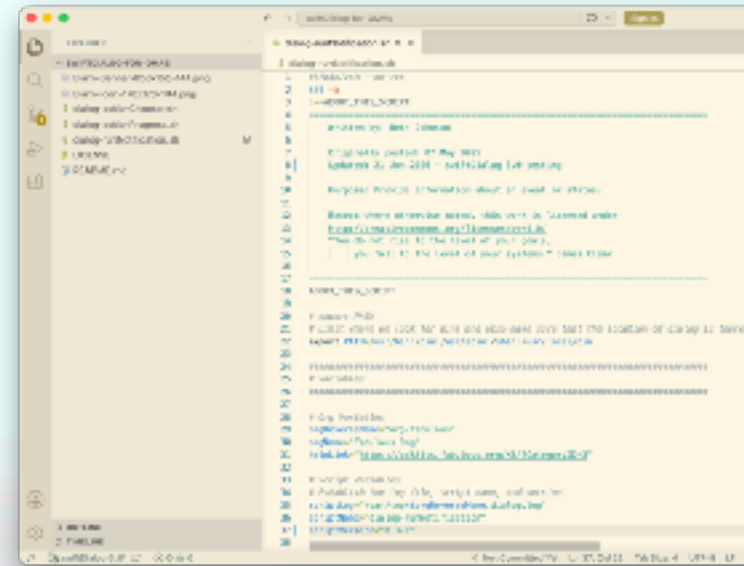


I open the branch in Visual Studio Code. Here we're going to make some changes to the files in the branch.

- First I'm going to edit a file to add a comment that I'm testing for compatibility.
- And I'm going to update my script version.
- And I'm going to save the file. Git is paying attention.
- When we save the file, VS Code adds this M next to the file name and the Version Control icon gets badged.
- This shows me that I have changes in the branch I'm working in that are not committed.

Let's get that change stowed away safely to origin. I could do this in Visual Studio Code, and I've so far found that confusing. But because the underlying technology is Git, the state of the branch is apparent to any Git-aware tool. Let's see what it looks like in GitHub Desktop.

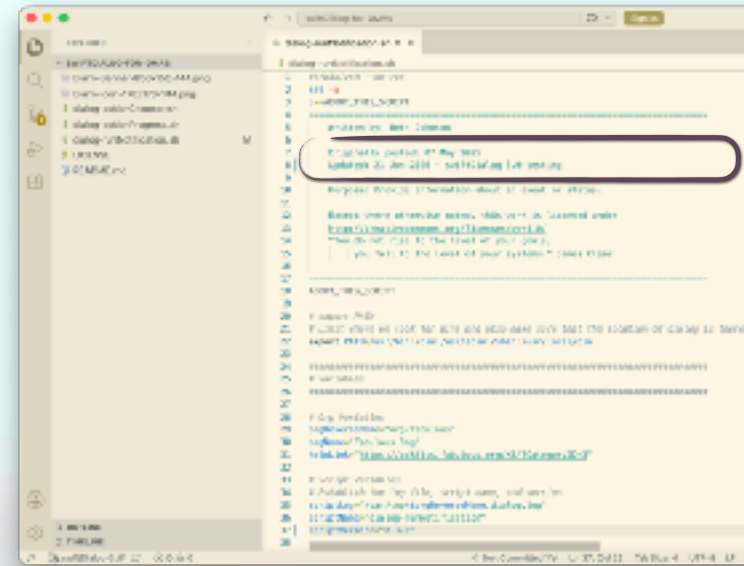
## Editing files



# Branch changes

## Editing files

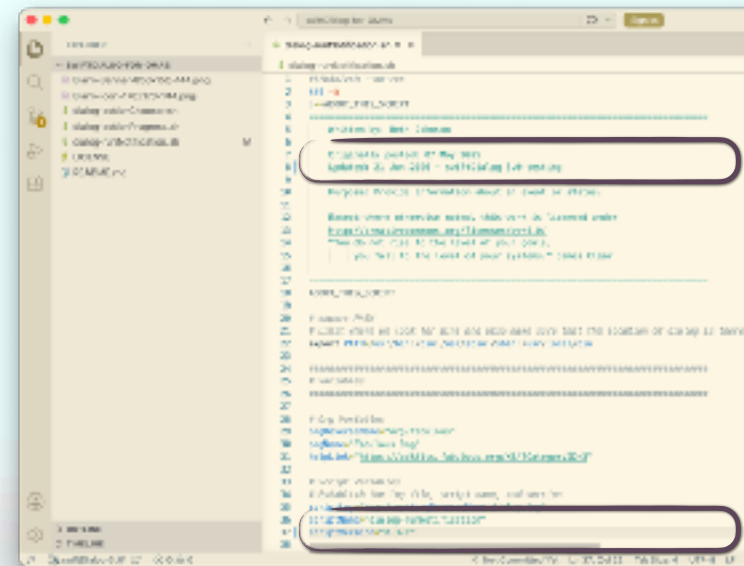
- Add comment



# Branch changes

## Editing files

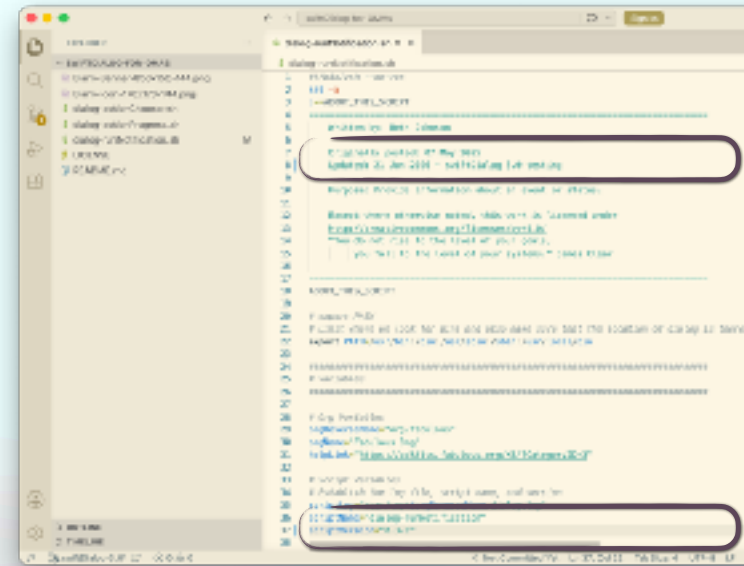
- Add comment
- Update version



# Branch changes

## Editing files

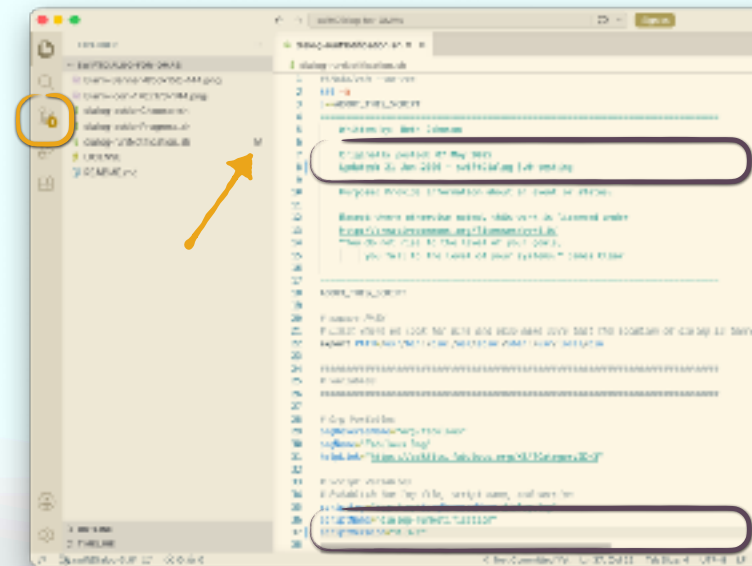
- Add comment
- Update version
- Git tracks changes on **save**



# Branch changes

## Editing files

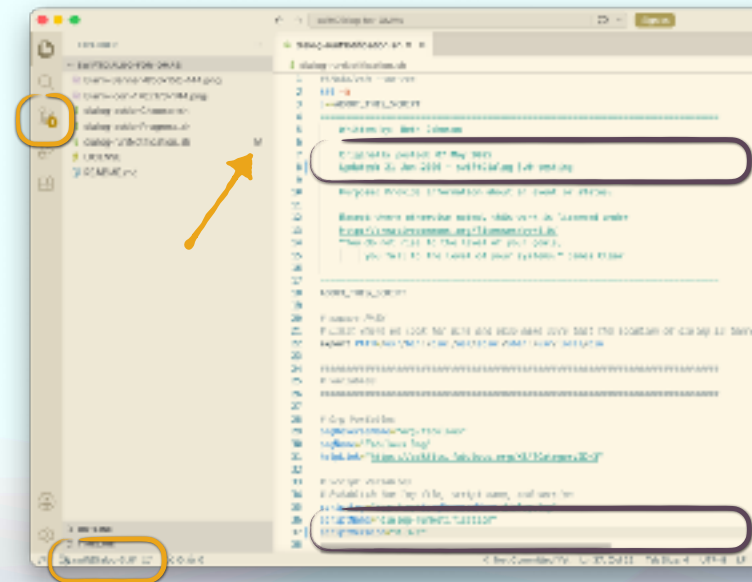
- Add comment
- Update version
- Git tracks changes on **save**
- Visual Studio Code updates from Git



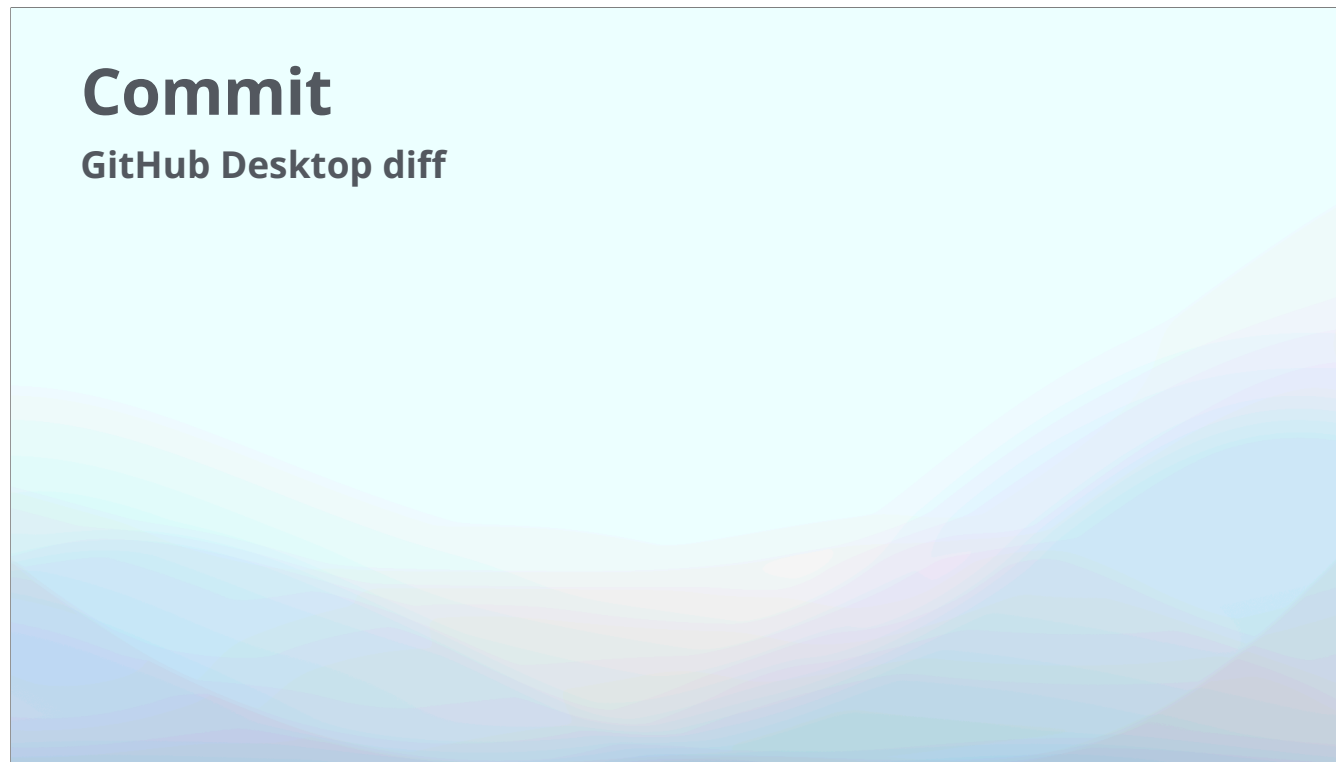
# Branch changes

## Editing files

- Add comment
- Update version
- Git tracks changes on **save**
- Visual Studio Code updates from Git







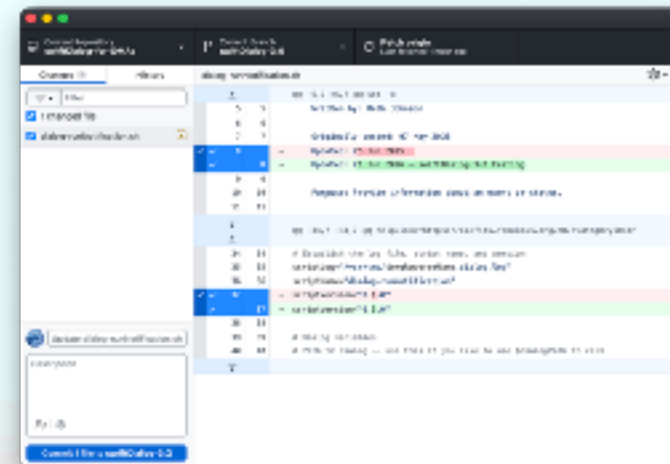
Oh, look. This is nice and simple. I

- check that my repo
- and my branch are correct.
- The diff view shows the changes I made.
- The summary says "Update" with the name of the script.
- I just need to add something more descriptive.
- Which I do.
- Notice that it's very clear about where the commit is going to: "Commit 1 file to swiftDialog-3.0" . . . Once you have multiple branches in play, you need to be aware of which branch is active.



# Commit

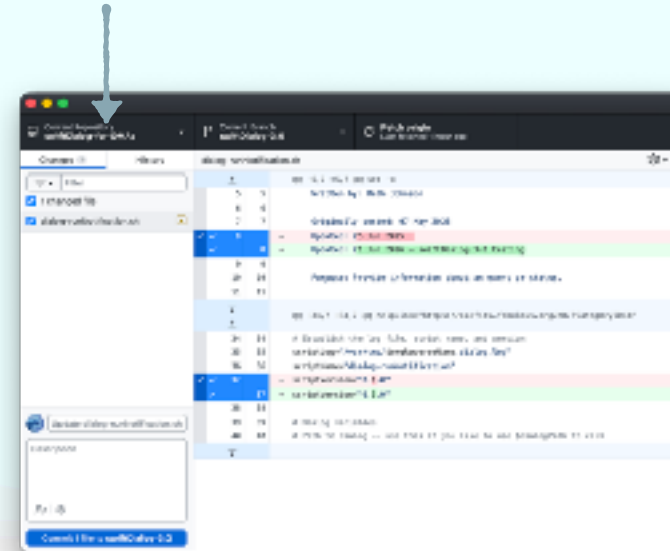
## GitHub Desktop diff



# Commit

## GitHub Desktop diff

- Check repo



# Commit

## GitHub Desktop diff

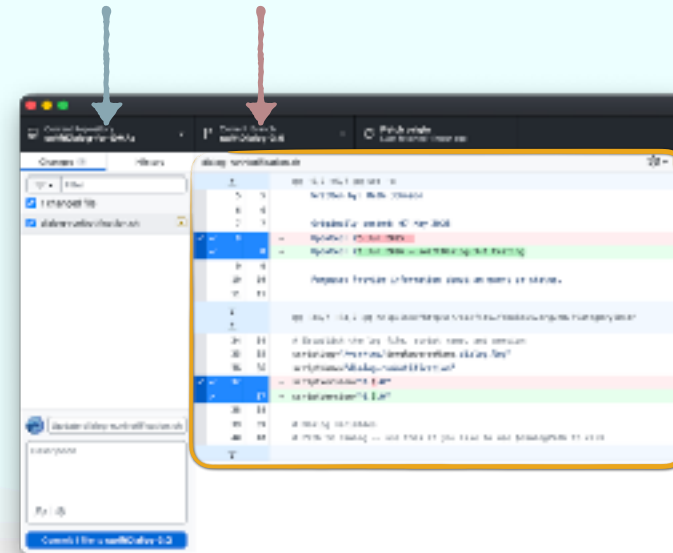
- Check repo
- and branch



# Commit

## GitHub Desktop diff

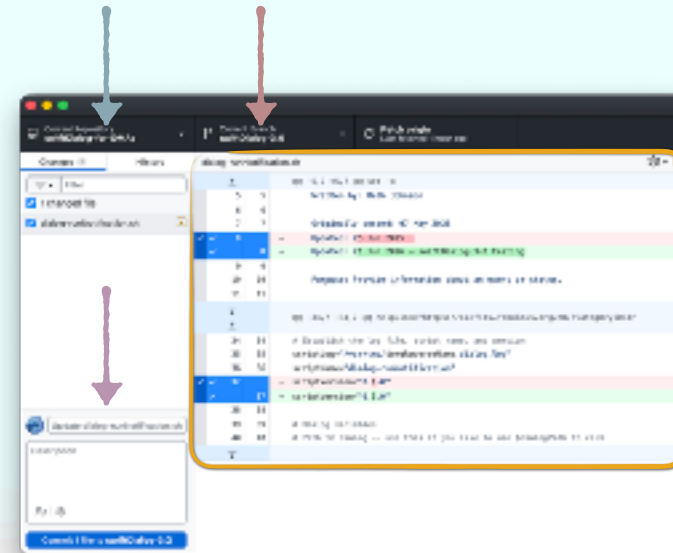
- Check repo
- and branch
- Diff view



# Commit

## GitHub Desktop diff

- Check repo
- and branch
- Diff view
- Summary pre-filled



# Commit

## GitHub Desktop diff

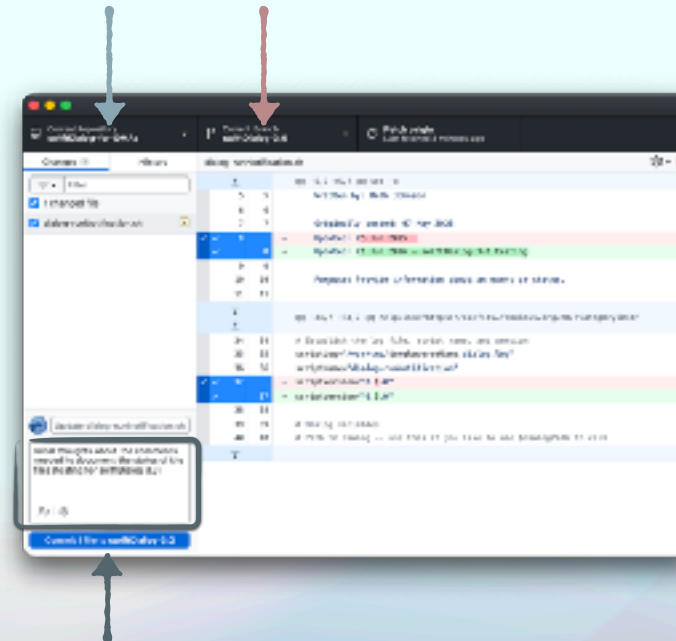
- Check repo
  - and branch
- Diff view
- Summary pre-filled
- Choose your own description



# Commit

## GitHub Desktop diff

- Check repo
  - and branch
- Diff view
- Summary pre-filled
- Choose your own description
- Destination clarity





# Commit

GitHub Desktop push

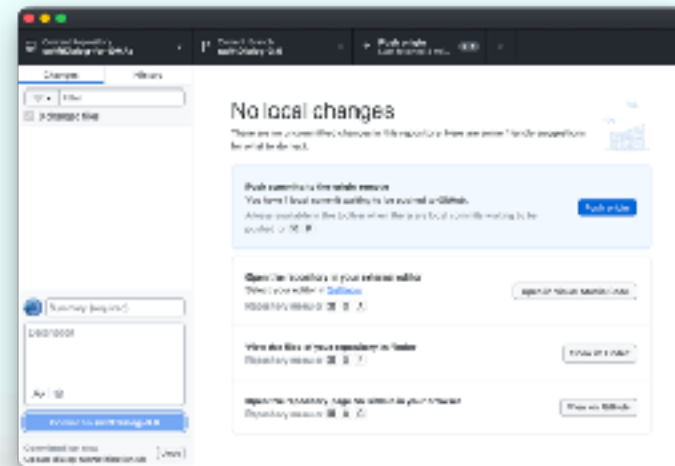
So we've committed the changes, but they're again just local to my workstation until

- I push them to origin,
- which is the branch I'm working in, not main.
- I push, and the app offers to help me with a request to pull my changes into main. But let's not get ahead of ourselves.



# Commit

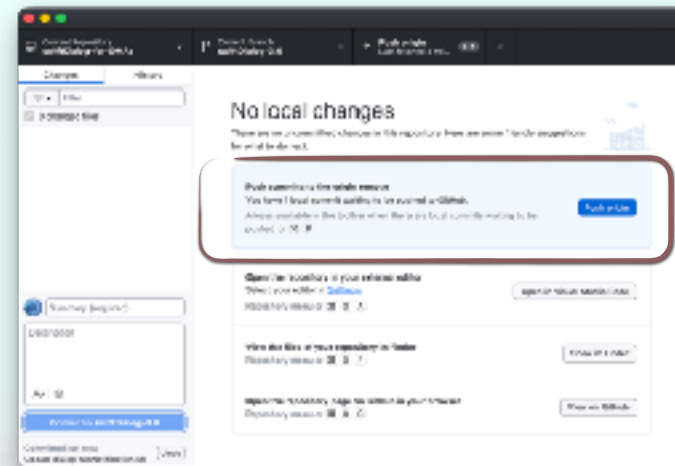
## GitHub Desktop push



# Commit

## GitHub Desktop push

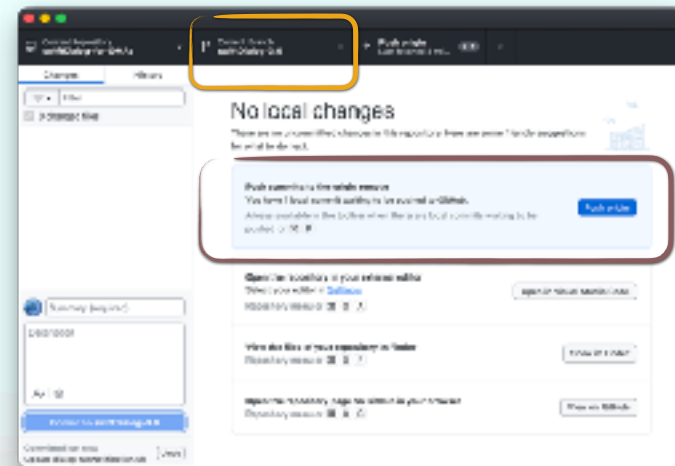
- Push to Origin



# Commit

## GitHub Desktop push

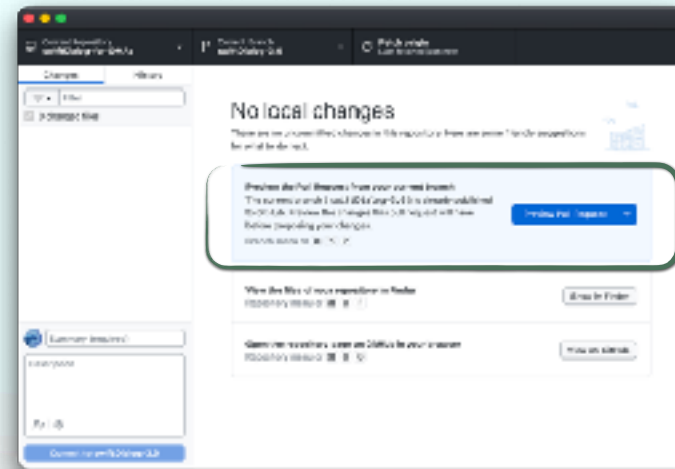
- Push to Origin
- Origin = branch



# Commit

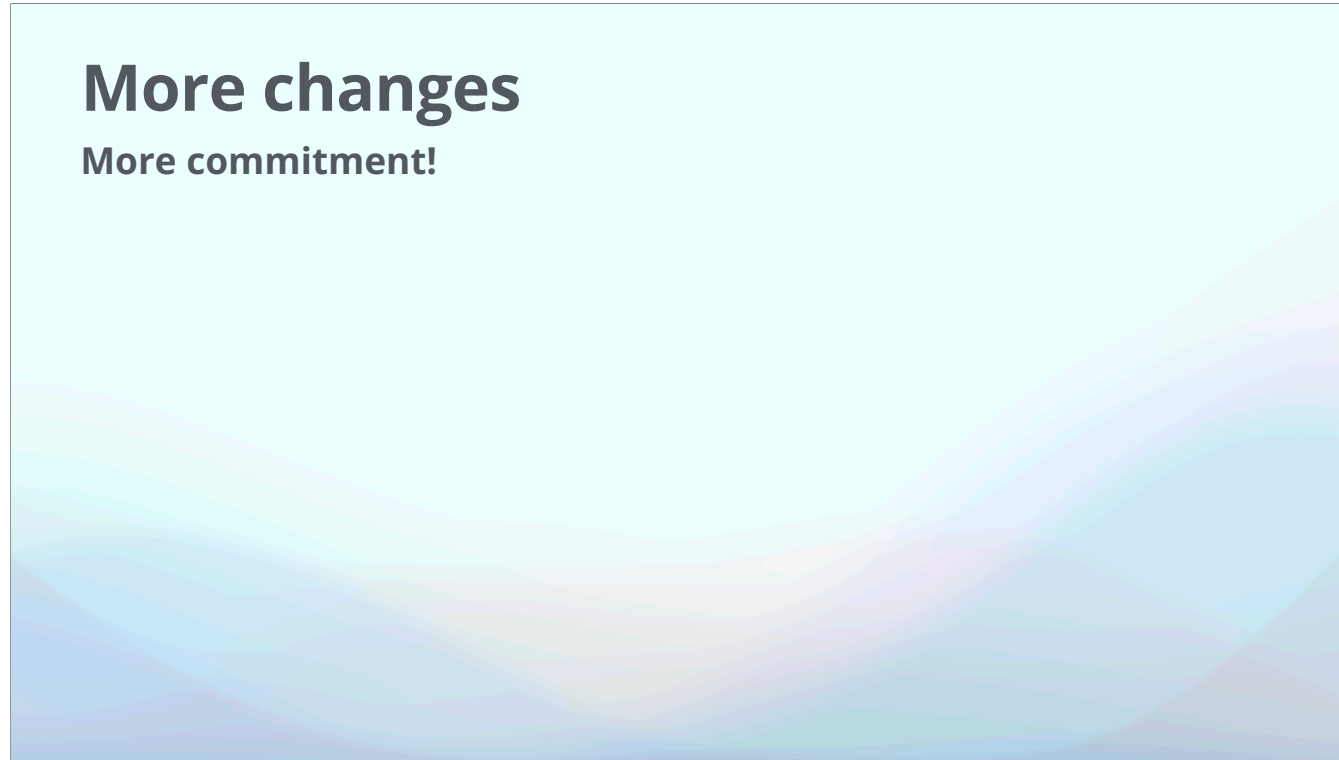
## GitHub Desktop push

- Push to Origin
  - Origin = branch
- Prompt to pull to main



# More changes

More commitment!



We're not done making changes, so we don't want to pull into main yet. So we can go back into the explorer in Visual Studio Code and make those changes to the rest of the files.

- As we save each, Git recognizes it as Modified and stages it for the commit.
- I'm also doing an edit to the repo's README file about what's going on with these changes. This is all part of my work to be more deliberate about changes and documenting them.
- I end up with three modified files that are staged for commit,
- but not yet committed to the branch.
- I can keep making as many changes and commits as I need to, in this branch.

But what about main? What's happening there?

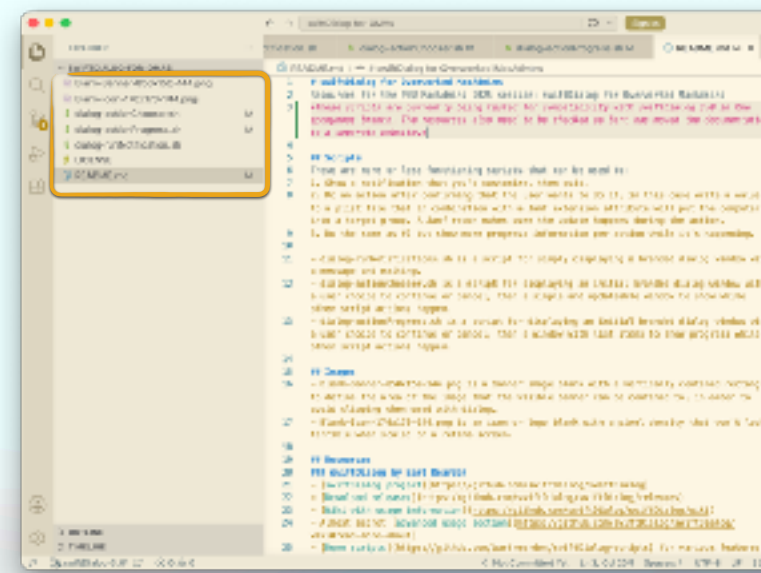
## More commitment!



# More changes

## More commitment!

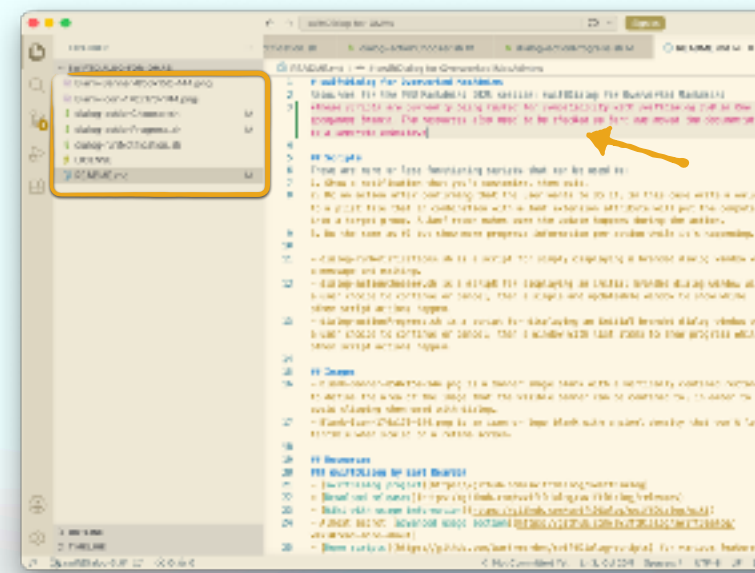
- Update other scripts





## More commitment!

- Update other scripts
- Add a comment in README

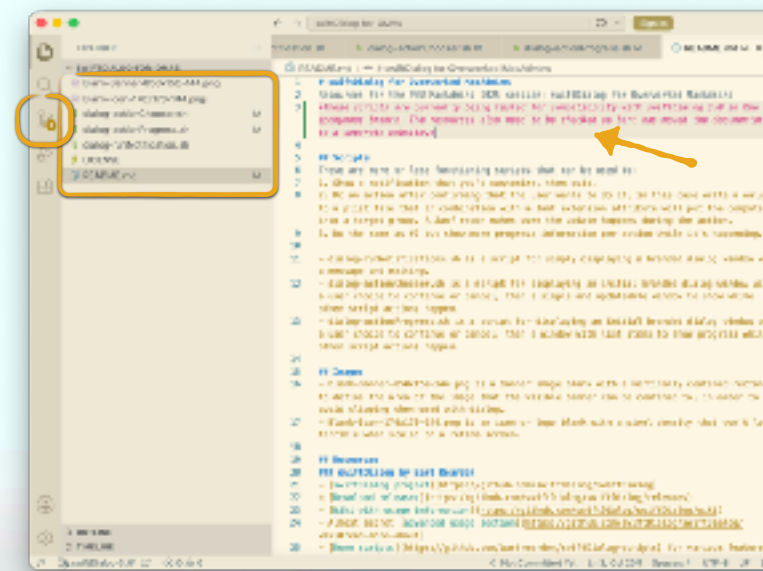




# More changes

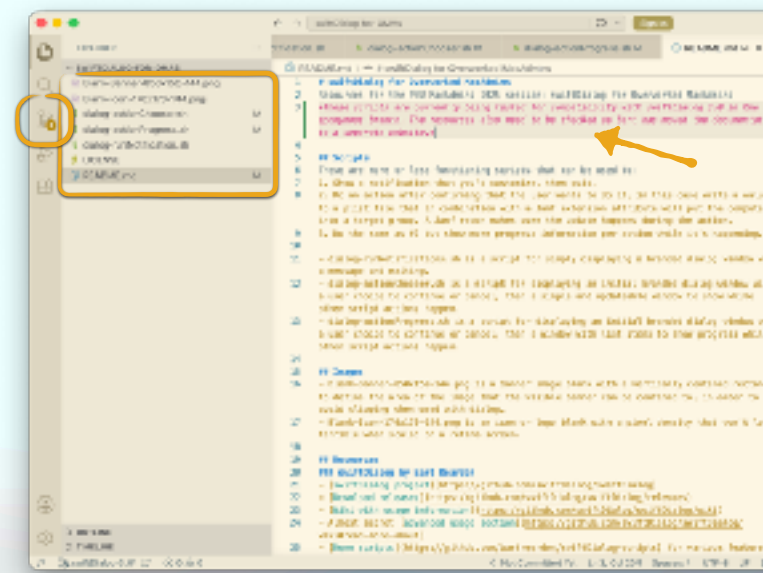
## More commitment!

- Update other scripts
- Add a comment in README
- Three modified files, staged



## More commitment!

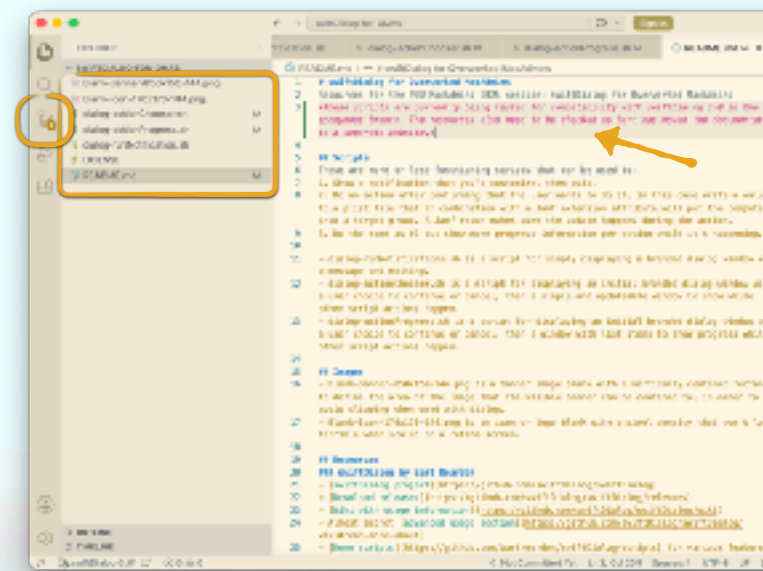
- Update other scripts
- Add a comment in README
- Three modified files, staged
- Not committed yet



# More changes

## More commitment!

- Update other scripts
- Add a comment in README
- Three modified files, staged
- Not committed yet
- Unlimited changes



# main in cloud

Original dates

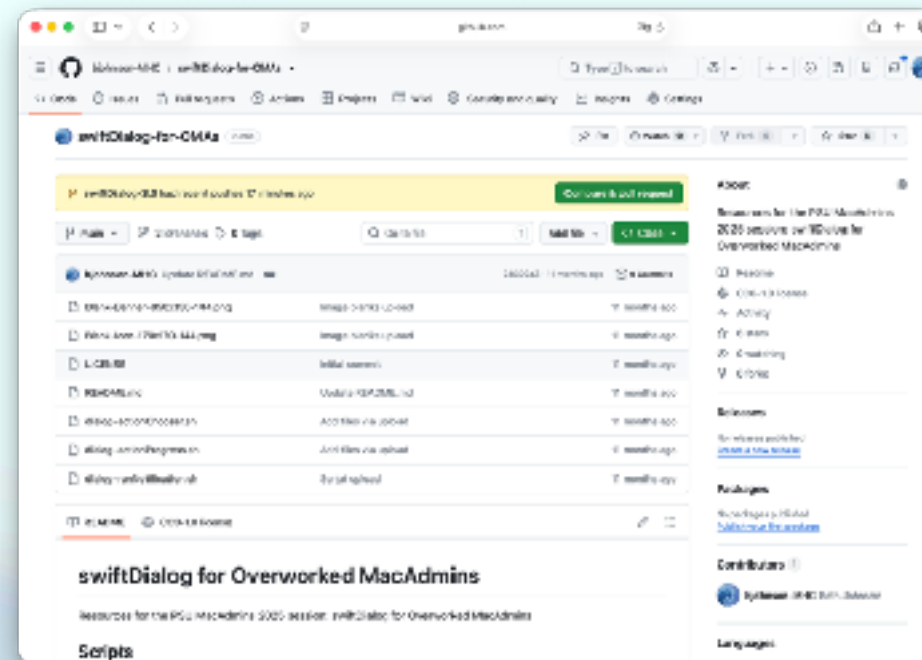
If we check main on the web, the

- UI also prompts us for a pull request.

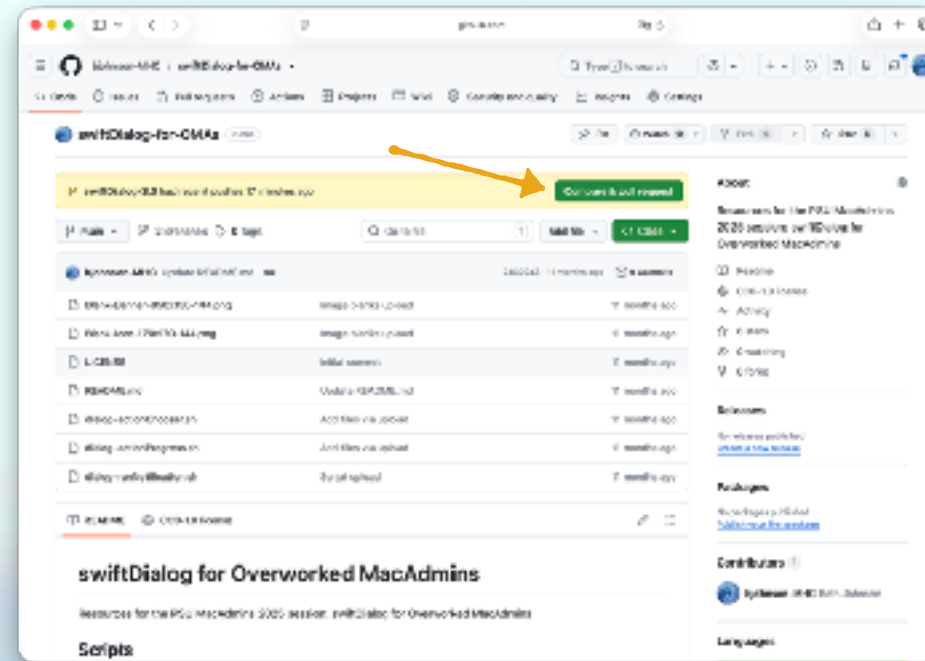
But in main,

- The file dates are the original dates.

main in  
cloud  
Original dates

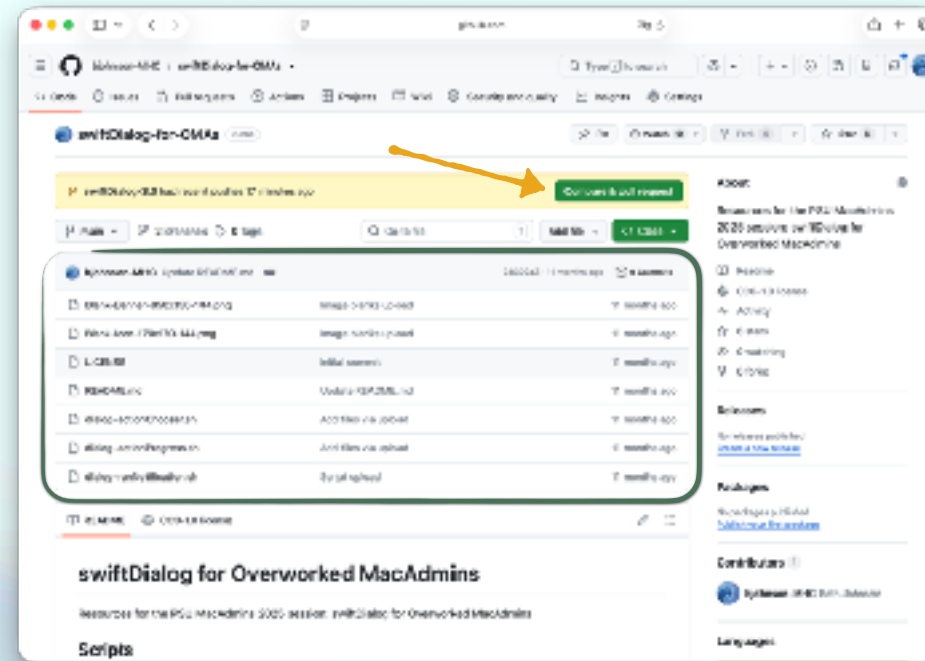


A screenshot of a web browser displaying the GitHub repository page for 'swiftDialog-for-MacOs'. The page shows the repository name, a description, and a list of files. A yellow arrow points to the 'Download source code' button. The page also includes a sidebar with 'About', 'Package', and 'Contributors' sections.

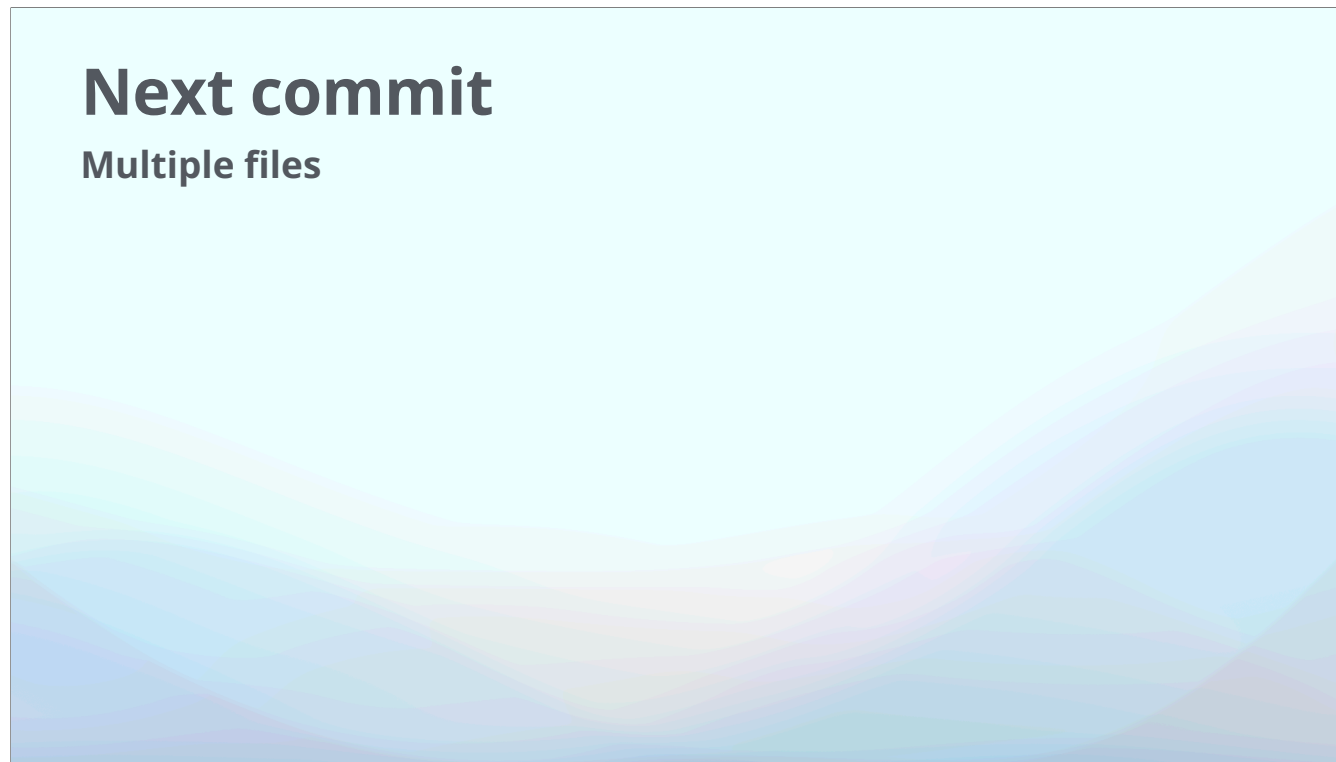


The screenshot shows the GitHub repository page for 'swiftDialog-for-MacOs'. The repository is owned by 'joshuawhite' and has 2020 stars and 10 forks. The description states: 'Resources for the PGU MacAdmins 2020 session: swiftDialog for Overworked MacAdmins'. A red arrow points to the 'Download SwiftDialog' button. The releases table lists versions from 1.0.0 to 1.10.0, with the latest version being 1.10.0, released 11 months ago. The repository has 2020 stars and 10 forks.

| Version | Download | Released      |
|---------|----------|---------------|
| 1.10.0  | Download | 11 months ago |
| 1.9.0   | Download | 11 months ago |
| 1.8.0   | Download | 11 months ago |
| 1.7.0   | Download | 11 months ago |
| 1.6.0   | Download | 11 months ago |
| 1.5.0   | Download | 11 months ago |
| 1.4.0   | Download | 11 months ago |
| 1.3.0   | Download | 11 months ago |
| 1.2.0   | Download | 11 months ago |
| 1.1.0   | Download | 11 months ago |
| 1.0.0   | Download | 11 months ago |







This next commit looks a bit different. Because there are multiple files that changed, GitHub Desktop doesn't make assumptions about what the summary should be,

- so we need to add one.
- Start with a verb, in this case Add is a good one
- And the description
- We can see all the differences in the files
- And which ones have changed and are staged for the commit
- We commit, then push to origin.
- And again a pull request is suggested. Let's take a look at what happens when we are ready to merge . . .

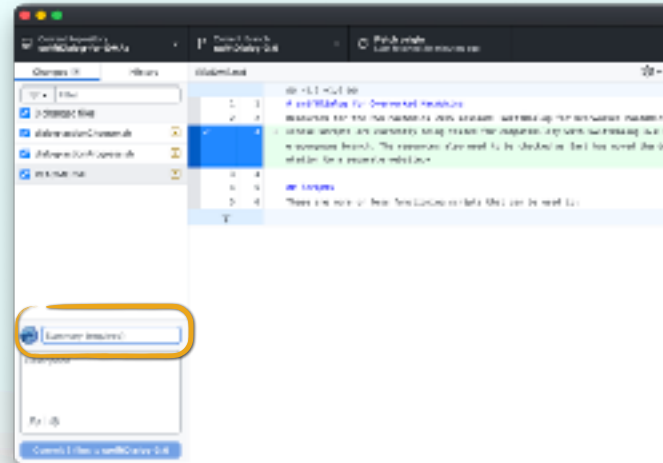




# Next commit

## Multiple files

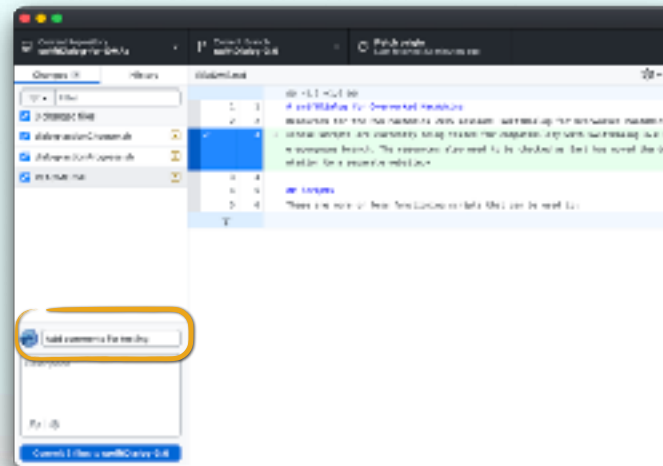
- Add summary



# Next commit

## Multiple files

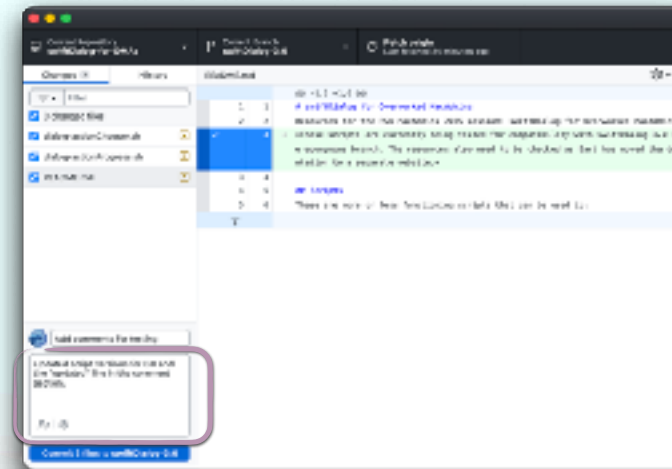
- Add summary
- Start with a verb



# Next commit

## Multiple files

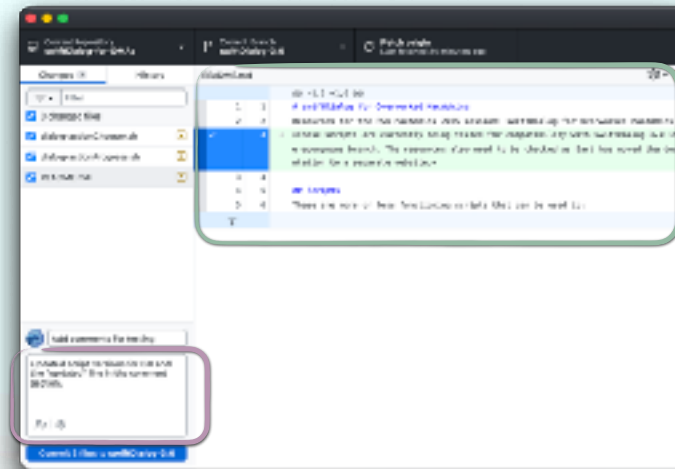
- Add summary
  - Start with a verb
- Add description



# Next commit

## Multiple files

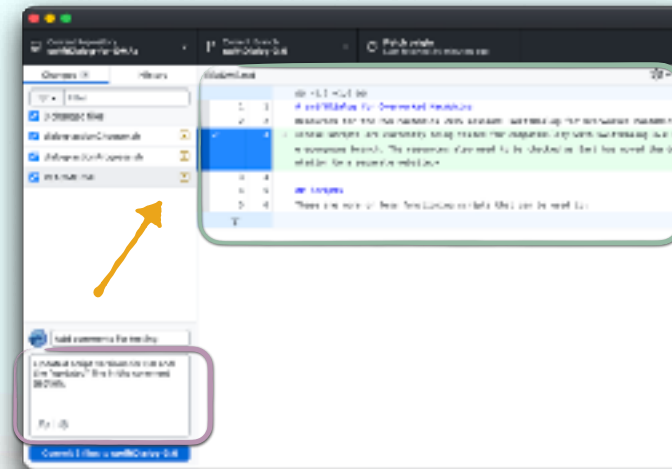
- Add summary
  - Start with a verb
- Add description
- Same diffs



# Next commit

## Multiple files

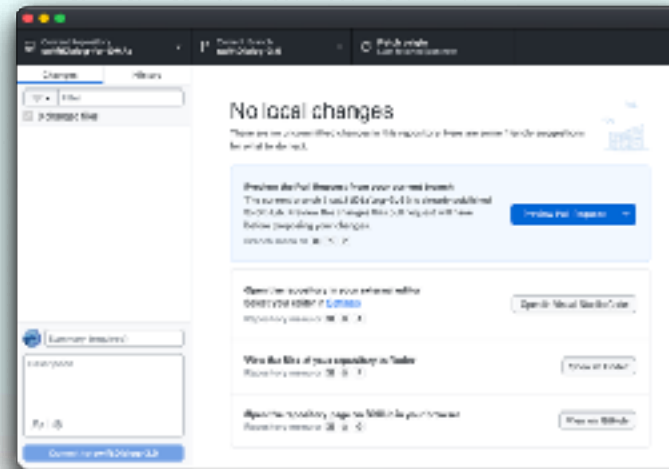
- Add summary
  - Start with a verb
- Add description
- Same diffs
- Shows all the changed files



# Next commit

## Multiple files

- Add summary
  - Start with a verb
- Add description
- Same diffs
- Shows all the changed files
- Commit and push

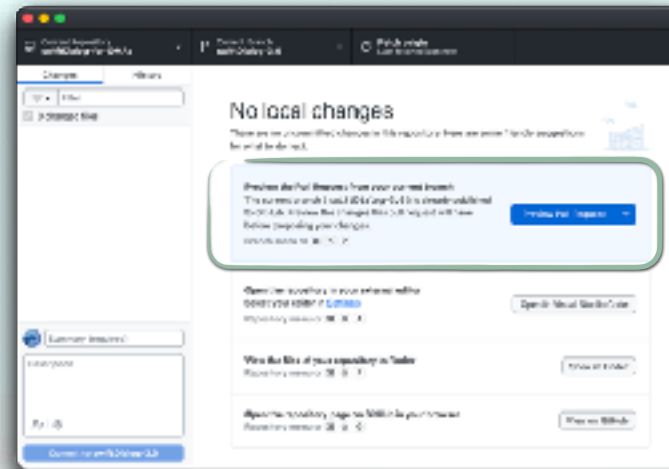




# Next commit

## Multiple files

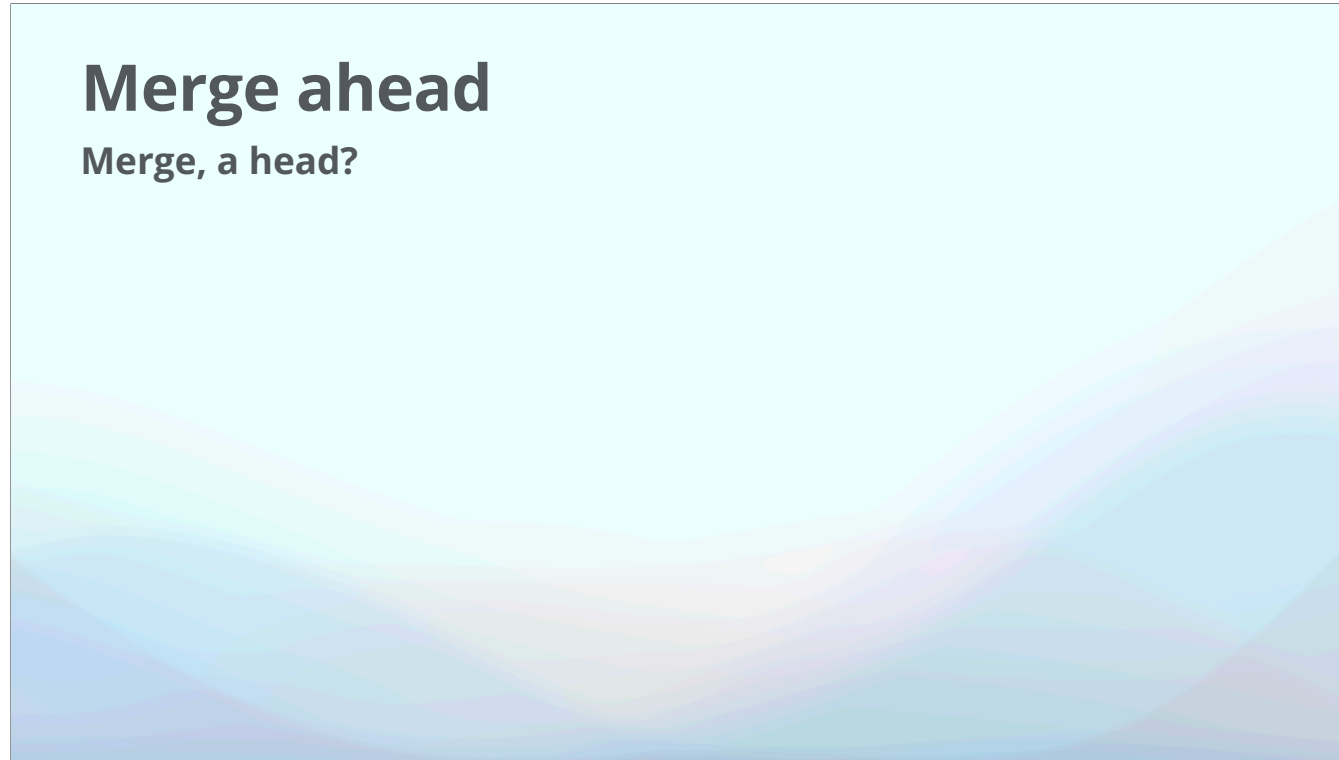
- Add summary
  - Start with a verb
- Add description
- Same diffs
- Shows all the changed files
- Commit and push
- Ignore pull request





# Merge ahead

Merge, a head?

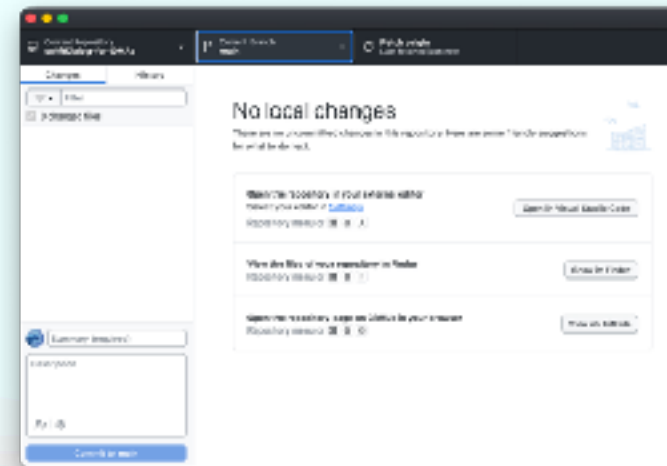


When I've tested my scripts and pushed any changes to the branch, I'm ready to fold the changes into my main branch via a merge.

- I switch my working branch to main and I check that everything is in sync (no changes in origin, no conflicts in main).
- In the branches menu,
- I select Choose branch to merge into main
- And I select my development branch
  - notice that this is going to be another commit!
- In the commit window I'm updating the description and doing the commit.
- And again I need to push to origin to have those changes communicated to the GitHub repository.

# Merge ahead

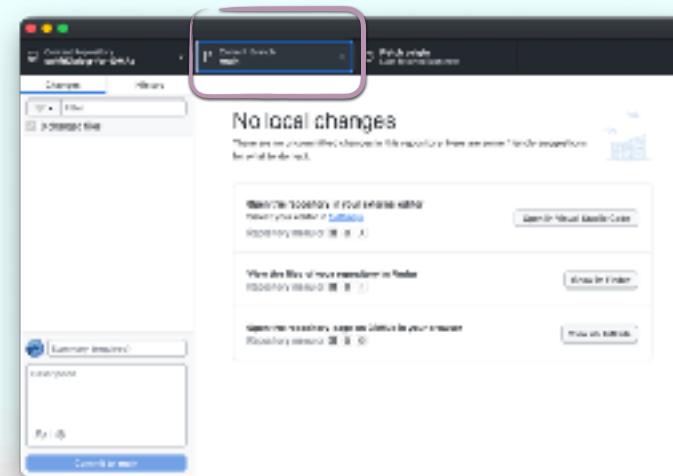
Merge, a head?



# Merge ahead

## Merge, a head?

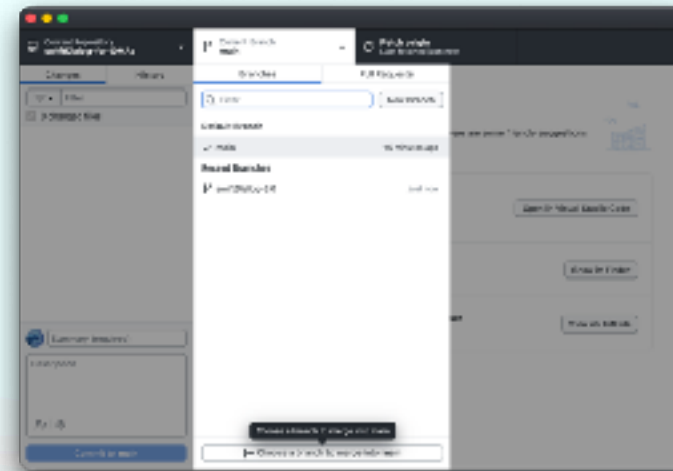
- Switch to main & verify



# Merge ahead

## Merge, a head?

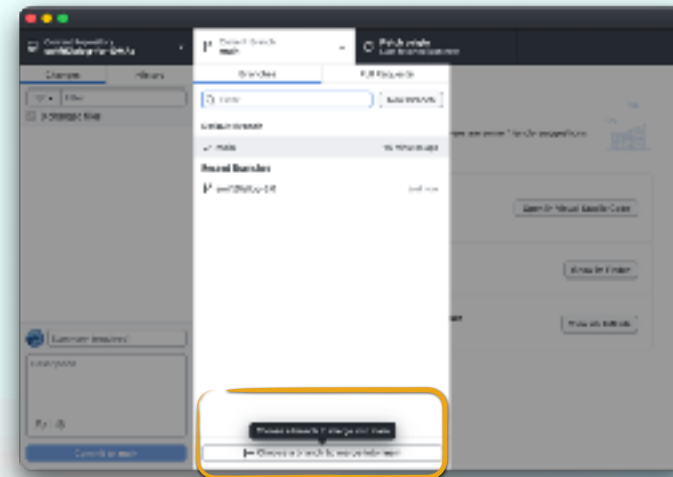
- Switch to main & verify
- Branches menu



# Merge ahead

## Merge, a head?

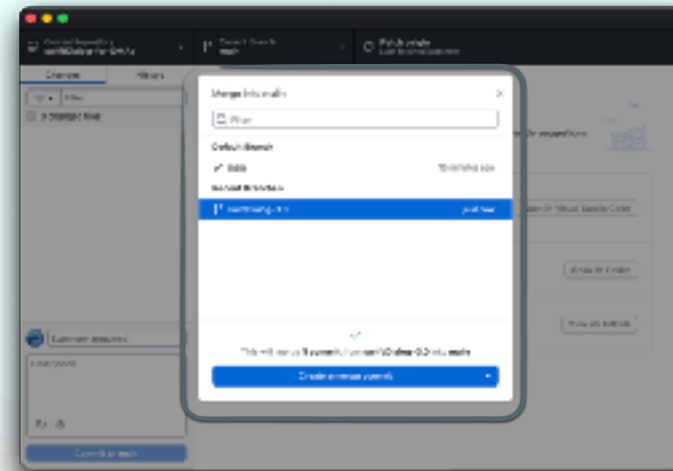
- Switch to main & verify
- Branches menu
- Choose merge branch



# Merge ahead

## Merge, a head?

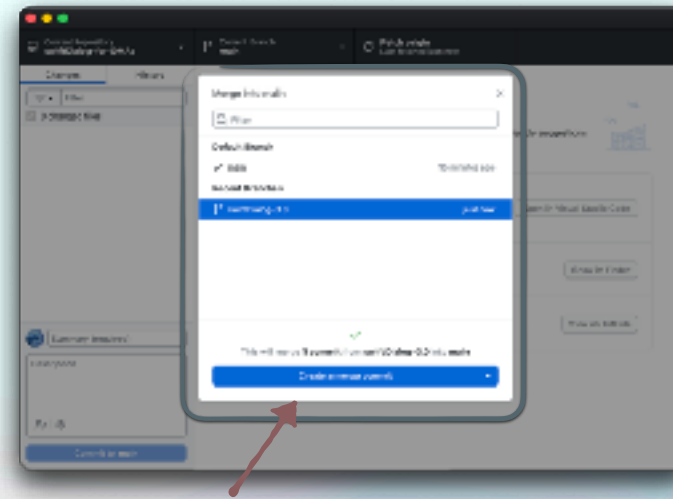
- Switch to main & verify
- Branches menu
  - Choose merge branch
  - Select branch



# Merge ahead

## Merge, a head?

- Switch to main & verify
- Branches menu
  - Choose merge branch
  - Select branch
  - Merge commit





# Merge ahead

## Merge, a head?

- Switch to main & verify
- Branches menu
  - Choose merge branch
  - Select branch
- Merge commit
- Then push origin





# Merge ahead

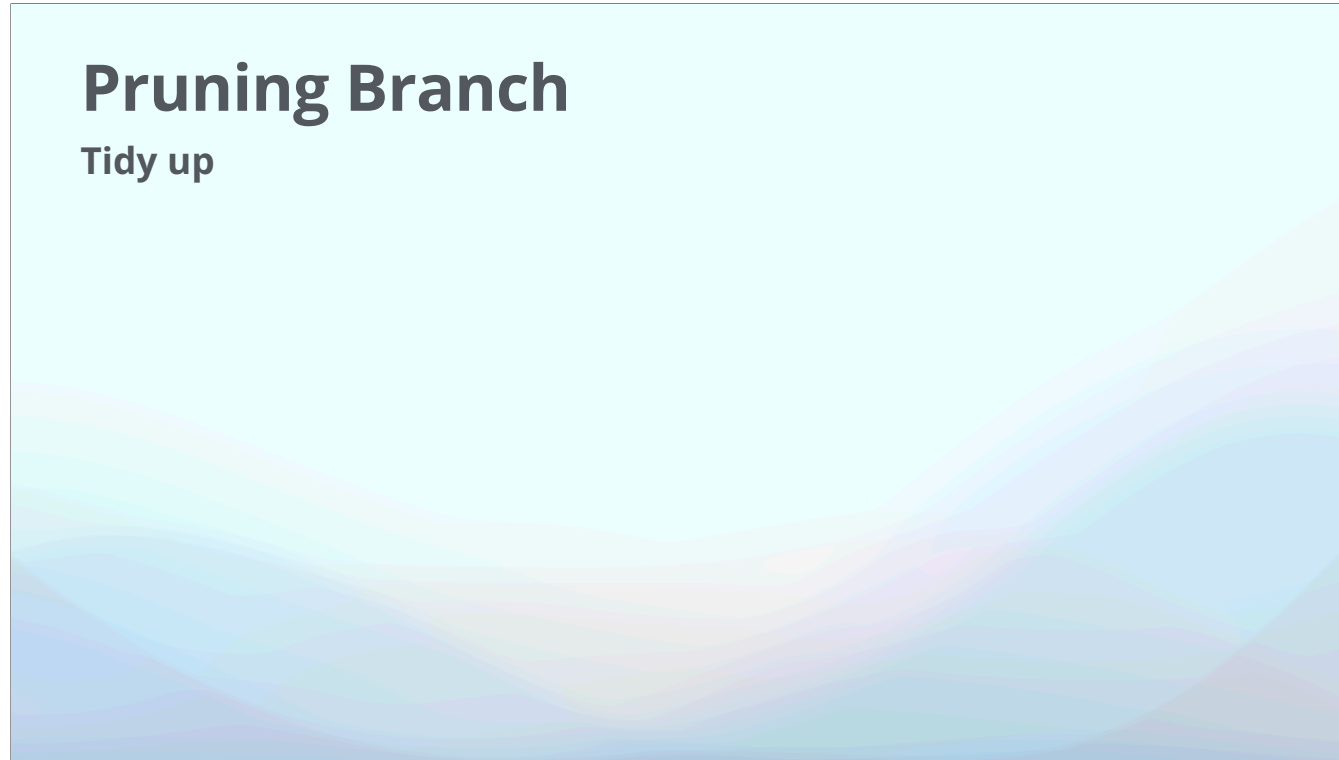
## Merge, a head?

- Switch to main & verify
- Branches menu
  - Choose merge branch
  - Select branch
  - Merge commit
- Then push origin



# Pruning Branch

Tidy up



Now we can simply delete that branch. We don't need it anymore.

- In GitHub Desktop with the branch active,
  - Go to the Branch menu and select Delete
  - Then we're prompted to delete on remote as well,
  - which is a good idea for the sake of clarity.
- There's also the option to delete from the web in the branches view,
  - but no option to also delete the remote -- that is, on the local workstation. If you leave the branch on local, and it's not on the web, you'll keep getting the suggestion to publish the branch again.

## Pruning Branch

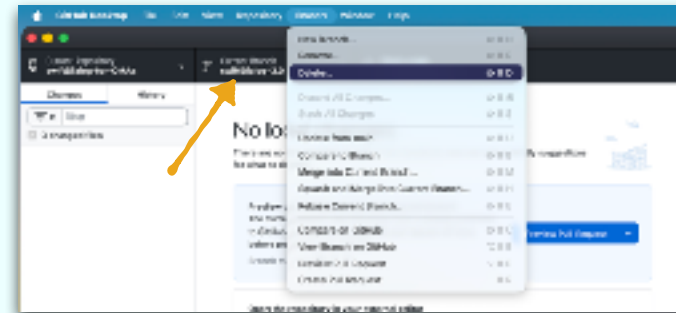
## Tidy up



# Pruning Branch

Tidy up

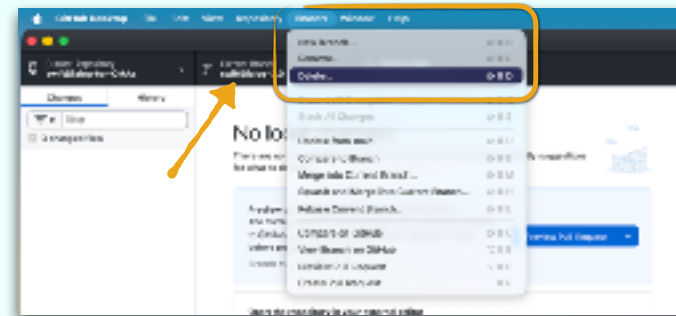
- GitHub Desktop



# Pruning Branch

## Tidy up

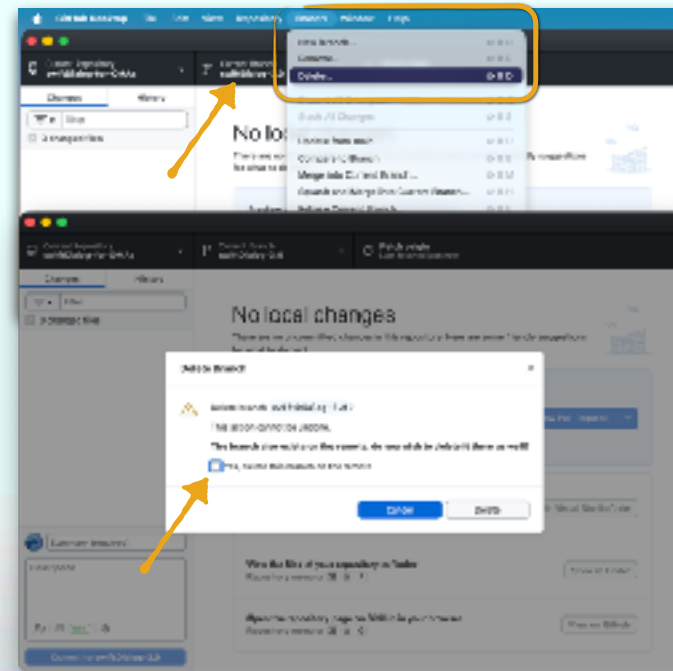
- GitHub Desktop
- Branch menu, Delete



# Pruning Branch

## Tidy up

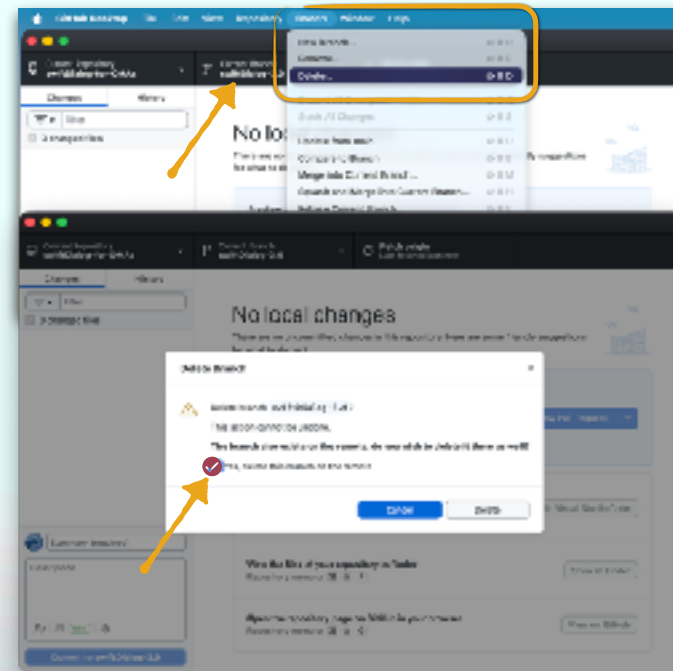
- GitHub Desktop
- Branch menu, Delete
- Option to delete on remote



# Pruning Branch

## Tidy up

- GitHub Desktop
- Branch menu, Delete
- Option to delete on remote

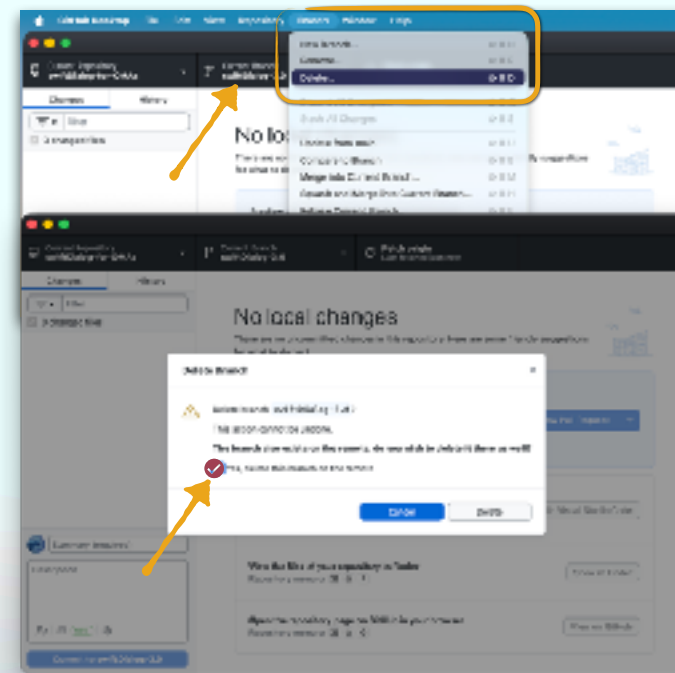




# Pruning Branch

## Tidy up

- GitHub Desktop
- Branch menu, Delete
- Option to delete on remote
- GitHub web: branches view

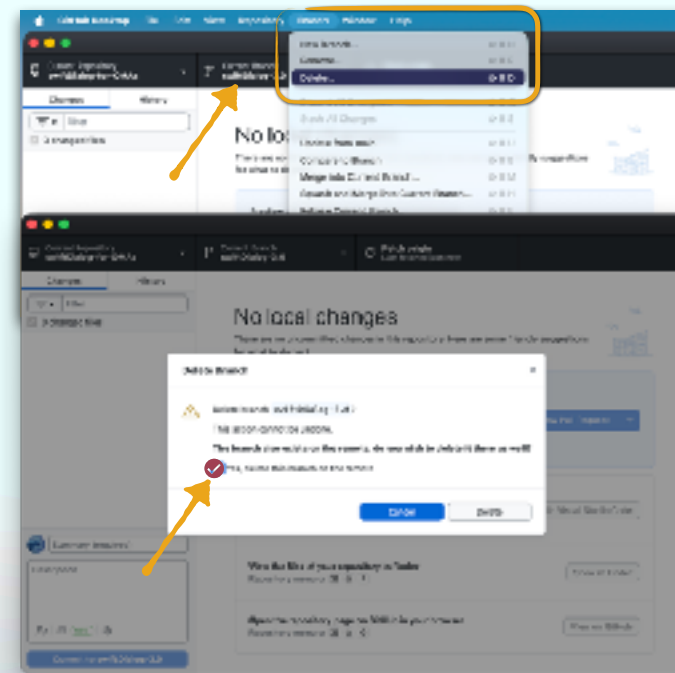


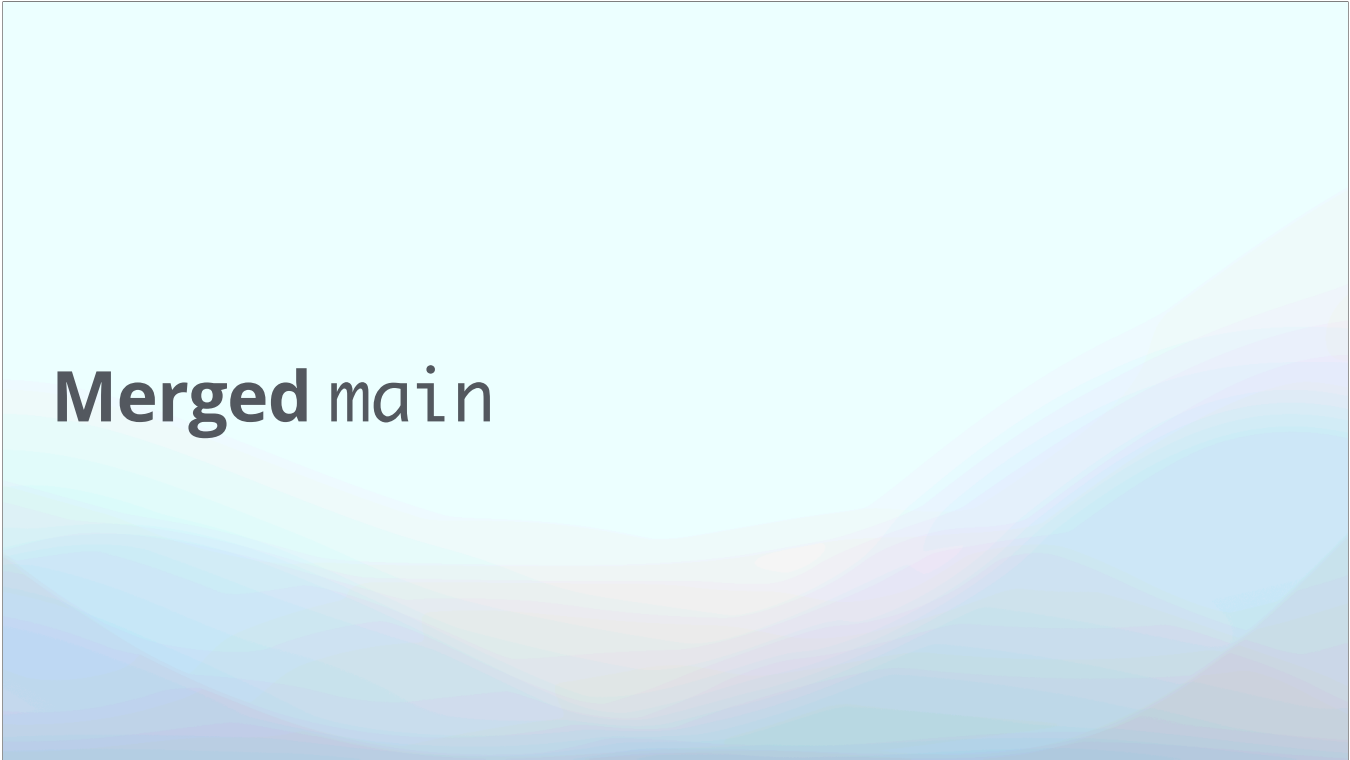


# Pruning Branch

## Tidy up

- GitHub Desktop
- Branch menu, Delete
- Option to delete on remote
- GitHub web: branches view
- Doesn't delete locally





**Merged main**

When we check out how it looks on the web,

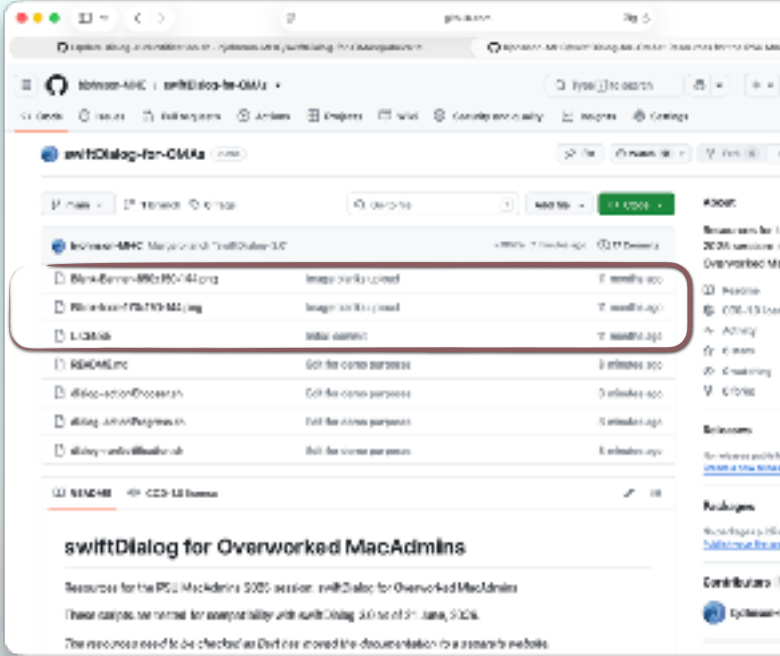
- We can see that we have both original files
- And newly updated files

In main, with a single branch.

We've got information from the merge commit about what changed, and they are all together at last!

I just did exactly this last month with my production repository, as I made a swiftDialog 3.0 branch, tested all my production scripts that use it against 3.0, and merged them into production after testing. The only change turned out to be a new comment that they had been tested with 3.0.

## A screenshot of the GitHub repository page for "swiftDialog-for-MacOs". The repository has 10 forks and 1 star. It lists several branches: main, develop, and feature branches like "fix-branch", "feature-branch", "feature-branch-2", "feature-branch-3", "feature-branch-4", "feature-branch-5", "feature-branch-6", "feature-branch-7", "feature-branch-8", "feature-branch-9", "feature-branch-10", "feature-branch-11", "feature-branch-12", "feature-branch-13", "feature-branch-14", "feature-branch-15", "feature-branch-16", "feature-branch-17", "feature-branch-18", "feature-branch-19", "feature-branch-20", "feature-branch-21", "feature-branch-22", "feature-branch-23", "feature-branch-24", "feature-branch-25", "feature-branch-26", "feature-branch-27", "feature-branch-28", "feature-branch-29", "feature-branch-30", "feature-branch-31", "feature-branch-32", "feature-branch-33", "feature-branch-34", "feature-branch-35", "feature-branch-36", "feature-branch-37", "feature-branch-38", "feature-branch-39", "feature-branch-40", "feature-branch-41", "feature-branch-42", "feature-branch-43", "feature-branch-44", "feature-branch-45", "feature-branch-46", "feature-branch-47", "feature-branch-48", "feature-branch-49", "feature-branch-50", "feature-branch-51", "feature-branch-52", "feature-branch-53", "feature-branch-54", "feature-branch-55", "feature-branch-56", "feature-branch-57", "feature-branch-58", "feature-branch-59", "feature-branch-60", "feature-branch-61", "feature-branch-62", "feature-branch-63", "feature-branch-64", "feature-branch-65", "feature-branch-66", "feature-branch-67", "feature-branch-68", "feature-branch-69", "feature-branch-70", "feature-branch-71", "feature-branch-72", "feature-branch-73", "feature-branch-74", "feature-branch-75", "feature-branch-76", "feature-branch-77", "feature-branch-78", "feature-branch-79", "feature-branch-80", "feature-branch-81", "feature-branch-82", "feature-branch-83", "feature-branch-84", "feature-branch-85", "feature-branch-86", "feature-branch-87", "feature-branch-88", "feature-branch-89", "feature-branch-90", "feature-branch-91", "feature-branch-92", "feature-branch-93", "feature-branch-94", "feature-branch-95", "feature-branch-96", "feature-branch-97", "feature-branch-98", "feature-branch-99", "feature-branch-100". The repository also shows a commit history and a README file. The README file contains information about the project, including its purpose and how to use it. The repository is licensed under the MIT license. The repository is created by the user "johndoe". The repository has 10 forks and 1 star. The repository is part of the "johndoe" organization. The repository is located at "https://github.com/johndoe/swiftDialog-for-MacOs". The repository is updated on 2023-10-27. The repository has 10 forks and 1 star. The repository is part of the "johndoe" organization. The repository is located at "https://github.com/johndoe/swiftDialog-for-MacOs". The repository is updated on 2023-10-27.



Merged main

The screenshot shows the GitHub repository page for 'swiftDialog-for-Overworked-MacAdmins'. The repository is owned by 'kbrnsn-MAC' and has a 'main' branch. The file list shows several files, with a red box highlighting the 'Assets' section. The assets are:

| Asset Name | Size   | Created        | Download |
|------------|--------|----------------|----------|
| Assets     | 1.1 MB | 10 minutes ago | Download |
| Assets     | 1.1 MB | 10 minutes ago | Download |
| Assets     | 1.1 MB | 10 minutes ago | Download |
| Assets     | 1.1 MB | 10 minutes ago | Download |
| Assets     | 1.1 MB | 10 minutes ago | Download |
| Assets     | 1.1 MB | 10 minutes ago | Download |
| Assets     | 1.1 MB | 10 minutes ago | Download |
| Assets     | 1.1 MB | 10 minutes ago | Download |
| Assets     | 1.1 MB | 10 minutes ago | Download |
| Assets     | 1.1 MB | 10 minutes ago | Download |

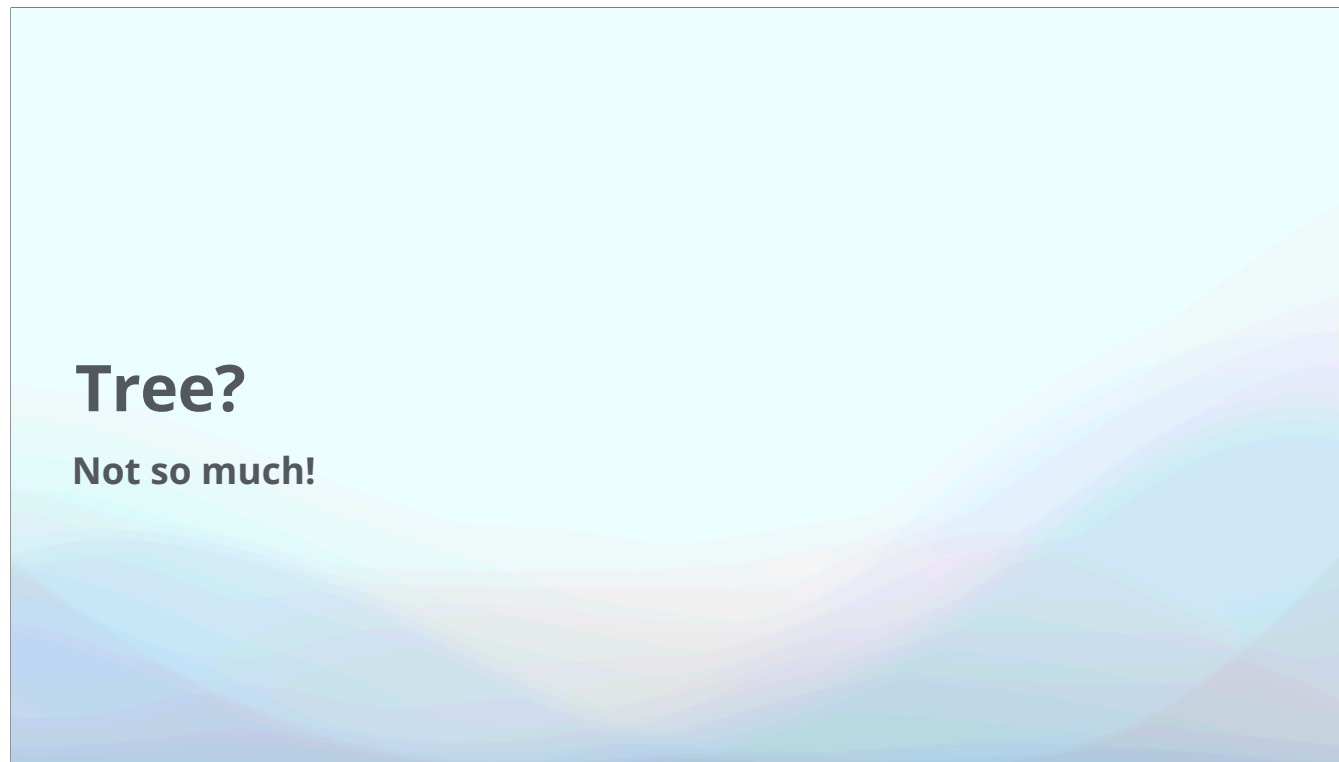
The repository description is:

**swiftDialog for Overworked MacAdmins**

Resources for the PSU MacAdmins S005 session: swiftDialog for Overworked MacAdmins

These scripts are meant for use with swiftDialog 3.0.0 as of 31 June, 2024.

The resources need to be checked as I've moved the documentation to a separate website.



Side note about the tree, branch terminology. This metaphor doesn't work for me at all. A tree's branches don't come back to the trunk, and you can't switch trunks except with grafting.

The way Git branches reminds me more of

- something from my childhood . . .

I don't know if this is just a United States thing or not, but in this classic board game you have

- a main path, with spaces to mark your position on the path, like making a commit.
- But also some branches that divert from the main path, then merge.

It's not a perfect analogy but works better for me than the tree thing.

**Tree?**  
Not so much!





# Tree?

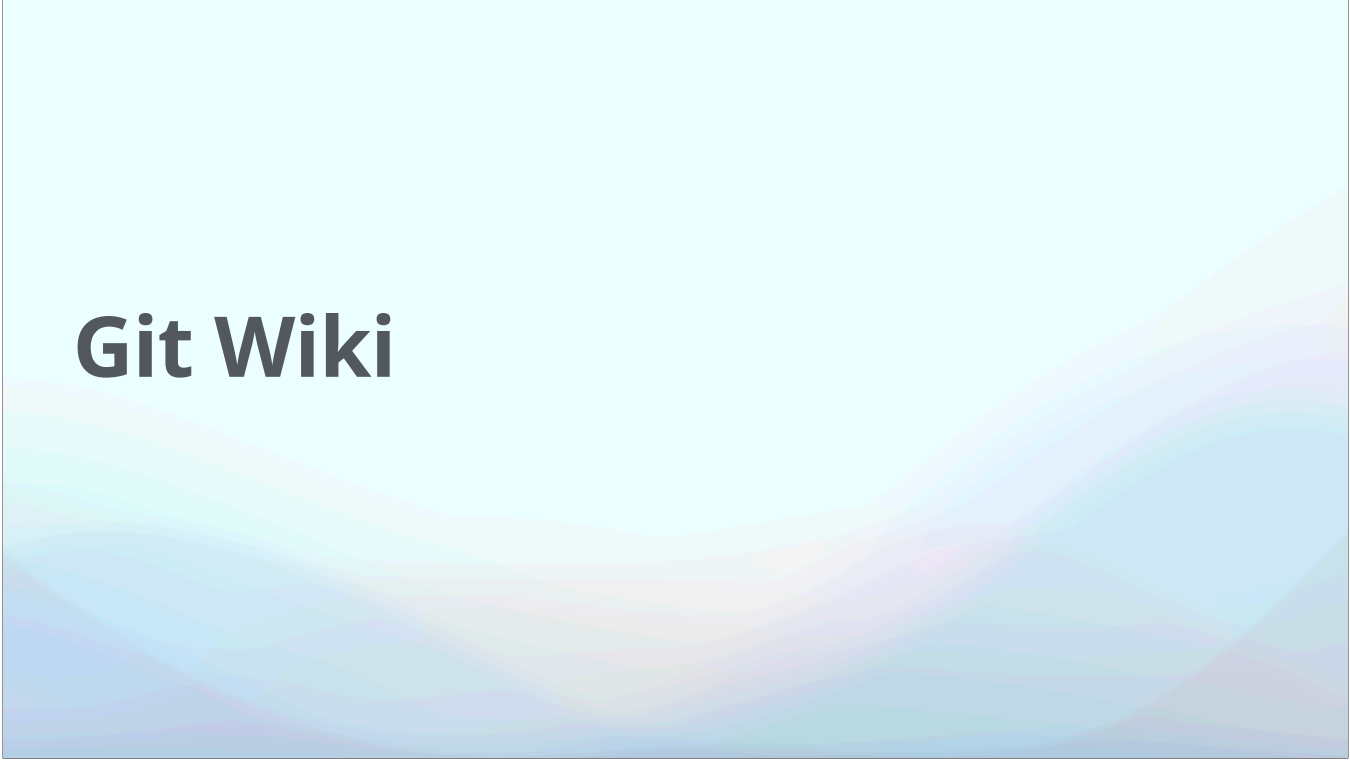
**Not so much!**





Tree?  
Not so much!





# Git Wiki

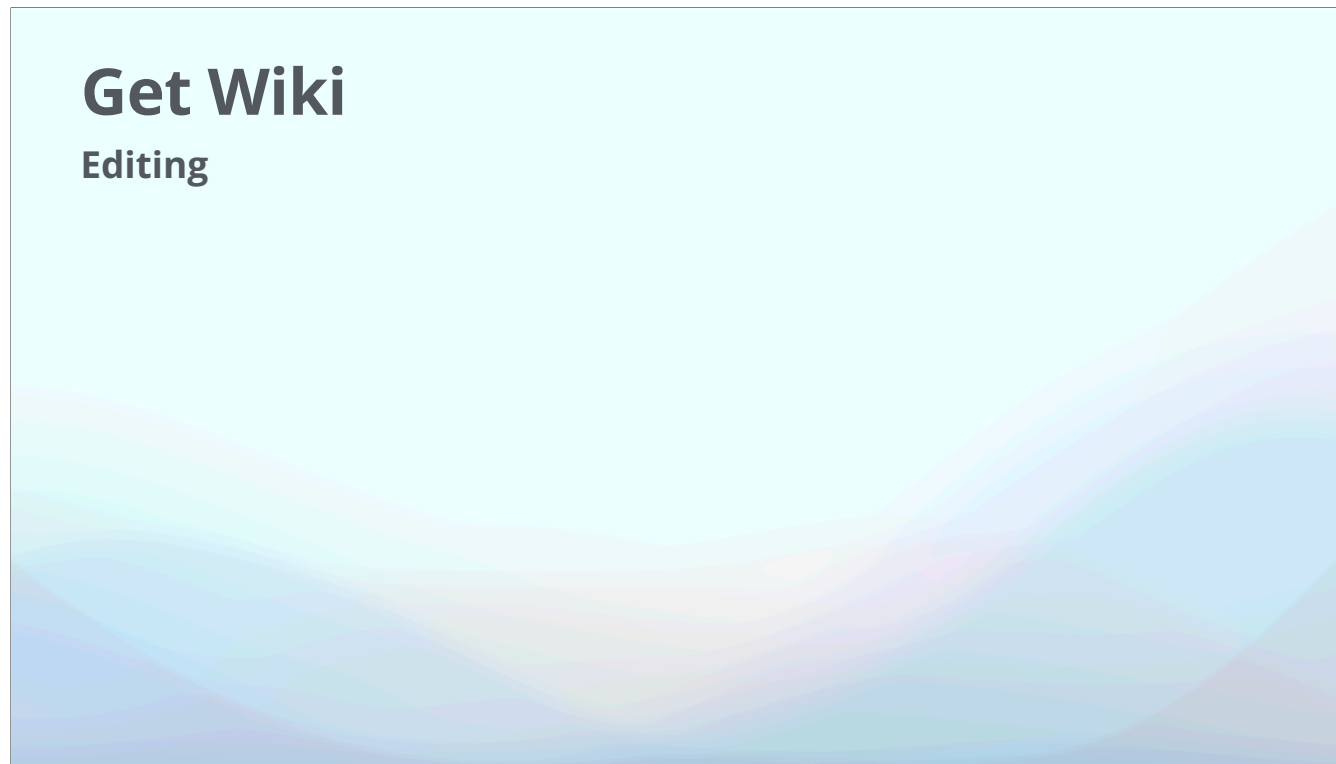
Okay, so that's two of my goals, the redundancy and the branching, done.

Another goal is to store documentation and allow other folks access to read and collaborate.

# Git Wiki



Berthe Morisot, Girl Writing, 1891



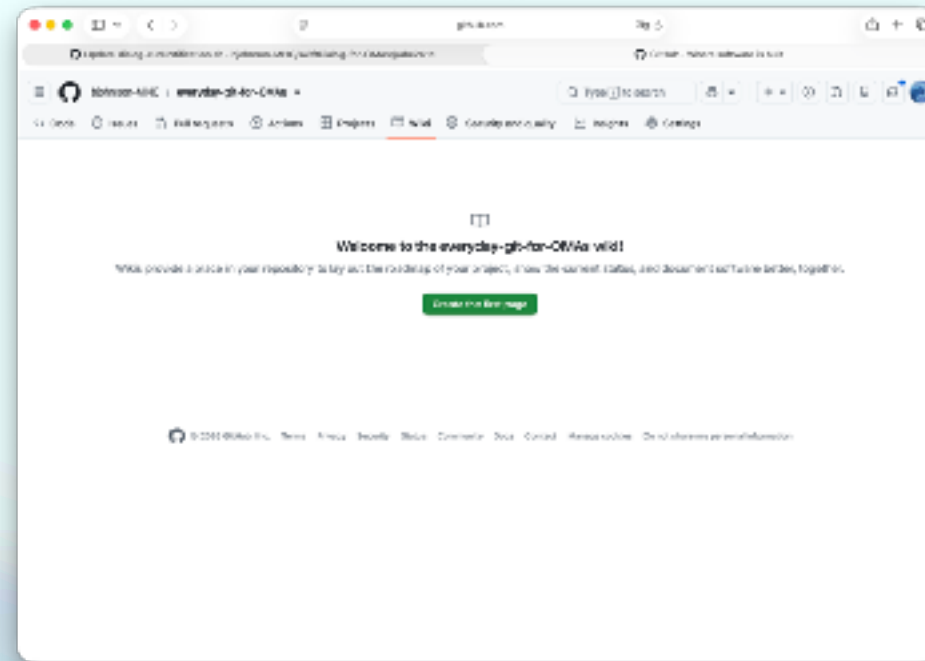
Access to a wiki varies depending on the platform and your account type.

- You can create and edit pages in the webUI
  - The format can be Markdown
  - And other formats are available
- But Markdown is pretty capable and quick for building pages.

You can totally build out a wiki online, but if you are fluent in Markdown it might be easier to just clone the Wiki down, edit, then push your changes.

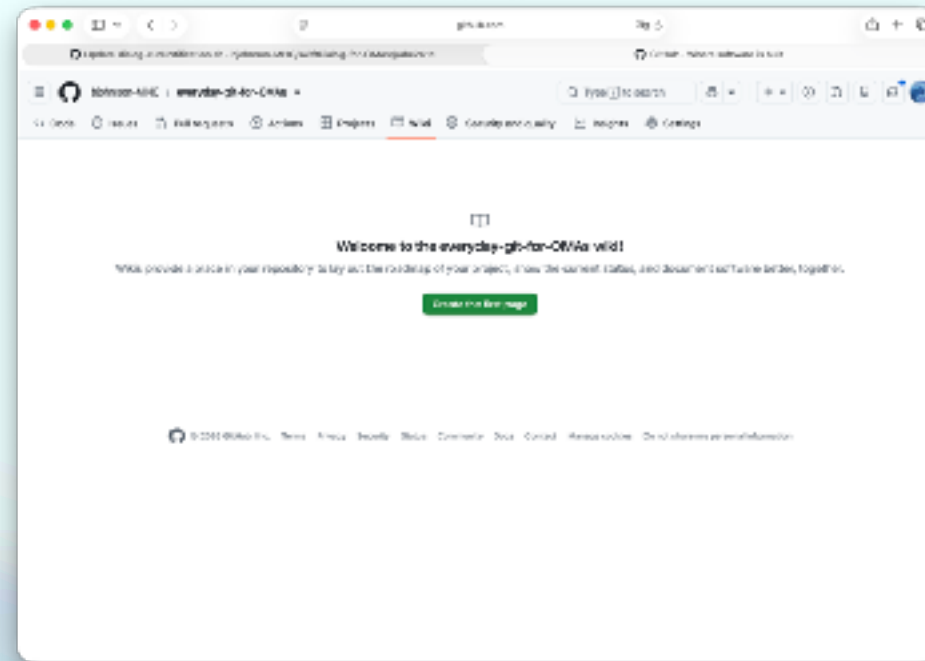
- Once you activate the wiki, you can clone it via the URL provided.

# Get Wiki Editing



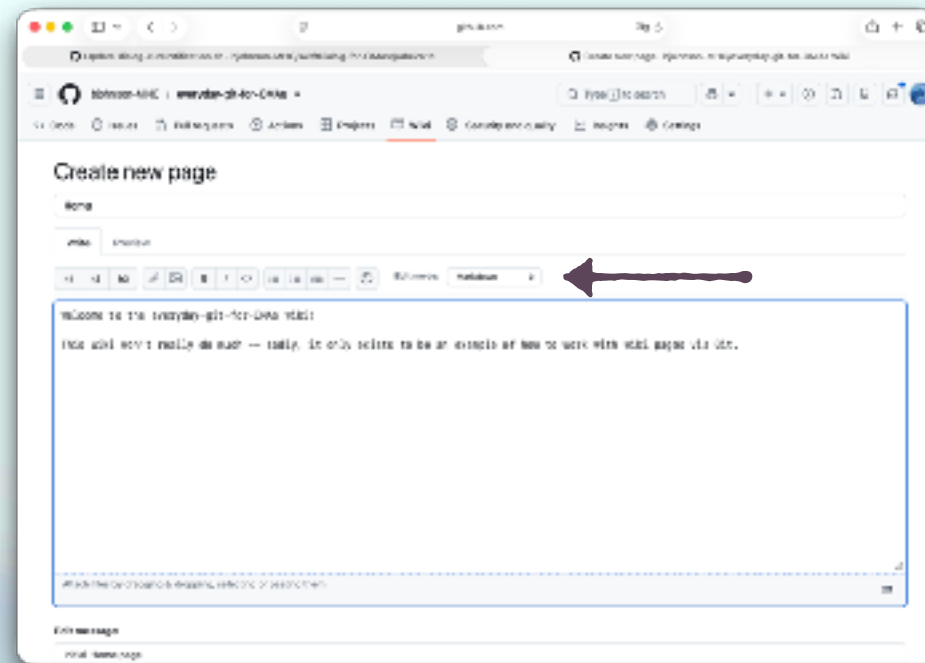
# Get Wiki Editing

- Edit in webUI



# Get Wiki Editing

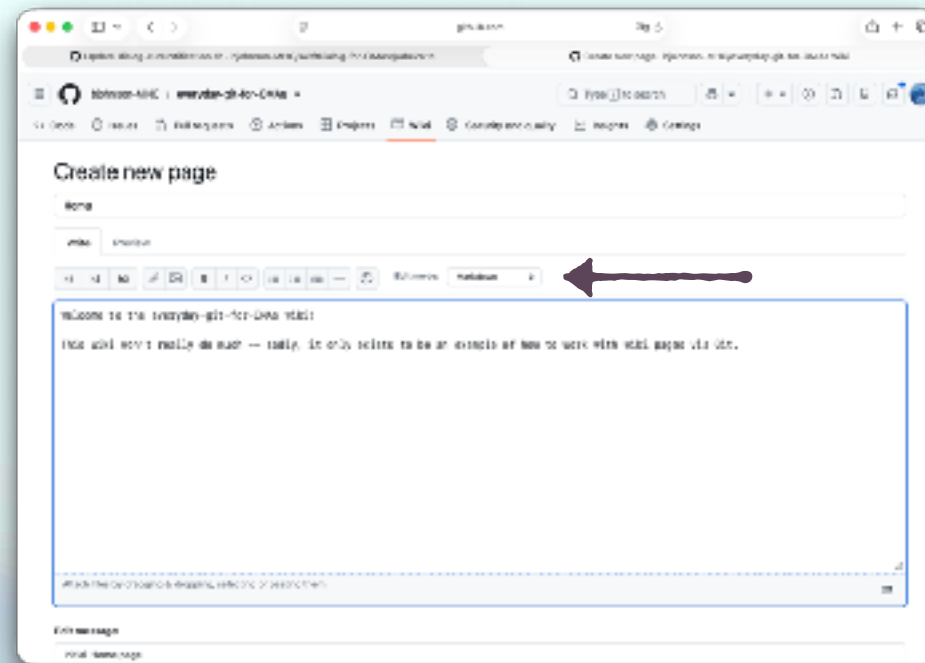
- Edit in webUI
- Markdown





# Get Wiki Editing

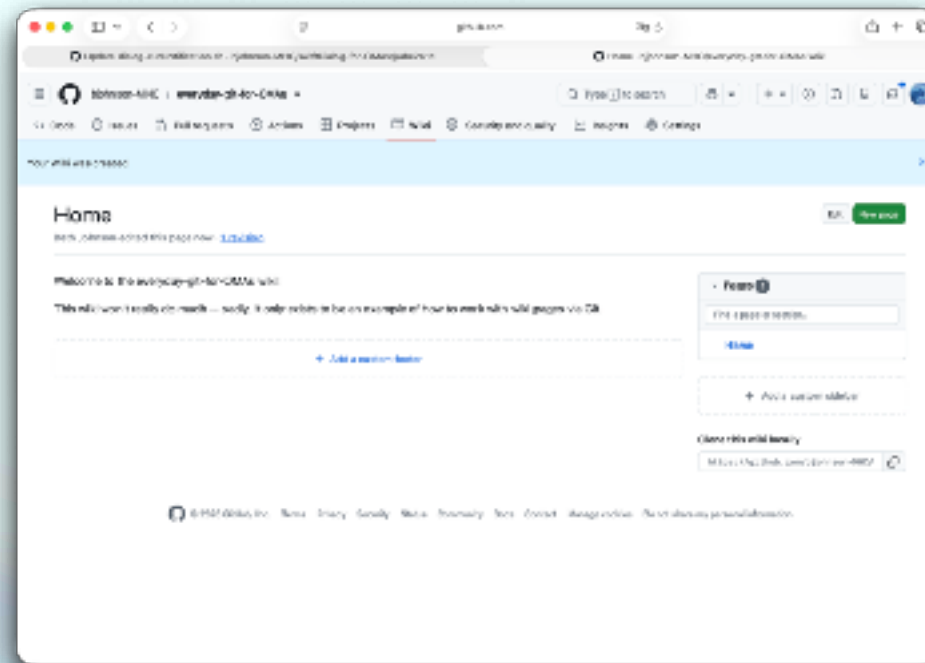
- Edit in webUI
- Markdown
- Others





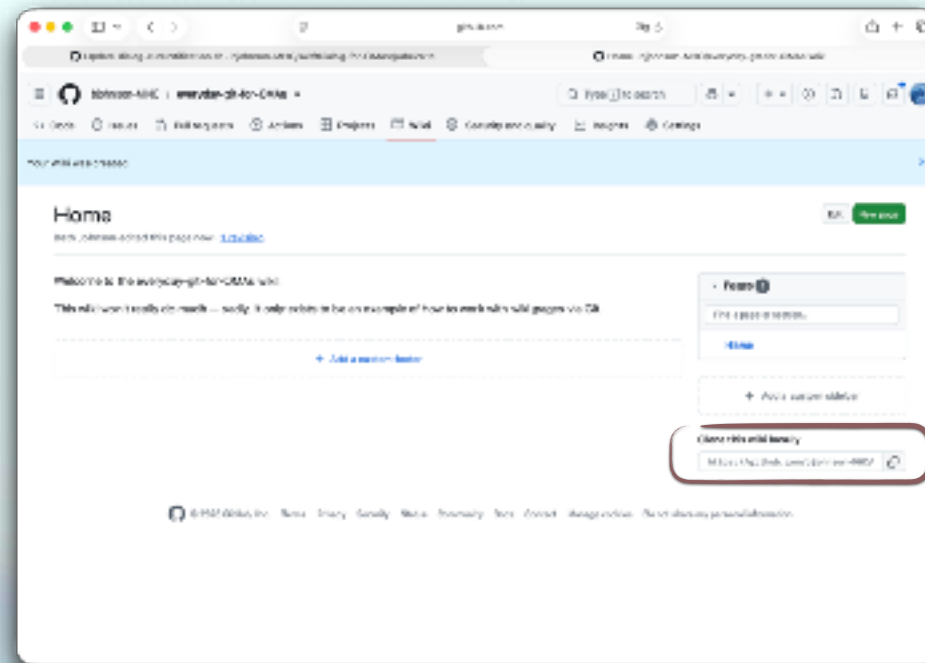
# Get Wiki Editing

- Edit in webUI
- Markdown
- Others
- Quick creation



# Get Wiki Editing

- Edit in webUI
- Markdown
- Others
- Quick creation
- Clone URL



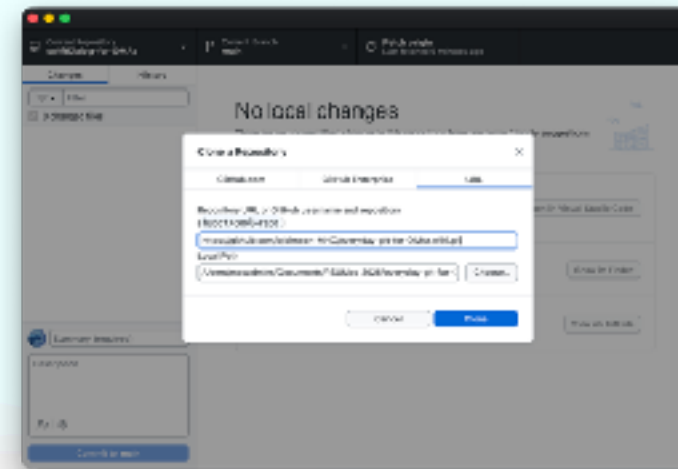
# Git Wiki

Repo by another name

- And plug that URL into GitHub Desktop's Clone a Repository window.
- The format is usually just the name of the repo with .wiki added.
- It's a repo, but it's not the same repo as your code repo.
- It will also need its own path.
- Once it's cloned, you can make commits and push back to origin.

# Git Wiki

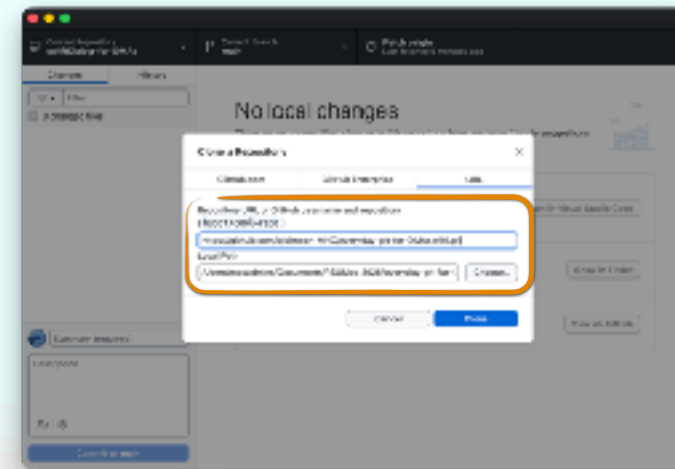
Repo by another name



# Git Wiki

## Repo by another name

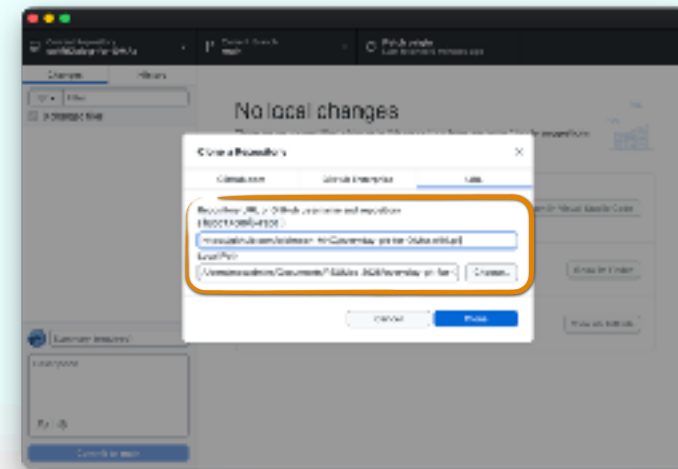
- Add via URL



# Git Wiki

## Repo by another name

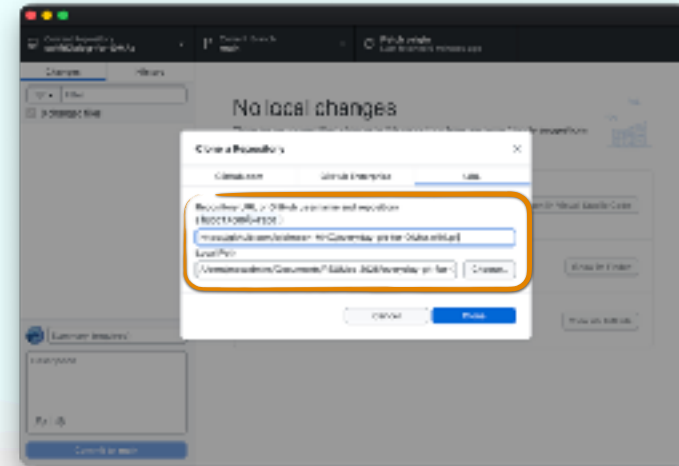
- Add via URL
  - `$yourRepo.wiki.git`



# Git Wiki

## Repo by another name

- Add via URL
  - `$yourRepo.wiki.git`
- Not included in normal clone/commit/push

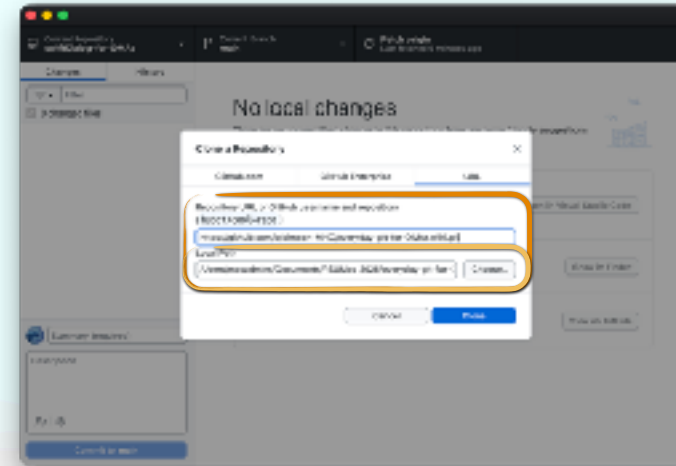




# Git Wiki

## Repo by another name

- Add via URL
  - `$yourRepo.wiki.git`
- Not included in normal clone/commit/push
- Separate local path

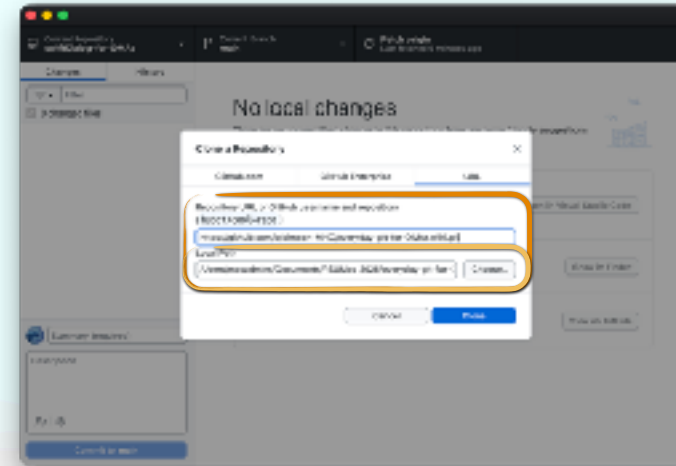


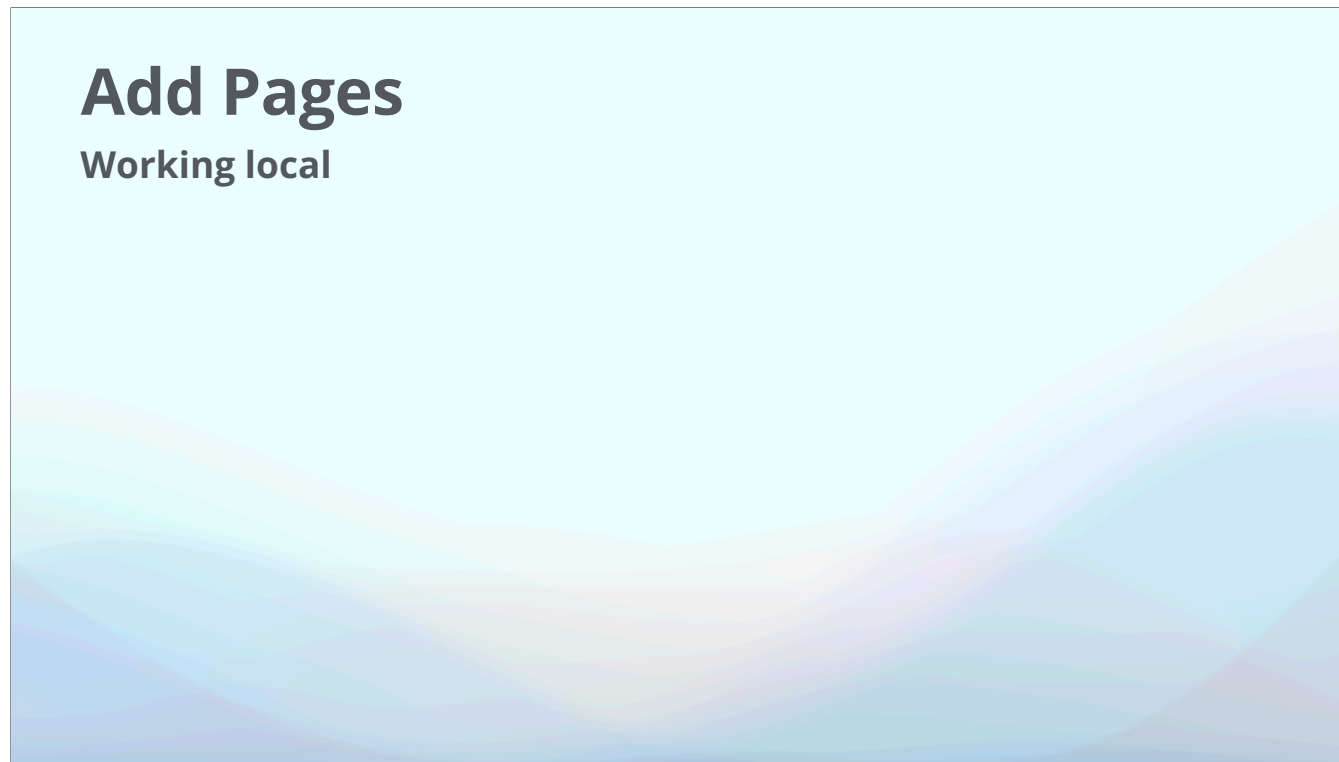


# Git Wiki

## Repo by another name

- Add via URL
  - `$yourRepo.wiki.git`
- Not included in normal clone/commit/push
- Separate local path
- Included in own clone/commit/push

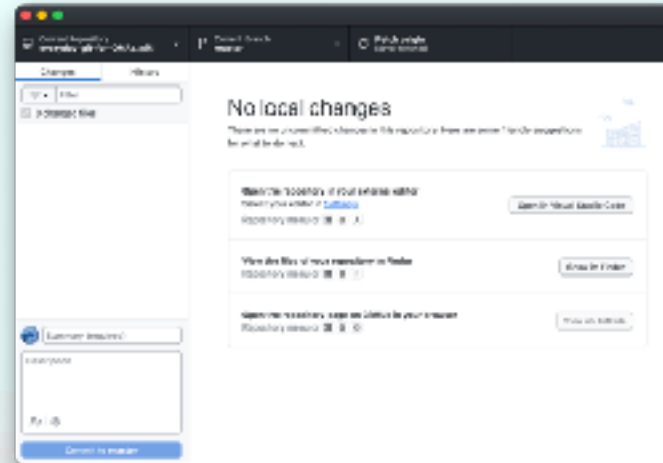




- I can open with VS Code, to directly edit the Markdown files.
  - Note though that platforms may use different flavors of Markdown. The title convention I used here is fine with GitLab
  - but GitHub doesn't know what to do with it.

# Add Pages

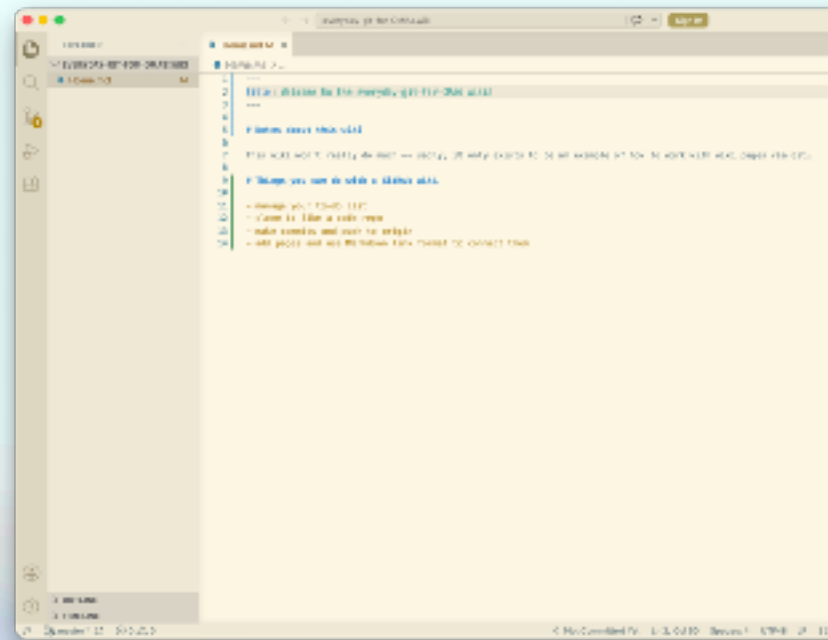
Working local



## Add Pages

## Working local

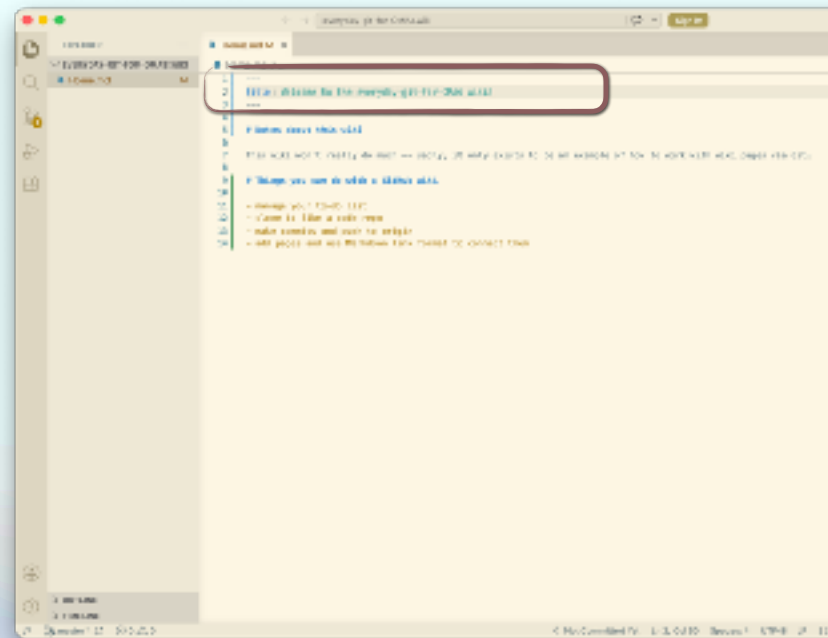
- Open in VS Code



# Add Pages

## Working local

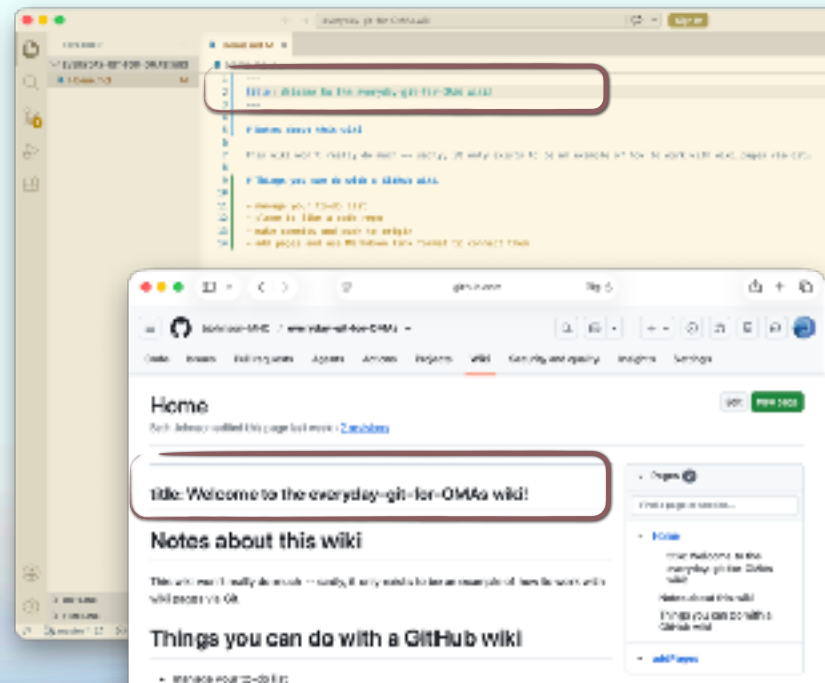
- Open in VS Code
- Markdown "flavors" vary by platform



# Add Pages

## Working local

- Open in VS Code
- Markdown "flavors" vary by platform



# Wiki Collab

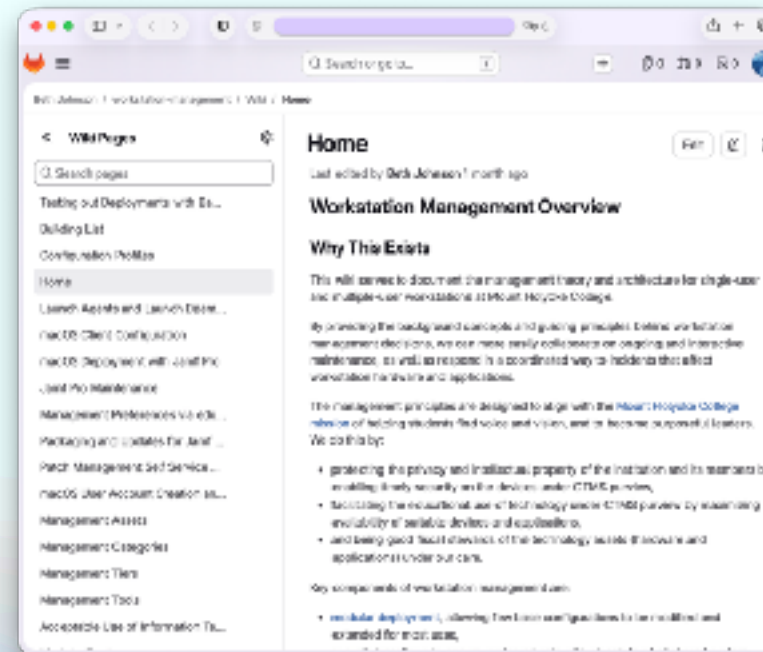


What I love about the Git based wiki is what it does for collaboration.

- I can add teammates to the repository and let them take responsibility for updating documentation.
- Hyperlinking between pages is super easy with Markdown
- It's so easy to update files and publish changes, it reduces the perceived barriers to making sure that documentation is current.
- And it looks professional! Which is more than can be said for many other documentation solutions.



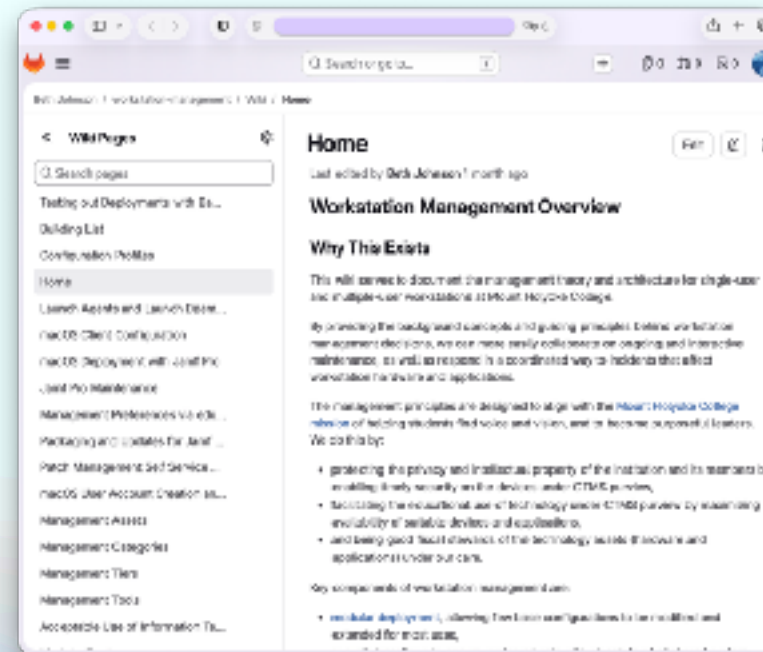
# Wiki Collab





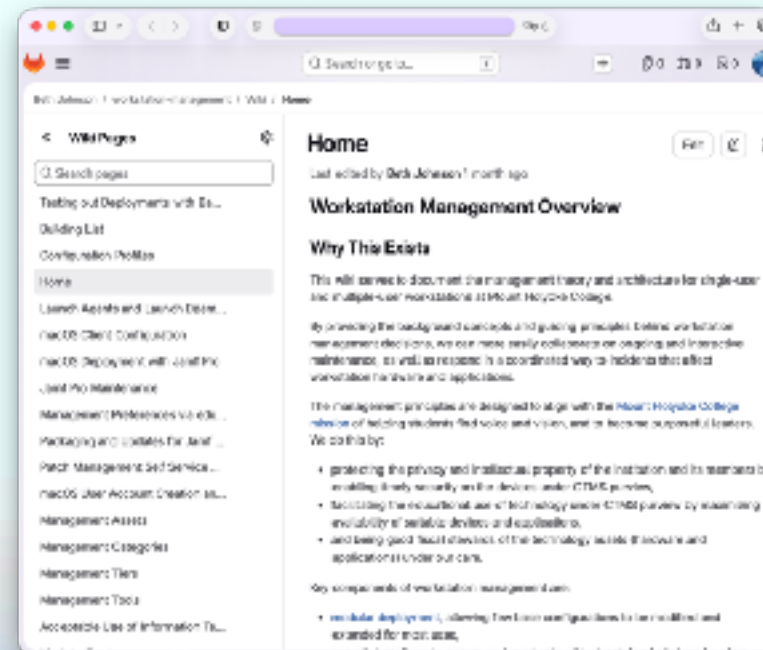
# Wiki Collab

- Sharing the load



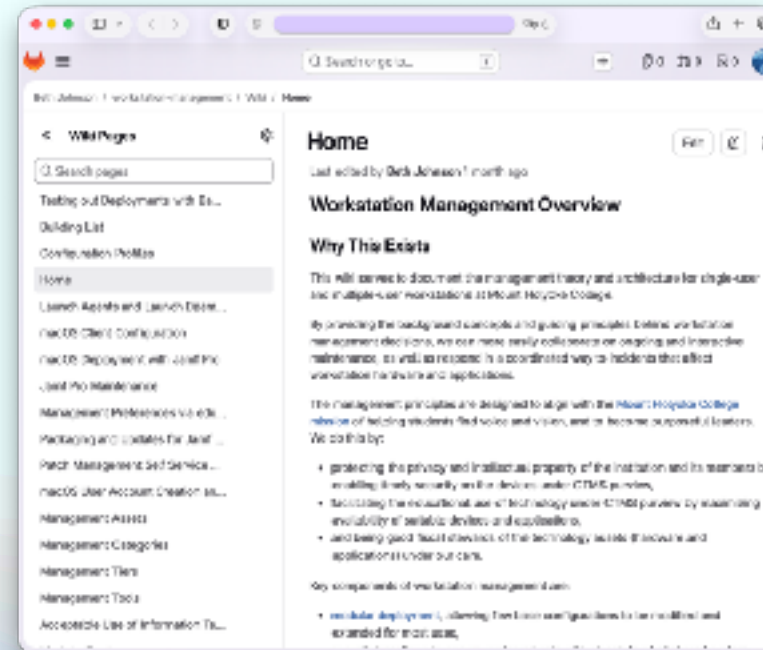
# Wiki Collab

- Sharing the load
- Simple page linking



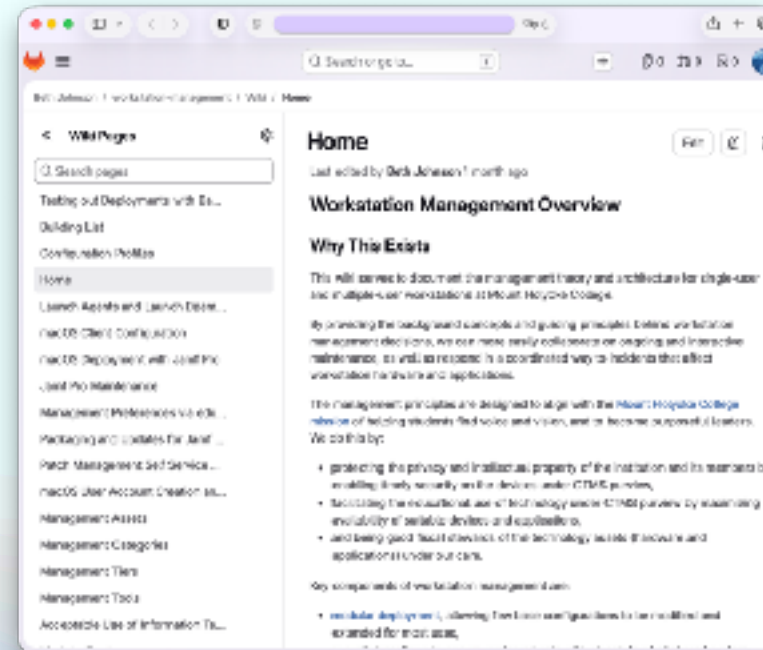
# Wiki Collab

- Sharing the load
- Simple page linking
- Reduced barriers to update documentation



# Wiki Collab

- Sharing the load
- Simple page linking
- Reduced barriers to update documentation
- Looks professional





# **Giving Back**

## **Git for Community Collabs**

Now for some bonus work, and that's how to use what I've learned about Git workflows to give back to an open source project that I've benefited from.

And what's great about this, is it's the same process that the other admins at my org can use to collaborate with me.

# Giving Back

Open source opportunity

I've been using Dan Snelson's Mac Health Check almost since he first published it. It's a great tool for end users and technicians alike.

- And there's a tiny typo in a section header that I've been fixing manually in each revision as I combine my customizations with his new releases.

But I don't have to keep doing that. It's a public repository. I can clone the repository, fix it, and request he pull in my tiny, one character change.

# Giving Back

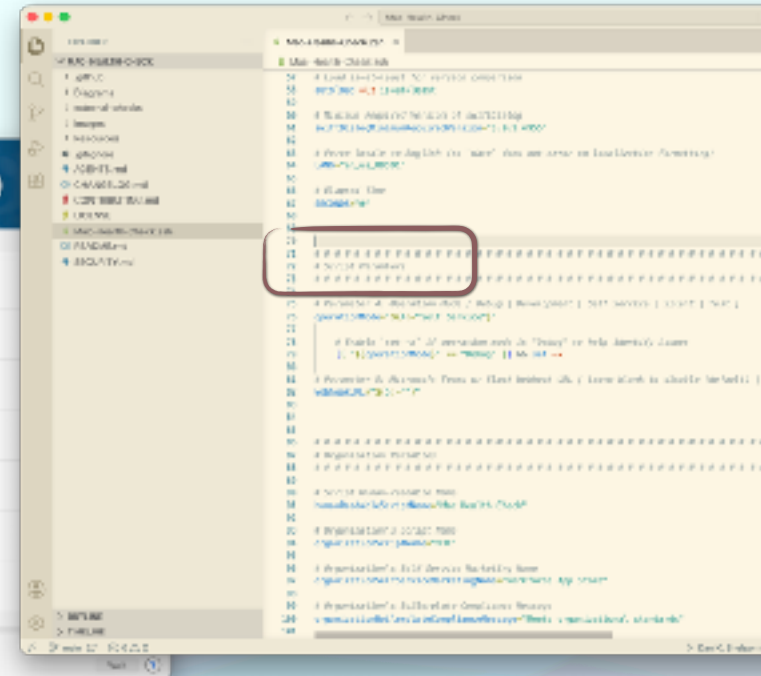
## Open source opportunity





# Giving Back

## Open source opportunity





# Giving Back

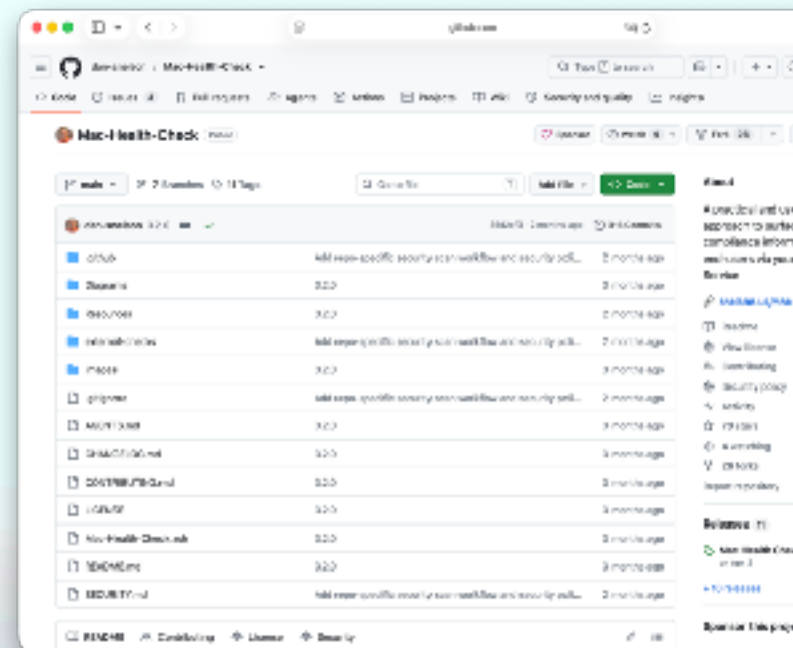
## Due diligence

First thing I want to do, though, is make sure that I'm going to contribute in a way that Dan, the repo owner, prefers to intake pull requests.

- After all, most open source project folks are volunteering their time and work to make useful stuff available to all of us, so it's the least I can do.
- Here I can see that Dan has a CONTRIBUTING file where he provides guidance on contributing to his project.
  - And when we open that document, we see that he would like pull requests against his development branch.
- Then I want to check to see if there's an open issue related to this typo -- and nothing seems related.
- Finally, I want to review any existing pull requests to see if I'm doing something that someone already did. And there's nothing there, so let's go!

# Giving Back

## Due diligence



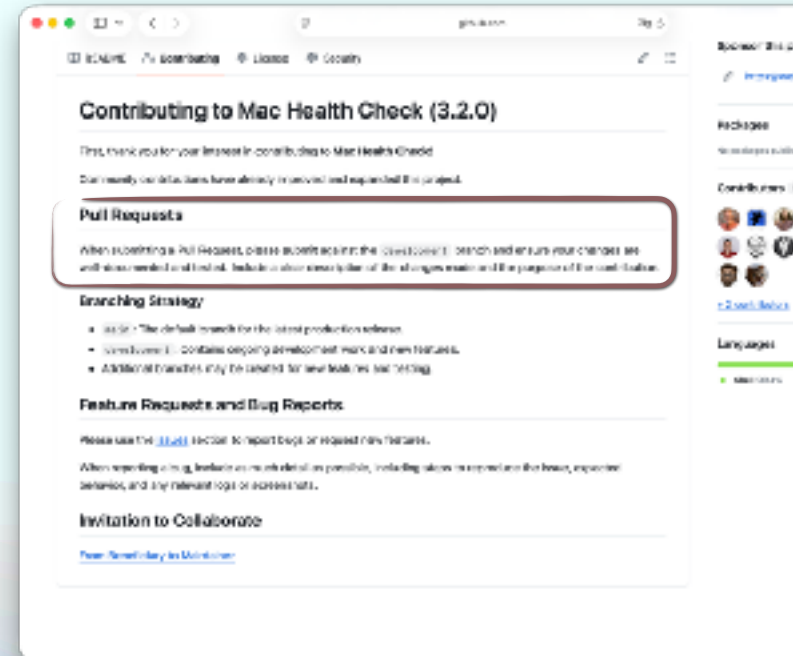




# Giving Back

## Due diligence

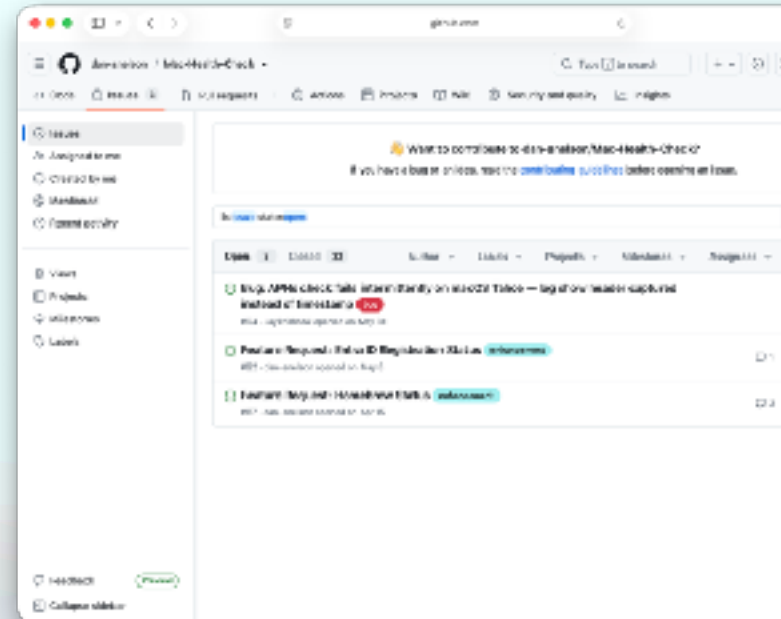
- Respect the maintainer
- Check CONTRIBUTING.MD



# Giving Back

## Due diligence

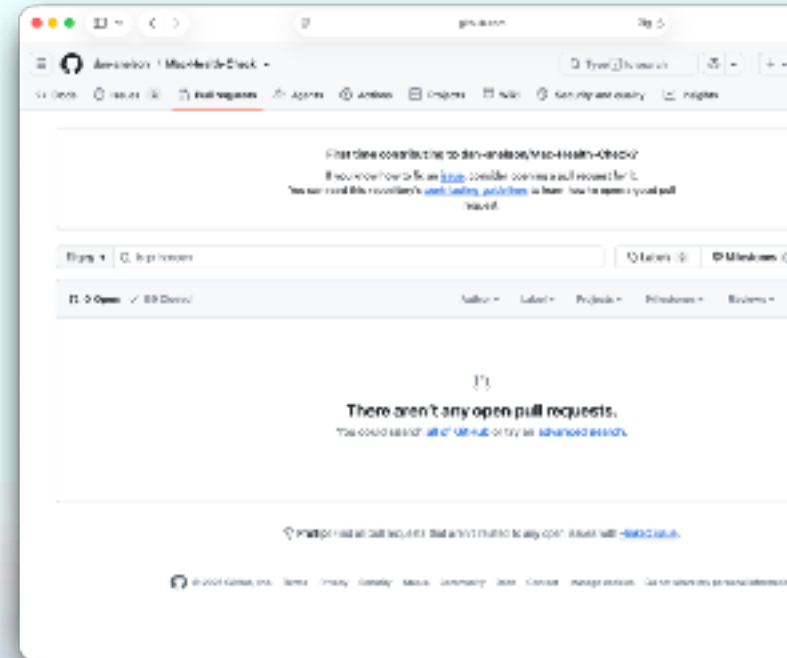
- Respect the maintainer
- Check CONTRIBUTING.MD
- Check for Issues



# Giving Back

## Due diligence

- Respect the maintainer
- Check CONTRIBUTING.MD
- Check for Issues
- Review any Pull Requests





# Giving Back

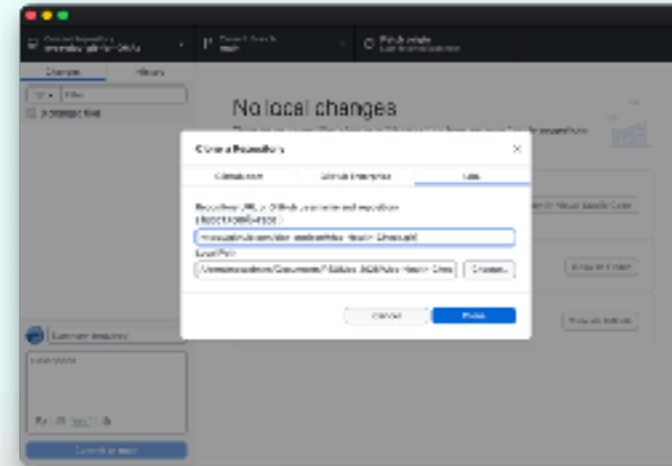
Clone and fork

- First thing is I'm going to clone the repository, which clones all the branches.
- I select or check out the development branch in GitHub Desktop. Because that's the branch for his pull requests.
- And I open it in Visual Studio Code.



# Giving Back

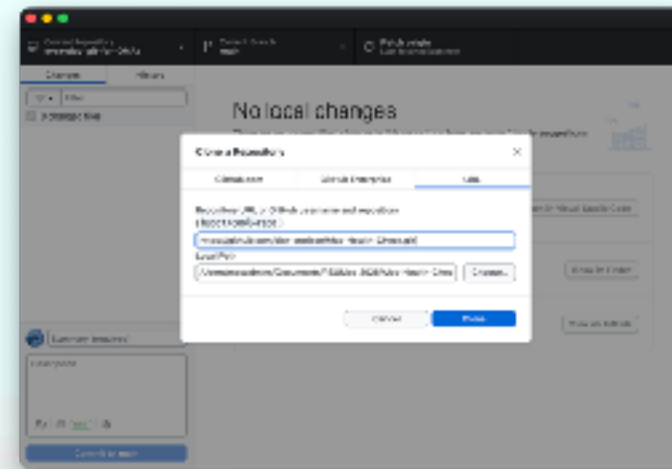
Clone and fork



# Giving Back

## Clone and fork

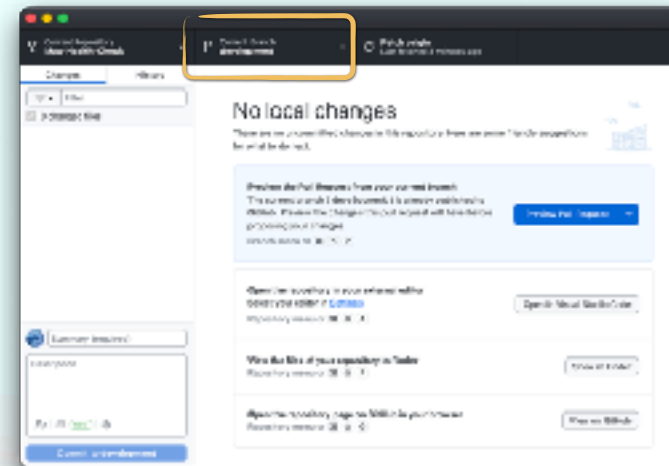
- Clone = all branches



# Giving Back

## Clone and fork

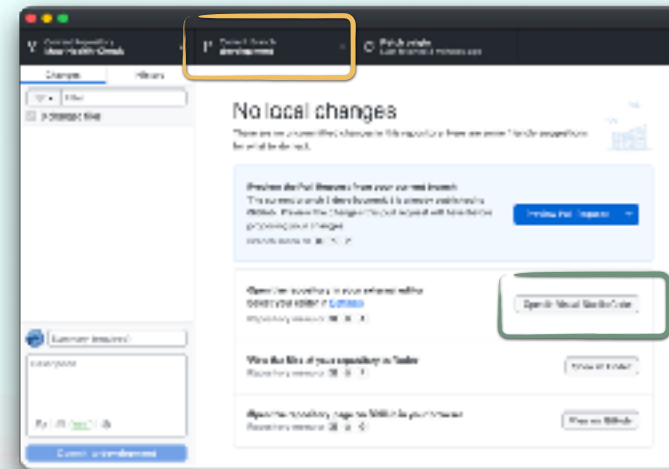
- Clone = all branches
- Select development



# Giving Back

## Clone and fork

- Clone = all branches
- Select development
- Open in Visual Studio Code



# Giving Back

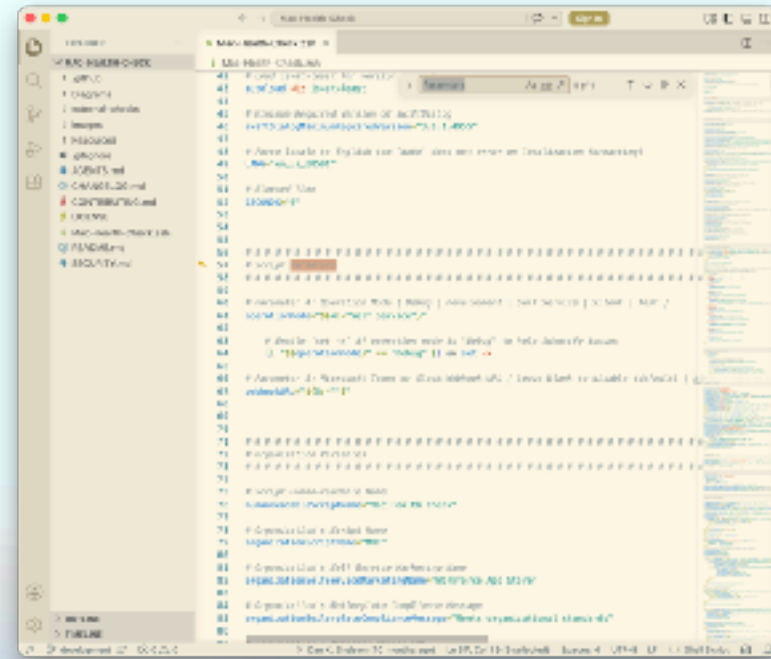
Smol typo

- I make my one tiny change (and double-check that it isn't replicated elsewhere in the script) and save the document.
- And the modified file is staged for commit.
- Before I commit, I'm notified that I can't write to the repository -- because of course I can't.
- I'm prompted to Fork the repo, which makes my own copy in my own account, with my own permissions.
- So I select Fork and am prompted for a purpose. If I select that the purpose of the fork is to contribute to the original project, GitHub will helpfully contextualize my commit and push options with that in mind.

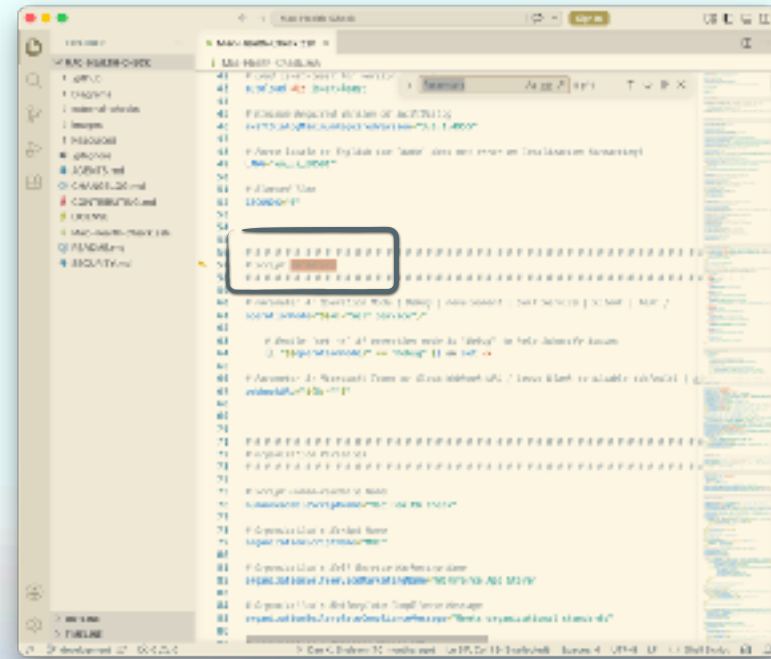
I continue ...

# Giving Back

Smol typo



- Edit script

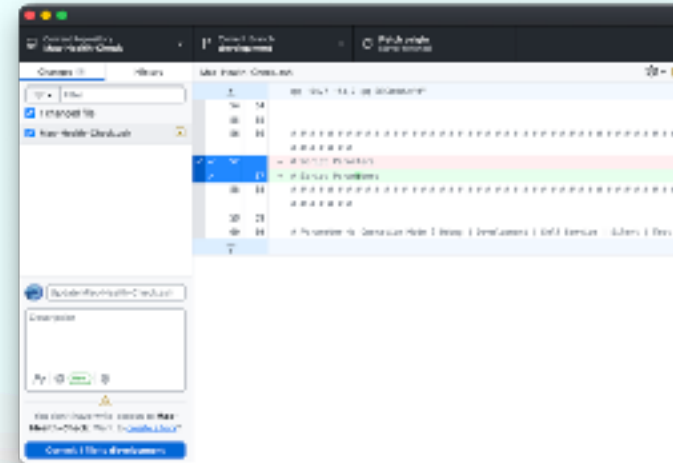




# Giving Back

## Smol typo

- Edit script
- Save and stage

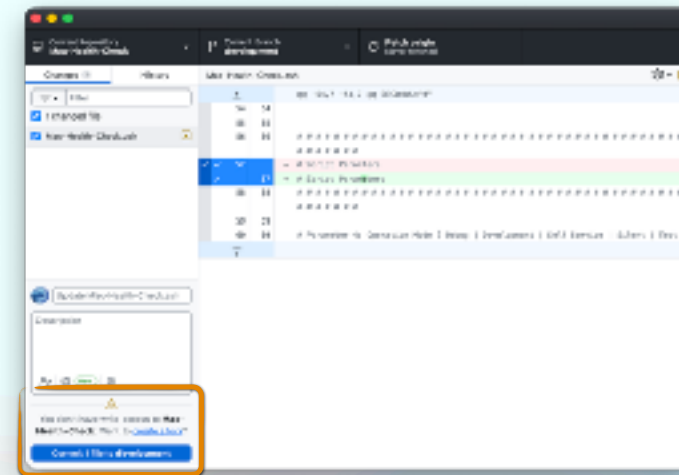




# Giving Back

## Smol typo

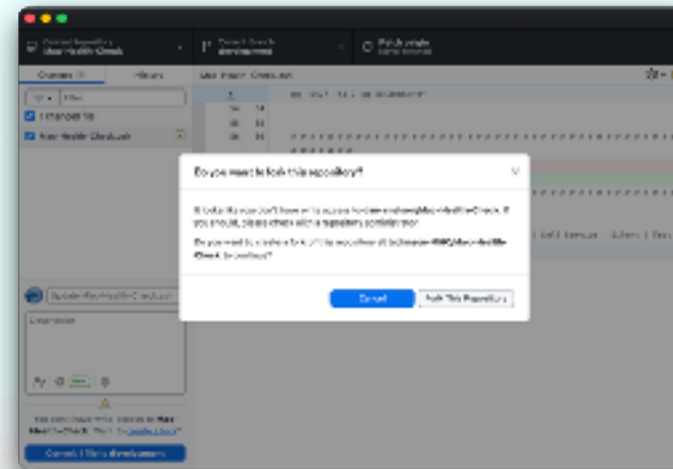
- Edit script
- Save and stage
- Can't write origin



# Giving Back

## Smol typo

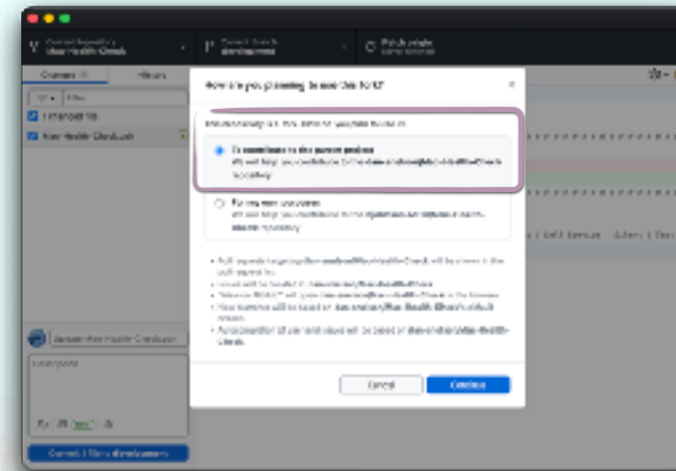
- Edit script
- Save and stage
- Can't write origin
  - Offered Fork



# Giving Back

## Smol typo

- Edit script
- Save and stage
- Can't write origin
  - Offered Fork
- Contextualizes actions



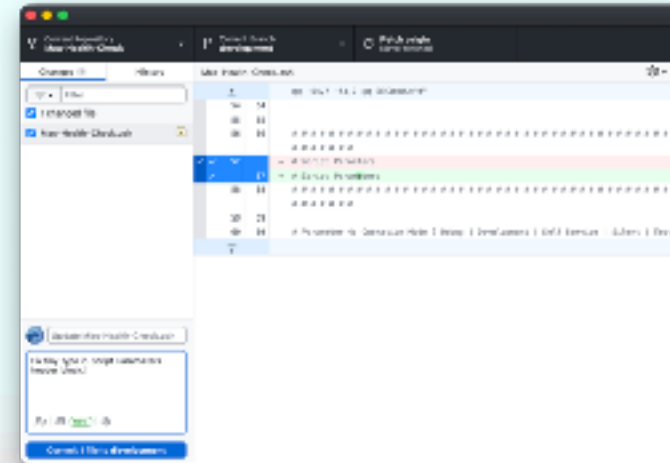
# Giving Back

Pull request!

- I'm going to make the commit with my description of changes,
- then push to origin -- which is my own copy of the repository. Then I'll be able to make the pull request.

# Giving Back

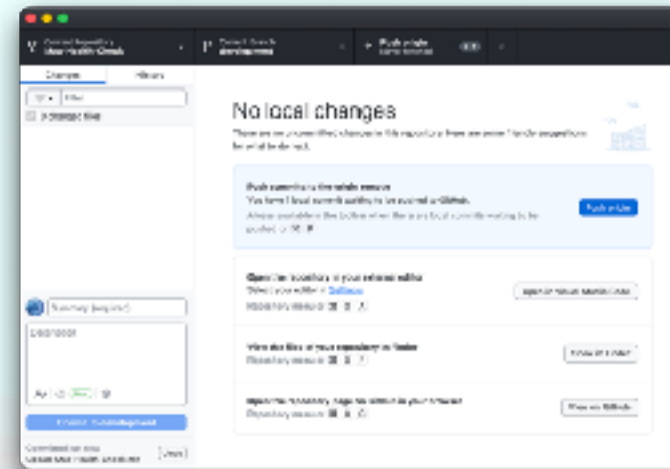
Pull request!



# Giving Back

## Pull request!

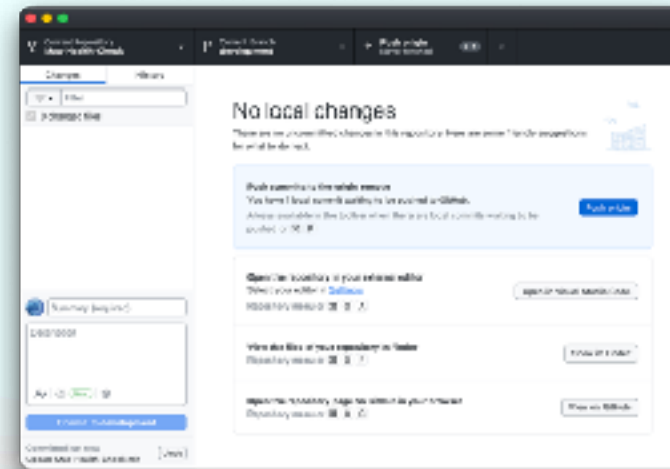
- Commit to my branch



# Giving Back

## Pull request!

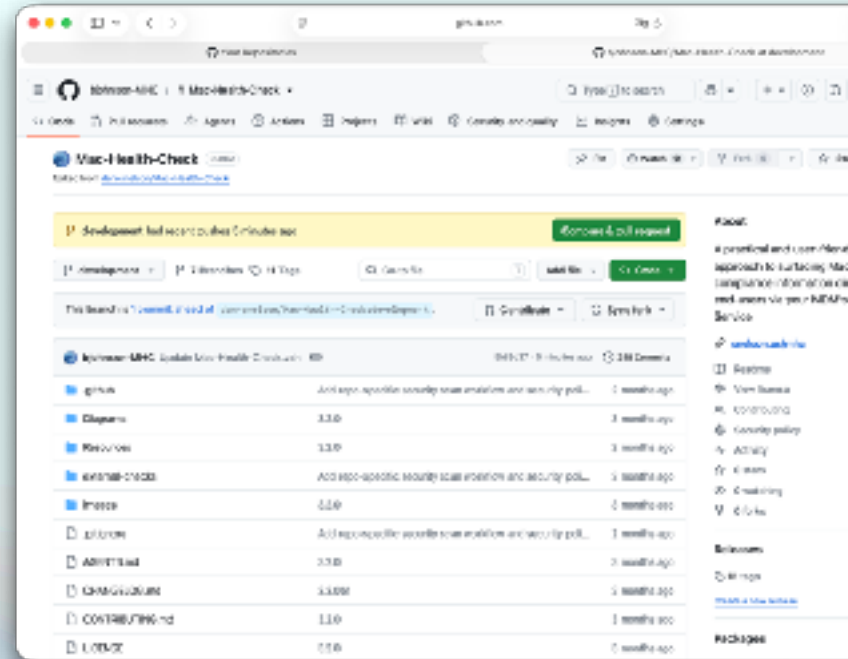
- Commit to my branch
- And push to my repo





# Giving Back

## Pull request!



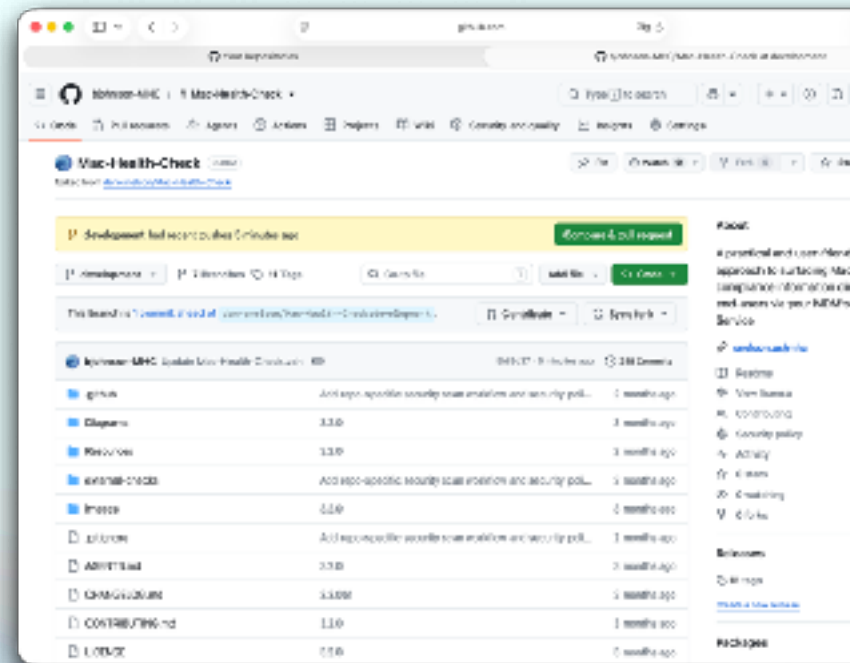
Let's take a look on the web after I push.

- Here it is in my fork.
- I'm a step ahead of his.
- From there, I can make a pull request to Dan to merge my tiny change in my copy of the development branch with his development branch. When I make this request, he'll have the diff view of the change, just like we saw when I did my own commit. This way a maintainer -- who could be you, approving or denying pull requests from your colleagues -- can see exactly what changes are being requested for addition.



# Giving Back

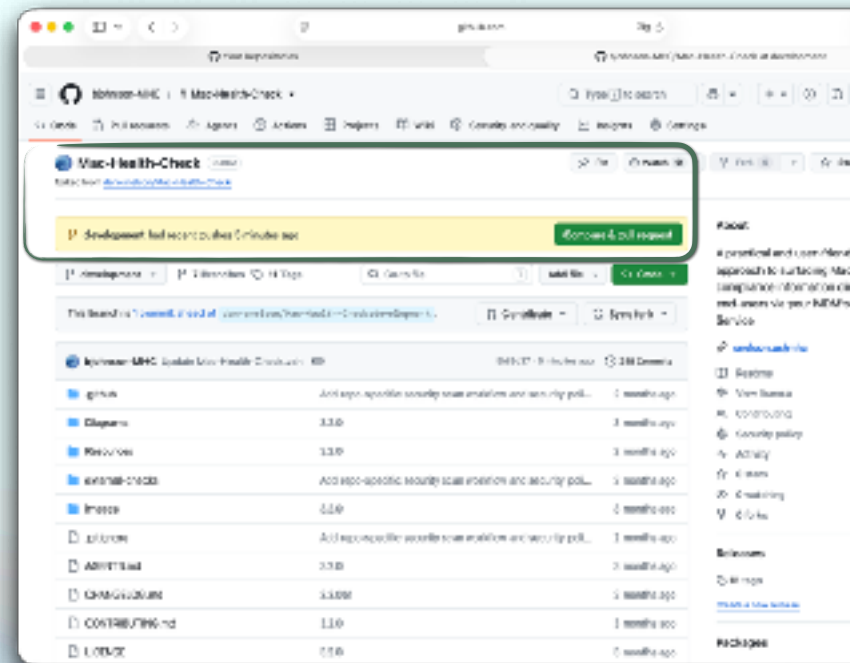
## Pull request!



# Giving Back

## Pull request!

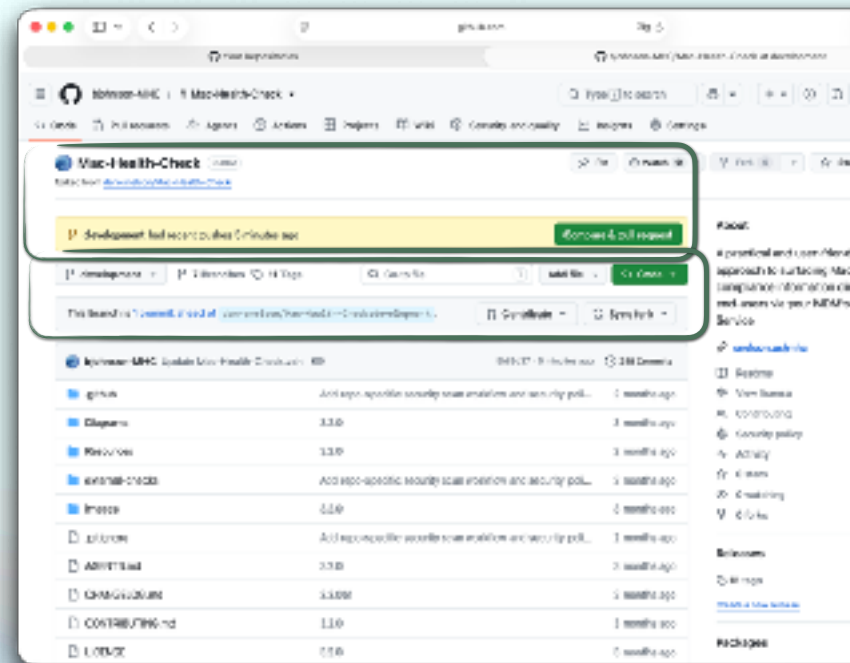
- My fork



# Giving Back

## Pull request!

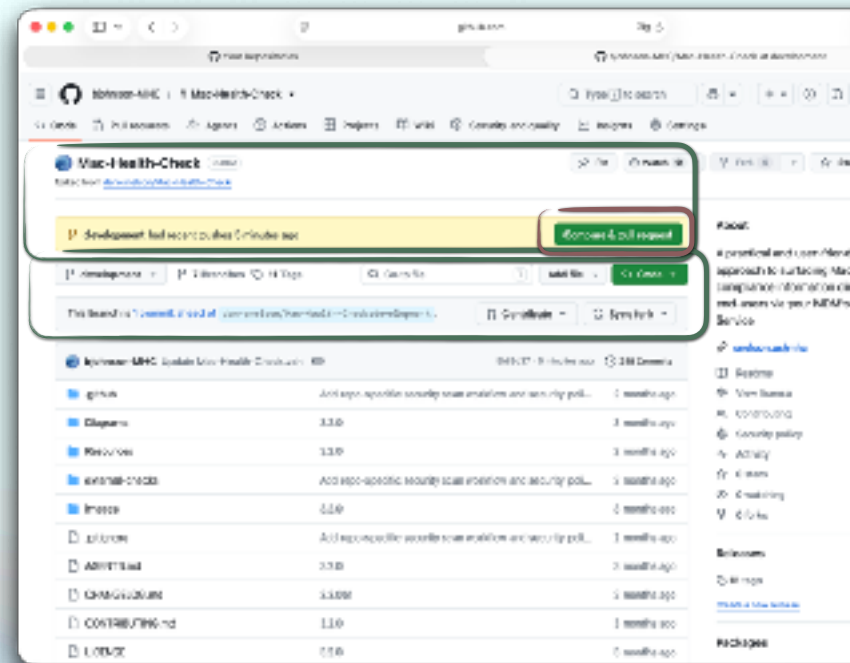
- My fork
- One commit ahead



# Giving Back

## Pull request!

- My fork
- One commit ahead
- Pull request time . . .



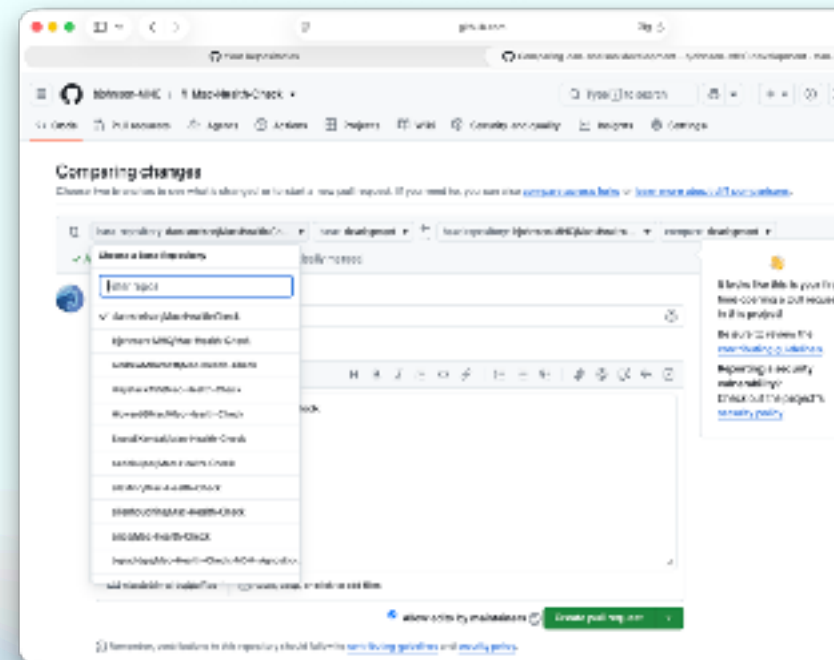
# Giving Back

## Pull Request

- I double check that target for my pull request,
- and I update my summary and description to provide more information.
- Then I submit the pull request!
- Dan's got some automations running,
- then it lands in the maintainer queue for them to approve or deny.

# Giving Back

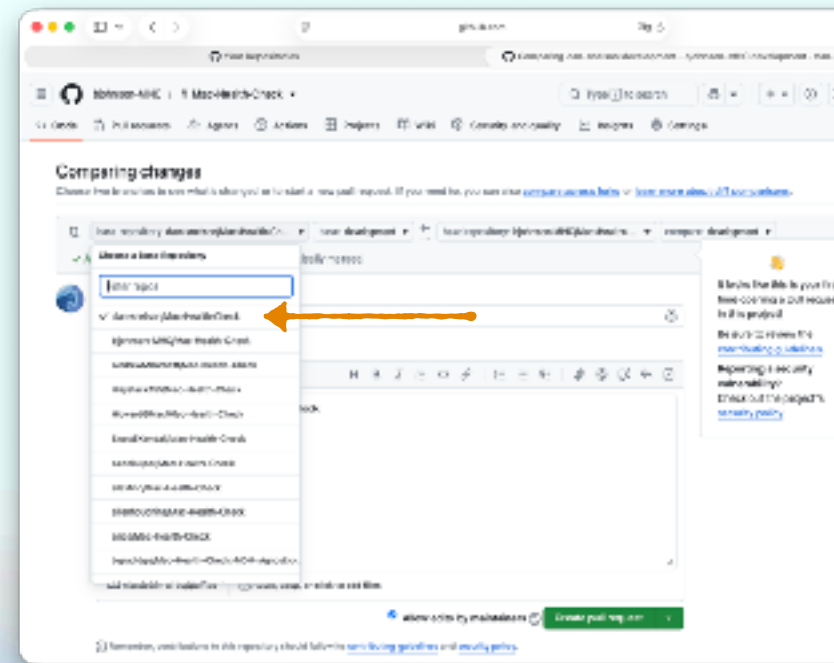
## Pull Request



# Giving Back

## Pull Request

- Double check origin

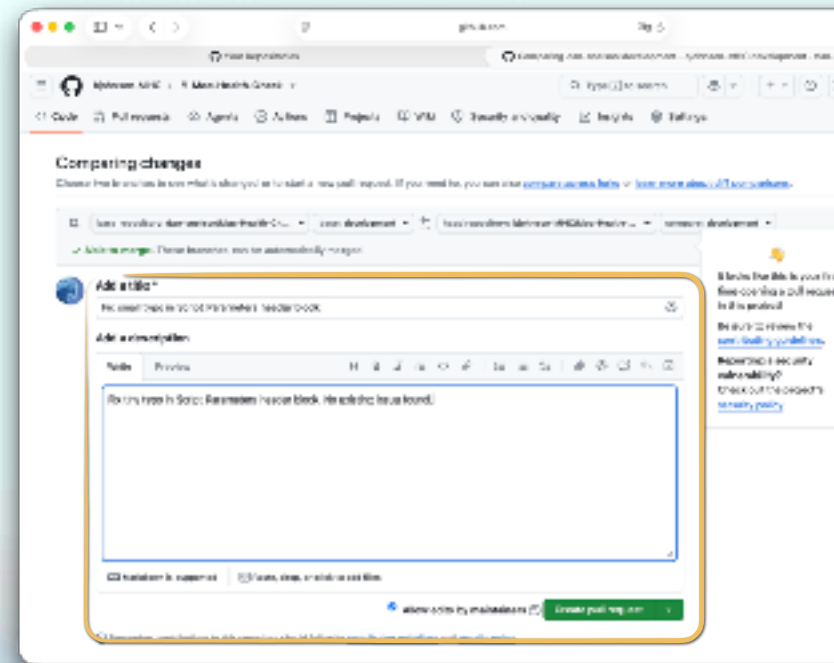




# Giving Back

## Pull Request

- Double check origin
- Summary & description

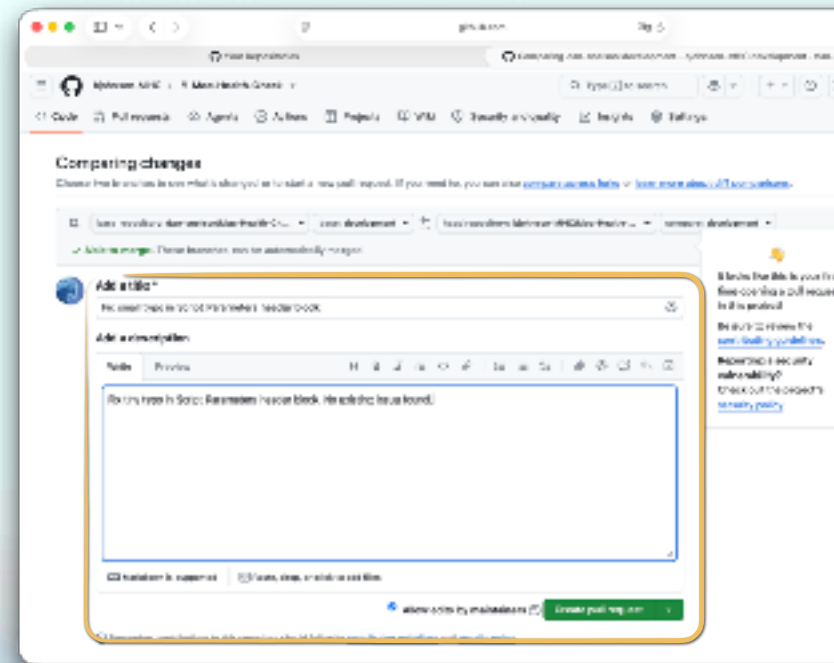




# Giving Back

## Pull Request

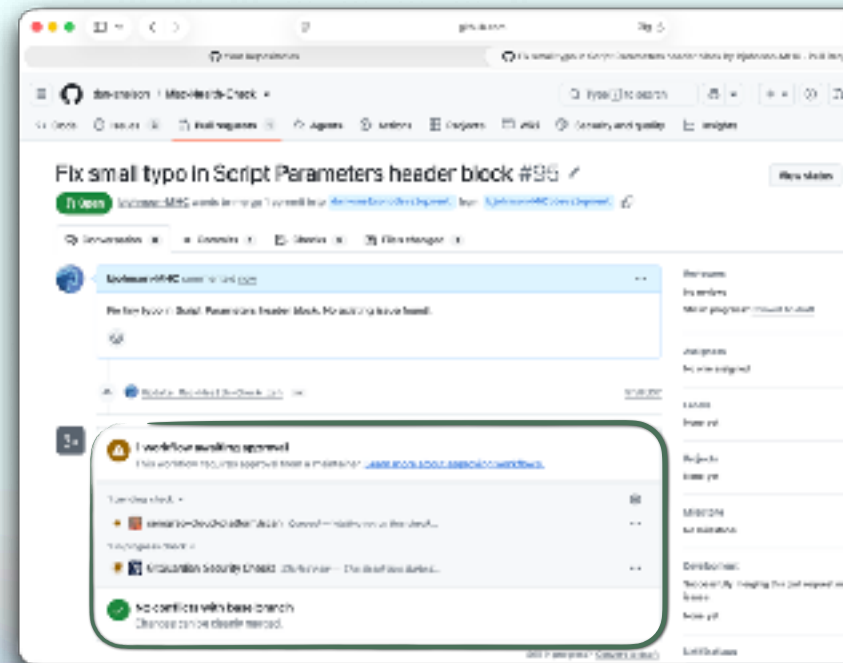
- Double check origin
- Summary & description
- Submit pull request



# Giving Back

## Pull Request

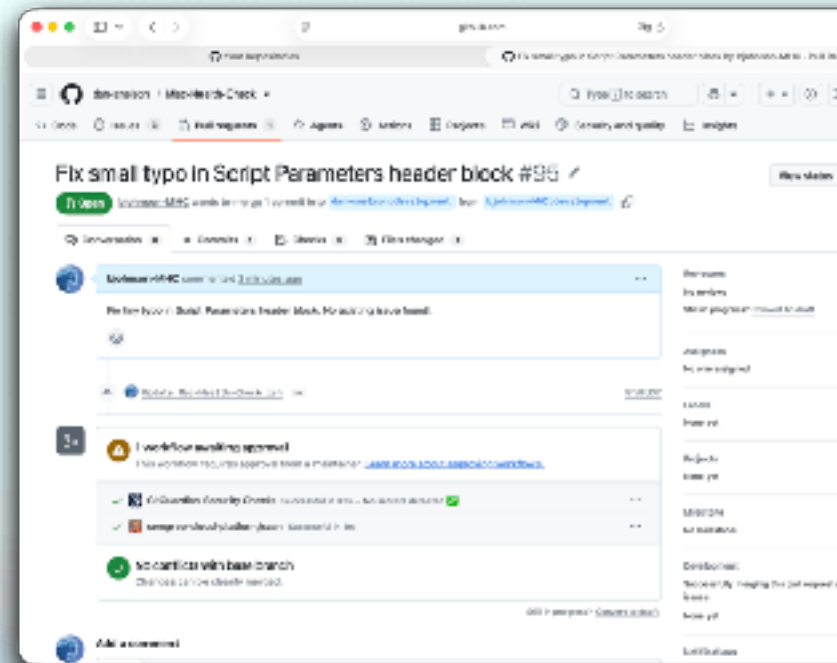
- Double check origin
- Summary & description
- Submit pull request
- Maintainer's workflow

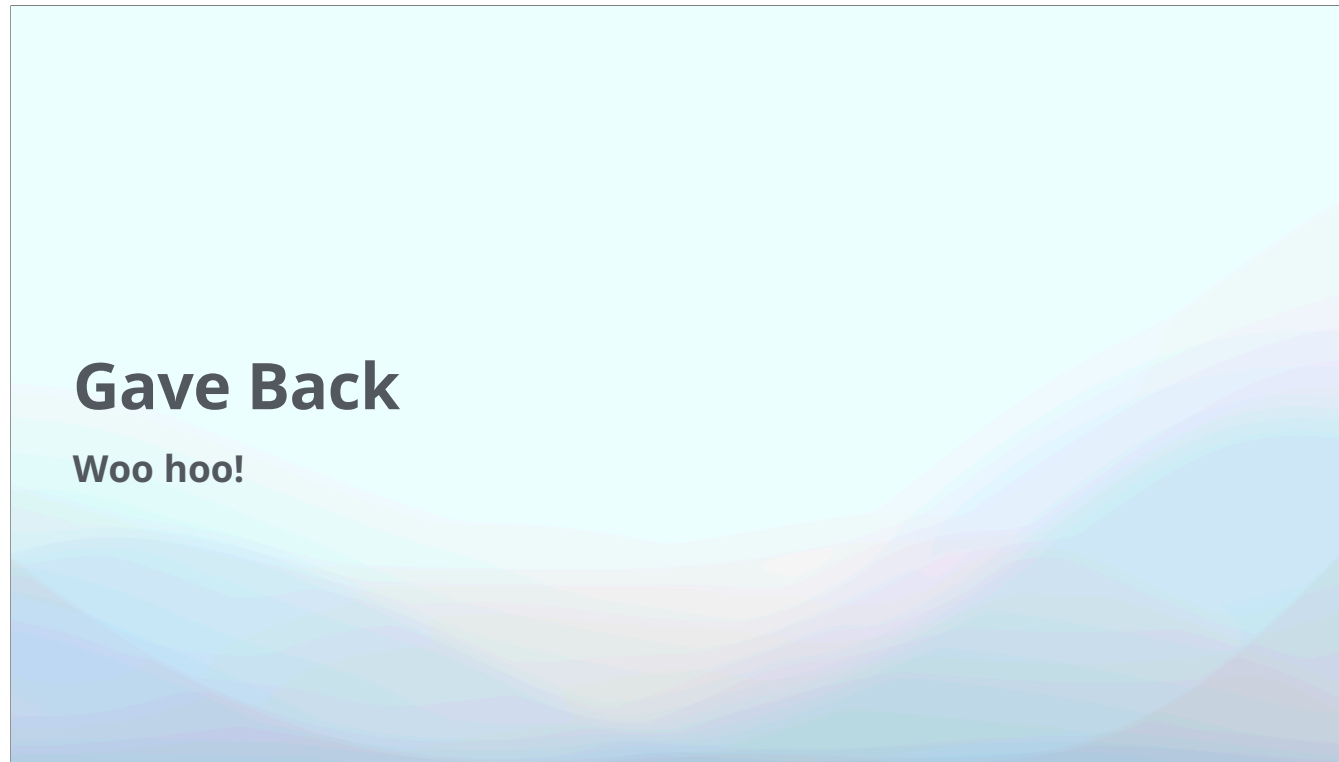


# Giving Back

## Pull Request

- Double check origin
- Summary & description
- Submit pull request
- Maintainer's workflow
- Submitted!

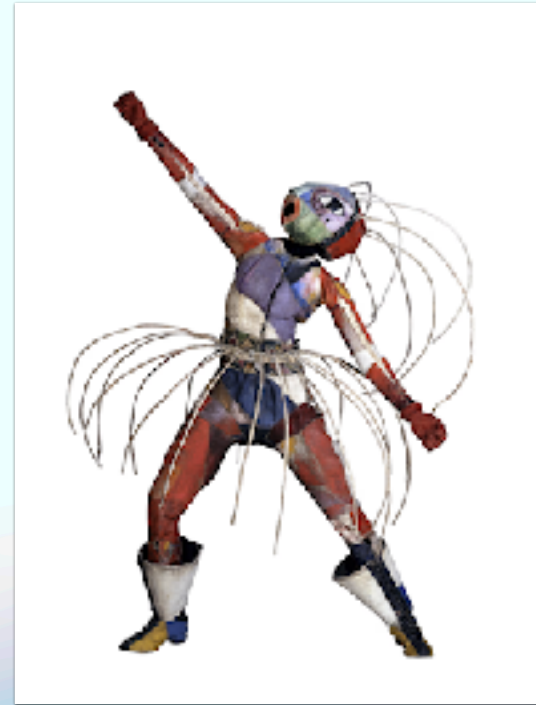




Folks, I can't tell you how excited I was to get this first pull request done. This just goes to show that even if you are not a hotshot coder or scripter you might also contribute to your favorite projects.

# Gave Back

Woo hoo!



Lavinia Schulz with Walter Holdt, Toboggan Frau, 1924

## Further Topics



Sophie Taeuber-Arp, Relief en bois peint, 1938

That is about all I can demonstrate for you. As you get comfortable with basic Git workflows, you will probably want to dig further into various topics.

# Git Security

Could be its own session . . .

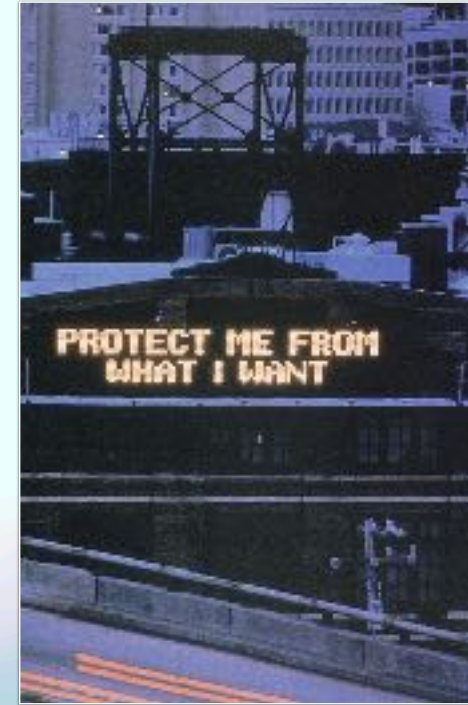
The first thing I want to talk about is perhaps something that's been an elephant in the room since the start, and that's securing Git. This topic could easily be its own session, and I'll cover just some highlights now.

- Use MFA on your account. GitHub is requiring this for most account types now, after several high profile incidents where malicious code was pushed to public repos.
- Use the best practices as provided by your platform. For example, Microsoft has a module on GitHub security that's shared by several of its courses.
- Use tokens for automations and take care to secure them. Different platforms have different methods for programmatically accessing repos, so check out their authentication docs.
- Mind that you are not inadvertently sharing credentials in the files in your repo
- And audit your permissions so folks only have the permissions they require.



# Git Security

Could be its own session . . .



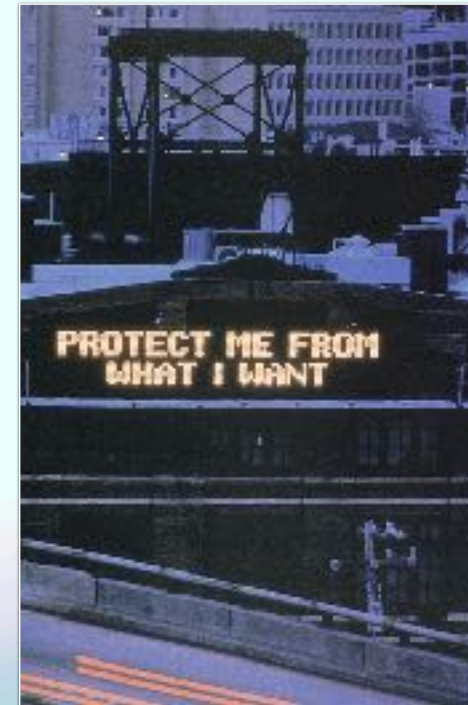
Jenny Holzer, Mixed Messages, 1987



# Git Security

Could be its own session . . .

- Use 2FA/MFA

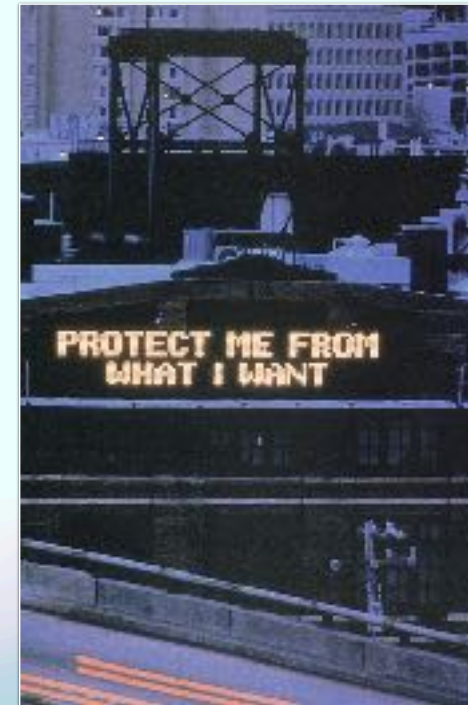


Jenny Holzer, Mixed Messages, 1987

# Git Security

Could be its own session . . .

- Use 2FA/MFA
- Platform best practices

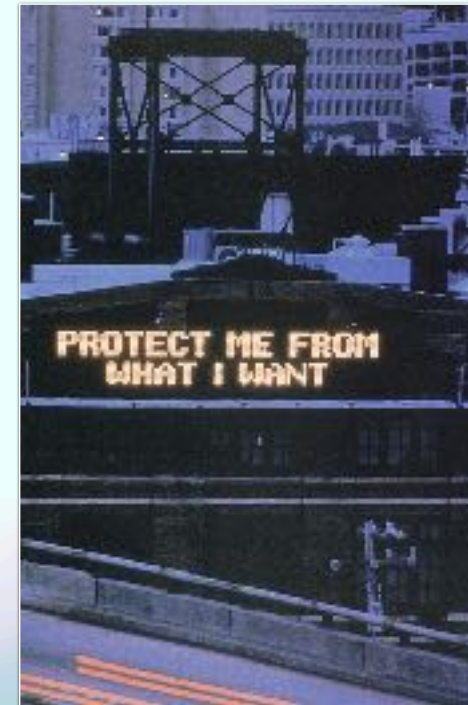


Jenny Holzer, Mixed Messages, 1987

# Git Security

Could be its own session . . .

- Use 2FA/MFA
- Platform best practices
- Account tokens/repo tokens

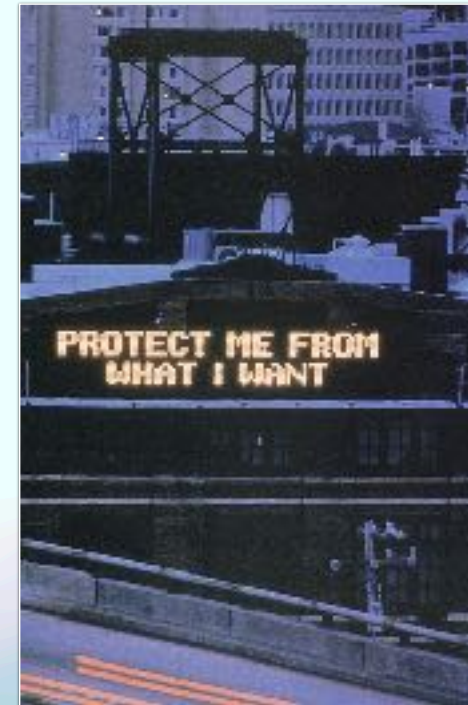


Jenny Holzer, Mixed Messages, 1987

# Git Security

Could be its own session . . .

- Use 2FA/MFA
- Platform best practices
- Account tokens/repo tokens
- Mind your credentials

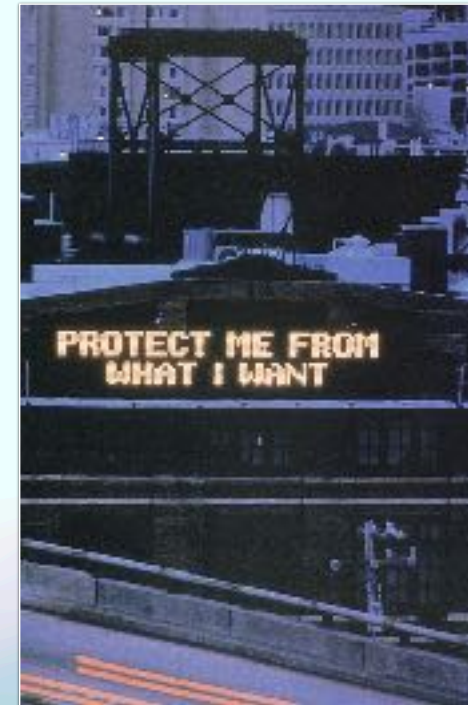


Jenny Holzer, Mixed Messages, 1987

# Git Security

Could be its own session . . .

- Use 2FA/MFA
- Platform best practices
- Account tokens/repo tokens
- Mind your credentials
- Audit permissions



Jenny Holzer, Mixed Messages, 1987

# Git More

## Advanced work

- As we saw with my pull request for Mac Health Check, maintainers can set up automations to run against submissions. If you have some repetitive task, there's probably an automation to do it. Account levels and platform choice will have something to do with what's available for you.
- There are several JNUC and MacAdmins sessions out there about automatically or manually pushing your production scripts to Jamf Pro via Git. And there was a session just this morning! I have links in the resources.
- You can also use the Fork, Commit, and Pull Request process to allow teammates to collaborate with your code -- and use appropriate permissions to encourage sharing or discourage YOLO-ing.
- You can also work your commits, merges, and other actions directly from your Git-enabled code editor, once you learn its conventions.

# Git More

Advanced work



Barbara Hepworth, Family of Man: Ancestor I, Anhe I, 1970



# Git More

## Advanced work

- Platform automations



Barbara Hepworth, Family of Man: Ancestor I, Anhe I, 1970



# Git More

## Advanced work

- Platform automations
- Push direct to Jamf



Barbara Hepworth, Family of Man: Ancestor I, Anhe I, 1970

# Git More

## Advanced work

- Platform automations
- Push direct to Jamf
- Collaborate with teammates



Barbara Hepworth, Family of Man: Ancestor I, Anhe I, 1970

# Git More

## Advanced work

- Platform automations
- Push direct to Jamf
- Collaborate with teammates
- Commit/push in your editor



Barbara Hepworth, Family of Man: Ancestor I, Anhe I, 1970

# Resources

Now for those resources . . .

All the stuff is in the readme in the repository, including the link to the Keynote.

# Resources - Git and Markdown

It's all in the repo!



- Git official page: <https://git-scm.com>
- Pro Git book: <https://git-scm.com/book/en/v2>
- Markdown: <https://daringfireball.net/projects/markdown/>
- Ignore .DS\_Store: [https://jonathanbossenger.com/2026/02/globally-ignore-ds\\_store-files-in-git/](https://jonathanbossenger.com/2026/02/globally-ignore-ds_store-files-in-git/)

Remember I said that the QR code would be shown again?

All the stuff is in the readme in the repository

I've got resources for Git itself and Markdown

As well as that source for the global ignore file.

# Resources - GitHub & GitLab

It's all in the repo!



- GitHub - Get Started - <https://docs.github.com/en/get-started>
- Microsoft Learning - GitHub Foundations Course, Part 1 - <https://learn.microsoft.com/en-us/training/paths/github-foundations/>
- and Part 2 - <https://learn.microsoft.com/en-us/training/paths/github-foundations-2/>
- GitLab - Get Started with GitLab - [https://docs.gitlab.com/user/get\\_started/](https://docs.gitlab.com/user/get_started/)
- 

Getting started courses for GitHub and GitLab

# Resources - Visual Studio Code

It's all in the repo!



- Visual Studio Code: [code.visualstudio.com/download](https://code.visualstudio.com/download)
- Overview: [code.visualstudio.com/docs/getstarted/overview](https://code.visualstudio.com/docs/getstarted/overview)
- Source control: [code.visualstudio.com/docs/sourcecontrol/overview](https://code.visualstudio.com/docs/sourcecontrol/overview)

Resources for Visual Studio Code



# Resources - Auto your admin

Sorry these are so tiny!



## PSU MacAdmins

- [Basic Git for non-Programmers – Chris Carr](#) | PSU MacAdmins 2025
- [Streamline Your macOS Administration with Github Actions - Jon Crain](#) - PSU MacAdmins 2023
- Sessions this week: Jamf as Code: Managing Scripts and Extension Attributes with GitLab CI (this morning),

## Jamf Resources

- [GitHub as the Source of Truth for Scripts in Jamf Pro | JNUC 2018](#)
- [GitHub as the source of truth for configuration in Jamf Pro | JNUC 2021](#) (in Japanese with English subtitles)
- Rich Trouton's script download - [Backing up macOS scripts from Jamf Pro | Der Flounder](#)

Of course there have been a bunch of sessions at PSU MacAdmins, and here are two recent ones.

And sessions at the Jamf Nation User Conference. And while we're on Jamf in particular, Rich Trouton has a script that I used to download all my production scripts from Jamf Pro.



# Resources - The Artists

With slide numbers



- Edmonia Lewis (2)
- Alice Aycock (3)
- Georgia O'Keefe (4)
- Käthe Kollwitz (5)
- Rosa Bonheur (9, 40)
- Artemisia Gentileschi (10)
- Ana Mendieta (11)
- Jenny Holzer (13, 100)
- Regina Cassolo Bracchi (15)
- Marcelle Cahn (22)
- Zubeida Agha (32)
- Anneke van der Feer (46)
- Tarsila do Amaral (60)
- Louise Nevelson (69)
- Berthe Morisot (85)
- Lavinia Schulz (98)
- Sophie Taeuber-Arp (99)
- Barbara Hepworth (101)

Here are all the artists whose work I featured with the slide number or numbers on which they appeared.

# Questions?



# Thank you!



Well, I know you had many choices for sessions today, and I appreciate that you choose mine. I hope that it proves useful to you. Please remember to provide feedback for the session, as it helps the conference organizers and me provide sessions that matter -- that's the QR code on the bottom left, and the link is also in Sched and in the shared session notes.

Don't hesitate to say hi in the hallway track or at meal times, to chat about this session in particular or the trials of the Mac Admin in higher education, or even one of my Special Interests.

Have a great rest of your conference!